

Designing Distributed Systems

Bản dịch song ngữ Anh–Việt · Patterns and Paradigms for Scalable, Reliable
Services

Brendan Burns (nguyên tác)

2026

Mục lục

Preface — Lời nói đầu	4
Who Should Read This Book — Ai nên đọc cuốn sách này	4
Why I Wrote This Book — Vì sao tôi viết cuốn sách này	5
The World of Distributed Systems Today — Thế giới của hệ phân tán ngày nay	6
Navigating This Book — Cách điều hướng cuốn sách này	6
Conventions Used in This Book — Các quy ước được dùng trong cuốn sách này	7
Online Resources — Tài nguyên trực tuyến	10
Using Code Examples — Sử dụng các ví dụ mã nguồn	11
O'Reilly Safari — O'Reilly Safari	12
How to Contact Us — Cách liên hệ với chúng tôi	12
Acknowledgments — Lời cảm ơn	13
CHAPTER 1 Introduction — CHAPTER 1 Giới thiệu	14
A Brief History of Systems Development — Lược sử phát triển hệ thống	15
A Brief History of Patterns in Software Development — Lược sử các mẫu trong phát triển phần mềm	16
Formalization of Algorithmic Programming — Sự hình thức hóa lập trình thuật toán	16
Patterns for Object-Oriented Programming — Các mẫu cho lập trình hướng đối tượng	17
The Rise of Open Source Software — Sự trỗi dậy của phần mềm mã nguồn mở	18
The Value of Patterns, Practices, and Components — Giá trị của các mẫu, thực hành và thành phần	20
Standing on the Shoulders of Giants — Đứng trên vai những người khổng lồ	20
A Shared Language for Discussing Our Practice — Một ngôn ngữ chung để bàn về thực hành của chúng ta	21
Shared Components for Easy Reuse — Các thành phần chung để dễ tái sử dụng	22
Summary — Tóm tắt	23
PART I Single-Node Patterns — PART I Các mẫu trên một nút đơn	23
Motivations — Động lực	23
Summary — Tóm tắt	25
CHAPTER 2 The Sidecar Pattern — CHAPTER 2 Mẫu Sidecar	27
An Example Sidecar: Adding HTTPS to a Legacy Service — Một ví dụ về Sidecar: Thêm HTTPS cho một dịch vụ kế thừa	28
Dynamic Configuration with Sidecars — Cấu hình động với Sidecar	29
Modular Application Containers — Container ứng dụng theo mô-đun	32
Hands On: Deploying the topz Container — Thực hành (Hands On): Triển khai container topz	33
Building a Simple PaaS with Sidecars — Xây dựng một PaaS đơn giản bằng Sidecar	34
Designing Sidecars for Modularity and Reusability — Thiết kế Sidecar cho tính mô-đun và khả năng tái sử dụng	36

Parameterized Containers — Container tham số hóa	36
Define Each Container's API — Định nghĩa API cho mỗi Container	37
Documenting Your Containers — Lập tài liệu cho các Container của bạn	39
Summary — Tóm tắt	40
CHAPTER 3 Ambassadors — CHAPTER 3 Ambassador (Đại sứ)	42
Using an Ambassador to Shard a Service — Dùng Ambassador để Phân mảnh một Dịch vụ	43
Hands On: Implementing a Sharded Redis — Thực hành: Hiện thực một Redis Phân mảnh	46
Using an Ambassador for Service Brokering — Dùng Ambassador để Làm Trung gian Dịch vụ	48
Using an Ambassador to Do Experimentation or Request Splitting — Dùng Ambassador để Thử nghiệm hoặc Tách Yêu cầu	50
Hands On: Implementing 10% Experiments — Thực hành: Hiện thực Thử nghiệm 10%	51
CHAPTER 4 Adapters — CHAPTER 4 Mẫu Adapter	55
Monitoring — Giám sát	56
Hands On: Using Prometheus for Monitoring — Thực hành (Hands On): Dùng Prometheus để giám sát	58
Logging — Ghi log	59
Hands On: Normalizing Different Logging Formats with Fluentd — Thực hành (Hands On): Chuẩn hóa các định dạng log khác nhau với Fluentd	62
Adding a Health Monitor — Thêm một bộ giám sát sức khỏe	64
Hands On: Adding Rich Health Monitoring for MySQL — Thực hành (Hands On): Thêm giám sát sức khỏe phong phú cho MySQL	65
PART II Serving Patterns — PART II Các mẫu phục vụ	67
Introduction to Microservices — Giới thiệu về Microservices	68
CHAPTER 5 Replicated Load-Balanced Services — CHAPTER 5 Dịch vụ nhân bản được cân bằng tải	72
Stateless Services — Dịch vụ không trạng thái	72
Readiness Probes for Load Balancing — Đầu dò sẵn sàng cho việc cân bằng tải	74
Hands On: Creating a Replicated Service in Kubernetes — Thực hành (Hands On): Tạo một dịch vụ nhân bản trong Kubernetes	75
Session Tracked Services — Dịch vụ theo dõi phiên	77
Application-Layer Replicated Services — Dịch vụ nhân bản ở tầng ứng dụng	80
Introducing a Caching Layer — Giới thiệu một tầng đệm (cache)	80
Deploying Your Cache — Triển khai cache của bạn	81
Hands On: Deploying the Caching Layer — Thực hành (Hands On): Triển khai tầng đệm	85
Expanding the Caching Layer — Mở rộng tầng đệm	88
Rate Limiting and Denial-of-Service Defense — Giới hạn tốc độ và phòng thủ chống từ chối dịch vụ	88
SSL Termination — Kết thúc SSL	89
Hands On: Deploying nginx and SSL Termination — Thực hành (Hands On): Triển khai nginx và kết thúc SSL	90
Summary — Tóm tắt	94
CHAPTER 6 Sharded Services — CHAPTER 6 Dịch vụ phân mảnh (Sharded Services)	95
Sharded Caching — Sharded Caching (Đệm phân mảnh)	96
Why You Might Need a Sharded Cache — Vì sao bạn có thể cần đến một cache phân mảnh	98
The Role of the Cache in System Performance — Vai trò của cache trong hiệu năng hệ thống	99
Replicated, Sharded Caches — Cache phân mảnh, nhân bản	101

Hands On: Deploying an Ambassador and Memcache for a Sharded Cache — Thực hành (Hands On): Triển khai một Ambassador và Memcache cho một Cache phân mảnh . . .	102
An Examination of Sharding Functions — Khảo sát về các hàm phân mảnh	106
Selecting a Key — Chọn khóa	107
Consistent Hashing Functions — Các hàm băm nhất quán (Consistent Hashing Functions) .	109
Hands On: Building a Consistent HTTP Sharding Proxy — Thực hành (Hands On): Xây dựng một Proxy phân mảnh HTTP nhất quán	110
Sharded, Replicated Serving — Phục vụ phân mảnh, nhân bản	111
Hot Sharding Systems — Các hệ thống phân mảnh nóng (Hot Sharding)	112
CHAPTER 7 Scatter/Gather — CHAPTER 7 Scatter/Gather	115
Scatter/Gather with Root Distribution — Scatter/Gather với Phân phối tại Gốc	116
Hands On: Distributed Document Search — Thực hành (Hands On): Tìm kiếm Tài liệu Phân tán	117
Scatter/Gather with Leaf Sharding — Scatter/Gather với Phân mảnh tại Lá	119
Hands On: Sharded Document Search — Thực hành (Hands On): Tìm kiếm Tài liệu có Phân mảnh	121
Choosing the Right Number of Leaves — Chọn Số lượng Lá Phù hợp	122
Scaling Scatter/Gather for Reliability and Scale — Mở rộng Scatter/Gather để Đạt Độ tin cậy và Quy mô	124
CHAPTER 8 Functions and Event-Driven Processing — CHAPTER 8 Hàm và Xử lý hướng Sự kiện	127
Determining When FaaS Makes Sense — Xác định Khi nào FaaS Hợp lý	129
The Benefits of FaaS — Lợi ích của FaaS	129
The Challenges of FaaS — Những Thách thức của FaaS	130
The Need for Background Processing — Nhu cầu Xử lý Nền	131
The Need to Hold Data in Memory — Nhu cầu Giữ Dữ liệu trong Bộ nhớ	132
The Costs of Sustained Request-Based Processing — Chi phí của Xử lý Dựa trên Yêu cầu Liên tục	132
Patterns for FaaS — Các Mẫu cho FaaS	133
The Decorator Pattern: Request or Response Transformation — Mẫu Decorator: Biến đổi Yêu cầu hoặc Phản hồi	133
Hands On: Adding Request Defaulting Prior to Request Processing — Thực hành: Thêm Gán mặc định cho Yêu cầu Trước khi Xử lý Yêu cầu	137
Handling Events — Xử lý Sự kiện	138
Hands On: Implementing Two-Factor Authentication — Thực hành: Hiện thực Xác thực Hai Yếu tố	139
Event-Based Pipelines — Pipeline Dựa trên Sự kiện	140
Hands On: Implementing a Pipeline for New-User Signup — Thực hành: Hiện thực một Pipeline cho Đăng ký Người dùng Mới	142
CHAPTER 9 Ownership Election — CHAPTER 9 Bầu chọn Quyền sở hữu	144
Determining If You Even Need Master Election — Xác định Liệu Bạn Có Thực Sự Cần Bầu chọn Master Hay Không	147
The Basics of Master Election — Những Điều Cơ Bản Về Bầu chọn Master	149
Hands On: Deploying etcd — Thực hành: Triển khai etcd	150
Implementing Locks — Triển khai Khóa (Lock)	153
Hands On: Implementing Locks in etcd — Thực hành: Triển khai Khóa trong etcd	157
Implementing Ownership — Triển khai Quyền sở hữu	158

Hands On: Implementing Leases in etcd — Thực hành: Triển khai Hợp đồng thuê (Lease) trong etcd	159
Handling Concurrent Data Manipulation — Xử lý Thao tác Dữ liệu Đồng thời	160
PART III Batch Computational Patterns — PART III Các mẫu tính toán theo lô	163
CHAPTER 10 Work Queue Systems — CHAPTER 10 Hệ thống hàng đợi công việc	164
A Generic Work Queue System — Một hệ thống hàng đợi công việc tổng quát	165
The Source Container Interface — Giao diện container nguồn	166
The Worker Container Interface — Giao diện container worker	169
The Shared Work Queue Infrastructure — Hạ tầng hàng đợi công việc dùng chung	171
Hands On: Implementing a Video Thumbnailer — Thực hành (Hands On): Triển khai bộ tạo thumbnail video	174
Dynamic Scaling of the Workers — Co giãn động số lượng worker	175
The Multi-Worker Pattern — Mẫu đa worker	178
CHAPTER 11 Event-Driven Batch Processing — CHAPTER 11 Xử Lý Theo Lô Hướng Sự Kiện	180
Patterns of Event-Driven Processing — Các Mẫu Xử Lý Hướng Sự Kiện	183
Copier — Copier	183
Filter — Filter	184
Splitter — Splitter	187
Sharder — Sharder	188
Merger — Merger	191
Hands On: Building an Event-Driven Flow for New User Sign-Up — Thực hành (Hands On): Xây Dựng Một Luồng Hướng Sự Kiện Cho Việc Đăng Ký Người Dùng Mới	194
Publisher/Subscriber Infrastructure — Hạ Tầng Publisher/Subscriber	198
Hands On: Deploying Kafka — Thực hành (Hands On): Triển Khai Kafka	199
CHAPTER 12 Coordinated Batch Processing — CHAPTER 12 Xử lý theo lô có điều phối	203
Join (or Barrier Synchronization) — Join (hay Đồng bộ hóa rào chắn - Barrier Synchronization)	205
Reduce — Reduce	208
Hands On: Count — Thực hành (Hands On): Đếm	208
Sum — Sum	210
Histogram — Histogram	211
Hands On: An Image Tagging and Processing Pipeline — Thực hành (Hands On): Một pipeline gắn thẻ và xử lý ảnh	212
CHAPTER 13 Conclusion: A New Beginning? — CHAPTER 13 Kết luận: Một khởi đầu mới?	217

Preface — Lời nói đầu

Who Should Read This Book — Ai nên đọc cuốn sách này

At this point, nearly every developer is a developer or consumer (or both) of distributed systems. Even relatively simple mobile applications are backed with cloud APIs so that their data can be present on whatever device the customer happens to be using. Whether you are new to developing distributed systems or an expert with scars on your hands to prove it, the patterns and components described in this book can transform your development of distributed systems from art to science. Reusable components and patterns for distributed systems will enable you to focus on the core details of your application. This book will help any developer become better, faster, and more efficient at building distributed systems.

Ở thời điểm hiện tại, gần như mọi lập trình viên đều là người phát triển hoặc người dùng (hoặc cả hai) của các hệ phân tán. Ngay cả những ứng dụng di động tương đối đơn giản cũng được hậu thuẫn bởi các API trên đám mây để dữ liệu có thể hiện diện trên bất kỳ thiết bị nào mà khách hàng đang dùng. Dù bạn mới bắt đầu phát triển hệ phân tán hay là một chuyên gia với đầy “vết sẹo trên tay” để chứng minh điều đó, các mẫu thiết kế (design pattern) và thành phần được mô tả trong cuốn sách này có thể biến việc phát triển hệ phân tán của bạn từ một nghệ thuật thành một khoa học. Các thành phần tái sử dụng và mẫu thiết kế cho hệ phân tán sẽ cho phép bạn tập trung vào những chi tiết cốt lõi của ứng dụng. Cuốn sách này sẽ giúp mọi lập trình viên trở nên giỏi hơn, nhanh hơn và hiệu quả hơn trong việc xây dựng hệ phân tán.

Why I Wrote This Book — Vì sao tôi viết cuốn sách này

Throughout my career as a developer of a variety of software systems from web search to the cloud, I have built a large number of scalable, reliable distributed systems. Each of these systems was, by and large, built from scratch. In general, this is true of all distributed applications. Despite having many of the same concepts and even at times nearly identical logic, the ability to apply patterns or reuse components is often very, very challenging. This forced me to waste time reimplementing systems, and each system ended up less polished than it might have otherwise been.

Trong suốt sự nghiệp phát triển nhiều loại hệ thống phần mềm khác nhau, từ tìm kiếm web cho đến đám mây, tôi đã xây dựng rất nhiều hệ phân tán có khả năng mở rộng và đáng tin cậy. Mỗi hệ thống trong số đó về cơ bản đều được xây dựng từ đầu. Nhìn chung, điều này đúng với tất cả các ứng dụng phân tán. Mặc dù có nhiều khái niệm giống nhau, và đôi khi thậm chí có logic gần như y hệt, khả năng áp dụng các mẫu thiết kế hay tái sử dụng các thành phần thường rất, rất khó khăn. Điều này buộc tôi phải lãng phí thời gian để hiện thực lại các hệ thống, và mỗi hệ thống cuối cùng lại kém hoàn thiện hơn so với mức đáng lẽ có thể đạt được.

The recent introduction of containers and **container** orchestrators fundamentally changed the landscape of **distributed system** development. Suddenly we have an object and **interface** for expressing core distributed system patterns and building reusable containerized components. I wrote this book to bring together all of the practitioners of distributed systems, giving us a shared language and common standard library so that we can all build better systems more quickly.

Sự xuất hiện gần đây của container và các bộ điều phối container (orchestrator) đã thay đổi căn bản bức tranh của việc phát triển hệ phân tán. Bỗng nhiên chúng ta có một đối tượng và một giao diện để diễn đạt các mẫu thiết kế cốt lõi của hệ phân tán và để xây dựng những thành phần đóng gói container tái sử dụng được. Tôi viết cuốn sách này nhằm quy tụ tất cả những người thực hành hệ phân tán lại với nhau, mang đến cho chúng ta một ngôn ngữ chung và một thư viện chuẩn chung để tất cả đều có thể xây dựng những hệ thống tốt hơn một cách nhanh hơn.

vii

vii

www.allitebooks.com

www.allitebooks.com

The World of Distributed Systems Today — Thế giới của hệ phân tán ngày nay

Once upon a time, people wrote programs that ran on one machine and were also accessed from that machine. The world has changed. Now, nearly every application is a distributed system running on multiple machines and accessed by multiple users from all over the world. Despite their prevalence, the design and development of these systems is often a black art practiced by a select group of wizards. But as with everything in technology, the world of distributed systems is advancing, regularizing, and abstracting. In this book I capture a collection of repeatable, generic patterns that can make the development of reliable distributed systems more approachable and efficient. The adoption of patterns and reusable components frees developers from reimplementing the same systems over and over again. This time is then freed to focus on building the core application itself.

Ngày xưa ngày xưa, người ta viết những chương trình chạy trên một máy và cũng được truy cập từ chính máy đó. Thế giới đã đổi thay. Giờ đây, gần như mọi ứng dụng đều là một hệ phân tán chạy trên nhiều máy và được truy cập bởi nhiều người dùng trên khắp thế giới. Mặc dù phổ biến như vậy, việc thiết kế và phát triển các hệ thống này thường vẫn là một thuật phù thủy được thực hành bởi một nhóm pháp sư chọn lọc. Nhưng cũng như mọi thứ trong công nghệ, thế giới của hệ phân tán đang tiến bộ, được chuẩn hóa và trừu tượng hóa. Trong cuốn sách này tôi đúc kết một tập hợp các mẫu thiết kế tổng quát, có thể lặp lại, giúp việc phát triển những hệ phân tán đáng tin cậy trở nên dễ tiếp cận và hiệu quả hơn. Việc áp dụng các mẫu thiết kế và thành phần tái sử dụng giải phóng các lập trình viên khỏi việc hiện thực lại cùng những hệ thống ấy hết lần này đến lần khác. Thời gian này nhờ đó được dành để tập trung vào việc xây dựng chính ứng dụng cốt lõi.

Navigating This Book — Cách điều hướng cuốn sách này

This book is organized into a 4 parts as follows:

Cuốn sách này được tổ chức thành 4 phần như sau:

Chapter 1, Introduction Introduces distributed systems and explains why patterns and reusable components can make such a difference in the rapid development of reliable distributed systems.

Chương 1, Giới thiệu Giới thiệu về hệ phân tán và giải thích vì sao các mẫu thiết kế và thành phần tái sử dụng có thể tạo ra khác biệt lớn đến vậy trong việc phát triển nhanh những hệ phân tán đáng tin cậy.

Part I, Single-**Node** Patterns Chapters 2 through 4 discuss reusable patterns and components that occur on individual nodes within a distributed system. It covers the side-car, **adapter**, and **ambassador** single-node patterns.

Phần I, Mẫu đơn nút (Single-Node Patterns) Các chương 2 đến 4 bàn về những mẫu thiết kế và thành phần tái sử dụng xuất hiện trên từng nút riêng lẻ trong một hệ phân tán. Phần này bao quát các mẫu đơn nút sidecar, adapter và ambassador.

Part II, Serving Patterns Chapters 8 and 9 cover multi-node distributed patterns for long-running serving systems like web applications. Patterns for replicating, scaling, and **master election** are discussed.

Phần II, Mẫu phục vụ (Serving Patterns) Các chương 8 và 9 bao quát những mẫu phân tán đa nút dành cho các hệ thống phục vụ chạy lâu dài như ứng dụng web. Phần này bàn về các mẫu nhân bản, mở rộng quy mô và bầu chọn master.

Part III, Batch Computational Patterns Chapters 10 through 12 cover distributed system patterns for large-scale batch data processing covering work queues, **event**-based processing, and coordinated workflows.

Phần III, Mẫu tính toán theo lô (Batch Computational Patterns) Các chương 10 đến 12 bao quát những mẫu hệ phân tán cho xử lý dữ liệu theo lô quy mô lớn, bao gồm hàng đợi công việc, xử lý dựa trên sự kiện và các luồng công việc được điều phối.

If you are an experienced distributed systems engineer, you can likely skip the first couple of chapters, though you may want to skim them to understand how we expect these patterns to be applied and why we think the general notion of distributed system patterns is so important.

Nếu bạn là một kỹ sư hệ phân tán có kinh nghiệm, có lẽ bạn có thể bỏ qua vài chương đầu, mặc dù bạn có thể muốn đọc lướt qua chúng để hiểu chúng tôi kỳ vọng các mẫu thiết kế này được áp dụng như thế nào và vì sao chúng tôi cho rằng khái niệm chung về mẫu thiết kế cho hệ phân tán lại quan trọng đến vậy.

Everyone will likely find utility in the single-node patterns as they are the most generic and most reusable patterns in the book.

Hầu như ai cũng sẽ thấy các mẫu đơn nút hữu ích, bởi chúng là những mẫu tổng quát nhất và tái sử dụng được nhiều nhất trong cả cuốn sách.

viii | Preface

viii | Lời nói đầu

www.allitebooks.com Depending on your goals and the systems you are interested in developing, you can choose to focus on either large-scale big data patterns, or patterns for long-running servers (or both). The two parts are largely independent from each other and can be read in any order.

www.allitebooks.com Tùy vào mục tiêu của bạn và những hệ thống bạn quan tâm phát triển, bạn có thể chọn tập trung vào các mẫu dữ liệu lớn quy mô lớn, hoặc các mẫu cho những máy chủ chạy lâu dài (hoặc cả hai). Hai phần này phần lớn độc lập với nhau và có thể đọc theo bất kỳ thứ tự nào.

Likewise, if you have extensive distributed system experience, you may find that some of the early patterns chapters (e.g., Part II on naming, discovery, and **load balancing**) are redundant with what you already know, so feel free to skim through to gain the high-level insights—but don't forget to look at all of the pretty pictures!

Tương tự, nếu bạn có nhiều kinh nghiệm về hệ phân tán, bạn có thể thấy một số chương mẫu thiết kế ở phần đầu (ví dụ, Phần II về đặt tên, khám phá và cân bằng tải) trùng lặp với những gì bạn đã biết, nên cứ tự nhiên đọc lướt để nắm các kiến thức ở mức tổng quan — nhưng đừng quên ngắm tất cả những hình minh họa đẹp đẽ nhé!

Conventions Used in This Book — Các quy ước được dùng trong cuốn sách này

The following typographical conventions are used in this book:

Những quy ước về kiểu chữ sau đây được dùng trong cuốn sách này:

Italic Indicates new terms, URLs, email addresses, filenames, and file extensions.

Chữ nghiêng Chỉ các thuật ngữ mới, URL, địa chỉ email, tên tệp và phần mở rộng tệp.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or **function** names, databases, data types, environment variables, statements, and keywords.

Dùng cho các đoạn liệt kê chương trình, cũng như trong các đoạn văn để chỉ những phần tử của chương trình như tên biến hoặc tên hàm, cơ sở dữ liệu, kiểu dữ liệu, biến môi trường, câu lệnh và từ khóa.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Hiển thị các lệnh hoặc văn bản khác mà người dùng cần gõ y nguyên.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.

Hiển thị văn bản cần được thay thế bằng các giá trị do người dùng cung cấp hoặc các giá trị được xác định theo ngữ cảnh.



This icon signifies a tip, suggestion, or general note.

Biểu tượng này biểu thị một mẹo, gợi ý, hoặc ghi chú chung.



This icon indicates a warning or caution.

Biểu tượng này biểu thị một cảnh báo hoặc lời nhắc cẩn trọng.

Online Resources — Tài nguyên trực tuyến

Though this book describes generally applicable distributed system patterns, it expects that readers are familiar with containers and container orchestration systems.

Mặc dù cuốn sách này mô tả các mẫu thiết kế hệ phân tán có tính ứng dụng tổng quát, nó kỳ vọng người đọc đã quen thuộc với container và các hệ thống điều phối container.

Preface | ix If you don't have a lot of pre-existing knowledge about these things, we recommend the following resources:

Lời nói đầu | ix Nếu bạn chưa có nhiều kiến thức sẵn có về những thứ này, chúng tôi

khuyến nghị các tài nguyên sau:

- <https://docker.io> • <https://kubernetes.io> • <https://dcos.io>
- <https://docker.io> • <https://kubernetes.io> • <https://dcos.io>

Using Code Examples — Sử dụng các ví dụ mã nguồn

Supplemental material (code examples, exercises, etc.) is available for download at <https://github.com/brendandburns/distributed-systems>.

Tài liệu bổ sung (ví dụ mã nguồn, bài tập, v.v.) có sẵn để tải về tại <https://github.com/brendandburns/designing-distributed-systems>.

This book is here to help you get your **job** done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing a CD-ROM of examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

Cuốn sách này có mặt để giúp bạn hoàn thành công việc của mình. Nói chung, nếu cuốn sách có kèm ví dụ mã nguồn, bạn có thể sử dụng nó trong các chương trình và tài liệu của bạn. Bạn không cần liên hệ với chúng tôi để xin phép trừ khi bạn sao chép lại một phần đáng kể của mã. Chẳng hạn, viết một chương trình sử dụng vài đoạn mã từ cuốn sách này thì không cần xin phép. Bán hoặc phân phối một đĩa CD-ROM chứa các ví dụ từ sách của O'Reilly thì cần xin phép. Trả lời một câu hỏi bằng cách trích dẫn cuốn sách này và dẫn lại mã ví dụ thì không cần xin phép. Tích hợp một lượng đáng kể mã ví dụ từ cuốn sách này vào tài liệu sản phẩm của bạn thì cần xin phép.

We appreciate, but do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “Designing Distributed Systems by Brendan Burns (O'Reilly). Copyright 2018 Brendan Burns, 978-1-491-98364-5.”

Chúng tôi trân trọng, nhưng không bắt buộc, việc ghi nhận nguồn. Phần ghi nhận nguồn thường gồm tiêu đề, tác giả, nhà xuất bản và ISBN. Ví dụ: “Designing Distributed Systems của Brendan Burns (O'Reilly). Copyright 2018 Brendan Burns, 978-1-491-98364-5.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

Nếu bạn cảm thấy việc sử dụng các ví dụ mã của mình nằm ngoài phạm vi sử dụng hợp lý hoặc ngoài sự cho phép nêu trên, cứ tự nhiên liên hệ với chúng tôi tại permissions@oreilly.com.

O'Reilly Safari — O'Reilly Safari



Safari (formerly Safari Books Online) is a membership-based training and reference platform for enterprise, government, educators, and individuals.

Safari (trước đây là Safari Books Online) là một nền tảng đào tạo và tra cứu dựa trên hình thức thành viên, dành cho doanh nghiệp, cơ quan chính phủ, nhà giáo dục và cá nhân.

Members have access to thousands of books, training videos, Learning Paths, interactive tutorials, and curated playlists from over 250 publishers, including O'Reilly Media, Harvard Business Review, Prentice Hall Professional, Addison-Wesley Professional, Microsoft Press, Sams, Que, Peachpit Press, Adobe, Focal Press, Cisco Press, John Wiley & Sons, Syngress, Morgan Kaufmann, IBM Redbooks, Packt, Adobe Press, FT Press, Apress, Manning, New Riders, McGraw-Hill, Jones & Bartlett, and Course Technology, among others.

Thành viên có quyền truy cập hàng nghìn cuốn sách, video đào tạo, Lộ trình Học tập (Learning Paths), hướng dẫn tương tác và các danh sách phát được tuyển chọn từ hơn 250 nhà xuất bản, bao gồm O'Reilly Media, Harvard Business Review, Prentice Hall Professional, Addison-Wesley Professional, Microsoft Press, Sams, Que, Peachpit Press, Adobe, Focal Press, Cisco Press, John Wiley & Sons, Syngress, Morgan Kaufmann, IBM Redbooks, Packt, Adobe Press, FT Press, Apress, Manning, New Riders, McGraw-Hill, Jones & Bartlett, và Course Technology, cùng nhiều nhà xuất bản khác.

x | Preface For more information, please visit <http://oreilly.com/safari>.

x | Lời nói đầu Để biết thêm thông tin, vui lòng truy cập <http://oreilly.com/safari>.

How to Contact Us — Cách liên hệ với chúng tôi

Please address comments and questions concerning this book to the publisher:

Vui lòng gửi các bình luận và câu hỏi liên quan đến cuốn sách này đến nhà xuất bản:

O'Reilly Media, Inc. 1005 Gravenstein Highway North Sebastopol, CA 95472 800-998-9938 (in the United States or Canada) 707-829-0515 (international or local) 707-829-0104 (fax)

O'Reilly Media, Inc. 1005 Gravenstein Highway North Sebastopol, CA 95472 800-998-9938 (tại Hoa Kỳ hoặc Canada) 707-829-0515 (quốc tế hoặc địa phương) 707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <http://bit.ly/designing-distributed-systems>.

Chúng tôi có một trang web cho cuốn sách này, nơi chúng tôi liệt kê các đính chính, ví dụ, và mọi thông tin bổ sung khác. Bạn có thể truy cập trang này tại <http://bit.ly/designing-distributed-systems>.

To comment or ask technical questions about this book, send email to bookquestions@oreilly.com.

Để bình luận hoặc đặt câu hỏi kỹ thuật về cuốn sách này, hãy gửi email đến bookquestions@oreilly.com.

For more information about our books, courses, conferences, and news, see our website at <http://www.oreilly.com>.

Để biết thêm thông tin về sách, khóa học, hội nghị và tin tức của chúng tôi, hãy xem trang web của chúng tôi tại <http://www.oreilly.com>.

Find us on Facebook: <http://facebook.com/oreilly>

Tìm chúng tôi trên Facebook: <http://facebook.com/oreilly>

Follow us on Twitter: <http://twitter.com/oreillymedia>

Theo dõi chúng tôi trên Twitter: <http://twitter.com/oreillymedia>

Watch us on YouTube: <http://www.youtube.com/oreillymedia>

Xem chúng tôi trên YouTube: <http://www.youtube.com/oreillymedia>

Acknowledgments — Lời cảm ơn

I'd like to thank my wife Robin and my children for everything they do to keep me happy and sane. To all of the people along the way who took the time to help me learn all of these things, many thanks! Also thanks to my parents for that first SE/30.

Tôi muốn cảm ơn vợ tôi, Robin, và các con tôi vì mọi điều họ làm để giữ cho tôi hạnh phúc và tỉnh táo. Gửi đến tất cả những người trên suốt chặng đường đã dành thời gian giúp tôi học được tất cả những điều này, xin chân thành cảm ơn! Cũng cảm ơn bố mẹ tôi vì chiếc SE/30 đầu tiên đó.

Preface | xi

Lời nói đầu | xi

CHAPTER 1 Introduction —

CHAPTER 1 Giới thiệu

Today's world of always-on applications and APIs have **availability** and **reliability** requirements that would have been required of only a handful of mission critical services around the globe only a few decades ago. Likewise, the potential for rapid, viral growth of a **service** means that every application has to be built to scale nearly instantly in response to user demand. These constraints and requirements mean that almost every application that is built—whether it is a consumer mobile app or a backend payments application—needs to be a **distributed system**.

Thế giới ngày nay với các ứng dụng và API luôn-bật (always-on) có những yêu cầu về tính sẵn sàng và độ tin cậy mà chỉ vài thập kỷ trước chỉ được đòi hỏi ở một số ít dịch vụ trọng yếu trên toàn cầu. Tương tự, khả năng một dịch vụ tăng trưởng nhanh chóng theo kiểu lan truyền nghĩa là mọi ứng dụng đều phải được xây dựng để mở rộng gần như tức thì nhằm đáp ứng nhu cầu người dùng. Những ràng buộc và yêu cầu này khiến hầu hết mọi ứng dụng được xây dựng — dù là một ứng dụng di động cho người tiêu dùng hay một ứng dụng thanh toán phía backend — đều cần phải là một hệ phân tán (distributed system).

But building distributed systems is challenging. Often, they are one-off bespoke solutions. In this way, distributed system development bears a striking resemblance to the world of software development prior to the development of modern **object-oriented programming** languages. Fortunately, as with the development of object-oriented languages, there have been technological advances that have dramatically reduced the challenges of building distributed systems. In this case, it is the rising popularity of containers and **container** orchestrators. As with the concept of objects within object-oriented programming, these containerized building blocks are the basis for the development of reusable components and patterns that dramatically simplify and make accessible the practices of building reliable distributed systems. In the following introduction, we give a brief history of the developments that have led to where we are today.

Nhưng việc xây dựng các hệ phân tán đầy thách thức. Thường thì chúng là những giải pháp riêng lẻ, đặt làm theo từng trường hợp. Theo nghĩa này, việc phát triển hệ phân tán có nét tương đồng đáng kinh ngạc với thế giới phát triển phần mềm trước khi các ngôn ngữ lập trình hướng đối tượng hiện đại ra đời. May mắn thay, cũng như sự ra đời của các ngôn ngữ hướng đối tượng, đã có những tiến bộ công nghệ làm giảm đáng kể các thách thức trong việc xây dựng hệ phân tán. Trong trường hợp này, đó là sự phổ biến ngày càng tăng của container và các bộ điều phối container (container orchestrator). Cũng như khái niệm về object trong lập trình hướng đối tượng, những khối xây dựng đóng gói container này là nền tảng cho việc phát triển các thành phần và mẫu tái sử dụng, giúp đơn giản hóa đáng kể và làm cho việc xây dựng các hệ phân tán đáng tin cậy trở nên dễ tiếp cận. Trong phần giới thiệu sau đây, chúng ta điểm qua ngắn gọn lịch sử của những bước phát triển đã đưa chúng ta đến vị trí hiện tại.

A Brief History of Systems Development — Lược sử phát triển hệ thống

In the beginning, there were machines built for specific purposes, such as calculating artillery tables or the tides, breaking codes, or other precise, complicated but rote mathematical applications. Eventually these purpose-built machines evolved into general-purpose programmable machines. And eventually they evolved from running

Thuở ban đầu, có những cỗ máy được chế tạo cho các mục đích cụ thể, chẳng hạn như tính toán bảng pháo binh hay thủy triều, giải mã, hoặc những ứng dụng toán học chính xác, phức tạp nhưng lặp đi lặp lại. Cuối cùng, những cỗ máy chuyên dụng này tiến hóa thành các máy lập trình đa năng. Và rồi chúng tiến hóa từ chỗ chạy

1 one program at a time to running multiple programs on a single machine via **time-sharing** operating systems, but these machines were still disjoint from each other.

một chương trình tại một thời điểm sang chạy nhiều chương trình trên một máy duy nhất thông qua các hệ điều hành chia sẻ thời gian (time-sharing), nhưng những cỗ máy này vẫn tách biệt với nhau.

Gradually, machines came to be networked together, and **client-server** architectures were born so that a relatively low-powered machine on someone's desk could be used to harness the greater power of a **mainframe** in another room or building. While this sort of client-server programming was somewhat more complicated than writing a program for a single machine, it was still fairly straightforward to understand. The client(s) made requests; the server(s) serviced those requests.

Dần dần, các máy được nối mạng với nhau, và kiến trúc client-server ra đời để một máy tương đối yếu trên bàn làm việc của ai đó có thể khai thác sức mạnh lớn hơn của một máy chủ lớn (mainframe) đặt ở phòng hay tòa nhà khác. Mặc dù kiểu lập trình client-server này phức tạp hơn đôi chút so với viết một chương trình cho một máy duy nhất, nó vẫn khá dễ hiểu. Client (các máy khách) gửi yêu cầu; server (các máy chủ) phục vụ những yêu cầu đó.

In the early 2000s, the rise of the internet and large-scale datacenters consisting of thousands of relatively low-cost commodity computers networked together gave rise to the widespread development of distributed systems . Unlike client-server architectures, distributed system applications are made up of multiple different applications running on different machines, or many replicas running across different machines, all communicating together to implement a system like web-search or a retail sales platform.

Đầu những năm 2000, sự trỗi dậy của internet và các trung tâm dữ liệu quy mô lớn gồm hàng nghìn máy tính phổ thông tương đối rẻ tiền được nối mạng với nhau đã làm nảy sinh việc phát triển rộng rãi các hệ phân tán. Khác với kiến trúc client-server, các ứng dụng hệ phân tán được tạo thành từ nhiều ứng dụng khác nhau chạy trên các máy khác nhau, hoặc nhiều bản sao (replica) chạy trên các máy khác nhau, tất cả cùng giao tiếp với nhau để hiện thực một hệ thống như tìm kiếm web hay một nền tảng bán lẻ.

Because of their distributed nature, when structured properly, distributed systems are inherently more reliable. And when architected correctly, they can lead to much more scalable organizational models for the teams of software engineers that built these systems. Unfortunately, these advantages come at a cost. These distributed systems can be significantly more complicated to design, build, and debug correctly. The engineering skills needed to build a reliable distributed system are significantly higher than those needed to build single-machine applications like mobile or web

frontends. Regardless, the need for reliable distributed systems only continues to grow. Thus there is a corresponding need for the tools, patterns, and practices for building them.

Do bản chất phân tán của mình, khi được cấu trúc đúng cách, các hệ phân tán vốn dĩ đáng tin cậy hơn. Và khi được kiến trúc đúng đắn, chúng có thể dẫn đến các mô hình tổ chức có khả năng mở rộng tốt hơn nhiều cho các nhóm kỹ sư phần mềm đã xây dựng những hệ thống này. Tiếc rằng, những ưu điểm này đi kèm cái giá phải trả. Các hệ phân tán này có thể phức tạp hơn đáng kể để thiết kế, xây dựng và gỡ lỗi cho đúng. Các kỹ năng kỹ thuật cần để xây dựng một hệ phân tán đáng tin cậy cao hơn đáng kể so với những kỹ năng cần để xây dựng các ứng dụng đơn máy như frontend di động hay web. Dù vậy, nhu cầu về các hệ phân tán đáng tin cậy vẫn không ngừng tăng lên. Vì thế cũng có một nhu cầu tương ứng về các công cụ, mẫu và thực hành để xây dựng chúng.

Fortunately, technology has also increased the ease with which you can build distributed systems. Containers, container images, and container orchestrators have all become popular in recent years because they are the foundation and building blocks for reliable distributed systems. Using containers and container orchestration as a foundation, we can establish a collection of patterns and reusable components. These patterns and components are a toolkit that we can use to build our systems more reliably and efficiently.

May mắn thay, công nghệ cũng đã làm tăng mức độ dễ dàng trong việc xây dựng hệ phân tán. Container, image container, và các bộ điều phối container đều trở nên phổ biến trong những năm gần đây bởi vì chúng là nền tảng và khối xây dựng cho các hệ phân tán đáng tin cậy. Lấy container và việc điều phối container làm nền tảng, chúng ta có thể thiết lập một tập hợp các mẫu và các thành phần tái sử dụng. Những mẫu và thành phần này là một bộ công cụ mà chúng ta có thể dùng để xây dựng hệ thống của mình một cách đáng tin cậy và hiệu quả hơn.

A Brief History of Patterns in Software Development — Lược sử các mẫu trong phát triển phần mềm

This is not the first time such a transformation has occurred in the software industry. For a better context on how patterns, practices, and reusable components have previously reshaped systems development, it is helpful to look at past moments when similar transformations have taken place.

Đây không phải lần đầu tiên một sự chuyển biến như vậy diễn ra trong ngành phần mềm. Để có bối cảnh tốt hơn về cách các mẫu, thực hành và thành phần tái sử dụng đã từng định hình lại việc phát triển hệ thống trước đây, ta nên nhìn lại những thời điểm trong quá khứ khi các chuyển biến tương tự đã xảy ra.

Formalization of Algorithmic Programming — Sự hình thức hóa lập trình thuật toán

Though people had been programming for more than a decade before its publication in 1962, Donald Knuth's collection, *The Art of Computer Programming* (Addison- Wesley Professional), marks an important chapter in the development of computer science. In particular, the books contain algorithms not designed for any specific computer, but rather to educate the reader on the algorithms themselves. These algorithms then could be adapted to the specific architecture of the machine being used or the specific problem that the reader was solving. This formalization was important because it provided users with a shared toolkit for building their programs, but also

because it showed that there was a general-purpose concept that programmers should learn and then subsequently apply in a variety of different contexts. The algorithms themselves, independent of any specific problem to solve, were worth understanding for their own sake.

Mặc dù người ta đã lập trình hơn một thập kỷ trước khi nó được xuất bản vào năm 1962, bộ sách *The Art of Computer Programming* (Addison-Wesley Professional) của Donald Knuth đánh dấu một chương quan trọng trong sự phát triển của khoa học máy tính. Cụ thể, các cuốn sách này chứa những thuật toán không được thiết kế cho bất kỳ máy tính cụ thể nào, mà nhằm giáo dục người đọc về chính các thuật toán đó. Những thuật toán này sau đó có thể được điều chỉnh cho phù hợp với kiến trúc cụ thể của máy đang dùng hoặc với vấn đề cụ thể mà người đọc đang giải quyết. Sự hình thức hóa này quan trọng vì nó cung cấp cho người dùng một bộ công cụ chung để xây dựng chương trình của họ, nhưng cũng vì nó cho thấy rằng có một khái niệm đa năng mà lập trình viên nên học rồi sau đó áp dụng vào nhiều bối cảnh khác nhau. Bản thân các thuật toán, độc lập với bất kỳ vấn đề cụ thể nào cần giải quyết, đều đáng được tìm hiểu vì giá trị tự thân của chúng.

Patterns for Object-Oriented Programming — Các mẫu cho lập trình hướng đối tượng

Knuth's books represent an important landmark in the thinking about computer programming, and algorithms represent an important component in the development of computer programming. However, as the complexity of programs grew, and the number of people writing a single program grew from the single digits to the double digits and eventually to the thousands, it became clear that procedural programming languages and algorithms were insufficient for the tasks of modern-day programming. These changes in computer programming led to the development of object-oriented programming languages, which elevated data, reusability, and **extensibility** to peers of the algorithm in the development of computer programs.

Các cuốn sách của Knuth là một cột mốc quan trọng trong tư duy về lập trình máy tính, và thuật toán là một thành phần quan trọng trong sự phát triển của lập trình máy tính. Tuy nhiên, khi độ phức tạp của các chương trình tăng lên, và số người viết một chương trình duy nhất tăng từ vài người lên hàng chục rồi cuối cùng đến hàng nghìn người, rõ ràng là các ngôn ngữ lập trình thủ tục và thuật toán đã không đủ cho các nhiệm vụ của lập trình hiện đại. Những thay đổi này trong lập trình máy tính đã dẫn đến sự phát triển của các ngôn ngữ lập trình hướng đối tượng, vốn nâng dữ liệu, khả năng tái sử dụng và khả năng mở rộng lên ngang hàng với thuật toán trong việc phát triển các chương trình máy tính.

In response to these changes to computer programming, there were changes to the patterns and practices for programming as well. Throughout the early to mid-1990s, there was an explosion of books on patterns for object-oriented programming. The most famous of these is the “gang of four” book, *Design Patterns: Elements of Reusable Object-Oriented Programming* by Erich Gamma et al. (Addison-Wesley Professional). *Design Patterns* gave a common language and framework to the task of programming. It described a series of **interface**-based patterns that could be reused in a variety of contexts. Because of advances in object-oriented programming and specifically interfaces, these patterns could also be implemented as generic reusable libraries. These libraries could be written once by a community of developers and reused repeatedly, saving time and improving reliability.

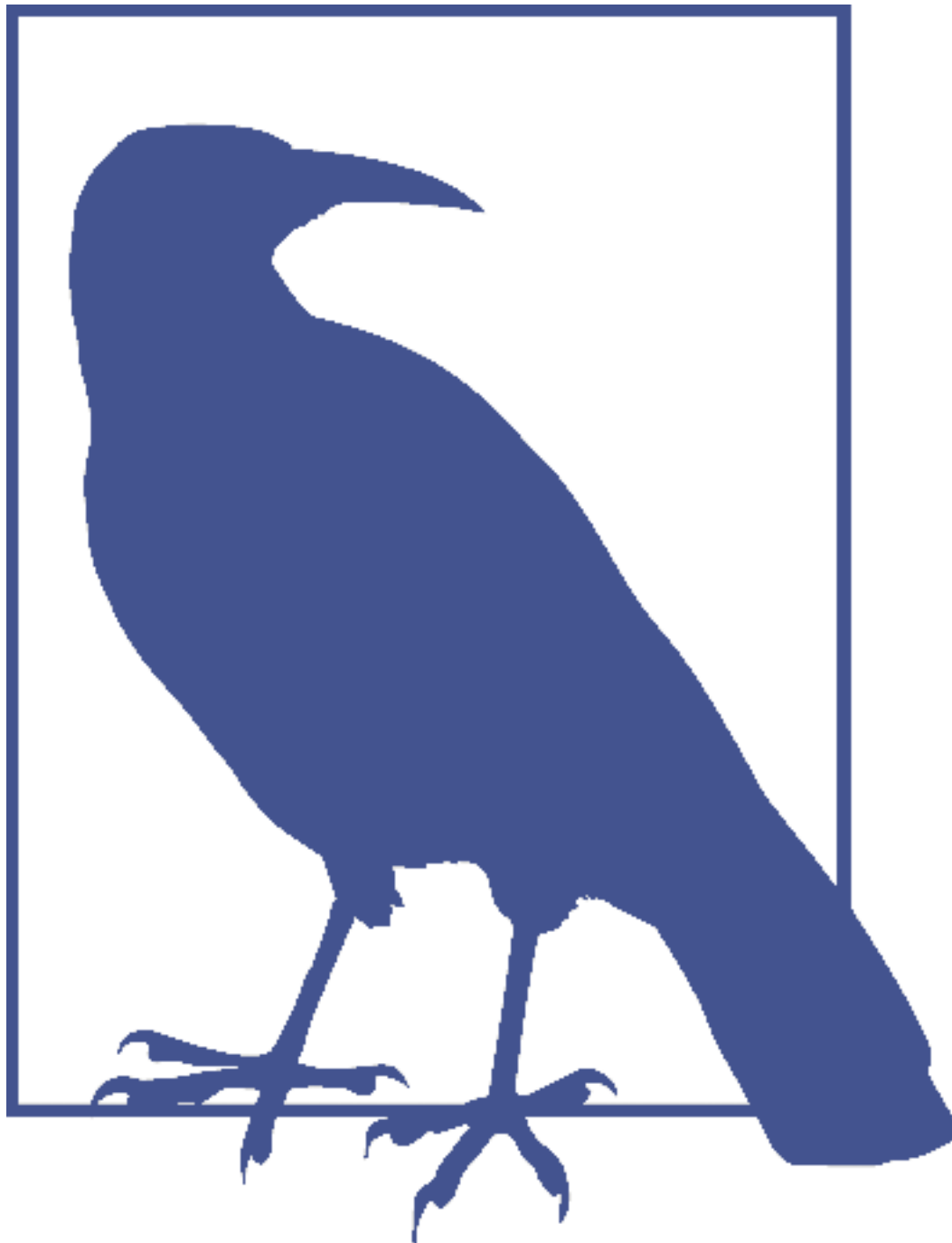
Để đáp lại những thay đổi này trong lập trình máy tính, cũng có những thay đổi đối với các mẫu và thực hành lập trình. Trong suốt đầu đến giữa thập niên 1990, đã có một sự bùng nổ của các cuốn sách về mẫu cho lập trình hướng đối tượng. Nổi tiếng

nhất trong số này là cuốn sách của “bộ tứ” (gang of four), Design Patterns: Elements of Reusable Object-Oriented Programming của Erich Gamma và cộng sự (Addison-Wesley Professional). Design Patterns đã mang lại một ngôn ngữ và khung làm việc chung cho công việc lập trình. Nó mô tả một loạt các mẫu dựa trên interface có thể được tái sử dụng trong nhiều bối cảnh khác nhau. Nhờ những tiến bộ trong lập trình hướng đối tượng và cụ thể là interface, các mẫu này cũng có thể được hiện thực thành các thư viện đa năng, tái sử dụng được. Những thư viện này có thể được viết một lần bởi một cộng đồng các nhà phát triển và được tái sử dụng nhiều lần, tiết kiệm thời gian và cải thiện độ tin cậy.

The Rise of Open Source Software — Sự trỗi dậy của phần mềm mã nguồn mở

Though the concept of developers sharing source code has been around nearly since the beginning of computing, and formal free software organizations have been in existence since the mid-1980s, the very late 1990s and the 2000s saw a dramatic increase in the development and distribution of **open source** software. Though open source is only tangentially related to the development of patterns for distributed systems, it is important in the sense that it was through the open source communities that it became increasingly clear that software development in general and distributed systems development in particular are community endeavors. It is important to note that all of the container technology that forms the foundation of the patterns described in this book has been developed and released as open source software. The value of patterns for both describing and improving the practice of distributed development is especially clear when you look at it from this community perspective.

Mặc dù khái niệm các nhà phát triển chia sẻ mã nguồn đã tồn tại gần như từ buổi sơ khai của máy tính, và các tổ chức phần mềm tự do chính thức đã ra đời từ giữa thập niên 1980, những năm rất cuối thập niên 1990 và thập niên 2000 đã chứng kiến sự gia tăng mạnh mẽ trong việc phát triển và phân phối phần mềm mã nguồn mở. Mặc dù mã nguồn mở chỉ liên quan một cách gián tiếp đến sự phát triển các mẫu cho hệ phân tán, nó quan trọng ở chỗ chính qua các cộng đồng mã nguồn mở mà người ta ngày càng thấy rõ rằng việc phát triển phần mềm nói chung và phát triển hệ phân tán nói riêng là những nỗ lực mang tính cộng đồng. Cần lưu ý rằng tất cả công nghệ container tạo nên nền tảng cho các mẫu được mô tả trong cuốn sách này đều đã được phát triển và phát hành dưới dạng phần mềm mã nguồn mở. Giá trị của các mẫu, cả trong việc mô tả lẫn cải thiện thực hành phát triển hệ phân tán, đặc biệt rõ ràng khi bạn nhìn nhận nó từ góc độ cộng đồng này.



What is a pattern for a distributed system? There are plenty of instructions out there that will tell you how to install specific distributed systems (such as a NoSQL database). Likewise, there are recipes for a specific collection of systems (like a MEAN stack). But when I speak of patterns, I'm referring to general blueprints for organizing distributed systems, without mandating any specific technology or application choices. The purpose of a pattern is to provide general advice or structure to guide your design. The hope is that such patterns will guide your thinking and also be generally applicable to a wide variety of applications and environments.

Một mẫu cho hệ phân tán là gì? Có rất nhiều hướng dẫn ngoài kia chỉ cho bạn cách cài đặt các hệ phân tán cụ thể (chẳng hạn một cơ sở dữ liệu NoSQL). Tương tự, có những công

thức cho một tập hợp hệ thống cụ thể (như một ngăn xếp MEAN). Nhưng khi tôi nói về các mẫu, ý tôi là những bản thiết kế tổng quát để tổ chức các hệ phân tán, mà không bắt buộc bất kỳ lựa chọn công nghệ hay ứng dụng cụ thể nào. Mục đích của một mẫu là cung cấp lời khuyên hay cấu trúc tổng quát để định hướng cho thiết kế của bạn. Hy vọng là những mẫu như vậy sẽ định hướng tư duy của bạn và cũng có thể áp dụng tổng quát cho nhiều loại ứng dụng và môi trường khác nhau.

The Value of Patterns, Practices, and Components — Giá trị của các mẫu, thực hành và thành phần

Before spending any of your valuable time reading about a series of patterns that I claim will improve your development practices, teach you new skills, and—let’s face it—change your life, it’s reasonable to ask: “Why?” What is it about the design patterns and practices that can change the way that we design and build software? In this section, I’ll lay out the reasons I think this is an important topic, and hopefully convince you to stick with me for the rest of the book.

Trước khi dành bất kỳ chút thời gian quý báu nào của bạn để đọc về một loạt các mẫu mà tôi tuyên bố sẽ cải thiện thực hành phát triển của bạn, dạy bạn những kỹ năng mới, và — thành thật mà nói — thay đổi cuộc đời bạn, sẽ hợp lý khi hỏi: “Tại sao?” Có điều gì ở các mẫu thiết kế và thực hành có thể thay đổi cách chúng ta thiết kế và xây dựng phần mềm? Trong phần này, tôi sẽ trình bày những lý do mà tôi cho rằng đây là một chủ đề quan trọng, và hy vọng thuyết phục được bạn đi cùng tôi trong phần còn lại của cuốn sách.

Standing on the Shoulders of Giants — Đứng trên vai những người khổng lồ

As a starting point, the value that patterns for distributed systems offer is the opportunity to figuratively stand on the shoulders of giants. It’s rarely the case that the problems we solve or the systems we build are truly unique. Ultimately, the combination of pieces that we put together and the overall business model that the software enables may be something that the world has never seen before. But the way the system is built and the problems it encounters as it aspires to be reliable, agile, and scalable are not new.

Để bắt đầu, giá trị mà các mẫu cho hệ phân tán mang lại là cơ hội đứng trên vai những người khổng lồ theo nghĩa bóng. Hiếm khi nào các vấn đề chúng ta giải quyết hay các hệ thống chúng ta xây dựng là thực sự độc nhất. Rốt cuộc, sự kết hợp các mảnh ghép mà chúng ta lắp ráp lại với nhau và mô hình kinh doanh tổng thể mà phần mềm cho phép có thể là thứ mà thế giới chưa từng thấy. Nhưng cách hệ thống được xây dựng và những vấn đề nó gặp phải khi nó hướng tới sự đáng tin cậy, linh hoạt và khả năng mở rộng thì không hề mới.

This, then, is the first value of patterns: they allow us to learn from the mistakes of others. Perhaps you have never built a distributed system before, or perhaps you have never built this type of distributed system. Rather than hoping that a colleague has some experience in this area or learning by making the same mistakes that others have already made, you can turn to patterns as your guide. Learning about patterns for distributed system development is the same as learning about any other best practice in computer programming. It accelerates your ability to build software without requiring that you have direct experience with the systems, mistakes, and firsthand learning that led to the codification of the pattern in the first place.

Đây, do đó, là giá trị đầu tiên của các mẫu: chúng cho phép chúng ta học hỏi từ sai lầm của người khác. Có thể bạn chưa bao giờ xây dựng một hệ phân tán trước đây, hoặc có thể bạn chưa bao giờ xây dựng loại hệ phân tán này. Thay vì hy vọng rằng một đồng nghiệp có chút kinh nghiệm trong lĩnh vực này, hay học bằng cách mắc cùng những sai lầm mà người khác đã mắc, bạn có thể tìm đến các mẫu như kim chỉ nam của mình. Học về các mẫu cho việc phát triển hệ phân tán cũng giống như học về bất kỳ thực hành tốt nhất nào khác trong lập trình máy tính. Nó tăng tốc khả năng xây dựng phần mềm của bạn mà không đòi hỏi bạn phải có kinh nghiệm trực tiếp với các hệ thống, sai lầm và sự học hỏi tận tay vốn đã dẫn đến việc hệ thống hóa mẫu đó ngay từ đầu.

A Shared Language for Discussing Our Practice — Một ngôn ngữ chung để bàn về thực hành của chúng ta

Learning about and accelerating our understanding of distributed systems is only the first value of having a shared set of patterns. Patterns have value even for experienced distributed system developers who already understand them well. Patterns provide a shared vocabulary that enables us to understand each other quickly. This understanding forms the basis for knowledge sharing and further learning.

Học hỏi và tăng tốc hiểu biết của chúng ta về các hệ phân tán mới chỉ là giá trị đầu tiên của việc có một tập hợp mẫu chung. Các mẫu có giá trị ngay cả đối với những nhà phát triển hệ phân tán dày dạn vốn đã hiểu rõ chúng. Các mẫu cung cấp một vốn từ vựng chung giúp chúng ta hiểu nhau nhanh chóng. Sự hiểu biết này tạo nền tảng cho việc chia sẻ tri thức và học hỏi sâu hơn.

To better understand this, imagine that we both are using the same object to build our house. I call that object a “Foo” while you call that object a “Bar.” How long will we spend arguing about the value of a Foo versus that of a Bar, or trying to explain the differing properties of Foo and Bar until we figure out that we’re speaking about the same object? Only once we determine that Foo and Bar are the same can we truly start learning from each other’s experience.

Để hiểu rõ hơn điều này, hãy tưởng tượng cả hai chúng ta đang dùng cùng một vật để xây nhà. Tôi gọi vật đó là “Foo” còn bạn gọi vật đó là “Bar.” Chúng ta sẽ mất bao lâu để tranh cãi về giá trị của một Foo so với một Bar, hay cố giải thích các thuộc tính khác nhau của Foo và Bar cho đến khi nhận ra rằng chúng ta đang nói về cùng một vật? Chỉ khi đã xác định được rằng Foo và Bar là một, chúng ta mới thực sự có thể bắt đầu học hỏi từ kinh nghiệm của nhau.

Without a common vocabulary, we waste time in arguments of “violent agreement” or in explaining concepts that others understand but know by another name. Consequently, another significant value of patterns is to provide a common set of names and definitions so that we don’t waste time worrying about naming, and instead get right down to discussing the details and implementation of the core concepts.

Không có một vốn từ vựng chung, chúng ta lãng phí thời gian vào các cuộc tranh cãi kiểu “đồng thuận một cách kịch liệt” (violent agreement) hoặc vào việc giải thích những khái niệm mà người khác đã hiểu nhưng biết đến dưới một cái tên khác. Do đó, một giá trị quan trọng nữa của các mẫu là cung cấp một tập hợp tên gọi và định nghĩa chung để chúng ta không phải lãng phí thời gian lo lắng về việc đặt tên, mà thay vào đó đi thẳng vào việc bàn luận chi tiết và cách hiện thực các khái niệm cốt lõi.

I have seen this happen in my short time working on containers. Along the way, the notion of a

sidecar container (described in Chapter 2 of this book) took hold within the container community. Because of this, we no longer have to spend time defining what it means to be a sidecar and can instead jump immediately to how the concept can be used to solve a particular problem. “If we just use a sidecar” ... “Yeah, and I know just the container we can use for that.” This example leads to the third value of patterns: the construction of reusable components.

Tôi đã thấy điều này xảy ra trong khoảng thời gian ngắn ngủi làm việc với container. Trong quá trình đó, khái niệm về một container sidecar (được mô tả trong Chương 2 của cuốn sách này) đã được cộng đồng container đón nhận. Nhờ vậy, chúng ta không còn phải mất thời gian định nghĩa sidecar nghĩa là gì nữa, mà có thể nhảy ngay đến việc khái niệm đó có thể được dùng ra sao để giải quyết một vấn đề cụ thể. “Nếu ta chỉ cần dùng một sidecar” ... “Ừ, và tôi biết chính xác cái container ta có thể dùng cho việc đó.” Ví dụ này dẫn đến giá trị thứ ba của các mẫu: việc xây dựng các thành phần tái sử dụng.

Shared Components for Easy Reuse — Các thành phần chung để dễ tái sử dụng

Beyond enabling people to learn from others and providing a shared vocabulary for discussing the art of building systems, patterns provide another important tool for computer programming: the ability to identify common components that can be implemented once.

Ngoài việc cho phép mọi người học hỏi từ người khác và cung cấp một vốn từ vựng chung để bàn về nghệ thuật xây dựng hệ thống, các mẫu còn cung cấp một công cụ quan trọng khác cho lập trình máy tính: khả năng xác định những thành phần chung có thể được hiện thực một lần.

If we had to create all of the code that our programs use ourselves, we would never get done. Indeed, we would barely get started. Today, every system ever written stands on the shoulders of thousands if not hundreds of thousands of years of human effort. Code for operating systems, printer drivers, distributed databases, container runtimes, and container orchestrators—indeed, the entirety of applications that we build today are built with reusable shared libraries and components.

Nếu chúng ta phải tự tạo toàn bộ mã mà các chương trình của mình sử dụng, chúng ta sẽ không bao giờ hoàn thành. Thực vậy, chúng ta sẽ chạt vật ngay từ vạch xuất phát. Ngày nay, mọi hệ thống từng được viết ra đều đứng trên vai hàng nghìn nếu không phải hàng trăm nghìn năm nỗ lực của con người. Mã cho hệ điều hành, trình điều khiển máy in, cơ sở dữ liệu phân tán, runtime container, và bộ điều phối container — thực vậy, toàn bộ các ứng dụng mà chúng ta xây dựng ngày nay đều được xây dựng từ các thư viện và thành phần chia sẻ, tái sử dụng được.

Patterns are the basis for the definition and development of such reusable components. The formalization of algorithms led to reusable implementations of sorting and other canonical algorithms. The identification of interface-based patterns gave rise to a collection of generic, object-oriented libraries implementing those patterns.

Các mẫu là cơ sở cho việc định nghĩa và phát triển những thành phần tái sử dụng như vậy. Sự hình thức hóa các thuật toán đã dẫn đến những hiện thực tái sử dụng được của thuật toán sắp xếp và các thuật toán kinh điển khác. Việc xác định các mẫu dựa trên interface đã làm nảy sinh một tập hợp các thư viện hướng đối tượng, đa năng, hiện thực những mẫu đó.

Identifying core patterns for distributed systems enables us to build shared common components. Implementing these patterns as container images with HTTP-based interfaces means they can be

reused across many different programming languages. And, of course, building reusable components improves the quality of each component because the shared code base gets sufficient usage to identify bugs and weaknesses, and sufficient attention to ensure that they get fixed.

Việc xác định các mẫu cốt lõi cho hệ phân tán cho phép chúng ta xây dựng các thành phần chung được chia sẻ. Hiện thực những mẫu này thành các image container với giao diện dựa trên HTTP nghĩa là chúng có thể được tái sử dụng trên nhiều ngôn ngữ lập trình khác nhau. Và, tất nhiên, việc xây dựng các thành phần tái sử dụng cải thiện chất lượng của từng thành phần bởi vì cơ sở mã được chia sẻ nhận được đủ lượng sử dụng để phát hiện lỗi và điểm yếu, cùng đủ sự quan tâm để đảm bảo chúng được khắc phục.

Summary — Tóm tắt

Distributed systems are required to implement the level of reliability, agility, and scale expected of modern computer programs. Distributed system design continues to be more of a black art practiced by wizards than a science applied by laypeople. The identification of common patterns and practices has regularized and improved the practice of algorithmic development and object-oriented programming. It is this book's goal to do the same for distributed systems. Enjoy!

Các hệ phân tán là cần thiết để hiện thực mức độ tin cậy, linh hoạt và quy mô được kỳ vọng ở các chương trình máy tính hiện đại. Việc thiết kế hệ phân tán vẫn tiếp tục mang dáng dấp của một thứ ma thuật được thực hành bởi các phù thủy hơn là một môn khoa học được áp dụng bởi người thường. Việc xác định các mẫu và thực hành chung đã quy chuẩn hóa và cải thiện thực hành phát triển thuật toán và lập trình hướng đối tượng. Mục tiêu của cuốn sách này là làm điều tương tự cho các hệ phân tán. Chúc bạn đọc vui vẻ!

www.allitebooks.com

www.allitebooks.com

PART I Single-Node Patterns — PART I Các mẫu trên một nút đơn

This book concerns itself with distributed systems, which are applications made up of many different components running on many different machines. However, the first section of this book is devoted to patterns that exist on a single **node**. The motivation for this is straightforward. Containers are the foundational building block for the patterns in this book, but in the end, groups of containers co-located on a single machine make up the atomic elements of **distributed system** patterns.

Cuốn sách này bàn về các hệ phân tán (distributed system), tức những ứng dụng được tạo thành từ nhiều thành phần khác nhau chạy trên nhiều máy khác nhau. Tuy nhiên, phần đầu tiên của cuốn sách lại dành cho các mẫu tồn tại trên một nút đơn (single node). Lý do cho điều này khá rõ ràng. Container là khối xây dựng nền tảng cho các mẫu trong cuốn sách này, nhưng rốt cuộc, các nhóm container nằm cùng chỗ (co-located) trên một máy đơn mới chính là những phần tử nguyên tử (atomic element) cấu thành nên các mẫu của hệ phân tán.

Motivations — Động lực

Though it is clear as to why you might want to break your distributed application into a collection of different containers running on different machines, it is perhaps somewhat less clear as to

why you might also want to break up the components running on a single machine into different containers. To understand the motivation for these groups of containers, it is worth considering the goals behind containerization. In general, the goal of a **container** is to establish boundaries around specific resources (e.g., this application needs two cores and 8 GB of memory). Likewise, the boundary delineates team **ownership** (e.g., this team owns this image). Finally, the boundary is intended to provide separation of concerns (e.g., this image does this one thing).

Dù đã rõ vì sao bạn có thể muốn chia ứng dụng phân tán của mình thành một tập hợp nhiều container chạy trên nhiều máy khác nhau, nhưng có lẽ vẫn chưa rõ lắm vì sao bạn cũng có thể muốn chia các thành phần đang chạy trên một máy đơn thành nhiều container khác nhau. Để hiểu động lực cho những nhóm container này, ta nên xét đến các mục tiêu đứng sau việc đóng gói container (containerization). Nhìn chung, mục tiêu của một container là thiết lập ranh giới quanh những tài nguyên cụ thể (ví dụ: ứng dụng này cần hai nhân (core) và 8 GB bộ nhớ). Tương tự, ranh giới này phân định quyền sở hữu của đội nhóm (ví dụ: đội này sở hữu image này). Cuối cùng, ranh giới còn nhằm tạo ra sự tách bạch mối quan tâm (separation of concerns) (ví dụ: image này chỉ làm đúng một việc này).

All of these reasons provide motivation for splitting up an application on a single machine into a group of containers. Consider resource isolation first. Your application may be made up of two components: one is a user-facing application server and the other is a background **configuration** file loader. Clearly, end-user-facing request **latency** is the highest priority, so the user-facing application needs to have sufficient resources to ensure that it is highly responsive. On the other hand, the background configuration loader is mostly a best-effort **service**; if it is delayed slightly during times of high user-request volume, the system will be okay. Likewise, the background configuration loader should not impact the quality of service that end users receive. For all of these reasons, you want to separate the user-facing service and the background **shard** loader into different containers. This allows you to attach different resource requirements and priorities to the two different containers and, for example, ensure that the background loader opportunistically steals cycles from the user-facing service whenever it is lightly loaded and the cycles are free. Likewise, separate resource requirements for the two containers ensure that the background loader will be terminated before the user-facing service if there is a resource contention issue caused by a memory leak or other overcommitment of memory resources.

Tất cả những lý do này tạo nên động lực cho việc tách một ứng dụng trên một máy đơn thành một nhóm container. Hãy xét đến sự cô lập tài nguyên (resource isolation) trước. Ứng dụng của bạn có thể gồm hai thành phần: một là máy chủ ứng dụng (application server) phục vụ trực tiếp người dùng, và thành phần kia là một trình nạp tệp cấu hình chạy nền (background configuration file loader). Rõ ràng, độ trễ của các yêu cầu hướng tới người dùng cuối là ưu tiên cao nhất, nên ứng dụng hướng người dùng cần có đủ tài nguyên để đảm bảo nó luôn phản hồi nhanh. Mặt khác, trình nạp cấu hình chạy nền chủ yếu là một dịch vụ kiểu nỗ-lực-tốt-nhất (best-effort); nếu nó bị trễ một chút trong những lúc lưu lượng yêu cầu của người dùng cao, hệ thống vẫn ổn. Tương tự, trình nạp cấu hình chạy nền không nên ảnh hưởng đến chất lượng dịch vụ mà người dùng cuối nhận được. Vì tất cả những lý do này, bạn muốn tách dịch vụ hướng người dùng và trình nạp mảnh (shard loader) chạy nền ra thành các container khác nhau. Điều này cho phép bạn gán các yêu cầu tài nguyên và mức ưu tiên khác nhau cho hai container khác nhau, và ví dụ, đảm bảo rằng trình nạp chạy nền tranh thủ “mượn” các chu kỳ xử lý từ dịch vụ hướng người dùng mỗi khi dịch vụ này tải nhẹ và còn chu kỳ rảnh. Tương tự, yêu cầu tài nguyên riêng cho hai container đảm bảo rằng trình nạp chạy nền sẽ bị chấm dứt trước dịch vụ hướng người dùng nếu xảy ra tranh chấp tài nguyên do rò rỉ bộ nhớ (memory leak) hoặc do cấp phát bộ nhớ vượt mức.

In addition to this resource isolation, there are other reasons to split your single-node application into multiple containers. Consider the task of scaling a team. There is good reason to believe that the ideal team size is six to eight people. In order to structure teams in this manner and yet still build significant systems, we need to have small, focused pieces for each team to own. Additionally, often some of the components, if factored properly, are reusable modules that can be used by many teams. Consider, for example, the task of keeping a local filesystem synchronized with a git source code repository. If you build this Git sync tool as a separate container, you can reuse it with PHP, HTML, JavaScript, Python, and numerous other web-serving environments. If you instead factor each environment as a single container where, for example, the Python runtime and the Git synchronization are inextricably bound, then this sort of modular reuse (and the corresponding small team that owns that reusable module) are impossible.

Ngoài sự cô lập tài nguyên này, còn có những lý do khác để chia ứng dụng đơn nút của bạn thành nhiều container. Hãy xét đến nhiệm vụ mở rộng quy mô một đội nhóm. Có lý do chính đáng để tin rằng quy mô đội lý tưởng là sáu đến tám người. Để tổ chức đội theo cách này mà vẫn xây dựng được những hệ thống đáng kể, ta cần có những phần nhỏ, tập trung để mỗi đội sở hữu. Thêm vào đó, thường thì một số thành phần, nếu được tách bóc (factor) đúng cách, sẽ trở thành các mô-đun tái sử dụng (reusable module) mà nhiều đội có thể dùng. Hãy xét, chẳng hạn, nhiệm vụ giữ cho một hệ thống tệp cục bộ luôn đồng bộ với một kho mã nguồn git. Nếu bạn xây công cụ đồng bộ Git này thành một container riêng, bạn có thể tái sử dụng nó với PHP, HTML, JavaScript, Python và vô số môi trường phục vụ web khác. Ngược lại, nếu bạn tách bóc mỗi môi trường thành một container đơn, trong đó, chẳng hạn, runtime Python và việc đồng bộ Git gắn chặt không thể tách rời nhau, thì kiểu tái sử dụng mô-đun như vậy (và đội nhỏ tương ứng sở hữu mô-đun tái sử dụng đó) là bất khả thi.

Finally, even if your application is small and all of your containers are owned by a single team, the separation of concerns ensures that your application is well understood and can easily be tested, updated, and deployed. Small, focused applications are easier to understand and have fewer couplings to other systems. This means, for example, that you can deploy the Git synchronization container without having to also redeploy your application server. This leads to rollouts with fewer dependencies and smaller scope. That, in turn, leads to more reliable rollouts (and rollbacks), which leads to greater agility and flexibility when deploying your application.

Cuối cùng, ngay cả khi ứng dụng của bạn nhỏ và toàn bộ container của bạn đều do một đội duy nhất sở hữu, việc tách bạch mối quan tâm vẫn đảm bảo rằng ứng dụng của bạn được hiểu rõ và dễ dàng kiểm thử, cập nhật cũng như triển khai. Các ứng dụng nhỏ, tập trung thì dễ hiểu hơn và ít ràng buộc (coupling) với các hệ thống khác hơn. Điều này có nghĩa là, ví dụ, bạn có thể triển khai container đồng bộ Git mà không phải triển khai lại máy chủ ứng dụng của mình. Việc này dẫn đến những lần tung ra (rollout) có ít phụ thuộc hơn và phạm vi nhỏ hơn. Điều đó, đến lượt nó, dẫn đến những lần tung ra (và hoàn tác - rollback) đáng tin cậy hơn, từ đó mang lại sự nhanh nhạy và linh hoạt hơn khi triển khai ứng dụng của bạn.

Summary — Tóm tắt

I hope that all of these examples have motivated you to think about breaking up your applications, even those on a single node, into multiple containers. The following chapters describe some patterns that can help guide you as you build modular groups of containers. In contrast to multi-node, distributed patterns, all of these patterns assume tight dependencies among all of the containers in the pattern. In particular, they assume that all of the containers in the pattern can

be reliably coscheduled onto a single machine. They also assume that all of the containers in the pattern can optionally share volumes or parts of their filesystems as well as other key container resources like network namespaces and shared memory. This tight grouping is called a **pod** in **Kubernetes**,¹ but the concept is generally applicable to different container orchestrators, though some support it more natively than others.

Tôi hy vọng tất cả những ví dụ này đã thúc đẩy bạn nghĩ đến việc chia nhỏ các ứng dụng của mình, kể cả những ứng dụng nằm trên một nút đơn, thành nhiều container. Các chương tiếp theo mô tả một số mẫu có thể giúp dẫn dắt bạn khi xây dựng các nhóm container theo mô-đun. Khác với các mẫu đa nút (multi-node), phân tán, tất cả những mẫu này đều giả định có sự phụ thuộc chặt chẽ giữa toàn bộ các container trong mẫu. Cụ thể, chúng giả định rằng tất cả các container trong mẫu có thể được lập lịch cùng nhau (coscheduled) một cách đáng tin cậy lên một máy đơn. Chúng cũng giả định rằng tất cả các container trong mẫu có thể tùy chọn chia sẻ các volume hoặc các phần của hệ thống tệp, cũng như các tài nguyên container then chốt khác như network namespace và bộ nhớ dùng chung (shared memory). Nhóm gắn kết chặt chẽ này được gọi là một Pod trong Kubernetes,¹ nhưng khái niệm này nhìn chung áp dụng được cho nhiều bộ điều phối container (orchestrator) khác nhau, dù một số hỗ trợ nó tự nhiên (natively) hơn những bộ khác.

¹ Kubernetes is an **open source** system for automating **deployment**, scaling, and management of containerized applications. Check out my book, *Kubernetes: Up and Running* (O'Reilly).

¹ Kubernetes là một hệ thống mã nguồn mở để tự động hóa việc triển khai, mở rộng quy mô và quản lý các ứng dụng đóng gói container. Hãy xem cuốn sách của tôi, *Kubernetes: Up and Running* (O'Reilly).

CHAPTER 2 The Sidecar Pattern —

CHAPTER 2 Mẫu Sidecar

The first **single-node pattern** is the **sidecar** pattern. The sidecar pattern is a single-node pattern made up of two containers. The first is the application container . It contains the core logic for the application. Without this container, the application would not exist. In addition to the application container, there is a sidecar container . The role of the sidecar is to augment and improve the application container, often without the application container's knowledge. In its simplest form, a sidecar container can be used to add functionality to a container that might otherwise be difficult to improve. Sidecar containers are coscheduled onto the same machine via an atomic container group , such as the pod **API** object in Kubernetes. In addition to being scheduled on the same machine, the application container and sidecar container share a number of resources, including parts of the filesystem, hostname and network, and many other namespaces. A generic image of this sidecar pattern is shown in Figure 2-1 .

Mẫu đơn nút (single-node) đầu tiên là mẫu sidecar. Mẫu sidecar là một mẫu đơn nút gồm hai container. Container thứ nhất là container ứng dụng (application container). Nó chứa logic cốt lõi của ứng dụng. Nếu không có container này, ứng dụng sẽ không tồn tại. Ngoài container ứng dụng, còn có một container sidecar (sidecar container). Vai trò của sidecar là bổ trợ và cải thiện container ứng dụng, thường là mà container ứng dụng không hề hay biết. Ở dạng đơn giản nhất, một container sidecar có thể được dùng để thêm chức năng cho một container mà nếu không thì rất khó cải thiện. Các container sidecar được lập lịch cùng nhau (coscheduled) lên cùng một máy thông qua một nhóm container nguyên tử (atomic container group), chẳng hạn như đối tượng API Pod trong Kubernetes. Ngoài việc được lập lịch trên cùng một máy, container ứng dụng và container sidecar còn chia sẻ một số tài nguyên, bao gồm các phần của hệ thống tệp, hostname và network, cùng nhiều namespace khác. Một hình ảnh tổng quát của mẫu sidecar này được thể hiện trong Hình 2-1.

Container Group (aka Pod)

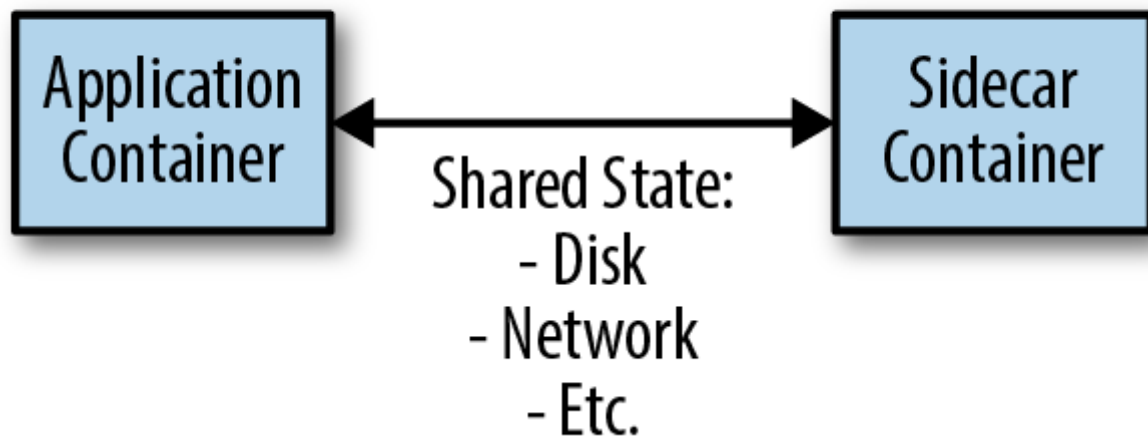


Figure 2-1. The generic sidecar pattern

Hình 2-1. Mẫu sidecar tổng quát

An Example Sidecar: Adding HTTPS to a Legacy Service — Một ví dụ về Sidecar: Thêm HTTPS cho một dịch vụ kế thừa

Consider, for example, a legacy web service. Years ago, when it was built, internal network security was not as high a priority for the company, and thus, the application only services requests over unencrypted HTTP, not HTTPS. Due to recent security incidents, the company has mandated the use of HTTPS for all company websites. To

Hãy xét, chẳng hạn, một dịch vụ web kế thừa (legacy). Nhiều năm trước, khi nó được xây dựng, an ninh mạng nội bộ chưa phải là ưu tiên cao của công ty, và do đó, ứng dụng chỉ phục vụ các yêu cầu qua HTTP không mã hóa, chứ không phải HTTPS. Do những sự cố an ninh gần đây, công ty đã bắt buộc sử dụng HTTPS cho tất cả các website của công ty. Để

11 compound the misery of the team sent to update this particular web service, the source code for this application was built with an old version of the company's build system, which no longer functions. Containerizing this HTTP application is simple enough: the binary can run in a container with a version of an old Linux distribution on top of a more modern kernel being run by the team's **container orchestrator**. However, the task of adding HTTPS to this application is significantly more challenging. The team is trying to decide between resurrecting the old build system versus porting the application's source code to the new build system, when one of the team members suggests that they use the sidecar pattern to resolve the situation more easily.

11 làm nỗi khổ của đội được cử đi cập nhật dịch vụ web cụ thể này thêm trầm trọng, mã nguồn của ứng dụng này lại được xây dựng bằng một phiên bản cũ của hệ thống build của công ty, vốn đã không còn hoạt động. Việc đóng gói container cho ứng dụng HTTP này khá đơn giản: tệp nhị phân (binary) có thể chạy trong một container với một phiên bản của bản phân phối Linux cũ nằm trên một nhân (kernel) hiện đại hơn do bộ điều phối container của đội vận hành. Tuy nhiên, nhiệm vụ thêm HTTPS cho ứng dụng này lại khó khăn hơn nhiều. Đội đang phân vân giữa việc hồi sinh hệ thống build cũ với việc chuyển

(port) mã nguồn của ứng dụng sang hệ thống build mới, thì một thành viên trong đội gợi ý họ dùng mẫu sidecar để giải quyết tình huống dễ dàng hơn.

The application of the sidecar pattern to this situation is straightforward. The legacy web service is configured to serve exclusively on localhost (127.0.0.1), which means that only services that share the local network with the server will be able to access the service. Normally, this wouldn't be a practical choice because it would mean that no one could access the web service. However, using the sidecar pattern, in addition to the legacy container, we will add an **nginx** sidecar container. This nginx container lives in the same network namespace as the legacy web application, so it can access the service that is running on localhost. At the same time, this nginx service can terminate HTTPS traffic on the external IP address of the pod and **proxy** that traffic to the legacy web application (see Figure 2-2). Since this unencrypted traffic is only sent via the local loopback **adapter** inside the container group, the network security team is satisfied that the data is safe. Likewise, by using the sidecar pattern, the team has modernized a legacy application without having to figure out how to rebuild a new application to serve HTTPS.

Việc áp dụng mẫu sidecar vào tình huống này khá đơn giản. Dịch vụ web kế thừa được cấu hình để chỉ phục vụ trên localhost (127.0.0.1), nghĩa là chỉ những dịch vụ chia sẻ mạng cục bộ với máy chủ mới có thể truy cập được dịch vụ này. Thông thường, đây sẽ không phải là một lựa chọn thực tế vì nó đồng nghĩa với việc không ai có thể truy cập dịch vụ web. Tuy nhiên, khi dùng mẫu sidecar, bên cạnh container kế thừa, ta sẽ thêm một container sidecar nginx. Container nginx này nằm trong cùng network namespace với ứng dụng web kế thừa, nên nó có thể truy cập dịch vụ đang chạy trên localhost. Đồng thời, dịch vụ nginx này có thể kết thúc (terminate) lưu lượng HTTPS trên địa chỉ IP bên ngoài của Pod và làm proxy cho lưu lượng đó tới ứng dụng web kế thừa (xem Hình 2-2). Vì lưu lượng không mã hóa này chỉ được gửi qua bộ điều hợp loopback cục bộ bên trong nhóm container, đội an ninh mạng yên tâm rằng dữ liệu được an toàn. Tương tự, bằng cách dùng mẫu sidecar, đội đã hiện đại hóa một ứng dụng kế thừa mà không phải tìm cách xây dựng lại một ứng dụng mới để phục vụ HTTPS.

Container Group (aka Pod)



Figure 2-2. The HTTPS sidecar

Hình 2-2. Sidecar HTTPS

Dynamic Configuration with Sidecars — Cấu hình động với Sidecar

Simply proxying traffic into an existing application is not the only use for a sidecar. Another common example is configuration synchronization. Many applications use a configuration file for parameterizing the application; this may be a raw text file or something more structured like XML, JSON, or YAML. Many pre-existing applications were written to assume that this file was present on the filesystem and read their configuration from there. However, in a cloud-native environment

it is often quite useful to use an API for updating configuration. This allows you to do a dynamic push of configuration information via an API instead of manually logging in to every server and updating the configuration file using imperative commands. The desire for such an API is driven both by ease of use as well as the ability to add automation like **rollback**, which makes configuring (and reconfiguring) safer and easier.

Đơn thuần làm proxy đưa lưu lượng vào một ứng dụng hiện có không phải là công dụng duy nhất của sidecar. Một ví dụ phổ biến khác là đồng bộ hóa cấu hình. Nhiều ứng dụng dùng một tệp cấu hình để tham số hóa ứng dụng; tệp này có thể là văn bản thô hoặc thứ gì đó có cấu trúc hơn như XML, JSON, hoặc YAML. Nhiều ứng dụng đã có sẵn được viết với giả định rằng tệp này hiện diện trên hệ thống tệp và đọc cấu hình từ đó. Tuy nhiên, trong một môi trường thuần đám mây (cloud-native), việc dùng một API để cập nhật cấu hình thường rất hữu ích. Điều này cho phép bạn đẩy (push) thông tin cấu hình một cách động qua API thay vì phải đăng nhập thủ công vào từng máy chủ và cập nhật tệp cấu hình bằng các lệnh mệnh lệnh (imperative command). Mong muốn có một API như vậy được thúc đẩy bởi cả sự tiện dụng lẫn khả năng thêm tự động hóa như hoàn tác (rollback), giúp việc cấu hình (và tái cấu hình) an toàn và dễ dàng hơn.

Similar to the case of HTTPS, new applications can be written with the expectation that configuration is a dynamic property that should be obtained using a cloud API, but adapting and updating an existing application can be significantly more challenging. Fortunately, the sidecar pattern again can be used to provide new functionality that augments a legacy application without changing the existing application. For the sidecar pattern shown in Figure 2-3, there again are two containers: the container that is the serving application and the container that is the configuration manager. The two containers are grouped together into a pod where they share a directory between themselves. This shared directory is where the configuration file is maintained.

Tương tự như trường hợp HTTPS, các ứng dụng mới có thể được viết với kỳ vọng rằng cấu hình là một thuộc tính động cần được lấy qua một API đám mây, nhưng việc thích nghi và cập nhật một ứng dụng hiện có có thể khó khăn hơn đáng kể. May thay, mẫu sidecar một lần nữa có thể được dùng để cung cấp chức năng mới hỗ trợ cho một ứng dụng kế thừa mà không thay đổi ứng dụng hiện có. Với mẫu sidecar thể hiện trong Hình 2-3, lại có hai container: container là ứng dụng phục vụ và container là trình quản lý cấu hình (configuration manager). Hai container được gom chung vào một Pod, nơi chúng chia sẻ một thư mục với nhau. Thư mục dùng chung này là nơi tệp cấu hình được duy trì.

When the legacy application starts, it loads its configuration from the filesystem, as expected. When the configuration manager starts, it examines the configuration API and looks for differences between the local filesystem and the configuration stored in the API. If there are differences, the configuration manager downloads the new configuration to the local filesystem and signals to the legacy application that it should reconfigure itself with this new configuration. The actual mechanism for this notification varies by application. Some applications actually watch the configuration file for changes, while others respond to a SIGHUP signal. In extreme cases, the configuration manager may send a SIGKILL signal to abort the legacy application. Once aborted, the container orchestration system will restart the legacy application, at which point it will load its new configuration. As with adding HTTPS to an existing application, this pattern illustrates how the sidecar pattern can help adapt pre-existing applications to more cloud-native scenarios.

Khi ứng dụng kế thừa khởi động, nó nạp cấu hình của mình từ hệ thống tệp, đúng như kỳ vọng. Khi trình quản lý cấu hình khởi động, nó kiểm tra API cấu hình và tìm những khác biệt giữa hệ thống tệp cục bộ và cấu hình được lưu trong API. Nếu có khác biệt, trình quản lý cấu hình tải cấu hình mới về hệ thống tệp cục bộ và phát tín hiệu cho ứng dụng kế thừa biết rằng nó nên tự tái cấu hình với cấu hình mới này. Cơ chế thông báo thực tế thay đổi

tùy theo ứng dụng. Một số ứng dụng thực sự theo dõi tệp cấu hình để phát hiện thay đổi, trong khi số khác phản ứng với tín hiệu SIGHUP. Trong các trường hợp cực đoan, trình quản lý cấu hình có thể gửi tín hiệu SIGKILL để hủy bỏ ứng dụng kế thừa. Sau khi bị hủy, hệ thống điều phối container sẽ khởi động lại ứng dụng kế thừa, và lúc đó nó sẽ nạp cấu hình mới của mình. Cũng giống như việc thêm HTTPS cho một ứng dụng hiện có, mẫu này minh họa cách mẫu sidecar có thể giúp thích nghi các ứng dụng đã có sẵn với những kịch bản thuần đám mây hơn.

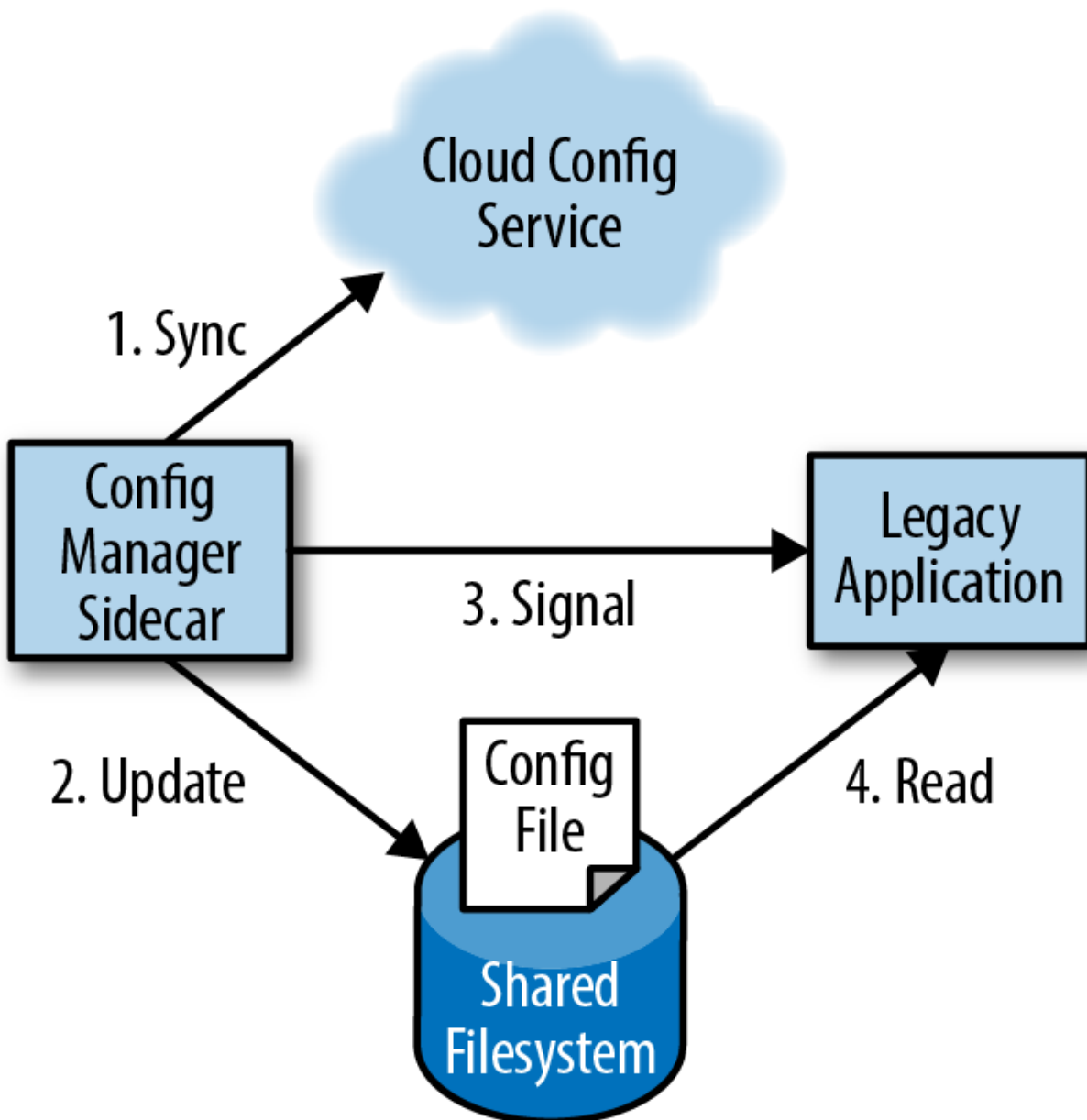


Figure 2-3. A sidecar example of managing a dynamic configuration

Hình 2-3. Một ví dụ về sidecar quản lý cấu hình động

Modular Application Containers — Container ứng dụng theo mô-đun

At this point, you might be forgiven if you thought that the sole reason for the sidecar pattern to exist was to adapt legacy applications where you no longer wanted to make modifications to the original source code. While this is a common use case for the pattern, there are many other motivations for designing things using sidecars. One of the other main advantages of using the sidecar pattern is modularity and reuse of the components used as sidecars. In deploying any real-world, reliable application, there is functionality that you need for debugging or other management of the application, such as giving a readout of all of the different processes using resources in the container, similar to the top command line tool.

Đến đây, cũng có thể thông cảm nếu bạn nghĩ rằng lý do duy nhất khiến mẫu sidecar tồn tại là để thích nghi các ứng dụng kế thừa khi bạn không còn muốn chỉnh sửa mã nguồn gốc nữa. Dù đây là một trường hợp sử dụng phổ biến của mẫu này, vẫn còn nhiều động lực khác để thiết kế mọi thứ bằng sidecar. Một trong những ưu điểm chính khác của việc dùng mẫu sidecar là tính mô-đun và khả năng tái sử dụng của các thành phần được dùng làm sidecar. Khi triển khai bất kỳ ứng dụng đáng tin cậy nào trong thực tế, có những chức năng bạn cần để gỡ lỗi (debug) hoặc quản lý ứng dụng theo cách khác, chẳng hạn như đưa ra một bản hiển thị tất cả các tiến trình khác nhau đang dùng tài nguyên trong container, tương tự công cụ dòng lệnh top.

One approach to providing this introspection is to require that each developer implement an HTTP /topz **interface** that provides a readout of resource usage. To make this easier, you might implement this **webhook** as a language-specific plugin that the developer could simply link into their application. But even if done this way, the developer would be forced to choose to link it in and your organization would be forced to implement the interface for every language it wants to support. Unless done with extreme rigor, this approach is bound to lead to variations among languages as well as a lack of support for the functionality when using new languages. Instead, this topz functionality can be deployed as a sidecar container that shares the process-id (PID) namespace with the application container. This topz container can **introspect** all running processes and provide a consistent user interface. Moreover, you can use the orchestration system to automatically add this container to all applications deployed via the orchestration system to ensure that there is a consistent set of tools available for all applications running in your infrastructure.

Một cách tiếp cận để cung cấp khả năng xem xét nội tại (introspection) này là yêu cầu mỗi lập trình viên hiện thực một giao diện HTTP /topz cung cấp bản hiển thị mức sử dụng tài nguyên. Để dễ dàng hơn, bạn có thể hiện thực webhook này dưới dạng một plugin đặc thù ngôn ngữ mà lập trình viên chỉ việc liên kết (link) vào ứng dụng của họ. Nhưng ngay cả khi làm theo cách này, lập trình viên vẫn buộc phải chọn liên kết nó vào, và tổ chức của bạn buộc phải hiện thực giao diện này cho mọi ngôn ngữ mà nó muốn hỗ trợ. Trừ khi được làm với sự nghiêm ngặt tột bậc, cách tiếp cận này chắc chắn sẽ dẫn đến sự khác biệt giữa các ngôn ngữ, cũng như thiếu hỗ trợ chức năng khi dùng ngôn ngữ mới. Thay vào đó, chức năng topz này có thể được triển khai dưới dạng một container sidecar chia sẻ namespace process-id (PID) với container ứng dụng. Container topz này có thể xem xét nội tại (introspect) tất cả các tiến trình đang chạy và cung cấp một giao diện người dùng nhất quán. Hơn nữa, bạn có thể dùng hệ thống điều phối để tự động thêm container này vào tất cả các ứng dụng được triển khai qua hệ thống điều phối, nhằm đảm bảo có một bộ công cụ nhất quán sẵn dùng cho mọi ứng dụng chạy trong hạ tầng của bạn.

Of course, with any technical choice, there are trade-offs between this modular container-based pattern and rolling your own code into your application. The library-based approach is always going

to be somewhat less tailored to the specifics of your application. This means that it may be less efficient in terms of size of performance, or that the API may require some adaptation to fit into your environment. I would compare these trade-offs to the difference between buying off-the-rack clothing versus bespoke fashion. The bespoke fashion will always fit you better, but it will take longer to arrive and cost more to acquire. As with clothes, for most of us it makes sense to buy the more general-purpose solution when it comes to coding. Of course, if your application demands extremes in terms of performance, you can always choose the handwritten solution.

Dĩ nhiên, với bất kỳ lựa chọn kỹ thuật nào, đều có những đánh đổi giữa mẫu dựa trên container theo mô-đun này với việc tự viết mã của riêng bạn vào ứng dụng. Cách tiếp cận dựa trên thư viện (library) luôn luôn ít được may đo theo những đặc thù của ứng dụng của bạn hơn một chút. Điều này nghĩa là nó có thể kém hiệu quả hơn về kích thước hay hiệu năng, hoặc API có thể cần một chút điều chỉnh để vừa với môi trường của bạn. Tôi ví những đánh đổi này như sự khác biệt giữa mua quần áo may sẵn với thời trang may đo. Thời trang may đo sẽ luôn vừa với bạn hơn, nhưng sẽ mất nhiều thời gian hơn để có và tốn nhiều tiền hơn để sở hữu. Cũng như với quần áo, với hầu hết chúng ta, việc mua giải pháp đa dụng hơn là hợp lý khi nói đến lập trình. Tất nhiên, nếu ứng dụng của bạn đòi hỏi mức tối cùng về hiệu năng, bạn luôn có thể chọn giải pháp viết tay.

Hands On: Deploying the topz Container — Thực hành (Hands On): Triển khai container topz

To see the topz sidecar in action, you first need to deploy some other container to act as the application container. Choose an existing application that you are running and deploy it using **Docker**:

Để thấy sidecar topz hoạt động, trước tiên bạn cần triển khai một container nào đó khác để đóng vai trò là container ứng dụng. Hãy chọn một ứng dụng hiện có mà bạn đang chạy và triển khai nó bằng Docker:

```
$docker run -d <my-app-image>
<container-hash-value>
```

After you run that image, you will receive the identifier for that specific container. It will look something like: cccf82b85000... If you don't have it, you can always look it up using the docker ps command, which will show all currently running containers. Assuming you have stashed that value in an environment variable named APP_ID , you can then run the topz container in the same PID namespace using:

Sau khi bạn chạy image đó, bạn sẽ nhận được định danh (identifier) cho container cụ thể đó. Nó sẽ trông như: cccf82b85000... Nếu bạn không có nó, bạn luôn có thể tra cứu nó bằng lệnh docker ps, lệnh này sẽ hiển thị tất cả các container đang chạy. Giả sử bạn đã cất giá trị đó vào một biến môi trường tên là APP_ID, bạn có thể chạy container topz trong cùng namespace PID bằng:

```
$docker run --pid=container:${APP_ID} \
-p 8080:8080 \
brendanburns/topz:db0fa58 \
/server --address=0.0.0.0:8080
```

This will launch the topz sidecar in the same PID namespace as the application container. Note that you may need to change the port that the sidecar uses for serving if your application container is also running on port 8080 . Once the sidecar is running, you can visit <http://localhost:8080/topz> to get

a complete readout of the processes that are running in the application container and their resource usage.

Lệnh này sẽ khởi chạy sidecar topz trong cùng namespace PID với container ứng dụng. Lưu ý rằng bạn có thể cần đổi cổng (port) mà sidecar dùng để phục vụ nếu container ứng dụng của bạn cũng đang chạy trên cổng 8080. Một khi sidecar đang chạy, bạn có thể truy cập <http://localhost:8080/topz> để có được bản hiển thị đầy đủ về các tiến trình đang chạy trong container ứng dụng cùng mức sử dụng tài nguyên của chúng.

You can mix this sidecar with any other existing container to easily get a view into how the container is using its resources via a web interface.

Bạn có thể trộn sidecar này với bất kỳ container hiện có nào khác để dễ dàng có được cái nhìn về cách container đang dùng tài nguyên của nó thông qua một giao diện web.

Building a Simple PaaS with Sidecars — Xây dựng một PaaS đơn giản bằng Sidecar

The sidecar pattern can be used for more than adaptation and monitoring. It can also be used as a means to implement the complete logic for your application in a simplified, modular manner. As an example, imagine building a simple platform as a service (PaaS) built around the git workflow. Once you deploy this PaaS, simply pushing new code up to a Git repository results in that code being deployed to the running servers. We'll see how the sidecar pattern makes building this PaaS remarkably straightforward.

Mẫu sidecar có thể được dùng cho nhiều việc hơn là thích nghi và giám sát. Nó cũng có thể được dùng như một phương tiện để hiện thực toàn bộ logic cho ứng dụng của bạn theo một cách đơn giản hóa, theo mô-đun. Ví dụ, hãy hình dung việc xây dựng một nền tảng-như-một-dịch-vụ (platform as a service - PaaS) đơn giản được xây quanh quy trình làm việc với git. Một khi bạn triển khai PaaS này, chỉ cần đẩy (push) mã mới lên một kho Git là khiến mã đó được triển khai tới các máy chủ đang chạy. Ta sẽ thấy mẫu sidecar khiến việc xây dựng PaaS này trở nên cực kỳ đơn giản như thế nào.

As previously stated, in the sidecar pattern there are two containers: the main application container and the sidecar. In our simple PaaS application, the main container is a Node.js server that implements a web server. The Node.js server is instrumented so that it automatically reloads the server when new files are updated. This is accomplished with the nodemon tool .

Như đã nói trước đó, trong mẫu sidecar có hai container: container ứng dụng chính và sidecar. Trong ứng dụng PaaS đơn giản của ta, container chính là một máy chủ Node.js hiện thực một máy chủ web. Máy chủ Node.js được trang bị sao cho nó tự động nạp lại máy chủ khi có tệp mới được cập nhật. Việc này được thực hiện bằng công cụ nodemon.

The sidecar container shares a filesystem with the main application container and runs a simple loop that synchronizes the filesystem with an existing Git repository:

Container sidecar chia sẻ một hệ thống tệp với container ứng dụng chính và chạy một vòng lặp đơn giản đồng bộ hệ thống tệp với một kho Git hiện có:

```
#!/bin/bash
while true; do
  git pull
  sleep 10
done
```

Obviously this script could be more complex, pulling from a specific branch instead of simply from HEAD. It is left purposefully simple to improve the readability of this example.

Hiển nhiên script này có thể phức tạp hơn, kéo (pull) từ một nhánh cụ thể thay vì chỉ đơn thuần từ HEAD. Nó được cố ý giữ đơn giản để cải thiện tính dễ đọc của ví dụ này.

The Node.js application and Git synchronization sidecar are coscheduled and deployed together to implement our simple PaaS (Figure 2-4). Once deployed, every time new code is pushed to a Git repository, the code is automatically updated by the sidecar and reloaded by the server.

Ứng dụng Node.js và sidecar đồng bộ Git được lập lịch cùng nhau và triển khai cùng nhau để hiện thực PaaS đơn giản của ta (Hình 2-4). Một khi đã triển khai, mỗi lần mã mới được đẩy lên một kho Git, mã đó được sidecar tự động cập nhật và được máy chủ nạp lại.

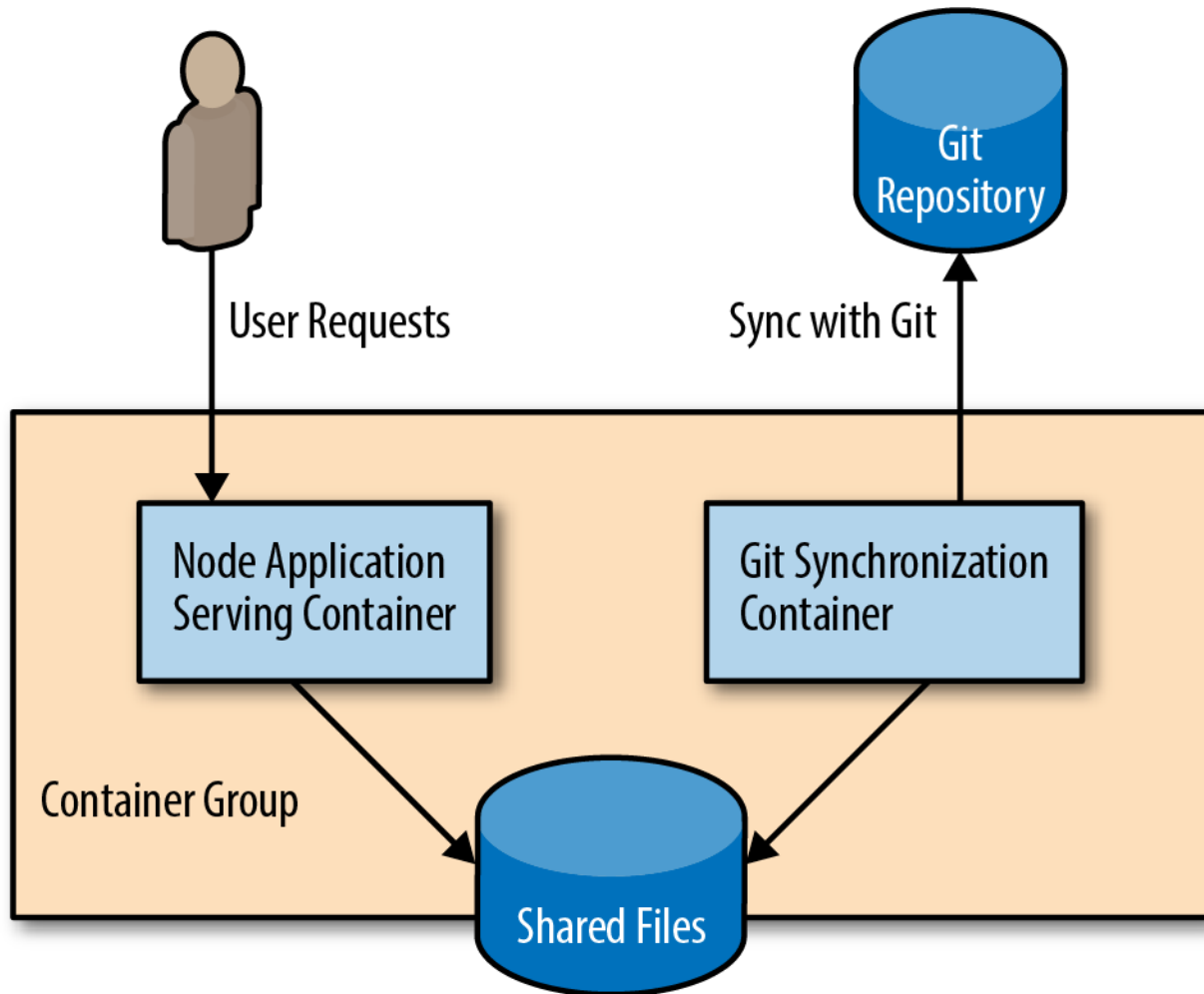


Figure 2-4. A simple sidecar-based PaaS

Hình 2-4. Một PaaS đơn giản dựa trên sidecar

Designing Sidecars for Modularity and Reusability — Thiết kế Sidecar cho tính mô-đun và khả năng tái sử dụng

In all of the examples of sidecars that we have detailed throughout this chapter, one of the most important themes is that every one was a modular, reusable **artifact**. To be successful, the sidecar should be reusable across a wide variety of applications and deployments. By achieving modular reuse, sidecars can dramatically speed up the building of your application.

Trong tất cả các ví dụ về sidecar mà ta đã trình bày chi tiết xuyên suốt chương này, một trong những chủ đề quan trọng nhất là mỗi cái đều là một sản phẩm có tính mô-đun, tái sử dụng được. Để thành công, sidecar nên tái sử dụng được trên nhiều ứng dụng và triển khai đa dạng. Bằng cách đạt được khả năng tái sử dụng theo mô-đun, sidecar có thể tăng tốc đáng kể việc xây dựng ứng dụng của bạn.

However, this modularity and reusability, just like achieving modularity in high-quality software development requires focus and discipline. In particular, you need to focus on developing three areas:

Tuy nhiên, tính mô-đun và khả năng tái sử dụng này, cũng giống như việc đạt được tính mô-đun trong phát triển phần mềm chất lượng cao, đòi hỏi sự tập trung và kỷ luật. Cụ thể, bạn cần tập trung phát triển ba mảng:

- Parameterizing your containers
- Creating the API surface of your container
- Documenting the operation of your container
 - Tham số hóa các container của bạn
 - Tạo ra bề mặt API (API surface) cho container của bạn
 - Lập tài liệu về cách vận hành của container của bạn

Parameterized Containers — Container tham số hóa

Parameterizing your containers is the most important thing you can do to make your containers modular and reusable regardless of whether they are sidecars or not, though sidecars and other add-on containers are especially important to parameterize.

Tham số hóa các container của bạn là điều quan trọng nhất bạn có thể làm để khiến container của mình có tính mô-đun và tái sử dụng được, bất kể chúng có phải là sidecar hay không, dù sidecar và các container bổ trợ (add-on) khác thì đặc biệt quan trọng để tham số hóa.

What do I mean when I say “parameterize”? Consider your container as a **function** in your program. How many parameters does it have? Each parameter represents an input that can customize a generic container to a specific situation. Consider, for example, the **SSL** add-on sidecar deployed previously. To be generally useful, it likely needs at least two parameters: the first is the name of the certificate being used to provide SSL, and the other is the port of the “legacy” application server running on localhost. Without these parameters, it is hard to imagine this sidecar container being usable for a broad array of applications. Similar parameters exist for all of the other sidecars described in this chapter.

Tôi muốn nói gì khi nói “tham số hóa”? Hãy coi container của bạn như một hàm trong chương trình của bạn. Nó có bao nhiêu tham số? Mỗi tham số đại diện cho một đầu vào có thể tùy biến một container chung thành một tình huống cụ thể. Hãy xét, chẳng hạn,

sidecar hỗ trợ SSL đã triển khai trước đó. Để hữu dụng một cách tổng quát, nó có lẽ cần ít nhất hai tham số: tham số thứ nhất là tên của chứng chỉ (certificate) được dùng để cung cấp SSL, và tham số kia là cổng của ứng dụng máy chủ “kế thừa” đang chạy trên localhost. Nếu không có những tham số này, khó hình dung container sidecar này lại có thể dùng được cho một dải rộng các ứng dụng. Các tham số tương tự tồn tại cho tất cả các sidecar khác được mô tả trong chương này.

Now that we know the parameters we want to expose, how do we actually expose them to users, and how do we consume them inside the container. There are two ways in which such parameters can be passed to your container: through environment variables or the command line. Though either is feasible, I have a general preference for passing parameters via environment variables. An example of passing such parameters to a sidecar container is:

Giờ khi đã biết những tham số mà ta muốn phơi ra (expose), làm thế nào ta thực sự phơi chúng cho người dùng, và làm thế nào ta tiêu thụ chúng bên trong container. Có hai cách để truyền những tham số như vậy vào container của bạn: thông qua biến môi trường hoặc dòng lệnh. Dù cách nào cũng khả thi, tôi nhìn chung thiên về việc truyền tham số qua biến môi trường. Một ví dụ về việc truyền những tham số như vậy vào một container sidecar là:

```
docker run -e=PORT=<port> -d <image>
```

Of course, delivering values into the container is only part of the battle. The other part is actually using these variables inside the container. Typically, to do that, a simple shell script is used that loads the environment variables supplied with the sidecar container and either adjusts the configuration files or parameterizes the underlying application.

Tất nhiên, việc đưa giá trị vào container chỉ là một phần của cuộc chiến. Phần còn lại là thực sự sử dụng những biến này bên trong container. Thông thường, để làm điều đó, ta dùng một script shell đơn giản nạp các biến môi trường được cung cấp cùng với container sidecar, rồi hoặc điều chỉnh các tệp cấu hình hoặc tham số hóa ứng dụng nền tảng bên dưới.

For example, you might pass in the certificate path and port as environment variables:

Ví dụ, bạn có thể truyền vào đường dẫn chứng chỉ và cổng dưới dạng biến môi trường:

```
docker run -e=PROXY_PORT=8080 -e=CERTIFICATE_PATH=/path/to/cert.crt ...
```

In your container, you would use those variables to configure an nginx.conf file that points the web server to the correct file and proxy location.

Trong container của bạn, bạn sẽ dùng những biến đó để cấu hình một tệp nginx.conf trở máy chủ web tới đúng tệp và vị trí proxy.

Define Each Container’s API — Định nghĩa API cho mỗi Container

Given that you are parameterizing your containers, it is obvious that your containers are defining a “function” that is called whenever the container is executed. This function is clearly a part of the API that is defined by your container, but there are other parts to this API as well, including calls that the container will make to other services as well as traditional HTTP or other APIs that the container provides.

Một khi bạn tham số hóa các container của mình, hiển nhiên là các container của bạn đang định nghĩa một “hàm” được gọi mỗi khi container được thực thi. Hàm này rõ ràng là

một phần của API mà container của bạn định nghĩa, nhưng cũng còn các phần khác của API này, bao gồm các lời gọi mà container sẽ thực hiện tới các dịch vụ khác cũng như các API HTTP truyền thống hay các API khác mà container cung cấp.

As you think about defining modular, reusable containers, it is important to realize that all aspects of how your container interacts with its world are part of the API defined by that reusable container. As in the world of microservices, these micro-containers rely on APIs to ensure that there is a clean separation between the main application container and the sidecar. Additionally the API exists to ensure that all consumers of the sidecar will continue to work correctly as subsequent versions are released. Likewise, having a clean API for a sidecar enables the sidecar developer to move more quickly since they have a clear definition (and hopefully unit tests) for the services they provide as a part of the sidecar.

Khi bạn nghĩ về việc định nghĩa các container có tính mô-đun, tái sử dụng được, điều quan trọng là nhận ra rằng mọi khía cạnh về cách container của bạn tương tác với thế giới đều là một phần của API do container tái sử dụng đó định nghĩa. Cũng như trong thế giới microservice, những micro-container này dựa vào API để đảm bảo có sự tách bạch sạch sẽ giữa container ứng dụng chính và sidecar. Thêm vào đó, API tồn tại để đảm bảo rằng tất cả những bên tiêu thụ sidecar sẽ tiếp tục hoạt động đúng khi các phiên bản sau được phát hành. Tương tự, có một API sạch sẽ cho một sidecar cho phép lập trình viên sidecar di chuyển nhanh hơn vì họ có một định nghĩa rõ ràng (và hy vọng là cả unit test) cho các dịch vụ mà họ cung cấp như một phần của sidecar.

To see a concrete example of why this API surface area is important, consider the configuration management sidecar we discussed earlier. A useful configuration for this sidecar might be `UPDATE_FREQUENCY`, which indicates how often the configuration should be synchronized with the filesystem. It is clear that if, at some later time, the name of the parameter is changed to `UPDATE_PERIOD`, this change would be a violation of the sidecar's API and clearly would break it for some users.

Để thấy một ví dụ cụ thể về vì sao diện tích bề mặt API này lại quan trọng, hãy xét sidecar quản lý cấu hình mà ta đã bàn trước đó. Một cấu hình hữu ích cho sidecar này có thể là `UPDATE_FREQUENCY`, biểu thị tần suất cấu hình nên được đồng bộ với hệ thống tệp. Rõ ràng là nếu, vào một lúc nào đó sau này, tên của tham số bị đổi thành `UPDATE_PERIOD`, thay đổi này sẽ là một sự vi phạm API của sidecar và rõ ràng sẽ làm hỏng nó đối với một số người dùng.

While that example is obvious, even more subtle changes can break the sidecar API. Imagine, for example, that `UPDATE_FREQUENCY` originally took a number in seconds. Over time and with feedback from users, the sidecar developer determined that specifying seconds for large time values (e.g., minutes) was annoying users and changed the parameter to accept strings (10 m, 5 s, etc.). Because old parameter values (e.g., 10, for 10 seconds) won't parse in this new scheme, this is a breaking API change. Suppose still that the developer anticipated this but made values without units parse to milliseconds where they had previously parsed to seconds. Even this change, despite not leading to an error, is a breaking API change for the sidecar since it will lead to significantly more frequent configuration checks and correspondingly more load on the cloud configuration server.

Dù ví dụ đó rõ ràng, ngay cả những thay đổi tinh tế hơn cũng có thể làm hỏng API của sidecar. Hãy hình dung, chẳng hạn, ban đầu `UPDATE_FREQUENCY` nhận một con số tính bằng giây. Theo thời gian và với phản hồi từ người dùng, lập trình viên sidecar xác định rằng việc chỉ định bằng giây cho các giá trị thời gian lớn (ví dụ, hàng phút) làm phiền người dùng và đã đổi tham số để nhận chuỗi (10 m, 5 s, v.v.). Vì các giá trị tham số cũ (ví dụ, 10, cho 10 giây) sẽ không phân tích (parse) được trong sơ đồ mới này, đây là một thay

đổi API gây vỡ (breaking). Giả sử thêm rằng lập trình viên đã lường trước điều này nhưng lại khiến các giá trị không có đơn vị được phân tích thành mili-giây ở chỗ trước đây chúng được phân tích thành giây. Ngay cả thay đổi này, dù không dẫn đến lỗi, cũng là một thay đổi API gây vỡ cho sidecar vì nó sẽ dẫn đến việc kiểm tra cấu hình thường xuyên hơn đáng kể và tương ứng là tải nhiều hơn lên máy chủ cấu hình đám mây.

I hope this discussion has shown you that for true modularity, you need to be very conscious of the API that your sidecar provides, and that “breaking” changes to that API may not always be as obvious as changing the name of a parameter.

Tôi hy vọng phần thảo luận này đã cho bạn thấy rằng để có tính mô-đun thực sự, bạn cần rất ý thức về API mà sidecar của bạn cung cấp, và rằng những thay đổi “gây vỡ” với API đó có thể không phải lúc nào cũng hiển nhiên như việc đổi tên một tham số.

Documenting Your Containers — Lập tài liệu cho các Container của bạn

By now, you’ve seen how you can parameterize your sidecar containers to make them modular and reusable. You’ve learned about the importance of maintaining a stable API to ensure that you don’t break sidecars for your users. But there’s one final step to building modular, reusable containers: ensuring that people can use them in the first place.

Đến giờ, bạn đã thấy cách bạn có thể tham số hóa các container sidecar của mình để khiến chúng có tính mô-đun và tái sử dụng được. Bạn đã học về tầm quan trọng của việc duy trì một API ổn định để đảm bảo bạn không làm hỏng sidecar đối với người dùng của mình. Nhưng còn một bước cuối cùng để xây dựng các container có tính mô-đun, tái sử dụng được: đảm bảo rằng người ta có thể dùng được chúng ngay từ đầu.

As with software libraries, the key to building something truly useful is explaining how to use it. There is little good in building a flexible, reliable modular container if no one can figure out how to use it. Sadly, there are few formal tools available for documenting container images, but there are some best practices you can use to accomplish this.

Cũng như với các thư viện phần mềm, chìa khóa để xây dựng một thứ thực sự hữu ích là giải thích cách dùng nó. Chẳng ích gì khi xây dựng một container mô-đun linh hoạt, đáng tin cậy mà không ai hình dung được cách dùng nó. Đáng tiếc, có rất ít công cụ chính quy sẵn có để lập tài liệu cho các image container, nhưng có một số thực hành tốt nhất bạn có thể dùng để làm điều này.

For every **container image**, the most obvious place to look for documentation is the Dockerfile from which the container was built. There are some parts of the Docker file that already document how the container works. One example of this is the EXPOSE directive, which indicates the ports that the image listens on. Even though EXPOSE is not necessary, it is a good practice to include it in your Dockerfile and also to add a comment that explains what exactly is listening on that port. For example:

Với mỗi image container, nơi hiển nhiên nhất để tìm tài liệu là Dockerfile mà từ đó container được xây dựng. Có một số phần của tệp Docker vốn đã lập tài liệu cho cách container hoạt động. Một ví dụ về điều này là chỉ thị EXPOSE, biểu thị các cổng mà image lắng nghe. Dù EXPOSE không bắt buộc, đưa nó vào Dockerfile của bạn là một thực hành tốt, và cũng nên thêm một chú thích giải thích chính xác cái gì đang lắng nghe trên cổng đó. Ví dụ:

```
...
# Main web server runs on port 8080
EXPOSE 8080
...
```

Additionally, if you use environment variables to parameterize your container, you can use the ENV directive to set default values for those parameters as well as document their usage:

Thêm vào đó, nếu bạn dùng biến môi trường để tham số hóa container, bạn có thể dùng chỉ thị ENV để đặt giá trị mặc định cho các tham số đó cũng như lập tài liệu về cách dùng chúng:

```
...
# The PROXY_PORT parameter indicates the port on localhost to redirect
# traffic to.
ENV PROXY_PORT 8000
...
```

Finally, you should always use the LABEL directive to add metadata for your image; for example, the maintainer's email address, web page, and version of the image:

Cuối cùng, bạn nên luôn dùng chỉ thị LABEL để thêm metadata cho image của mình; ví dụ, địa chỉ email của người bảo trì (maintainer), trang web và phiên bản của image:

```
...
LABEL "org.label-schema.vendor"="name@company.com"
LABEL "org.label.url"="http://images.company.com/my-cool-image"
LABEL "org.label-schema.version"="1.0.3"
...
```

The names for these labels are drawn from the schema established by the Label Schema project . The project is working to establish a shared set of well-known labels. By using a common taxonomy of image labels, multiple different tools can rely on the same meta information in order to visualize, monitor, and correctly use an application. By adopting shared terms, you can use the set of tools developed in the community without modifying your image. Of course, you can also add whatever additional labels make sense in the context of your image.

Tên của các nhãn (label) này được lấy từ sơ đồ (schema) do dự án Label Schema thiết lập. Dự án này đang nỗ lực thiết lập một tập hợp dùng chung các nhãn nổi tiếng. Bằng cách dùng một phân loại học (taxonomy) chung về nhãn image, nhiều công cụ khác nhau có thể dựa vào cùng thông tin meta để trực quan hóa, giám sát và sử dụng đúng một ứng dụng. Bằng cách áp dụng các thuật ngữ dùng chung, bạn có thể dùng bộ công cụ được phát triển trong cộng đồng mà không cần sửa đổi image của mình. Tất nhiên, bạn cũng có thể thêm bất kỳ nhãn bổ sung nào hợp lý trong ngữ cảnh image của bạn.

Summary — Tóm tắt

Over the course of this chapter, we've introduced the sidecar pattern for combining containers on a single machine. In the sidecar pattern, a sidecar container augments and extends an application container to add functionality. Sidecars can be used to update existing legacy applications when changing the application is too costly. Likewise, they can be used to create modular utility containers that standardize implementations of common functionality. These utility containers can be reused in a large number of applications, increasing consistency and reducing the cost of developing each

application. Subsequent chapters introduce other single-node patterns that demonstrate other uses for modular, reusable containers.

Xuyên suốt chương này, ta đã giới thiệu mẫu sidecar để kết hợp các container trên một máy đơn. Trong mẫu sidecar, một container sidecar hỗ trợ và mở rộng một container ứng dụng để thêm chức năng. Sidecar có thể được dùng để cập nhật các ứng dụng kế thừa hiện có khi việc thay đổi ứng dụng quá tốn kém. Tương tự, chúng có thể được dùng để tạo ra các container tiện ích theo mô-đun nhằm chuẩn hóa cách hiện thực của những chức năng phổ biến. Những container tiện ích này có thể được tái sử dụng trong một lượng lớn ứng dụng, làm tăng tính nhất quán và giảm chi phí phát triển từng ứng dụng. Các chương tiếp theo giới thiệu những mẫu đơn nút khác minh họa những công dụng khác của các container có tính mô-đun, tái sử dụng được.

CHAPTER 3 Ambassadors —

CHAPTER 3 Ambassador (Đại sứ)

The previous chapter introduced the **sidecar** pattern, where one **container** augments a pre-existing container to add functionality. This chapter introduces the **ambassador** pattern, where an ambassador container brokers interactions between the application container and the rest of the world. As with other single-**node** patterns, the two containers are tightly linked in a symbiotic pairing that is scheduled to a single machine. A canonical diagram of this pattern is shown in Figure 3-1.

Chương trước đã giới thiệu mẫu sidecar, trong đó một container bổ sung cho một container có sẵn để thêm chức năng. Chương này giới thiệu mẫu ambassador, trong đó một container ambassador làm trung gian cho các tương tác giữa container ứng dụng và phần còn lại của thế giới. Cũng như những mẫu đơn nút (single-node) khác, hai container được liên kết chặt chẽ trong một cặp cộng sinh được lập lịch chạy trên cùng một máy. Sơ đồ tiêu biểu của mẫu này được trình bày trong Hình 3-1.

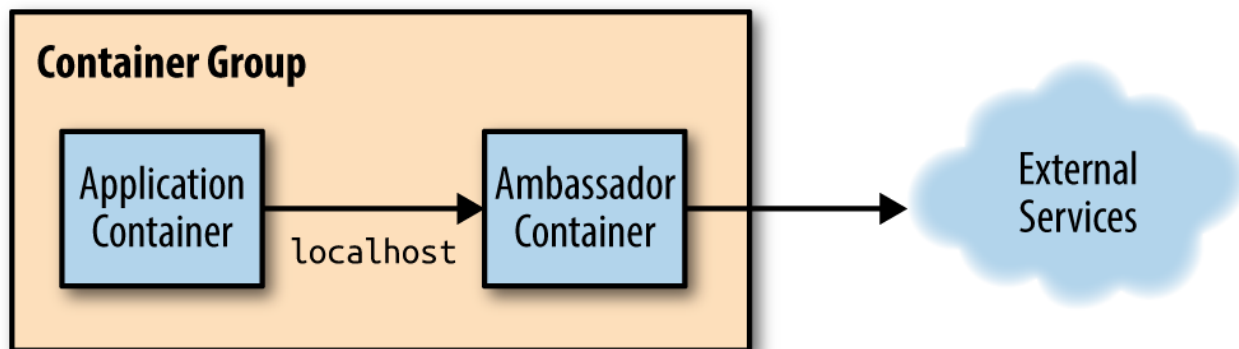


Figure 3-1. Generic ambassador pattern

Hình 3-1. Mẫu ambassador tổng quát

The value of the ambassador pattern is twofold. First, as with the other single-node patterns, there is inherent value in building modular, reusable containers. The separation of concerns makes the containers easier to build and maintain. Likewise, the ambassador container can be reused with a number of different application containers. This reuse speeds up application development because the container's code can be reused in a number of places. Additionally the implementation is both more consistent and of a higher quality because it is built once and used in many different contexts.

Giá trị của mẫu ambassador có hai mặt. Thứ nhất, cũng như các mẫu đơn nút khác, bản thân việc xây dựng những container mô-đun, có thể tái sử dụng đã mang lại giá trị. Việc

tách bạch các mối quan tâm khiến container dễ xây dựng và dễ bảo trì hơn. Tương tự, container ambassador có thể được tái sử dụng với nhiều container ứng dụng khác nhau. Sự tái sử dụng này giúp đẩy nhanh quá trình phát triển ứng dụng vì mã của container có thể được dùng lại ở nhiều nơi. Ngoài ra, phần triển khai cũng nhất quán hơn và chất lượng cao hơn vì nó được xây dựng một lần và dùng trong nhiều bối cảnh khác nhau.

The rest of this chapter provides a number of examples of using the ambassador pattern to implement a series of real-world applications.

Phần còn lại của chương này cung cấp một số ví dụ về việc dùng mẫu ambassador để hiện thực một loạt ứng dụng thực tế.

21

21

Using an Ambassador to Shard a Service — Dùng Ambassador để Phân mảnh một Dịch vụ

Sometimes the data that you want to store in a storage layer becomes too big for a single machine to handle. In such situations, you need to **shard** your storage layer. Sharding splits up the layer into multiple disjoint pieces, each hosted by a separate machine. This chapter focuses on a **single-node pattern** for adapting an existing **service** to talk to a **sharded service** that exists somewhere in the world. It does not discuss how the sharded service came to exist. Sharding and a multi-node sharded service **design pattern** are discussed in great detail in Chapter 6 . A diagram of a sharded service is shown in Figure 3-2 .

Đôi khi dữ liệu mà bạn muốn lưu trong tầng lưu trữ trở nên quá lớn để một máy đơn lẻ có thể xử lý. Trong những tình huống như vậy, bạn cần phân mảnh (shard) tầng lưu trữ của mình. Việc phân mảnh chia tầng này thành nhiều phần rời nhau, mỗi phần do một máy riêng biệt lưu giữ. Chương này tập trung vào một mẫu đơn nút (single-node) để thích ứng một dịch vụ có sẵn sao cho nó nói chuyện được với một dịch vụ phân mảnh (sharded) tồn tại đâu đó ngoài kia. Nó không bàn về việc dịch vụ phân mảnh đó ra đời như thế nào. Phân mảnh và mẫu thiết kế dịch vụ phân mảnh đa nút (multi-node) được bàn rất chi tiết trong Chương 6. Sơ đồ một dịch vụ phân mảnh được trình bày trong Hình 3-2.

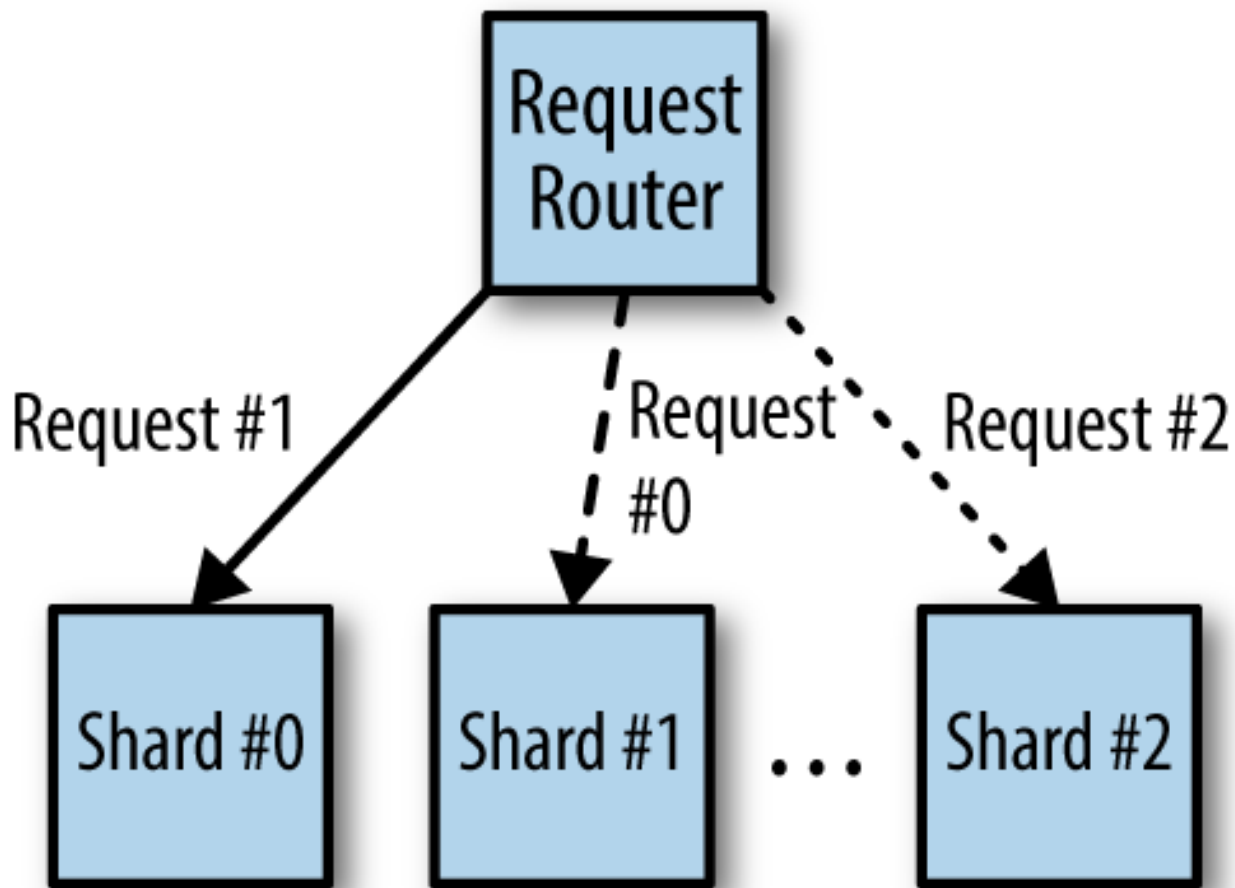


Figure 3-2. A generic sharded service

Hình 3-2. Một dịch vụ phân mảnh tổng quát

When deploying a sharded service, one question that arises is how to integrate it with the frontend or middleware code that stores data. Clearly there needs to be logic that routes a particular request to a particular shard, but often it is difficult to retrofit such a sharded client into existing source code that expects to connect to a single storage backend. Additionally, sharded services make it difficult to share **configuration** between development environments (where there is often only a single storage shard) and production environments (where there are often many storage shards).

Khi triển khai một dịch vụ phân mảnh, một câu hỏi nảy sinh là làm thế nào để tích hợp nó với mã frontend hoặc middleware vốn lưu dữ liệu. Rõ ràng cần có logic định tuyến một yêu cầu cụ thể tới một mảnh (shard) cụ thể, nhưng thường rất khó để gắn thêm một client phân mảnh như vậy vào mã nguồn có sẵn vốn chỉ kỳ vọng kết nối tới một backend lưu trữ duy nhất. Ngoài ra, các dịch vụ phân mảnh khiến việc chia sẻ cấu hình giữa môi trường phát triển (nơi thường chỉ có một mảnh lưu trữ duy nhất) và môi trường sản xuất (nơi thường có nhiều mảnh lưu trữ) trở nên khó khăn.

One approach is to build all of the sharding logic into the sharded service itself. In this approach, the sharded service also has a **stateless load balancer** that directs traffic to the appropriate shard. Effectively, this load balancer is a distributed ambassador as a service. This makes a client-side ambassador unnecessary at the expense of a more complicated **deployment** for the sharded service.

The alternative is to integrate a single-node ambassador on the client side to route traffic to the appropriate shard. This makes deploying the client somewhat more complicated but simplifies the deployment of the sharded service. As is always the case with trade-offs, it is up to the particulars of your specific application to determine which approach makes the most sense. Some factors to consider include where team lines fall in your architecture, as well as where you are writing code versus simply deploying off-the-shelf software. Ultimately, either choice is valid. The following section describes how to use the single-node ambassador pattern for client-side sharding.

Một cách tiếp cận là xây dựng toàn bộ logic phân mảnh vào ngay bên trong bản thân dịch vụ phân mảnh. Trong cách này, dịch vụ phân mảnh cũng có một bộ cân bằng tải không trạng thái (stateless) chuyển hướng lưu lượng tới mảnh phù hợp. Trên thực tế, bộ cân bằng tải này chính là một ambassador phân tán dưới dạng dịch vụ. Điều này khiến không cần ambassador phía client, đổi lại là việc triển khai dịch vụ phân mảnh phức tạp hơn. Phương án thay thế là tích hợp một ambassador đơn nút ở phía client để định tuyến lưu lượng tới mảnh phù hợp. Cách này khiến việc triển khai client phức tạp hơn đôi chút nhưng đơn giản hóa việc triển khai dịch vụ phân mảnh. Như thường thấy với các đánh đổi, việc xác định cách tiếp cận nào hợp lý nhất tùy thuộc vào đặc thù riêng của ứng dụng của bạn. Một số yếu tố cần cân nhắc bao gồm các ranh giới giữa các nhóm (team) trong kiến trúc của bạn nằm ở đâu, cũng như chỗ nào bạn tự viết mã so với chỗ nào bạn chỉ đơn thuần triển khai phần mềm có sẵn. Rốt cuộc, cả hai lựa chọn đều hợp lệ. Phần tiếp theo mô tả cách dùng mẫu ambassador đơn nút để phân mảnh phía client.

When adapting an existing application to a sharded backend, you can introduce an ambassador container that contains all of the logic needed to route requests to the appropriate storage shard. Thus, your frontend or middleware application only connects to what appears to be a single storage backend running on localhost. However, this server is in fact actually a sharding ambassador **proxy**, which receives all of the requests from your application code, sends a request to the appropriate storage shard, and then returns the result to your application. This use of an ambassador is diagrammed in Figure 3-3.

Khi thích ứng một ứng dụng có sẵn với một backend phân mảnh, bạn có thể đưa vào một container ambassador chứa toàn bộ logic cần thiết để định tuyến yêu cầu tới mảnh lưu trữ phù hợp. Nhờ đó, ứng dụng frontend hoặc middleware của bạn chỉ kết nối tới thứ trông như một backend lưu trữ duy nhất chạy trên localhost. Tuy nhiên, máy chủ này trên thực tế lại là một proxy ambassador phân mảnh (sharding ambassador proxy), thứ nhận tất cả yêu cầu từ mã ứng dụng của bạn, gửi yêu cầu tới mảnh lưu trữ phù hợp, rồi trả kết quả về cho ứng dụng. Việc sử dụng ambassador này được minh họa trong Hình 3-3.

The net result of applying the ambassador pattern to sharded services is a separation of concerns between the application container, which simply knows it needs to talk to a storage service and discovers that service on localhost, and the sharding ambassador proxy, which only contains the code necessary to perform appropriate sharding. As with all good single-node patterns, this ambassador can be reused between many different applications. Or, as we'll see in the following hands-on example, an off-the shelf **open source** implementation can be used for the ambassador, speeding up the development of the overall **distributed system**.

Kết quả cuối cùng của việc áp dụng mẫu ambassador cho các dịch vụ phân mảnh là sự tách bạch các mối quan tâm giữa container ứng dụng, vốn chỉ biết rằng nó cần nói chuyện với một dịch vụ lưu trữ và phát hiện dịch vụ đó trên localhost, với proxy ambassador phân mảnh, vốn chỉ chứa mã cần thiết để thực hiện việc phân mảnh phù hợp. Như với mọi mẫu đơn nút tốt, ambassador này có thể được tái sử dụng giữa nhiều ứng dụng khác nhau. Hoặc, như chúng ta sẽ thấy trong ví dụ thực hành tiếp theo, một phần triển khai mã nguồn mở có sẵn có thể được dùng làm ambassador, giúp đẩy nhanh quá trình phát triển toàn bộ hệ

phân tán.

Hands On: Implementing a Sharded Redis — Thực hành: Hiện thực một Redis Phân mảnh

Redis is a fast **key-value store** that can be used as a cache or for more persistent storage. In this example, we'll be using it as a cache. We'll begin by deploying a sharded Redis service to a **Kubernetes** cluster. We'll use the **StatefulSet API** object to deploy it, since it will give us unique DNS names for each shard that we can use when configuring the proxy.

Redis là một kho key-value nhanh, có thể dùng làm cache hoặc để lưu trữ bền vững hơn. Trong ví dụ này, chúng ta sẽ dùng nó làm cache. Chúng ta sẽ bắt đầu bằng việc triển khai một dịch vụ Redis phân mảnh lên một cụm Kubernetes. Chúng ta sẽ dùng đối tượng API StatefulSet để triển khai nó, vì nó sẽ cho chúng ta những tên DNS duy nhất cho từng mảnh mà ta có thể dùng khi cấu hình proxy.

The StatefulSet for Redis looks like this:

StatefulSet cho Redis trông như sau:

```
apiVersion: apps/v1beta1
kind: StatefulSet
metadata:
  name: sharded-redis
spec:
  serviceName: "redis"
  replicas: 3
  template:
    metadata:
      labels:
        app: redis
    spec:
      terminationGracePeriodSeconds: 10
      containers:
      - name: redis
        image: redis
        ports:
        - containerPort: 6379
          name: redis
```

Save this to a file named redis-shards.yaml and you can deploy this with **kubectl create -f redis-shards.yaml**. This will create three containers running redis. You can see these by running **kubectl get pods**; you should see sharded-redis-

Lưu nội dung này vào một tệp tên là redis-shards.yaml và bạn có thể triển khai nó bằng **kubectl create -f redis-shards.yaml**. Lệnh này sẽ tạo ba container chạy redis. Bạn có thể xem chúng bằng cách chạy **kubectl get pods**; bạn sẽ thấy sharded-redis-

[0,1,2].

Of course, just running the replicas isn't sufficient; we also need names by which we can refer to the replicas. In this case, we'll use a Kubernetes Service, which will create DNS names for the replicas we have created. The Service looks like this:

Tất nhiên, chỉ chạy các bản sao thôi thì chưa đủ; chúng ta cũng cần những tên để tham chiếu tới các bản sao đó. Trong trường hợp này, chúng ta sẽ dùng một Service của Kubernetes, thứ sẽ tạo các tên DNS cho những bản sao mà ta đã tạo. Service trông như sau:

```
apiVersion: v1
kind: Service
metadata:
  name: redis
  labels:
    app: redis
spec:
  ports:
    - port: 6379
      name: redis
  clusterIP: None
  selector:
    app: redis
```

Save this to a file named `redis-service.yaml` and deploy with `kubectl create -f redis-service.yaml`. You should now have DNS entries for `sharded-redis-0.redis`, `sharded-redis-1.redis`, etc. We can use these names to configure **twemproxy**. `twemproxy` is a lightweight, highly performant proxy for **memcached** and Redis, which was originally developed by Twitter and is open source and available on GitHub. We can configure `twemproxy` to point to the replicas we created by using the following configuration:

Lưu nội dung này vào một tệp tên là `redis-service.yaml` và triển khai bằng `kubectl create -f redis-service.yaml`. Bây giờ bạn sẽ có các mục DNS cho `sharded-redis-0.redis`, `sharded-redis-1.redis`, v.v. Chúng ta có thể dùng những tên này để cấu hình `twemproxy`. `twemproxy` là một proxy nhẹ, hiệu năng cao cho `memcached` và Redis, ban đầu được Twitter phát triển và là mã nguồn mở, có sẵn trên GitHub. Chúng ta có thể cấu hình `twemproxy` trở tới những bản sao đã tạo bằng cách dùng cấu hình sau:

```
redis:
  listen: 127.0.0.1:6379
  hash: fnv1a_64
  distribution: ketama
  auto_eject_hosts: true
  redis: true
  timeout: 400
  server_retry_timeout: 2000
  server_failure_limit: 1
  servers:
    - sharded-redis-0.redis:6379:1
    - sharded-redis-1.redis:6379:1
    - sharded-redis-2.redis:6379:1
```

In this config, you can see that we are serving the Redis protocol on `localhost:6379` so that the application container can access the ambassador. We will deploy this into our ambassador **pod** using a Kubernetes **ConfigMap** object that we can create with:

Trong cấu hình này, bạn có thể thấy chúng ta đang phục vụ giao thức Redis trên `localhost:6379` để container ứng dụng có thể truy cập ambassador. Chúng ta sẽ triển khai

nó vào pod ambassador của mình bằng một đối tượng ConfigMap của Kubernetes mà ta có thể tạo bằng:

```
kubectl create configmap --from-file=nutcracker.yaml
```

Finally, all of the preparations are done and we can deploy our ambasssador example. We define a pod that looks like:

Cuối cùng, mọi sự chuẩn bị đã xong và chúng ta có thể triển khai ví dụ ambassador của mình. Chúng ta định nghĩa một pod trông như sau:

```
apiVersion: v1
kind: Pod
metadata:
  name: ambassador-example
spec:
  containers:
    # This is where the application container would go, for example
    # - name: nginx
    #   image: nginx
    # This is the ambassador container
    - name: twemproxy
      image: ganomede/twemproxy
      command:
        - nutcracker
        - -c
        - /etc/config/nutcracker.yaml
        - -v
        - 7
        - -s
        - 6222
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
  volumes:
    - name: config-volume
      configMap:
        name: twem-config
```

This pod defines the ambassador; then the specific user's application container can be injected to complete the pod.

Pod này định nghĩa ambassador; sau đó container ứng dụng cụ thể của người dùng có thể được tiêm vào để hoàn thiện pod.

Using an Ambassador for Service Brokering — Dùng Ambassador để Làm Trung gian Dịch vụ

When trying to render an application **portable** across multiple environments (e.g., public cloud, physical datacenter, or private cloud), one of the primary challenges is **service discovery** and configuration. To understand what this means, imagine a frontend that relies on a MySQL database to store its data. In the public cloud, this MySQL service might be provided as software-as-a-service

(SaaS), whereas in a private cloud it might be necessary to dynamically spin up a new virtual machine or container running MySQL.

Khi cố gắng làm cho một ứng dụng có thể di động (portable) qua nhiều môi trường (ví dụ: public cloud, trung tâm dữ liệu vật lý, hoặc private cloud), một trong những thách thức chính là phát hiện dịch vụ (service discovery) và cấu hình. Để hiểu điều này nghĩa là gì, hãy hình dung một frontend dựa vào một cơ sở dữ liệu MySQL để lưu dữ liệu của nó. Trên public cloud, dịch vụ MySQL này có thể được cung cấp dưới dạng phần mềm như một dịch vụ (software-as-a-service, SaaS), trong khi trên private cloud có thể cần khởi tạo động một máy ảo hoặc container mới chạy MySQL.

Consequently, building a portable application requires that the application know how to **introspect** its environment and find the appropriate MySQL service to connect to. This process is called service discovery, and the system that performs this discovery and linking is commonly called a **service broker**. As with previous examples, the ambassador pattern enables a system to separate the logic of the application container from the logic of the service broker ambassador. The application simply always connects to an instance of the service (e.g., MySQL) running on localhost. It is the responsibility of the service broker ambassador to introspect its environment and broker the appropriate connection. This process is shown in Figure 3-3.

Do đó, việc xây dựng một ứng dụng di động đòi hỏi ứng dụng biết cách xem xét nội tại (introspect) môi trường của nó và tìm ra dịch vụ MySQL phù hợp để kết nối. Quá trình này gọi là phát hiện dịch vụ (service discovery), và hệ thống thực hiện việc phát hiện và liên kết này thường được gọi là bộ trung gian dịch vụ (service broker). Như trong các ví dụ trước, mẫu ambassador cho phép một hệ thống tách logic của container ứng dụng khỏi logic của ambassador trung gian dịch vụ. Ứng dụng đơn giản luôn kết nối tới một thực thể của dịch vụ (ví dụ: MySQL) chạy trên localhost. Trách nhiệm của ambassador trung gian dịch vụ là xem xét nội tại môi trường của nó và làm trung gian cho kết nối phù hợp. Quá trình này được minh họa trong Hình 3-3.

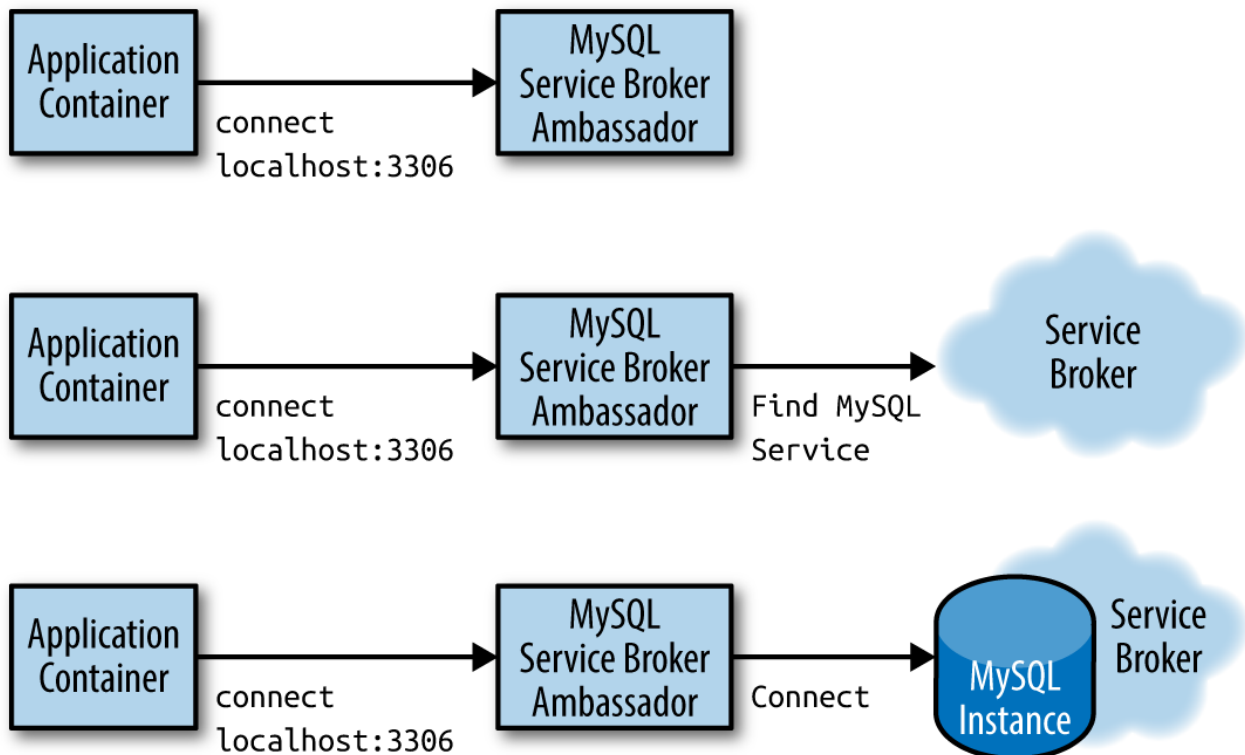


Figure 3-3. A service broker ambassador creating a MySQL service

Hình 3-3. Một ambassador trung gian dịch vụ tạo ra một dịch vụ MySQL

Using an Ambassador to Do Experimentation or Request Splitting — Dùng Ambassador để Thử nghiệm hoặc Tách Yêu cầu

A final example application of the ambassador pattern is to perform experimentation or other forms of **request splitting**. In many production systems, it is advantageous to be able to perform request splitting, where some fraction of all requests are not serviced by the main production service but rather are redirected to a different implementation of the service. Most often, this is used to perform experiments with new, beta versions of the service to determine if the new version of the software is reliable or comparable in performance to the currently deployed version.

Một ví dụ ứng dụng cuối cùng của mẫu ambassador là thực hiện thử nghiệm hoặc các hình thức tách yêu cầu (request splitting) khác. Trong nhiều hệ thống sản xuất, việc có khả năng tách yêu cầu là một lợi thế, theo đó một phần nhỏ trong tất cả các yêu cầu không được phục vụ bởi dịch vụ sản xuất chính mà thay vào đó được chuyển hướng tới một phần triển khai khác của dịch vụ. Thông thường nhất, điều này được dùng để thực hiện các thử nghiệm với những phiên bản beta mới của dịch vụ nhằm xác định xem phiên bản phần mềm mới có đáng tin cậy hay có hiệu năng tương đương với phiên bản đang triển khai hiện tại hay không.

Additionally, request splitting is sometimes used to **tee** or split traffic such that all traffic goes to both the production system as well as a newer, undeployed version. The responses from the production system are returned to the user, while the responses from the tee-d service are ignored. Most often, this form of request splitting is used to simulate production load on the new version of the service without risking impact to existing production users.

Ngoài ra, tách yêu cầu đôi khi được dùng để rẽ nhánh (tee) hoặc tách lưu lượng sao cho toàn bộ lưu lượng đi tới cả hệ thống sản xuất lẫn một phiên bản mới hơn, chưa triển khai. Các phản hồi từ hệ thống sản xuất được trả về cho người dùng, trong khi các phản hồi từ dịch vụ được rẽ nhánh (tee) thì bị bỏ qua. Thông thường nhất, hình thức tách yêu cầu này được dùng để mô phỏng tải sản xuất trên phiên bản mới của dịch vụ mà không gây rủi ro ảnh hưởng tới những người dùng sản xuất hiện hữu.

Given the previous examples, it is straightforward to see how a request-splitting ambassador can interact with an application container to implement request splitting. As before, the application container simply connects to the service on localhost, while the ambassador container receives the requests, proxies responses to both the production and experimental systems, and then returns the production responses back as if it had performed the work itself.

Với các ví dụ trước, có thể dễ dàng thấy được làm thế nào một ambassador tách yêu cầu tương tác với một container ứng dụng để hiện thực việc tách yêu cầu. Như trước, container ứng dụng đơn giản kết nối tới dịch vụ trên localhost, trong khi container ambassador nhận các yêu cầu, làm proxy cho các phản hồi tới cả hệ thống sản xuất lẫn hệ thống thử nghiệm, rồi trả các phản hồi sản xuất về như thể chính nó đã thực hiện công việc.

This separation of concerns keeps the code in each container slim and focused, and the modular factoring of the application ensures that the request-splitting ambassador can be reused for a variety of different applications and settings.

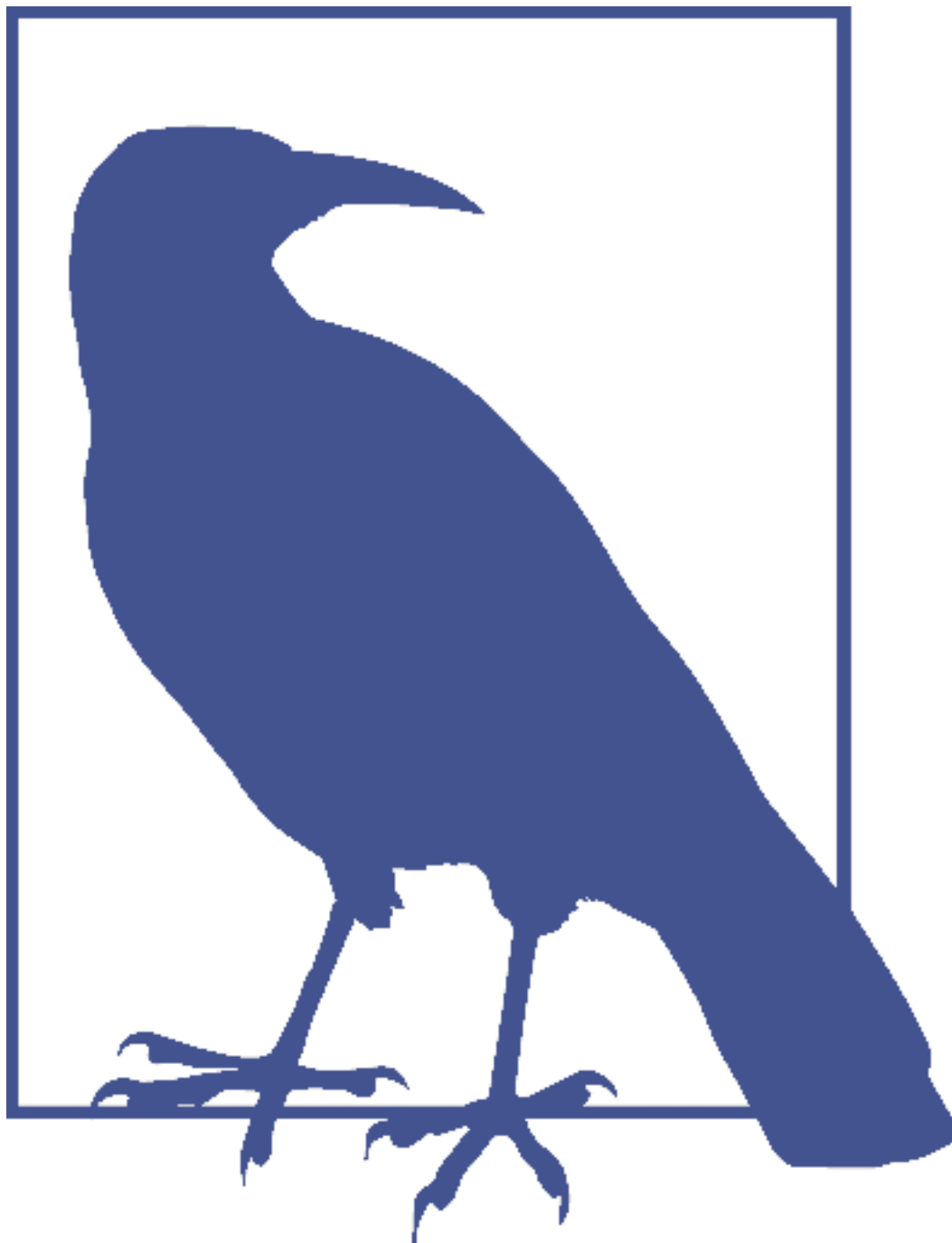
Sự tách bạch các mối quan tâm này giữ cho mã trong mỗi container gọn nhẹ và tập trung, và việc phân tách mô-đun của ứng dụng đảm bảo rằng ambassador tách yêu cầu có thể được tái sử dụng cho nhiều ứng dụng và bối cảnh khác nhau.

Hands On: Implementing 10% Experiments — Thực hành: Hiện thực Thử nghiệm 10%

To implement our request-splitting experiment, we're going to use the **nginx** web server. Nginx is a powerful, richly featured open source server. To configure nginx as the ambassador, we'll use the following configuration (note that this is for HTTP but it could easily be adapted for HTTPS as well).

Để hiện thực thử nghiệm tách yêu cầu của mình, chúng ta sẽ dùng máy chủ web nginx. Nginx là một máy chủ mã nguồn mở mạnh mẽ, giàu tính năng. Để cấu hình nginx làm ambassador, chúng ta sẽ dùng cấu hình sau (lưu ý rằng đây là cho HTTP nhưng nó cũng có thể dễ dàng được thích ứng cho HTTPS).

```
worker_processes 5;
error_log error.log;
pid nginx.pid;
worker_rlimit_nofile 8192;
events {
    worker_connections 1024;
}
http {
    upstream backend {
        ip_hash;
        server web weight=9;
        server experiment;
    }
    server {
        listen localhost:80;
        location / {
            proxy_pass http://backend;
        }
    }
}
```



As with the previous discussion of sharded services, it's also possible to deploy the experiment framework as a separate **microservice** in front of your application instead of integrating it as a part of your client pods. Of course, by doing this you are introducing another service that needs to be maintained, scaled, monitored, etc. If experimentation is likely to be a longstanding component in your architecture, this might be worthwhile. If it is used more occasionally, then a client-side ambassador might make more sense.

Cũng như phần thảo luận trước về các dịch vụ phân mảnh, ta cũng có thể triển khai khung thử nghiệm như một microservice riêng biệt đặt trước ứng dụng của bạn thay vì tích hợp nó như một phần của các pod client. Tất nhiên, khi làm vậy bạn đang đưa vào thêm một

dịch vụ cần được bảo trì, mở rộng quy mô, giám sát, v.v. Nếu việc thử nghiệm có khả năng trở thành một thành phần lâu dài trong kiến trúc của bạn, thì điều này có thể đáng làm. Nếu nó chỉ được dùng thỉnh thoảng, thì một ambassador phía client có thể hợp lý hơn.

You'll note that I'm using **IP hashing** in this configuration. This is important because it ensures that the user doesn't flip-flop back and forth between the experiment and the main site. This assures that every user has a consistent experience with the application.

Bạn sẽ nhận thấy rằng tôi đang dùng băm theo IP (IP hashing) trong cấu hình này. Điều này quan trọng vì nó đảm bảo người dùng không bị nhảy qua nhảy lại giữa thử nghiệm và trang chính. Việc này đảm bảo mỗi người dùng có một trải nghiệm nhất quán với ứng dụng.

The weight parameter is used to send 90% of the traffic to the main existing application, while 10% of the traffic is redirected to the experiment.

Tham số weight được dùng để gửi 90% lưu lượng tới ứng dụng chính hiện hữu, trong khi 10% lưu lượng được chuyển hướng tới thử nghiệm.

As with other examples, we'll deploy this configuration as a ConfigMap object in Kubernetes:

Như với các ví dụ khác, chúng ta sẽ triển khai cấu hình này như một đối tượng ConfigMap trong Kubernetes:

```
kubectl create configmaps --from-file=nginx.conf
```

Of course, this assumes that you have both a web and experiment service defined. If you don't, you need to create them now before you try to create the ambassador container, since nginx doesn't like to start if it can't find the services it is proxying to. Here are some example service configs:

Tất nhiên, điều này giả định rằng bạn đã định nghĩa cả dịch vụ web và experiment. Nếu chưa, bạn cần tạo chúng ngay bây giờ trước khi cố tạo container ambassador, vì nginx không thích khởi động nếu nó không tìm thấy các dịch vụ mà nó làm proxy tới. Dưới đây là một số ví dụ cấu hình dịch vụ:

```
# This is the 'experiment' service
apiVersion: v1
kind: Service
metadata:
  name: experiment
  labels:
    app: experiment
spec:
  ports:
    - port: 80
      name: web
  selector:
    app: experiment
# Change this selector to match your application's labels
---
# This is the 'prod' service
apiVersion: v1
kind: Service
metadata:
  name: web
```

```

    labels:
      app: web
spec:
  ports:
    - port: 80
      name: web
  selector:
# Change this selector to match your application's labels
    app: web

```

And then we will deploy nginx itself as the ambassador container within a pod:

Và sau đó chúng ta sẽ triển khai chính nginx như container ambassador bên trong một pod:

```

apiVersion: v1
kind: Pod
metadata:
  name: experiment-example
spec:
  containers:
# This is where the application container would go, for example
# - name: some-name
#   image: some-image
# This is the ambassador container
    - name: nginx
      image: nginx
      volumeMounts:
        - name: config-volume
          mountPath: /etc/nginx
  volumes:
    - name: config-volume
      configMap:
        name: experiment-config

```

You can add a second (or third, or fourth) container to the pod to take advantage of the ambassador.

Bạn có thể thêm một container thứ hai (hoặc thứ ba, hoặc thứ tư) vào pod để tận dụng ambassador.

CHAPTER 4 Adapters — CHAPTER 4

Mẫu Adapter

In the preceding chapters, we saw how the **sidecar** pattern can extend and augment existing application containers. We also saw how ambassadors can alter and broker how an application **container** communicates with the external world. This chapter describes the final **single-node**** pattern: **the adapter**** pattern. In the adapter pattern, the adapter container is used to modify the **interface** of the application container so that it conforms to some predefined interface that is expected of all applications. For example, an adapter might ensure that an application implements a consistent monitoring interface. Or it might ensure that log files are always written to stdout or any number of other conventions.

Ở các chương trước, chúng ta đã thấy mẫu sidecar có thể mở rộng và bổ trợ cho các container ứng dụng hiện có như thế nào. Chúng ta cũng đã thấy ambassador có thể thay đổi và làm trung gian cho cách một container ứng dụng giao tiếp với thế giới bên ngoài. Chương này mô tả mẫu đơn nút (single-node) cuối cùng: mẫu adapter. Trong mẫu adapter, container adapter được dùng để chỉnh sửa giao diện của container ứng dụng sao cho nó tuân theo một giao diện định sẵn nào đó mà mọi ứng dụng đều được kỳ vọng phải có. Ví dụ, một adapter có thể đảm bảo rằng một ứng dụng hiện thực một giao diện giám sát nhất quán. Hoặc nó có thể đảm bảo rằng các tệp log luôn được ghi ra stdout, hay bất kỳ quy ước nào khác.

Real-world application development is a heterogeneous, hybrid exercise. Some parts of your application might be written from scratch by your team, some supplied by vendors, and some might consist entirely of off-the-shelf **open source** or proprietary software that you consume as precompiled binary. The net effect of this heterogeneity is that any real-world application you deploy will have been written in a variety of languages, with a variety of conventions for logging, monitoring, and other common services.

Việc phát triển ứng dụng trong thực tế là một bài toán không đồng nhất, mang tính lai ghép. Một số phần trong ứng dụng của bạn có thể được nhóm của bạn viết từ đầu, một số do nhà cung cấp cung cấp, và một số có thể hoàn toàn là phần mềm nguồn mở hoặc phần mềm độc quyền có sẵn (off-the-shelf) mà bạn dùng dưới dạng nhị phân đã biên dịch sẵn. Hệ quả của sự không đồng nhất này là bất kỳ ứng dụng thực tế nào bạn triển khai cũng đều sẽ được viết bằng nhiều ngôn ngữ khác nhau, với nhiều quy ước khác nhau về ghi log, giám sát và các dịch vụ chung khác.

Yet, to effectively monitor and operate your application, you need common interfaces. When each application provides metrics using a different format and interface, it is very difficult to collect all of those metrics in a single place for visualization and alerting. This is where the adapter pattern is relevant. Like other single-node patterns, the adapter pattern is made up of modular containers.

Different application containers can present many different monitoring interfaces while the adapter container adapts this heterogeneity to present a consistent interface. This enables you to deploy a single tool that expects this single interface. Figure 4-1 illustrates this general pattern.

Tuy vậy, để giám sát và vận hành ứng dụng của mình một cách hiệu quả, bạn cần các giao diện chung. Khi mỗi ứng dụng cung cấp các chỉ số (metric) theo một định dạng và giao diện khác nhau, việc thu thập tất cả các chỉ số đó về một nơi để trực quan hóa và cảnh báo trở nên rất khó khăn. Đây chính là nơi mẫu adapter phát huy tác dụng. Giống như các mẫu đơn nút khác, mẫu adapter được tạo thành từ các container mô-đun. Các container ứng dụng khác nhau có thể trình ra nhiều giao diện giám sát khác nhau, trong khi container adapter sẽ thích ứng sự không đồng nhất này để trình ra một giao diện nhất quán. Điều này cho phép bạn triển khai một công cụ duy nhất kỳ vọng vào giao diện thống nhất đó. Hình 4-1 minh họa mẫu tổng quát này.

31

31

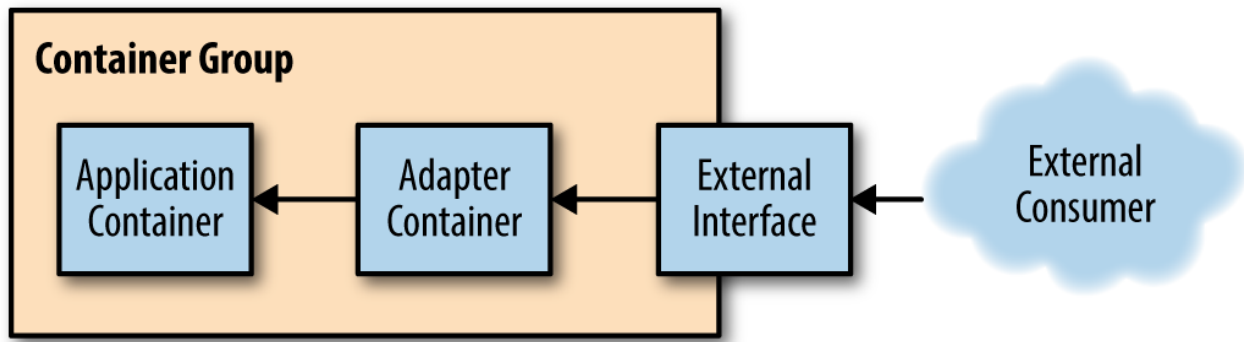


Figure 4-1. The generic adapter pattern

Hình 4-1. Mẫu adapter tổng quát

The remainder of this chapter gives several different applications of the adapter pattern.

Phần còn lại của chương này trình bày một số ứng dụng khác nhau của mẫu adapter.

Monitoring — Giám sát

When monitoring your software, you want a single solution that can automatically discover and monitor any application that is deployed into your environment. To make this feasible, every application has to implement the same monitoring interface. There are numerous examples of standardized monitoring interfaces, such as syslog , **event** tracing on Windows (etw), JMX for Java applications, and many, many other protocols and interfaces. However, each of these is unique in both protocol for communication as well as the style of communication (push versus pull).

Khi giám sát phần mềm của mình, bạn muốn có một giải pháp duy nhất có thể tự động phát hiện và giám sát bất kỳ ứng dụng nào được triển khai vào môi trường của bạn. Để điều này khả thi, mọi ứng dụng đều phải hiện thực cùng một giao diện giám sát. Có rất nhiều ví dụ về các giao diện giám sát chuẩn hóa, chẳng hạn như syslog, event tracing trên Windows (etw), JMX cho các ứng dụng Java, và rất, rất nhiều giao thức cùng giao diện khác. Tuy nhiên, mỗi cái lại độc nhất cả về giao thức giao tiếp lẫn phong cách giao tiếp (đẩy hay kéo — push versus pull).

Sadly, applications in your **distributed system** are likely to span the gamut from code that you have written yourself to off-the-shelf open source components. As a result, you will find yourself with a wide range of different monitoring interfaces that you need to integrate into a single well-understood system.

Đáng tiếc là các ứng dụng trong hệ phân tán của bạn nhiều khả năng sẽ trải dài từ mã do chính bạn tự viết cho đến các thành phần nguồn mở có sẵn. Kết quả là bạn sẽ thấy mình phải đối mặt với một loạt giao diện giám sát khác nhau mà bạn cần tích hợp vào một hệ thống thống nhất, dễ hiểu.

Fortunately, most monitoring solutions understand that they need to be widely applicable, and thus they have implemented a variety of plugins that can adapt one monitoring format to a common interface. Given this set of tools, how can we deploy and manage our applications in an agile and stable manner? Fortunately, the adapter pattern can provide us with the answers. Applying the adapter pattern to monitoring, we see that the application container is simply the application that we want to monitor. The adapter container contains the tools for transforming the monitoring interface exposed by the application container into the interface expected by the general-purpose monitoring system.

May mắn thay, hầu hết các giải pháp giám sát đều hiểu rằng chúng cần áp dụng được trên diện rộng, và do đó chúng đã hiện thực nhiều plugin khác nhau có thể thích ứng một định dạng giám sát sang một giao diện chung. Với bộ công cụ này, làm thế nào chúng ta có thể triển khai và quản lý các ứng dụng của mình một cách linh hoạt và ổn định? May thay, mẫu adapter có thể cung cấp cho chúng ta câu trả lời. Áp dụng mẫu adapter vào việc giám sát, ta thấy container ứng dụng đơn giản chính là ứng dụng mà ta muốn giám sát. Container adapter chứa các công cụ để biến đổi giao diện giám sát mà container ứng dụng phơi ra thành giao diện mà hệ thống giám sát đã dụng kỳ vọng.

Decoupling the system in this fashion makes for a more comprehensible, maintainable system. Rolling out new versions of the application doesn't require a **rollout** of the monitoring adapter. Additionally, the monitoring container can be reused with multiple different application containers. The monitoring container may even have been supplied by the monitoring system maintainers independent of the application developers. Finally, deploying the monitoring adapter as a separate container ensures that each container gets its own dedicated resources in terms of both CPU and memory.

Việc tách rời hệ thống theo cách này tạo nên một hệ thống dễ hiểu và dễ bảo trì hơn. Việc tung ra phiên bản mới của ứng dụng không đòi hỏi phải tung ra lại adapter giám sát. Ngoài ra, container giám sát có thể được tái sử dụng với nhiều container ứng dụng khác nhau. Container giám sát thậm chí có thể đã được những người duy trì hệ thống giám sát cung cấp một cách độc lập với các nhà phát triển ứng dụng. Cuối cùng, việc triển khai adapter giám sát như một container riêng biệt đảm bảo rằng mỗi container nhận được tài nguyên riêng dành cho nó, cả về CPU lẫn bộ nhớ.

This ensures that a misbehaving monitoring adapter cannot cause problems with a user-facing **service**.

Điều này đảm bảo rằng một adapter giám sát hoạt động sai lệch không thể gây ra vấn đề cho một dịch vụ hướng tới người dùng.

Hands On: Using Prometheus for Monitoring — Thực hành (Hands On): Dùng Prometheus để giám sát

As an example, consider monitoring your containers via the **Prometheus** open source project . Prometheus is a monitoring aggregator, which collects metrics and aggregates them into a single time-series database. On top of this database, Prometheus provides visualization and query language for introspecting the collected metrics. To collect metrics from a variety of different systems, Prometheus expects every container to expose a specific metrics **API**. This enables Prometheus to monitor a wide variety of different programs through a single interface.

Ví dụ, hãy xét việc giám sát các container của bạn thông qua dự án nguồn mở Prometheus. Prometheus là một bộ tổng hợp giám sát (monitoring aggregator), nó thu thập các chỉ số và tổng hợp chúng vào một cơ sở dữ liệu chuỗi thời gian (time-series database) duy nhất. Trên nền cơ sở dữ liệu này, Prometheus cung cấp khả năng trực quan hóa và một ngôn ngữ truy vấn để xem xét các chỉ số đã thu thập. Để thu thập chỉ số từ nhiều hệ thống khác nhau, Prometheus kỳ vọng mỗi container phơi ra một API chỉ số cụ thể. Điều này cho phép Prometheus giám sát nhiều chương trình khác nhau thông qua một giao diện duy nhất.

However, many popular programs, such as the **Redis key-value store**, do not export metrics in a format that is compatible with Prometheus. Consequently, the adapter pattern is quite useful for taking an existing service like Redis and adapting it to the Prometheus metrics-collection interface.

Tuy nhiên, nhiều chương trình phổ biến, chẳng hạn như kho khóa-giá trị Redis, lại không xuất các chỉ số theo định dạng tương thích với Prometheus. Do đó, mẫu adapter khá hữu ích trong việc lấy một dịch vụ hiện có như Redis và thích ứng nó cho phù hợp với giao diện thu thập chỉ số của Prometheus.

Consider a simple **Kubernetes pod** definition for a Redis server:

Hãy xét một định nghĩa Pod Kubernetes đơn giản cho một máy chủ Redis:

```
apiVersion: v1
kind: Pod
metadata:
  name: adapter-example
  namespace: default
spec:
  containers:
  - image: redis
    name: redis
```

At this point, this container is not capable of being monitored by Prometheus because it does not export the right interface. However, if we simply add an adapter container (in this case, an open source Prometheus exporter), we can modify this pod to export the correct interface and thus adapt it to fit Prometheus's expectations:

Tại thời điểm này, container này chưa thể được Prometheus giám sát vì nó không xuất ra giao diện phù hợp. Tuy nhiên, nếu ta chỉ đơn giản thêm vào một container adapter (trong trường hợp này là một Prometheus exporter nguồn mở), ta có thể chỉnh sửa Pod này để xuất ra giao diện đúng và nhờ đó thích ứng nó cho khớp với kỳ vọng của Prometheus:

```
apiVersion: v1
kind: Pod
metadata:
```

```

name: adapter-example
namespace: default
spec:
  containers:
  - image: redis
    name: redis
  # Provide an adapter that implements the Prometheus interface
  - image: oliver006/redis_exporter
    name: adapter

```

This example illustrates not only the value of the adapter pattern for ensuring a consistent interface, but also the value of container patterns in general for modular container reuse. In this case, the example shown combines an existing Redis container with an existing Prometheus adapter. The net effect is a monitorable Redis server, with little work on our part to deploy it. In the absence of the adapter pattern, the same **deployment** would have required significantly more custom work and would have resulted in a much less operable solution, since any updates to either Redis or the adapter would have required work to apply the update.

Ví dụ này không chỉ minh họa giá trị của mẫu adapter trong việc đảm bảo một giao diện nhất quán, mà còn minh họa giá trị của các mẫu container nói chung trong việc tái sử dụng container theo kiểu mô-đun. Ở đây, ví dụ được trình bày kết hợp một container Redis có sẵn với một adapter Prometheus có sẵn. Hiệu quả thu được là một máy chủ Redis có thể giám sát được, với rất ít công sức từ phía chúng ta để triển khai nó. Nếu không có mẫu adapter, cùng một việc triển khai như vậy sẽ đòi hỏi nhiều công việc tùy chỉnh hơn đáng kể, và sẽ cho ra một giải pháp ít vận hành được hơn nhiều, vì bất kỳ bản cập nhật nào cho Redis hoặc adapter cũng sẽ đòi hỏi công sức để áp dụng bản cập nhật đó.

Logging — Ghi log

Much like monitoring, there is a wide variety of heterogeneity in how systems log data to an output stream. Systems might divide their logs into different levels (such as debug, info, warning, and error) with each level going into a different file. Some might simply log to stdout and stderr. This is especially problematic in the world of containerized applications where there is a general expectation that your containers will log to stdout, because that is what is available via commands like **docker logs** or

Cũng giống như giám sát, có sự không đồng nhất rất lớn trong cách các hệ thống ghi log dữ liệu ra một luồng đầu ra. Các hệ thống có thể chia log của chúng thành các mức khác nhau (như debug, info, warning và error) với mỗi mức đi vào một tệp khác nhau. Một số hệ thống có thể chỉ đơn giản ghi log ra stdout và stderr. Điều này đặc biệt rắc rối trong thế giới các ứng dụng đóng gói container, nơi có một kỳ vọng chung rằng các container của bạn sẽ ghi log ra stdout, bởi đó là những gì truy cập được qua các lệnh như docker logs hoặc

kubectl logs.

Adding further complexity, the information logged generally has structured information (e.g., the date/time of the log), but this information varies widely between different logging libraries (e.g., Java's built-in logging versus the log package for Go).

Làm cho mọi thứ phức tạp hơn nữa, thông tin được ghi log thường có thông tin có cấu trúc (ví dụ, ngày/giờ của log), nhưng thông tin này lại khác nhau rất nhiều giữa các thư viện

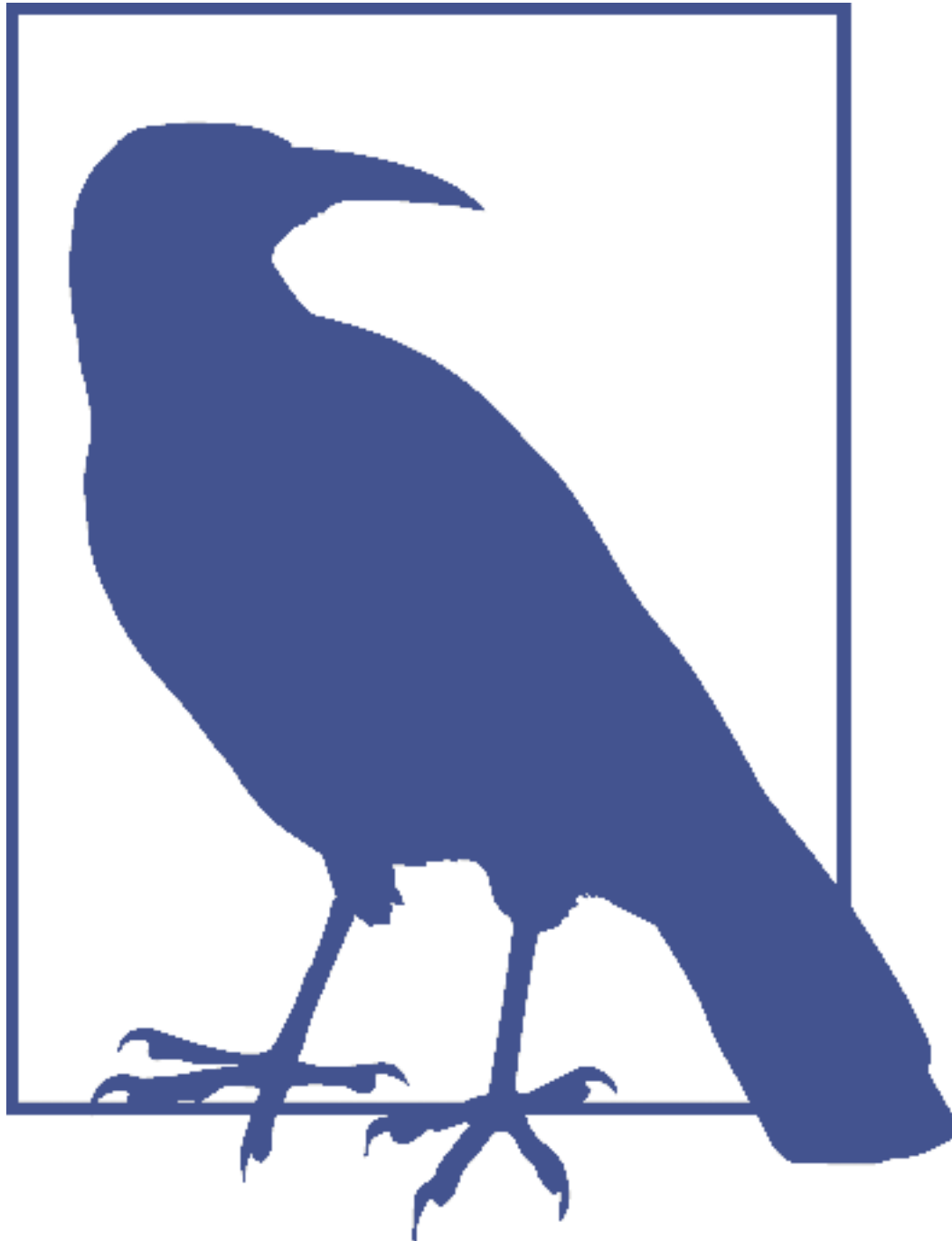
ghi log khác nhau (ví dụ, hệ thống ghi log tích hợp sẵn của Java so với gói glog cho Go).

Of course, when you are storing and querying the logs for your distributed system, you don't really care about these differences in logging format. You want to ensure that despite different structures for the data, every log ends up with the appropriate timestamp.

Tất nhiên, khi bạn lưu trữ và truy vấn log cho hệ phân tán của mình, bạn thực sự không quan tâm đến những khác biệt về định dạng ghi log này. Bạn muốn đảm bảo rằng dù dữ liệu có cấu trúc khác nhau, mọi log đều có dấu thời gian phù hợp.

Fortunately, as with monitoring, the adapter pattern can help provide a modular, re-usable design for both of these situations. While the application container may log to a file, the adapter container can redirect that file to stdout. Different application containers can log information in different formats, but the adapter container can transform that data into a single structured representation that can be consumed by your log aggregator. Again, the adapter is taking a heterogeneous world of applications and creating a homogenous world of common interfaces.

May mắn thay, cũng như với giám sát, mẫu adapter có thể giúp cung cấp một thiết kế mô-đun, tái sử dụng được cho cả hai tình huống này. Trong khi container ứng dụng có thể ghi log ra một tệp, container adapter có thể chuyển hướng tệp đó ra stdout. Các container ứng dụng khác nhau có thể ghi log thông tin theo các định dạng khác nhau, nhưng container adapter có thể biến đổi dữ liệu đó thành một biểu diễn có cấu trúc duy nhất mà bộ tổng hợp log của bạn có thể tiêu thụ. Một lần nữa, adapter đang lấy một thế giới ứng dụng không đồng nhất và tạo ra một thế giới đồng nhất gồm các giao diện chung.



One question that often comes up when considering adapter patterns is: Why not simply modify the application container itself? If you are the developer responsible for the application container, then this might actually be a good solution. Adapting your code or your container to implement a consistent interface can work well. However, in many cases we are reusing a container produced by another party. In such cases, deriving a slightly modified image that we have to maintain (patch, rebase, etc.) is significantly more expensive than developing an adapter container that can run alongside the other party's image. Additionally, decoupling the adapter into its own container allows for the possibility of sharing and reuse, which isn't possible when you modify the application container.

Một câu hỏi thường nảy sinh khi xét đến các mẫu adapter là: Tại sao không đơn giản chỉnh sửa luôn chính container ứng dụng? Nếu bạn là nhà phát triển chịu trách nhiệm cho container ứng dụng, thì đây thực sự có thể là một giải pháp tốt. Việc thích ứng mã của bạn hoặc container của bạn để hiện thực một giao diện nhất quán có thể hoạt động tốt. Tuy nhiên, trong nhiều trường hợp chúng ta đang tái sử dụng một container do bên khác sản xuất. Trong những trường hợp như vậy, việc tạo ra một image được chỉnh sửa đôi chút mà ta phải duy trì (vá lỗi, rebase, v.v.) tốn kém hơn đáng kể so với việc phát triển một container adapter có thể chạy song song với image của bên kia. Ngoài ra, việc tách rời adapter thành container riêng của nó mở ra khả năng chia sẻ và tái sử dụng, điều không thể có được khi bạn chỉnh sửa chính container ứng dụng.

Hands On: Normalizing Different Logging Formats with Fluentd

— Thực hành (Hands On): Chuẩn hóa các định dạng log khác nhau với Fluentd

One common task for an adapter is to normalize log metrics into a standard set of events. Many different applications have different output formats, but you can use a standard logging tool deployed as an adapter to normalize them all to a consistent format. In this example, we will use the **fluentd** monitoring agent as well as some community-supported plugins to obtain logs from a variety of different sources.

Một nhiệm vụ phổ biến của adapter là chuẩn hóa các chỉ số log thành một tập sự kiện chuẩn. Nhiều ứng dụng khác nhau có các định dạng đầu ra khác nhau, nhưng bạn có thể dùng một công cụ ghi log chuẩn được triển khai dưới dạng adapter để chuẩn hóa tất cả chúng về một định dạng nhất quán. Trong ví dụ này, chúng ta sẽ dùng tác nhân giám sát fluentd cùng với một số plugin do cộng đồng hỗ trợ để lấy log từ nhiều nguồn khác nhau.

fluentd is one of the more popular open source logging agents available. One of its major features is a rich set of community-supported plugins that enable a great deal of flexibility in monitoring a variety of applications.

fluentd là một trong những tác nhân ghi log nguồn mở phổ biến hơn cả hiện có. Một trong những tính năng chính của nó là một bộ plugin phong phú do cộng đồng hỗ trợ, mang lại sự linh hoạt rất lớn trong việc giám sát nhiều loại ứng dụng.

The first application that we will monitor is Redis. Redis is a popular key-value store; one of the commands it offers is the SLOWLOG command. This command lists recent queries that exceeded a particular time interval. Such information is quite useful in debugging your application's performance. Unfortunately, SLOWLOG is only available as a command on the Redis server, which means that it is difficult to use retrospectively if a problem happens when someone isn't available to debug the server. To fix this limitation, we can use fluentd and the adapter pattern to add slow-query logging to Redis.

Ứng dụng đầu tiên chúng ta sẽ giám sát là Redis. Redis là một kho khóa-giá trị phổ biến; một trong những lệnh nó cung cấp là lệnh SLOWLOG. Lệnh này liệt kê các truy vấn gần đây đã vượt quá một khoảng thời gian nhất định. Thông tin như vậy khá hữu ích trong việc gỡ lỗi hiệu năng ứng dụng của bạn. Đáng tiếc, SLOWLOG chỉ có sẵn dưới dạng một lệnh trên máy chủ Redis, nghĩa là rất khó dùng nó hồi cứu nếu sự cố xảy ra khi không có ai sẵn sàng để gỡ lỗi máy chủ. Để khắc phục hạn chế này, ta có thể dùng fluentd và mẫu adapter để bổ sung khả năng ghi log truy vấn chậm cho Redis.

To do this, we use the adapter pattern with a redis container as the main application container, and the fluentd container as our adapter container. In this case, we will also use the fluent-plugin-redis-slowlog fluentd plugin to listen to the slow queries. We can configure this plugin using the following snippet:

Để làm điều này, ta dùng mẫu adapter với một container redis làm container ứng dụng chính, và container fluentd làm container adapter của ta. Trong trường hợp này, ta cũng sẽ dùng plugin fluentd fluent-plugin-redis-slowlog để lắng nghe các truy vấn chậm. Ta có thể cấu hình plugin này bằng đoạn mã sau:

```
<source>
  type redis_slowlog
  host localhost
  port 6379
  tag redis.slowlog
</source>
```

Because we are using an adapter and the containers both share a network namespace, configuring the logging simply uses localhost and the default Redis port (6379). Given this application of the adapter pattern, logging will always be available whenever we want to debug slow Redis queries.

Vì ta đang dùng một adapter và cả hai container đều chia sẻ chung một network namespace, việc cấu hình ghi log chỉ đơn giản dùng localhost và cổng Redis mặc định (6379). Với cách áp dụng mẫu adapter này, khả năng ghi log sẽ luôn có sẵn bất cứ khi nào ta muốn gỡ lỗi các truy vấn Redis chậm.

A similar exercise can be done to monitor logs from the Apache Storm system. Again, Storm provides data via a RESTful API, which is useful but has limitations if we are not currently monitoring the system when a problem occurs. Like Redis, we can use a fluentd adapter to transfer the Storm process into a time series of queryable logs. To do this, we deploy a fluentd adapter with the fluent-plugin-storm plugin enabled. We can configure this plugin with a fluentd config pointed at localhost (because again, we are running as a container group with a shared localhost); the config for the plugin looks like:

Một bài tập tương tự có thể được thực hiện để giám sát log từ hệ thống Apache Storm. Một lần nữa, Storm cung cấp dữ liệu thông qua một API RESTful, điều này hữu ích nhưng có hạn chế nếu ta không đang giám sát hệ thống vào lúc sự cố xảy ra. Giống như Redis, ta có thể dùng một adapter fluentd để biến đổi tiến trình Storm thành một chuỗi thời gian gồm các log có thể truy vấn được. Để làm điều này, ta triển khai một adapter fluentd với plugin fluent-plugin-storm được bật. Ta có thể cấu hình plugin này bằng một cấu hình fluentd trỏ tới localhost (vì một lần nữa, ta đang chạy như một nhóm container chia sẻ chung localhost); cấu hình cho plugin trông như sau:

```
<source>
  type storm
  tag storm
  url http://localhost:8080
  window 600
  sys 0
</source>
```

Adding a Health Monitor — Thêm một bộ giám sát sức khỏe

One last example of applying the adapter pattern is derived from monitoring the health of an application container. Consider the task of monitoring the health of an off-the-shelf database container. In this case, the container for the database is supplied by the database project, and we would rather not modify that container simply to add health checks. Of course, a **container orchestrator** will allow us to add simple health checks to ensure that the process is running and that it is listening on a particular port, but what if we want to add richer health checks that actually run queries against the database?

Một ví dụ cuối cùng về việc áp dụng mẫu adapter bắt nguồn từ việc giám sát sức khỏe của một container ứng dụng. Hãy xét nhiệm vụ giám sát sức khỏe của một container cơ sở dữ liệu có sẵn (off-the-shelf). Trong trường hợp này, container cho cơ sở dữ liệu do dự án cơ sở dữ liệu cung cấp, và ta không muốn chỉnh sửa container đó chỉ để thêm các kiểm tra sức khỏe (health check). Tất nhiên, một bộ điều phối container sẽ cho phép ta thêm các kiểm tra sức khỏe đơn giản nhằm đảm bảo rằng tiến trình đang chạy và nó đang lắng nghe trên một cổng cụ thể, nhưng nếu ta muốn thêm các kiểm tra sức khỏe phong phú hơn, thực sự chạy các truy vấn vào cơ sở dữ liệu thì sao?

Container orchestration systems like Kubernetes enable us to use shell scripts as health checks as well. Given this capability, we can write a rich shell script that runs a number of different diagnostic queries against the database to determine its health. But where can we store such a script and how can we version it?

Các hệ thống điều phối container như Kubernetes cũng cho phép ta dùng các shell script làm kiểm tra sức khỏe. Với khả năng này, ta có thể viết một shell script phong phú chạy một số truy vấn chẩn đoán khác nhau vào cơ sở dữ liệu để xác định sức khỏe của nó. Nhưng ta có thể lưu một script như vậy ở đâu và làm thế nào để quản lý phiên bản của nó?

The answer to these problems should be easy to guess by now: we can use an adapter container. The database runs in the application container and shares a network interface with the adapter container. The adapter container is a simple container that only contains the shell script for determining the health of the database. This script can then be set up as the **health check** for the database container and can perform whatever rich health checks our application requires. If these checks ever fail, the database will be automatically restarted.

Câu trả lời cho những vấn đề này lẽ ra giờ đã dễ đoán: ta có thể dùng một container adapter. Cơ sở dữ liệu chạy trong container ứng dụng và chia sẻ chung một giao diện mạng với container adapter. Container adapter là một container đơn giản chỉ chứa shell script để xác định sức khỏe của cơ sở dữ liệu. Script này sau đó có thể được thiết lập làm kiểm tra sức khỏe cho container cơ sở dữ liệu và có thể thực hiện bất kỳ kiểm tra sức khỏe phong phú nào mà ứng dụng của ta yêu cầu. Nếu các kiểm tra này thất bại, cơ sở dữ liệu sẽ tự động được khởi động lại.

www.allitebooks.com

www.allitebooks.com

Hands On: Adding Rich Health Monitoring for MySQL — Thực hành (Hands On): Thêm giám sát sức khỏe phong phú cho MySQL

Suppose then that you want to add deep monitoring on a MySQL database where you actually run a query that was representative of your workload. In this case, one option would be to update the MySQL container to contain a health check that is specific to your application. However, this is generally an unattractive idea because it requires that you both modify some existing MySQL base image as well as update that image as new MySQL images are released.

Vậy giả sử rằng bạn muốn thêm giám sát sâu trên một cơ sở dữ liệu MySQL, nơi bạn thực sự chạy một truy vấn đại diện cho khối lượng công việc của mình. Trong trường hợp này, một lựa chọn sẽ là cập nhật container MySQL để chứa một kiểm tra sức khỏe đặc thù cho ứng dụng của bạn. Tuy nhiên, đây nói chung là một ý tưởng kém hấp dẫn vì nó đòi hỏi bạn vừa phải chỉnh sửa một số image MySQL nên có sẵn, vừa phải cập nhật image đó mỗi khi có các image MySQL mới được phát hành.

Using the adapter pattern is a much more attractive approach to adding health checks to your database container. Instead of modifying the existing MySQL container, you can add an additional adapter container to the pre-existing MySQL container, which runs the appropriate query to test the database health. Given that this adapter container implements the expected HTTP health check, it is simply a case of defining the MySQL database process's health check in terms of the interface exposed by this database adapter.

Dùng mẫu adapter là một cách tiếp cận hấp dẫn hơn nhiều để thêm các kiểm tra sức khỏe vào container cơ sở dữ liệu của bạn. Thay vì chỉnh sửa container MySQL hiện có, bạn có thể thêm một container adapter bổ sung vào container MySQL có sẵn, container này chạy truy vấn phù hợp để kiểm tra sức khỏe cơ sở dữ liệu. Vì container adapter này hiện thực kiểm tra sức khỏe HTTP được kỳ vọng, việc còn lại đơn giản chỉ là định nghĩa kiểm tra sức khỏe của tiến trình cơ sở dữ liệu MySQL theo giao diện mà adapter cơ sở dữ liệu này phơi ra.

The source code for this adapter is relatively straightforward and looks like this in Go (though clearly other language implementations are possible as well):

Mã nguồn cho adapter này tương đối đơn giản và trong Go trông như thế này (dù rõ ràng các bản hiện thực bằng ngôn ngữ khác cũng khả thi):

```
package main
import (
    "database/sql"
    "flag"
    "fmt"
    "net/http"
    _ "github.com/go-sql-driver/mysql"
)
var (
    user    = flag.String("user", "", "The database user name")
    passwd  = flag.String("password", "", "The database password")
    db      = flag.String("database", "", "The database to connect to")
    query   = flag.String("query", "", "The test query")
    addr    = flag.String("address", "localhost:8080",
        "The address to listen on")
)
```

```
// Basic usage:
// db-check --query="SELECT * from my-cool-table" \
//          --user=bdburns \
//          --passwd="you wish"
//
func main() {
flag.Parse()
db, err := sql.Open("localhost",
                    fmt.Sprintf("%s:%s@/%s", *user, *passwd, *db))
if err != nil {
fmt.Printf("Error opening database: %v", err)
}
// Simple web handler that runs the query
http.HandleFunc("/", func(res http.ResponseWriter, req *http.Request) {
_, err := db.Exec(*query)
if err != nil {
res.WriteHeader(http.StatusInternalServerError)
res.Write([]byte(err.Error()))
return
}
res.WriteHeader(http.StatusOK)
res.Write([]byte("OK"))
return
})
// Startup the server
http.ListenAndServe(*addr, nil)
}
```

We can then build this into a **container image** and pull it into a pod that looks like:

Sau đó ta có thể dựng nó thành một image container và kéo nó vào một Pod trông như thế này:

```
apiVersion: v1
kind: Pod
metadata:
  name: adapter-example-health
  namespace: default
spec:
  containers:
  - image: mysql
    name: mysql
  - image: brendanburns/mysql-adapter
    name: adapter
```

That way, the mysql container is unchanged, but the desired feedback about the health of the mysql server can still be obtained from the adapter container.

Bằng cách đó, container mysql không bị thay đổi, nhưng phản hồi mong muốn về sức khỏe của máy chủ mysql vẫn có thể lấy được từ container adapter.

When looking at this application of the adapter pattern, it may seem like applying the pattern is superfluous. Clearly we could have built our own custom image that knew how to health check the

mysql instance itself.

Khi nhìn vào cách áp dụng mẫu adapter này, có vẻ như việc áp dụng mẫu là thừa thãi. Rõ ràng ta có thể đã tự dựng image tùy chỉnh của riêng mình biết cách tự kiểm tra sức khỏe của thực thể mysql.

While this is true, this method ignores the strong benefits that derive from modularity. If every developer implements their own specific container with health checking built in, there are no opportunities for reuse or sharing.

Dù điều này đúng, cách làm đó bỏ qua những lợi ích mạnh mẽ bắt nguồn từ tính mô-đun. Nếu mọi nhà phát triển đều hiện thực container cụ thể của riêng mình với kiểm tra sức khỏe tích hợp sẵn, thì không có cơ hội nào cho việc tái sử dụng hay chia sẻ.

In contrast, if we use patterns like the adapter to develop modular solutions comprised of multiple containers, the work is inherently decoupled and more easily shared. An adapter that is developed to health check mysql is a module that can be shared and reused by a variety of people. Further, people can apply the adapter pattern using this shared health-checking container, without having deep knowledge of how to health check a mysql database. Thus the modularity and adapter pattern serve not to just facilitate sharing, but also to empower people to take advantage of the knowledge of others.

Ngược lại, nếu ta dùng các mẫu như adapter để phát triển những giải pháp mô-đun gồm nhiều container, công việc về bản chất được tách rời và dễ chia sẻ hơn. Một adapter được phát triển để kiểm tra sức khỏe mysql là một mô-đun có thể được nhiều người chia sẻ và tái sử dụng. Hơn nữa, mọi người có thể áp dụng mẫu adapter bằng cách dùng container kiểm tra sức khỏe được chia sẻ này, mà không cần kiến thức sâu về cách kiểm tra sức khỏe một cơ sở dữ liệu mysql. Như vậy, tính mô-đun và mẫu adapter không chỉ phục vụ việc tạo điều kiện cho chia sẻ, mà còn trao quyền cho mọi người tận dụng kiến thức của người khác.

Sometimes design patterns aren't just for the developers who apply them, but lead to the development of communities that can collaborate and share solutions between members of the community as well as the broader developer ecosystem.

Đôi khi các mẫu thiết kế không chỉ dành cho những nhà phát triển áp dụng chúng, mà còn dẫn đến sự hình thành các cộng đồng có thể cộng tác và chia sẻ giải pháp giữa các thành viên trong cộng đồng cũng như với hệ sinh thái nhà phát triển rộng lớn hơn.

PART II Serving Patterns — PART II Các mẫu phục vụ

The previous chapter described patterns for grouping collections of containers that are scheduled on the same machine. These groups are tightly coupled, symbiotic systems. They depend on local, shared resources like disk, network **interface**, or inter-process communications. Such collections of containers are important patterns, but they are also building blocks for larger systems. **Reliability**, **scalability**, and separation of concerns dictate that real-world systems are built out of many different components, spread across multiple machines. In contrast to single-**node** patterns, the multi-node distributed patterns are more loosely coupled. While the patterns dictate patterns of communication between the components, this communication is based on network calls. Furthermore, many calls are issued in parallel, and systems coordinate via loose synchronization rather than tight constraints.

Chương trước đã mô tả các mẫu để nhóm những tập hợp container được lập lịch trên cùng một máy. Các nhóm này là những hệ thống cộng sinh, gắn kết chặt chẽ. Chúng phụ thuộc

vào các tài nguyên cục bộ, dùng chung như đĩa, giao diện mạng hay các phương tiện giao tiếp liên tiến trình. Những tập hợp container như vậy là các mẫu quan trọng, nhưng đồng thời cũng là các khối xây dựng cho những hệ thống lớn hơn. Độ tin cậy, khả năng mở rộng và sự tách bạch các mối quan tâm đòi hỏi rằng các hệ thống thực tế phải được xây dựng từ nhiều thành phần khác nhau, trải rộng trên nhiều máy. Trái ngược với các mẫu đơn nút (single-node), các mẫu phân tán đa nút (multi-node) gắn kết lỏng lẻo hơn. Mặc dù các mẫu này quy định cách thức giao tiếp giữa các thành phần, giao tiếp đó dựa trên những lời gọi mạng. Hơn nữa, nhiều lời gọi được phát đi song song, và các hệ thống phối hợp với nhau thông qua đồng bộ lỏng lẻo thay vì những ràng buộc chặt chẽ.

Introduction to Microservices — Giới thiệu về Microservices

Recently, the term microservices has become a buzzword for describing multi-node distributed software architectures. Microservices describe a system built out of many different components running in different processes and communicating over defined APIs. Microservices stand in contrast to monolithic systems, which tend to place all of the functionality for a **service** within a single, tightly coordinated application. These two different architectural approaches are shown in Figures II-1 and II-2 .

Gần đây, thuật ngữ microservices đã trở thành một từ thời thượng để mô tả các kiến trúc phần mềm phân tán đa nút. Microservices mô tả một hệ thống được xây dựng từ nhiều thành phần khác nhau chạy trên các tiến trình khác nhau và giao tiếp qua các API đã được định nghĩa. Microservices đối lập với các hệ thống nguyên khối (monolithic), vốn có xu hướng đặt toàn bộ chức năng của một dịch vụ vào trong một ứng dụng đơn nhất, phối hợp chặt chẽ. Hai cách tiếp cận kiến trúc khác nhau này được minh họa ở Hình II-1 và II-2.

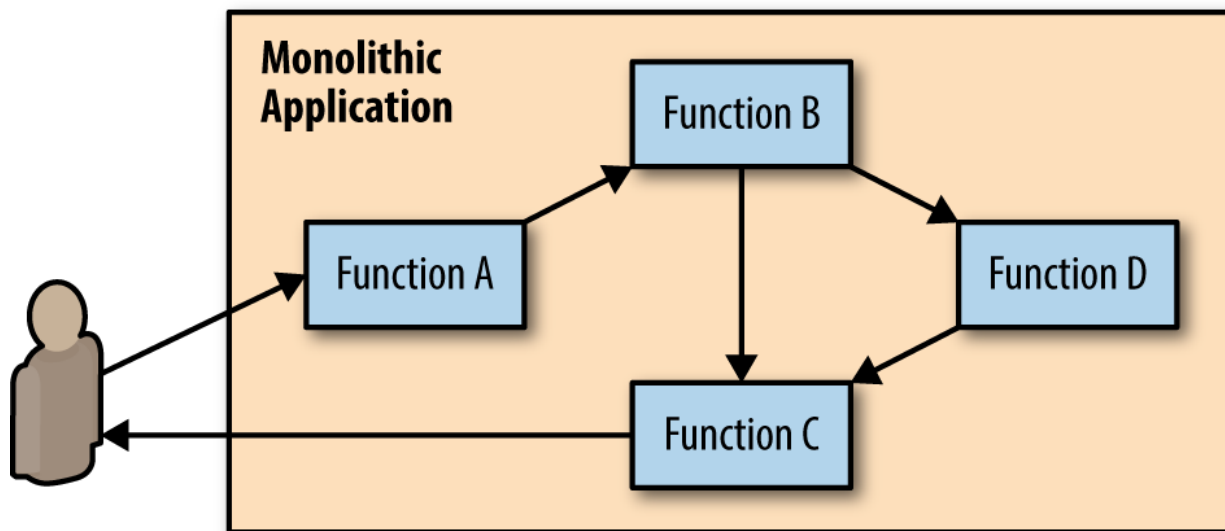


Figure II-1. A monolithic service with all functions in a single **container**

Hình II-1. Một dịch vụ nguyên khối với tất cả chức năng nằm trong một container duy nhất

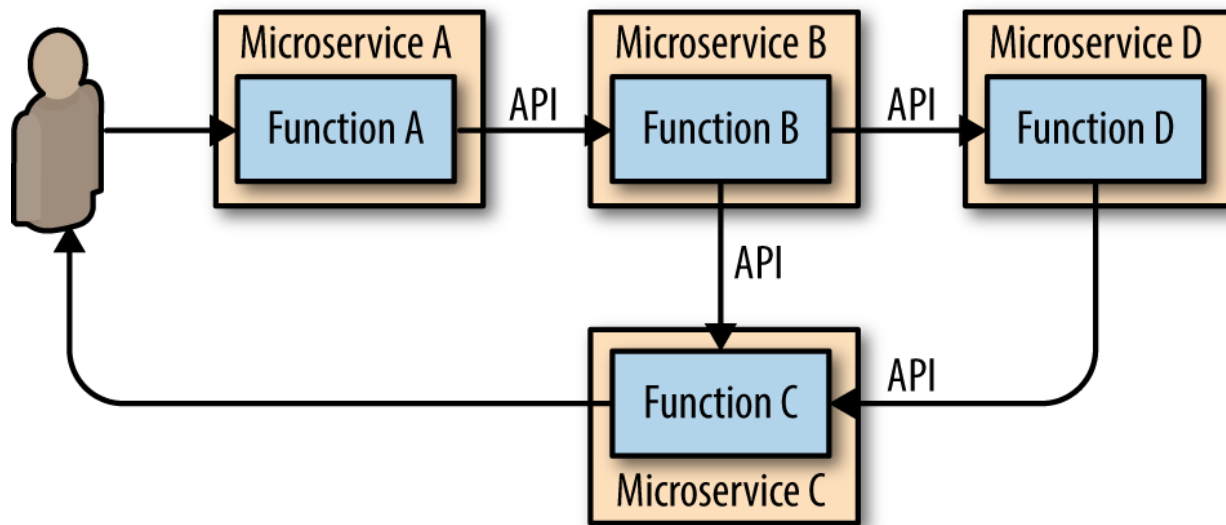


Figure II-2. A **microservice** architecture with each **function** broken out as a separate microservice

Hình II-2. Một kiến trúc microservice với mỗi chức năng được tách ra thành một microservice riêng

There are numerous benefits to the microservices approach, most of them are centered around reliability and agility. Microservices break down an application into small pieces, each focused on providing a single service. This reduced scope enables each service to be built and maintained by a single “two pizza” team. Reduced team size also reduces the overhead associated with keeping a team focused and moving in one direction.

Có rất nhiều lợi ích từ cách tiếp cận microservices, hầu hết tập trung vào độ tin cậy và sự linh hoạt. Microservices chia nhỏ một ứng dụng thành những phần nhỏ, mỗi phần tập trung cung cấp một dịch vụ duy nhất. Phạm vi thu hẹp này cho phép mỗi dịch vụ được xây dựng và bảo trì bởi một nhóm “hai chiếc pizza” duy nhất. Quy mô nhóm nhỏ hơn cũng làm giảm chi phí phụ trợ liên quan đến việc giữ cho một nhóm tập trung và cùng tiến theo một hướng.

Additionally, the introduction of formal APIs in between different microservices decouples the teams from one another and provides a reliable contract between the different services. This formal contract reduces the need for tight synchronization among the teams because the team providing the **API** understands the surface area that it needs to keep stable, and the team consuming the API can rely on a stable service without worrying about its details. This decoupling enables teams to independently manage their code and release schedules, which in turn improves each team’s ability to iterate and improve their code.

Thêm vào đó, việc đưa ra các API chính thức giữa các microservice khác nhau giúp tách rời các nhóm khỏi nhau và cung cấp một giao kèo đáng tin cậy giữa các dịch vụ khác nhau. Giao kèo chính thức này làm giảm nhu cầu đồng bộ chặt chẽ giữa các nhóm, bởi vì nhóm cung cấp API hiểu rõ phần bề mặt mà họ cần giữ ổn định, còn nhóm sử dụng API có thể dựa vào một dịch vụ ổn định mà không phải lo lắng về các chi tiết của nó. Sự tách rời này cho phép các nhóm quản lý mã nguồn và lịch phát hành của mình một cách độc lập, từ đó cải thiện khả năng lặp đi lặp lại và hoàn thiện mã nguồn của mỗi nhóm.

Finally, the decoupling of microservices enables better scaling. Because each component has been broken out into its own service, it can be scaled independently. It is rare for each service within

a larger application to grow at the same rate, or have the same way of scaling. Some systems are **stateless** and can simply scale horizontally, whereas other systems maintain state and require sharding or other approaches to scale. By separating each service out, each service can use the approach to scaling that suits it best. This is not possible when all services are part of a single monolith.

Cuối cùng, sự tách rời của microservices cho phép mở rộng tốt hơn. Vì mỗi thành phần đã được tách ra thành dịch vụ riêng, nó có thể được mở rộng độc lập. Hiếm khi mỗi dịch vụ trong một ứng dụng lớn lại tăng trưởng cùng một tốc độ, hay có cùng cách mở rộng. Một số hệ thống là không trạng thái và có thể đơn giản mở rộng theo chiều ngang, trong khi các hệ thống khác duy trì trạng thái và đòi hỏi phân mảnh (sharding) hoặc những cách tiếp cận khác để mở rộng. Bằng cách tách riêng từng dịch vụ, mỗi dịch vụ có thể dùng cách mở rộng phù hợp nhất với nó. Điều này là không thể khi tất cả các dịch vụ là một phần của cùng một khối nguyên khối.

But of course there are downsides to the microservices approach to system design as well. The two foremost disadvantages are that because the system has become more loosely coupled, debugging the system when failures occur is significantly more difficult. You can no longer simply load a single application into a debugger and determine what went wrong. Any errors are the byproducts of a large number of systems often running on different machines. This environment is quite challenging to reproduce in a debugger. As a corollary, microservices-based systems are also difficult to design and architect. A microservices-based system uses multiple methods of communicating between services; different patterns (e.g., synchronous, asynchronous, message-passing, etc.); and multiple different patterns of coordination and control among the services.

Nhưng tất nhiên cũng có những nhược điểm đối với cách tiếp cận microservices trong thiết kế hệ thống. Hai bất lợi hàng đầu là vì hệ thống đã trở nên gắn kết lỏng lẻo hơn, việc gỡ lỗi hệ thống khi xảy ra sự cố trở nên khó khăn hơn đáng kể. Bạn không còn có thể đơn giản nạp một ứng dụng duy nhất vào trình gỡ lỗi và xác định điều gì đã sai. Bất kỳ lỗi nào cũng là sản phẩm phụ của một số lượng lớn hệ thống thường chạy trên các máy khác nhau. Môi trường này khá khó tái tạo trong một trình gỡ lỗi. Như một hệ quả, các hệ thống dựa trên microservices cũng khó thiết kế và kiến trúc. Một hệ thống dựa trên microservices sử dụng nhiều phương thức giao tiếp giữa các dịch vụ; các mẫu khác nhau (ví dụ: đồng bộ, bất đồng bộ, truyền thông điệp, v.v.); và nhiều mẫu phối hợp và điều khiển khác nhau giữa các dịch vụ.

These challenges are the motivation for distributed patterns. If a microservices architecture is made up of well-known patterns, then it is easier to design because many of the design practices are specified by the patterns. Additionally, patterns make the systems easier to debug because they enable developers to apply lessons learned across a number of different systems that use the same patterns.

Những thách thức này chính là động lực cho các mẫu phân tán. Nếu một kiến trúc microservices được tạo nên từ những mẫu quen thuộc, thì nó sẽ dễ thiết kế hơn vì nhiều thực hành thiết kế đã được các mẫu quy định sẵn. Thêm vào đó, các mẫu giúp hệ thống dễ gỡ lỗi hơn vì chúng cho phép các nhà phát triển áp dụng những bài học rút ra từ nhiều hệ thống khác nhau cùng sử dụng các mẫu đó.

With that in mind, this section introduces a number of multi-node patterns for building distributed systems. These patterns are not mutually exclusive. Any real-world system will be built from a collection of these patterns working together to produce a single higher-level application.

Với suy nghĩ đó, phần này giới thiệu một số mẫu đa nút để xây dựng các hệ phân tán. Các mẫu này không loại trừ lẫn nhau. Bất kỳ hệ thống thực tế nào cũng sẽ được xây dựng từ

một tập hợp các mẫu này phối hợp với nhau để tạo ra một ứng dụng cấp cao duy nhất.

CHAPTER 5 Replicated Load-Balanced Services — CHAPTER 5 Dịch vụ nhân bản được cân bằng tải

The simplest distributed pattern, and one that most are familiar with, is a replicated load-balanced service. In such a service, every server is identical to every other server and all are capable of supporting traffic. The pattern consists of a scalable number of servers with a **load balancer** in front of them. The load balancer is typically either completely round-robin or uses some form of session stickiness. The chapter will give a concrete example of how to deploy such a service in **Kubernetes**.

Mẫu phân tán đơn giản nhất, và là mẫu mà hầu hết mọi người quen thuộc, là dịch vụ nhân bản được cân bằng tải (replicated load-balanced service). Trong một dịch vụ như vậy, mọi máy chủ đều giống hệt nhau và tất cả đều có khả năng phục vụ lưu lượng. Mẫu này bao gồm một số lượng máy chủ có thể mở rộng được với một bộ cân bằng tải đặt phía trước chúng. Bộ cân bằng tải thường hoặc hoàn toàn xoay vòng (round-robin) hoặc sử dụng một dạng phiên dính (session stickiness) nào đó. Chương này sẽ đưa ra một ví dụ cụ thể về cách triển khai một dịch vụ như vậy trong Kubernetes.

Stateless Services — Dịch vụ không trạng thái

Stateless services are ones that don't require saved state to operate correctly. In the simplest stateless applications, even individual requests may be routed to separate instances of the service (see Figure 5-1). Examples of stateless services include things like static content servers and complex middleware systems that receive and aggregate responses from numerous different backend systems.

Dịch vụ không trạng thái là những dịch vụ không cần trạng thái được lưu lại để hoạt động đúng đắn. Trong những ứng dụng không trạng thái đơn giản nhất, ngay cả các yêu cầu riêng lẻ cũng có thể được định tuyến tới các thực thể (instance) riêng biệt của dịch vụ (xem Hình 5-1). Ví dụ về các dịch vụ không trạng thái bao gồm những thứ như máy chủ nội dung tĩnh và các hệ thống middleware phức tạp tiếp nhận và tổng hợp phản hồi từ nhiều hệ thống backend khác nhau.

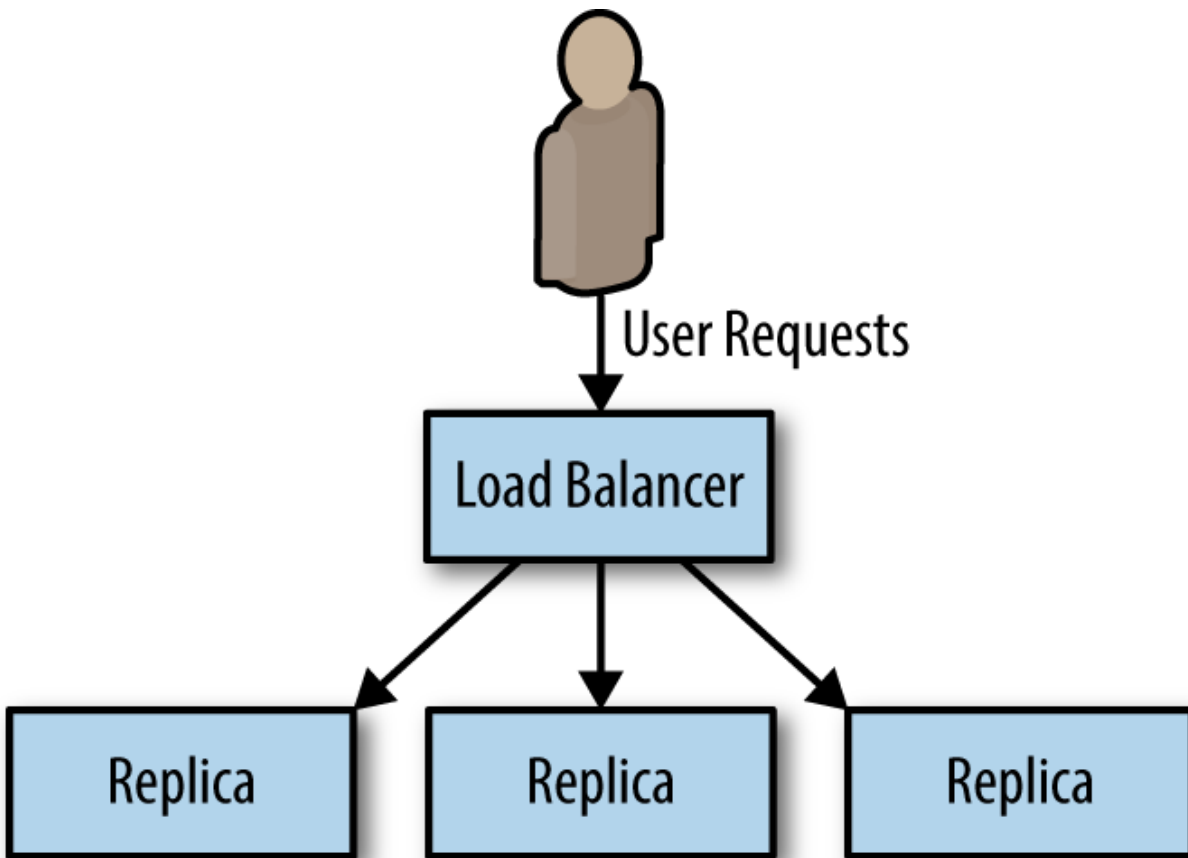


Figure 5-1. Basic replicated stateless service

Hình 5-1. Dịch vụ không trạng thái nhân bản cơ bản

45 Stateless systems are replicated to provide redundancy and scale. No matter how small your service is, you need at least two replicas to provide a service with a “highly available” service level agreement (**SLA**). To understand why this is true, consider trying to deliver a **three-nines (99.9% availability)**. In a three-nines service, you get 1.4 minutes of downtime per day ($24 \times 60 \times 0.001$). Assuming that you have a service that never crashes, that still means you need to be able to do a software upgrade in less than 1.4 minutes in order to hit your SLA with a single instance. And that’s assuming that you do daily software rollouts. If your team is really embracing **continuous delivery** and you’re pushing a new version of software every hour, you need to be able to do a software **rollout** in 3.6 seconds to achieve your 99.9% uptime SLA with a single instance. Any longer than that and you will have more than 0.01% downtime from those 3.6 seconds.

45 Các hệ thống không trạng thái được nhân bản để cung cấp sự dư thừa và khả năng mở rộng. Bất kể dịch vụ của bạn nhỏ đến đâu, bạn cần ít nhất hai bản sao để cung cấp một dịch vụ với thỏa thuận mức dịch vụ (SLA) “tính sẵn sàng cao”. Để hiểu tại sao điều này đúng, hãy thử hình dung việc cung cấp một dịch vụ ba số chín (99.9% tính sẵn sàng). Trong một dịch vụ ba số chín, bạn có 1.4 phút thời gian ngừng hoạt động mỗi ngày ($24 \times 60 \times 0.001$). Giả sử bạn có một dịch vụ không bao giờ sập, điều đó vẫn có nghĩa là bạn cần có khả năng thực hiện một bản nâng cấp phần mềm trong chưa đầy 1.4 phút để đạt được SLA của mình với một thực thể duy nhất. Và đó là giả sử bạn thực hiện việc tung phần mềm hằng ngày. Nếu nhóm của bạn thực sự áp dụng phân phối liên tục (continuous delivery) và bạn đẩy một phiên bản phần mềm mới mỗi giờ, bạn cần có khả năng thực hiện việc tung phần

mềm trong 3.6 giây để đạt được SLA 99.9% thời gian hoạt động với một thực thể duy nhất. Lâu hơn thế thì bạn sẽ có hơn 0.01% thời gian ngừng hoạt động từ 3.6 giây đó.

Of course, instead of all of that work, you could just have two replicas of your service with a load balancer in front of them. That way, while you are doing a rollout, or in the—unlikely, I’m sure—**event** that your software crashes, your users will be served by the other **replica** of the service and never know anything was going on.

Tất nhiên, thay vì tất cả công việc đó, bạn chỉ cần có hai bản sao của dịch vụ với một bộ cân bằng tải đặt phía trước chúng. Bằng cách đó, trong khi bạn đang tung một bản phát hành, hoặc trong trường hợp — chắc chắn là khó xảy ra rồi — phần mềm của bạn sập, người dùng của bạn sẽ được phục vụ bởi bản sao kia của dịch vụ và không bao giờ biết là có chuyện gì đang xảy ra.

As services grow larger, they are also replicated to support additional users. Horizontally scalable systems handle more and more users by adding more replicas; see Figure 5-2 . They achieve this with the load-balanced replicated serving pattern.

Khi các dịch vụ phát triển lớn hơn, chúng cũng được nhân bản để hỗ trợ thêm người dùng. Các hệ thống có khả năng mở rộng theo chiều ngang xử lý ngày càng nhiều người dùng bằng cách thêm nhiều bản sao hơn; xem Hình 5-2. Chúng đạt được điều này bằng mẫu phục vụ nhân bản được cân bằng tải.

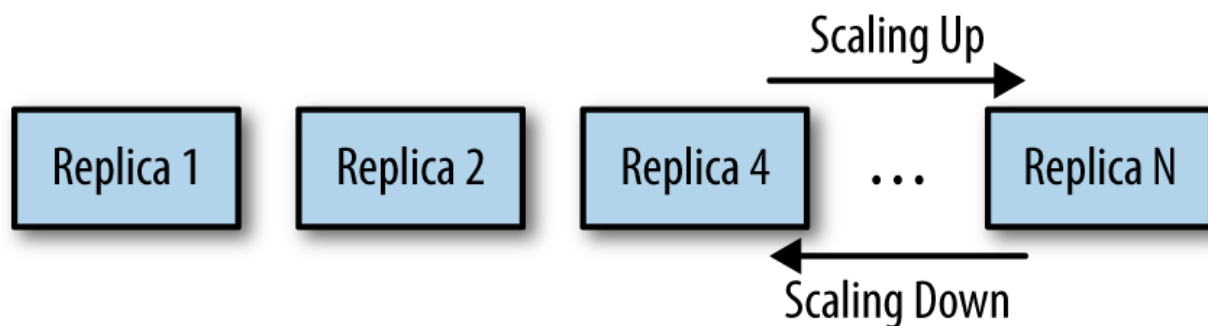


Figure 5-2. **Horizontal scaling** of a replicated stateless application

Hình 5-2. Mở rộng theo chiều ngang của một ứng dụng không trạng thái nhân bản

Readiness Probes for Load Balancing — Đầu dò sẵn sàng cho việc cân bằng tải

Of course, simply replicating your service and adding a load balancer is only part of a complete pattern for stateless replicated serving. When designing a **replicated service**, it is equally important to build and deploy a **readiness probe** to inform the load balancer. We have discussed how health probes can be used by a container orchestration system to determine when an application needs to be restarted. In contrast, a readiness probe determines when an application is ready to serve user requests. The reason for the differentiation is that many applications require some time to become initialized before they are ready to serve. They may need to connect to databases, load plugins, or download serving files from the network. In all of these cases, the containers are alive , but they are not ready . When building an application for a replicated service pattern, be sure to include a special URL that implements this readiness check.

Tất nhiên, chỉ đơn giản nhân bản dịch vụ và thêm một bộ cân bằng tải mới chỉ là một phần

của mẫu hoàn chỉnh cho việc phục vụ nhân bản không trạng thái. Khi thiết kế một dịch vụ nhân bản, việc xây dựng và triển khai một đầu dò sẵn sàng (readiness probe) để thông báo cho bộ cân bằng tải cũng quan trọng không kém. Chúng ta đã thảo luận về cách các đầu dò sức khỏe có thể được một hệ thống điều phối container sử dụng để xác định khi nào một ứng dụng cần được khởi động lại. Trái lại, một đầu dò sẵn sàng xác định khi nào một ứng dụng đã sẵn sàng phục vụ các yêu cầu của người dùng. Lý do cho sự phân biệt này là nhiều ứng dụng cần một khoảng thời gian để khởi tạo trước khi sẵn sàng phục vụ. Chúng có thể cần kết nối tới cơ sở dữ liệu, nạp các plugin, hoặc tải các tệp phục vụ từ mạng về. Trong tất cả những trường hợp này, các container đang sống, nhưng chúng chưa sẵn sàng. Khi xây dựng một ứng dụng cho mẫu dịch vụ nhân bản, hãy chắc chắn bao gồm một URL đặc biệt triển khai việc kiểm tra sẵn sàng này.

Hands On: Creating a Replicated Service in Kubernetes — Thực hành (Hands On): Tạo một dịch vụ nhân bản trong Kubernetes

The instructions below give a concrete example of how to deploy a stateless, replicated service behind a load balancer. These directions use the Kubernetes **container orchestrator**, but the pattern can be implemented on top of a number of different container orchestrators.

Các hướng dẫn dưới đây đưa ra một ví dụ cụ thể về cách triển khai một dịch vụ nhân bản, không trạng thái đặt phía sau một bộ cân bằng tải. Các hướng dẫn này sử dụng bộ điều phối container Kubernetes, nhưng mẫu này có thể được triển khai trên nhiều bộ điều phối container khác nhau.

To begin with, we will create a small NodeJS application that serves definitions of words from the dictionary.

Để bắt đầu, chúng ta sẽ tạo một ứng dụng NodeJS nhỏ phục vụ định nghĩa của các từ trong từ điển.

To try this service out, you can run it using a **container image**:

Để thử nghiệm dịch vụ này, bạn có thể chạy nó bằng một image container:

```
docker run -p 8080:8080 brendanburns/dictionary-server
```

This runs a simple dictionary server on your local machine. For example, you can visit <http://localhost:8080/dog> to see the definition for dog .

Lệnh này chạy một máy chủ từ điển đơn giản trên máy cục bộ của bạn. Ví dụ, bạn có thể truy cập <http://localhost:8080/dog> để xem định nghĩa của từ dog.

If you look at the logs for the container, you'll see that it starts serving immediately but only reports readiness after the dictionary (which is approximately 8 MB) has been downloaded over the network.

Nếu bạn nhìn vào nhật ký (log) của container, bạn sẽ thấy nó bắt đầu phục vụ ngay lập tức nhưng chỉ báo cáo trạng thái sẵn sàng sau khi từ điển (khoảng 8 MB) đã được tải xuống qua mạng.

To deploy this in Kubernetes, you create a **Deployment** :

Để triển khai cái này trong Kubernetes, bạn tạo một Deployment:

```
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
```

```

name: dictionary-server
spec:
  replicas: 3
  template:
    metadata:
      labels:
        app: dictionary-server
    spec:
      containers:
      - name: server
        image: brendanburns/dictionary-server
        ports:
        - containerPort: 8080
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 5

```

You can create this replicated, stateless service with:

Bạn có thể tạo dịch vụ nhân bản, không trạng thái này với:

```
kubectl create -f dictionary-deploy.yaml
```

Now that you have a number of replicas, you need a load balancer to bring requests to your replicas. The load balancer serves to distribute the load as well as to provide an abstraction to separate the replicated service from the consumers of the service. The load balancer also provides a resolvable name that is independent of any of the specific replicas.

Giờ khi bạn đã có một số bản sao, bạn cần một bộ cân bằng tải để đưa các yêu cầu tới các bản sao của bạn. Bộ cân bằng tải phục vụ việc phân phối tải cũng như cung cấp một lớp trừu tượng để tách dịch vụ nhân bản khỏi những bên tiêu dùng dịch vụ. Bộ cân bằng tải cũng cung cấp một tên có thể phân giải được, độc lập với bất kỳ bản sao cụ thể nào.

With Kubernetes, you can create this load balancer with a Service object:

Với Kubernetes, bạn có thể tạo bộ cân bằng tải này bằng một đối tượng Service:

```

kind: Service
apiVersion: v1
metadata:
  name: dictionary-server-service
spec:
  selector:
    app: dictionary-server
  ports:
  - protocol: TCP
    port: 8080
    targetPort: 8080

```

Once you have the **configuration** file, you can create the dictionary service with:

Một khi bạn đã có tệp cấu hình, bạn có thể tạo dịch vụ từ điển với:

```
kubectl create -f dictionary-service.yaml
```

Session Tracked Services — Dịch vụ theo dõi phiên

The previous examples of the stateless replicated pattern routed requests from all users to all replicas of a service. While this ensures an even distribution of load and fault tolerance, it is not always the preferred solution. Often there are reasons for wanting to ensure that a particular user's requests always end up on the same machine. Sometimes this is because you are caching that user's data in memory, so landing on the same machine ensures a higher **cache hit** rate. Sometimes it is because the interaction is long-running in nature, so some amount of state is maintained between requests. Regardless of the reason, an adaption of the stateless replicated service pattern is to use session tracked services, which ensure that all requests for a single user **map** to the same replica, as illustrated in Figure 5-3 .

Các ví dụ trước về mẫu nhân bản không trạng thái định tuyến các yêu cầu từ tất cả người dùng tới tất cả các bản sao của một dịch vụ. Mặc dù điều này đảm bảo phân phối tải đồng đều và khả năng chịu lỗi, nó không phải lúc nào cũng là giải pháp được ưa chuộng. Thường có những lý do để muốn đảm bảo rằng các yêu cầu của một người dùng cụ thể luôn kết thúc trên cùng một máy. Đôi khi điều này là vì bạn đang lưu đệm (cache) dữ liệu của người dùng đó trong bộ nhớ, nên việc rơi vào cùng một máy đảm bảo tỷ lệ trúng cache cao hơn. Đôi khi là vì tương tác có bản chất chạy dài, nên một lượng trạng thái nào đó được duy trì giữa các yêu cầu. Bất kể lý do là gì, một biến thể của mẫu dịch vụ nhân bản không trạng thái là sử dụng dịch vụ theo dõi phiên, đảm bảo rằng tất cả yêu cầu của một người dùng đều ánh xạ tới cùng một bản sao, như được minh họa ở Hình 5-3.

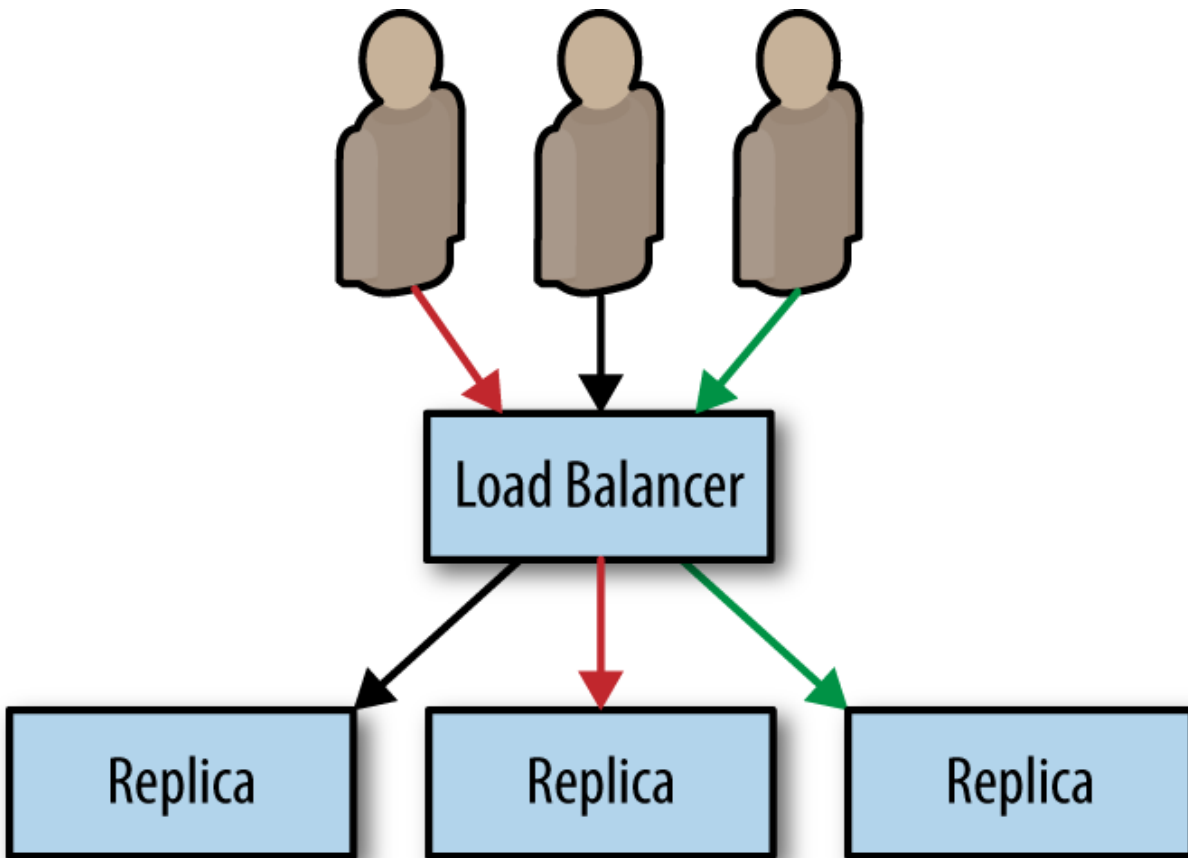
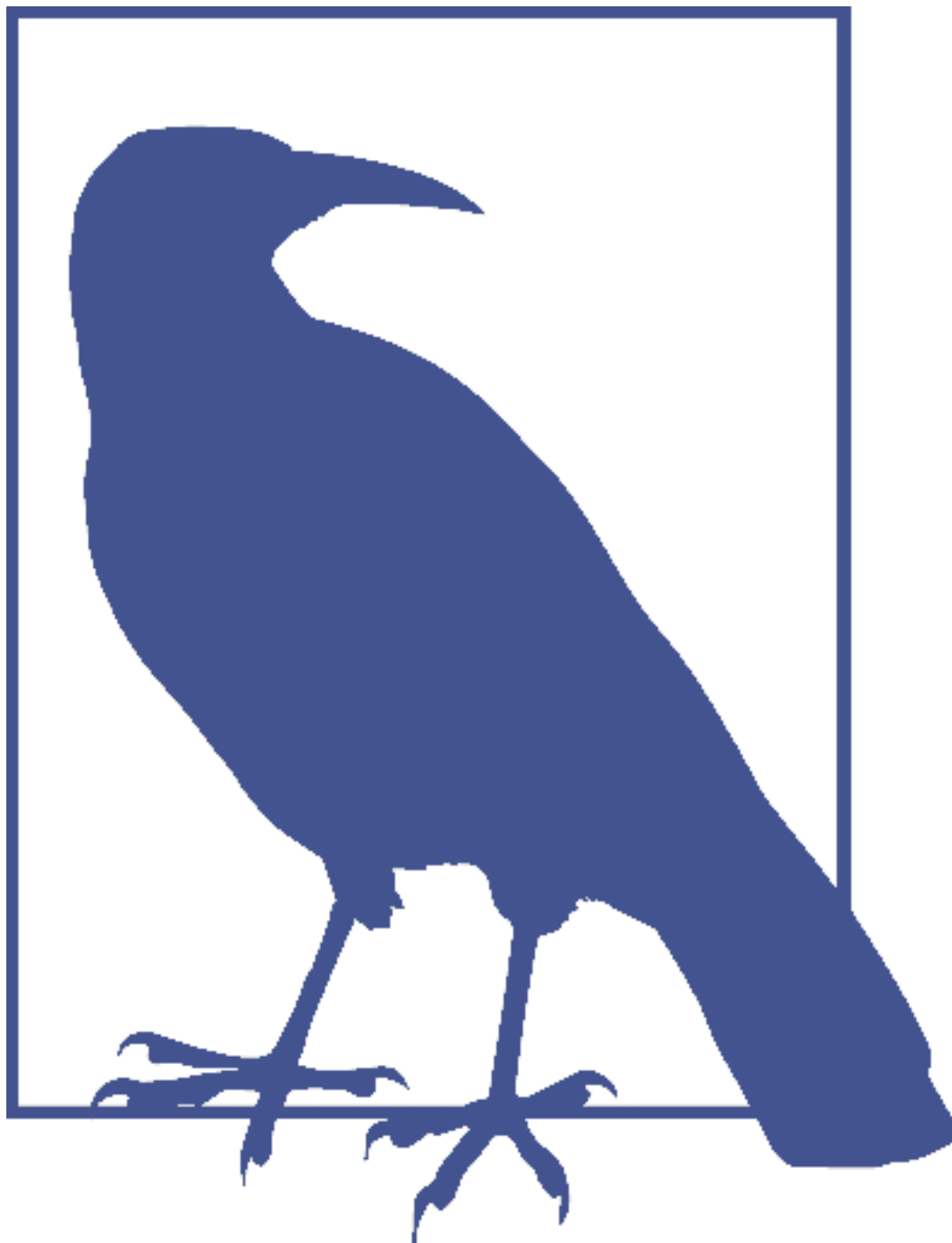


Figure 5-3. A session tracked service where all requests for a specific user are routed to a single instance. Generally speaking, this **session tracking** is performed by hashing the source and destination IP addresses and using that key to identify the server that should service the requests. So long as the source and destination IP addresses remain constant, all requests are sent to the same replica.

Hình 5-3. Một dịch vụ theo dõi phiên trong đó tất cả yêu cầu của một người dùng cụ thể được định tuyến tới một thực thể duy nhất. Nói chung, việc theo dõi phiên này được thực hiện bằng cách băm địa chỉ IP nguồn và đích và dùng khóa đó để xác định máy chủ nào sẽ phục vụ các yêu cầu. Chừng nào địa chỉ IP nguồn và đích còn không đổi, tất cả yêu cầu đều được gửi tới cùng một bản sao.



IP-based session tracking works within a cluster (internal IPs) but generally doesn't work well with external IP addresses because of network address translation (NAT). For external session tracking, application-level tracking (e.g., via cookies) is preferred.

Việc theo dõi phiên dựa trên IP hoạt động được trong phạm vi một cụm (IP nội bộ) nhưng nói chung không hoạt động tốt với các địa chỉ IP bên ngoài vì biên dịch địa chỉ mạng (NAT). Đối với việc theo dõi phiên bên ngoài, việc theo dõi ở mức ứng dụng (ví dụ: qua cookie) được ưa chuộng hơn.

Often, session tracking is accomplished via a **consistent hashing** function . The benefit of a consistent hashing function becomes evident when the service is scaled up or down. Obviously,

when the number of replicas changes, the mapping of a particular user to a replica may change. Consistent hashing functions minimize the number of users that actually change which replica they are mapped to, reducing the impact of scaling on your application.

Thường thì việc theo dõi phiên được thực hiện thông qua một hàm băm nhất quán (consistent hashing). Lợi ích của hàm băm nhất quán trở nên rõ ràng khi dịch vụ được mở rộng lên hoặc thu hẹp xuống. Rõ ràng, khi số lượng bản sao thay đổi, việc ánh xạ một người dùng cụ thể tới một bản sao có thể thay đổi. Các hàm băm nhất quán giảm thiểu số lượng người dùng thực sự thay đổi bản sao mà họ được ánh xạ tới, làm giảm tác động của việc mở rộng lên ứng dụng của bạn.

Application-Layer Replicated Services — Dịch vụ nhân bản ở tầng ứng dụng

In all of the preceding examples, the replication and **load balancing** takes place in the network layer of the service. The load balancing is independent of the actual protocol that is being spoken over the network, beyond TCP/IP. However, many applications use HTTP as the protocol for speaking with each other, and knowledge of the application protocol that is being spoken enables further refinements to the replicated stateless serving pattern for additional functionality.

Trong tất cả các ví dụ trước, việc nhân bản và cân bằng tải diễn ra ở tầng mạng của dịch vụ. Việc cân bằng tải độc lập với giao thức thực sự đang được nói trên mạng, ngoài TCP/IP ra. Tuy nhiên, nhiều ứng dụng dùng HTTP làm giao thức để giao tiếp với nhau, và sự hiểu biết về giao thức ứng dụng đang được sử dụng cho phép tinh chỉnh thêm mẫu phục vụ nhân bản không trạng thái để có thêm tính năng.

Introducing a Caching Layer — Giới thiệu một tầng đệm (cache)

Sometimes the code in your stateless service is still expensive despite being stateless. It might make queries to a database to service requests or do a significant amount of rendering or data mixing to service the request. In such a world, a **caching layer** can make a great deal of sense. A cache exists between your stateless application and the end-user request. The simplest form of caching for web applications is a **caching web proxy**. The **caching proxy** is simply an HTTP server that maintains user requests in memory state. If two users request the same web page, only one request will go to your backend; the other will be serviced out of memory in the cache. This is illustrated in Figure 5-4.

Đôi khi mã trong dịch vụ không trạng thái của bạn vẫn tốn kém dù không trạng thái. Nó có thể thực hiện các truy vấn tới cơ sở dữ liệu để phục vụ các yêu cầu hoặc thực hiện một lượng đáng kể việc kết xuất (rendering) hay trộn dữ liệu để phục vụ yêu cầu. Trong một thế giới như vậy, một tầng đệm (cache) có thể rất hợp lý. Một cache tồn tại giữa ứng dụng không trạng thái của bạn và yêu cầu của người dùng cuối. Dạng đệm đơn giản nhất cho các ứng dụng web là một caching web proxy. Caching proxy đơn giản là một máy chủ HTTP duy trì các yêu cầu của người dùng trong trạng thái bộ nhớ. Nếu hai người dùng yêu cầu cùng một trang web, chỉ một yêu cầu sẽ đi tới backend của bạn; yêu cầu kia sẽ được phục vụ từ bộ nhớ trong cache. Điều này được minh họa ở Hình 5-4.

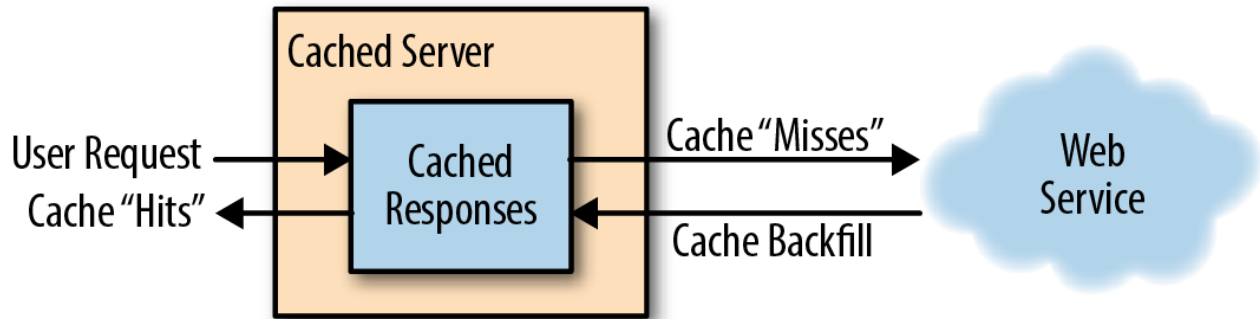


Figure 5-4. The operation of a cache server

Hình 5-4. Hoạt động của một máy chủ cache

For our purposes, we will use **Varnish**, an **open source** web cache.

Cho mục đích của chúng ta, chúng ta sẽ dùng Varnish, một bộ đệm web (web cache) mã nguồn mở.

Deploying Your Cache — Triển khai cache của bạn

The simplest way to deploy the web cache is alongside each instance of your web server using the **sidecar** pattern (see Figure 5-5).

Cách đơn giản nhất để triển khai web cache là đặt nó kế bên mỗi thực thể của máy chủ web bằng mẫu sidecar (xem Hình 5-5).

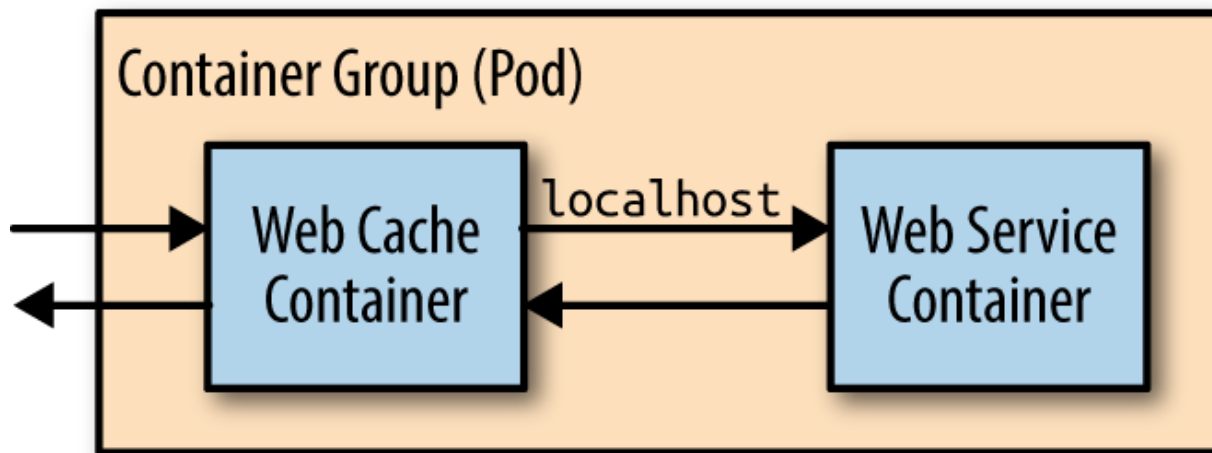


Figure 5-5. Adding the web cache server as a sidecar

Hình 5-5. Thêm máy chủ web cache dưới dạng một sidecar

Though this approach is simple, it has some disadvantages, namely that you will have to scale your cache at the same scale as your web servers. This is often not the approach you want. For your cache, you want as few replicas as possible with lots of resources for each replica (e.g., rather than 10 replicas with 1 GB of RAM each, you'd want two replicas with 5 GB of RAM each). To understand why this is preferable, consider that every page will be stored in every replica. With 10 replicas, you will

store every page 10 times, reducing the overall set of pages that you can keep in memory in the cache. This causes a reduction in the **hit rate**, the fraction of the time that a request can be served out of cache, which in turn decreases the utility of the cache. Though you do want a few large caches, you might also want lots of small replicas of your web servers. Many languages (e.g., NodeJS) can really only utilize a single core, and thus you want many replicas to be able to take advantages of multiple cores, even on the same machine. Therefore, it makes the most sense to configure your caching layer as a second stateless replicated serving tier above your web-serving tier, as illustrated in Figure 5-6 .

Mặc dù cách tiếp cận này đơn giản, nó có một số nhược điểm, cụ thể là bạn sẽ phải mở rộng cache của mình với cùng quy mô như các máy chủ web. Đây thường không phải là cách tiếp cận bạn muốn. Đối với cache, bạn muốn càng ít bản sao càng tốt với nhiều tài nguyên cho mỗi bản sao (ví dụ: thay vì 10 bản sao mỗi cái 1 GB RAM, bạn sẽ muốn hai bản sao mỗi cái 5 GB RAM). Để hiểu tại sao điều này tốt hơn, hãy xét rằng mỗi trang sẽ được lưu trong mọi bản sao. Với 10 bản sao, bạn sẽ lưu mỗi trang 10 lần, làm giảm tổng tập hợp các trang mà bạn có thể giữ trong bộ nhớ trong cache. Điều này gây ra việc giảm tỷ lệ trúng cache (hit rate), tức tỷ lệ thời gian mà một yêu cầu có thể được phục vụ từ cache, từ đó làm giảm tính hữu dụng của cache. Mặc dù bạn muốn có một vài cache lớn, bạn cũng có thể muốn có nhiều bản sao nhỏ của các máy chủ web. Nhiều ngôn ngữ (ví dụ: NodeJS) thực sự chỉ có thể tận dụng một lõi đơn, và do đó bạn muốn có nhiều bản sao để có thể tận dụng nhiều lõi, ngay cả trên cùng một máy. Vì thế, hợp lý nhất là cấu hình tầng đệm của bạn như một tầng phục vụ nhân bản không trạng thái thứ hai nằm trên tầng phục vụ web của bạn, như được minh họa ở Hình 5-6.

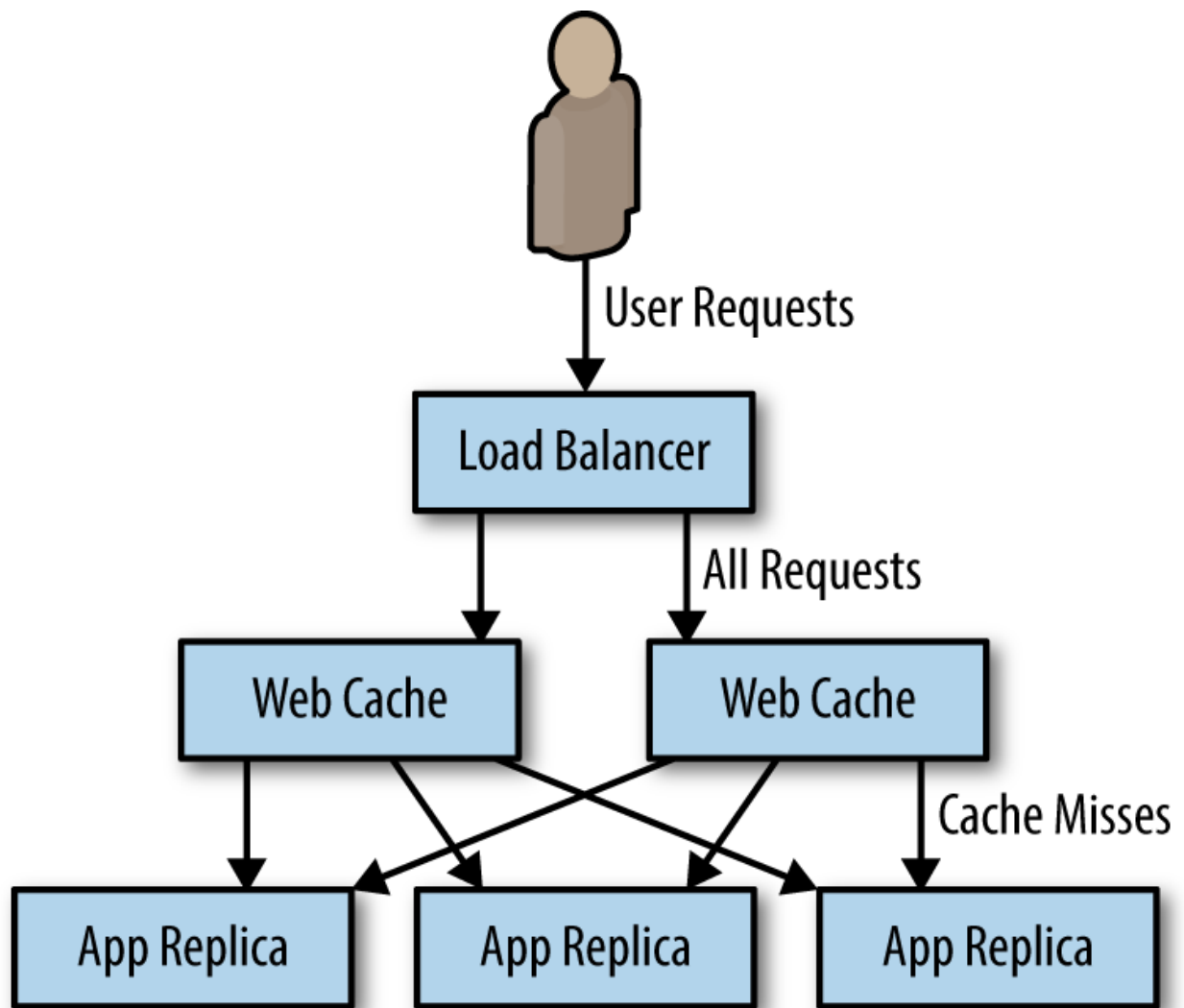
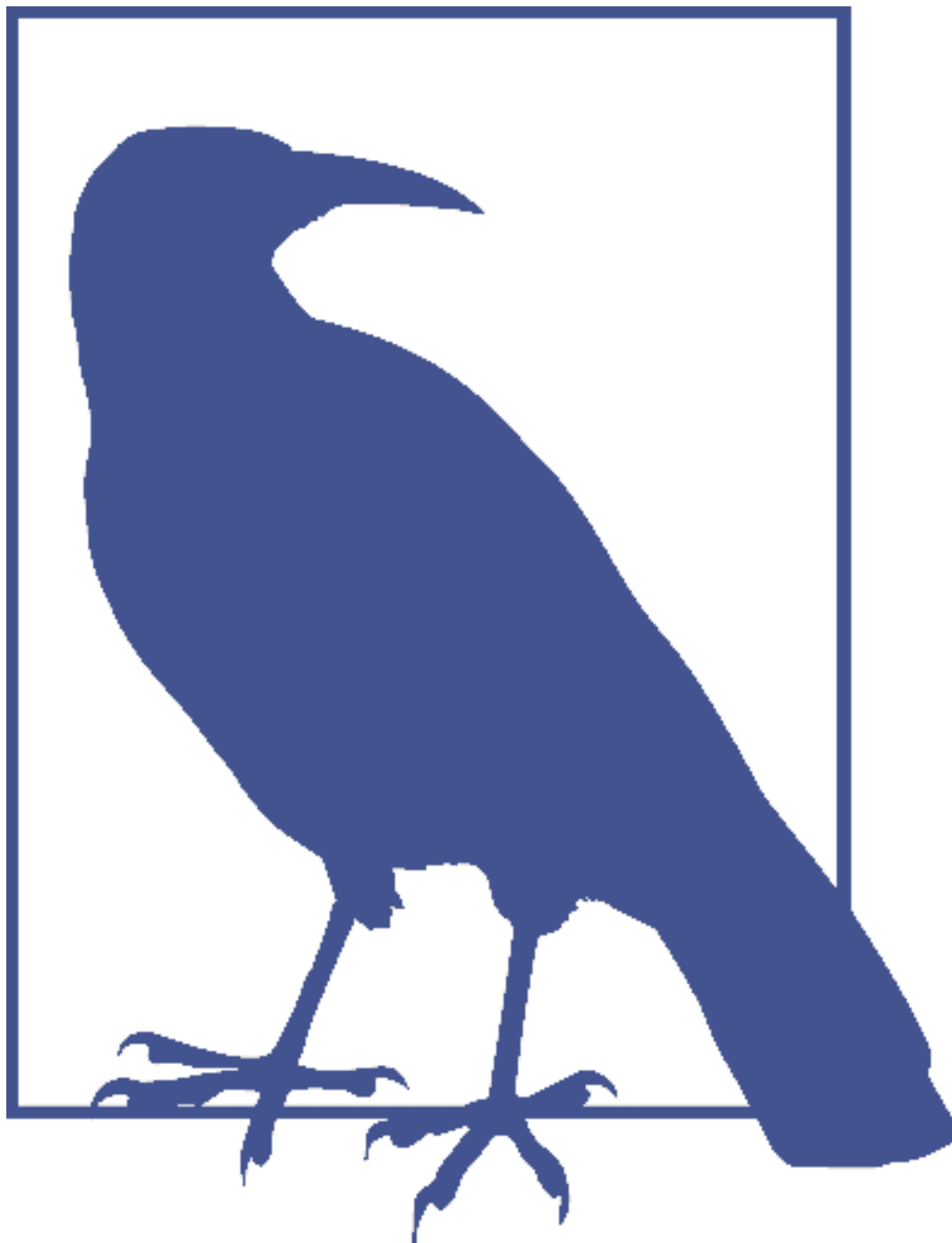


Figure 5-6. Adding the caching layer to our replicated service

Hình 5-6. Thêm tầng đệm vào dịch vụ nhân bản của chúng ta



Unless you are careful, caching can break session tracking. The reason for this is that if you use default IP address **affinity** and load balancing, all requests will be sent from the IP addresses of the cache, not the end user of your service. If you've followed the advice previously given and deployed a few large caches, your IP-address-based affinity may in fact mean that some replicas of your web layer see no traffic. Instead, you need to use something like a cookie or HTTP header for session tracking.

Trừ khi bạn cẩn thận, việc đệm có thể phá vỡ việc theo dõi phiên. Lý do cho điều này là nếu bạn dùng tính tương thuộc (affinity) địa chỉ IP mặc định và cân bằng tải, tất cả yêu cầu sẽ được gửi từ các địa chỉ IP của cache, chứ không phải của người dùng cuối của dịch

vụ. Nếu bạn đã làm theo lời khuyên đã đưa ra trước đó và triển khai một vài cache lớn, tính tương thuộc dựa trên địa chỉ IP của bạn trên thực tế có thể đồng nghĩa với việc một số bản sao của tầng web không nhận được lưu lượng nào. Thay vào đó, bạn cần dùng một thứ gì đó như cookie hoặc HTTP header để theo dõi phiên.

Hands On: Deploying the Caching Layer — Thực hành (Hands On): Triển khai tầng đệm

The dictionary-server service we built earlier distributes traffic to the dictionary server and is discoverable as the DNS name dictionary-server-service . This pattern is illustrated in Figure 5-7 .

Dịch vụ dictionary-server mà chúng ta xây dựng trước đó phân phối lưu lượng tới máy chủ từ điển và có thể được khám phá qua tên DNS dictionary-server-service. Mẫu này được minh họa ở Hình 5-7.

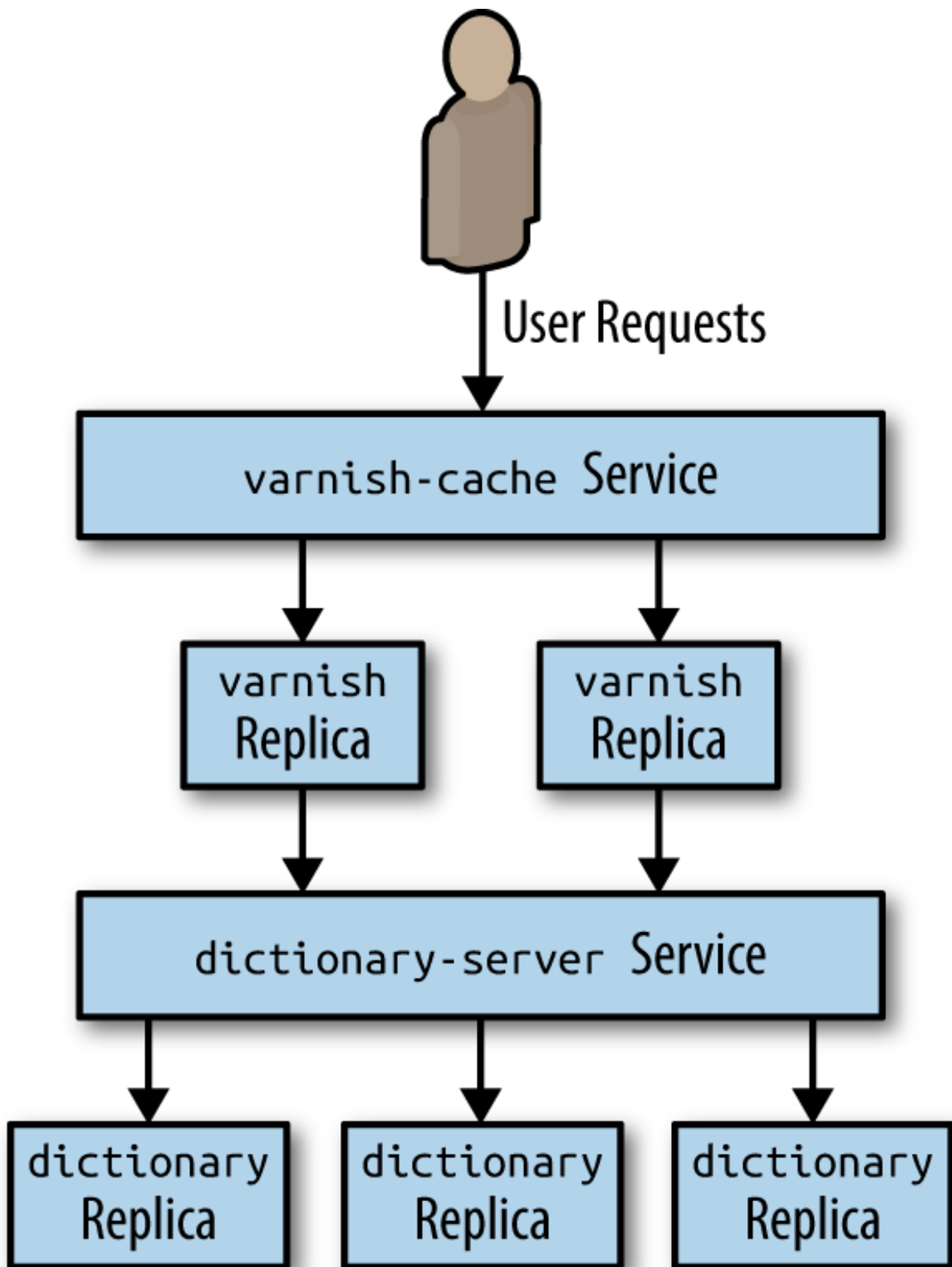


Figure 5-7. Adding a caching layer to the dictionary server

Hình 5-7. Thêm một tầng đệm vào máy chủ từ điển

We can begin building this with the following Varnish cache configuration:

Chúng ta có thể bắt đầu xây dựng cái này với cấu hình Varnish cache sau:

```
vcl 4.0;
backend default {
    .host = "dictionary-server-service";
    .port = "8080";
}
```

Create a **ConfigMap** object to hold this configuration:

Tạo một đối tượng ConfigMap để chứa cấu hình này:

```
kubectl create configmap varnish-config --from-file=default.vcl
```

Now we can deploy the replicated Varnish cache, which will load this configuration:

Giờ chúng ta có thể triển khai Varnish cache nhân bản, nó sẽ nạp cấu hình này:

```
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: varnish-cache
spec:
  replicas: 2
  template:
    metadata:
      labels:
        app: varnish-cache
    spec:
      containers:
        - name: cache
          resources:
            requests:
              # We'll use two gigabytes for each varnish cache
              memory: 2Gi
          image: brendanburns/varnish
          command:
            - varnishd
            - -F
            - -f
            - /etc/varnish-config/default.vcl
            - -a
            - 0.0.0.0:8080
            - -s
          # This memory allocation should match the memory request above
          - malloc,2G
          ports:
            - containerPort: 8080
          volumeMounts:
            - name: varnish
              mountPath: /etc/varnish-config
          volumes:
```

```
- name: varnish
  configMap:
    name: varnish-config
```

You can deploy the replicated Varnish servers with:

Bạn có thể triển khai các máy chủ Varnish nhân bản với:

```
kubectl create -f varnish-deploy.yaml
```

And then finally deploy a load balancer for this Varnish cache:

Và rồi cuối cùng triển khai một bộ cân bằng tải cho Varnish cache này:

```
kind: Service
apiVersion: v1
metadata:
  name: varnish-service
spec:
  selector:
    app: varnish-cache
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

which you can create with:

mà bạn có thể tạo với:

```
kubectl create -f varnish-service.yaml
```

Expanding the Caching Layer — Mở rộng tầng đệm

Now that we have inserted a caching layer into our stateless, replicated service, let's look at what this layer can provide beyond standard caching. HTTP reverse proxies like Varnish are generally pluggable and can provide a number of advanced features that are useful beyond caching.

Giờ khi chúng ta đã chèn một tầng đệm vào dịch vụ nhân bản, không trạng thái của mình, hãy xem tầng này có thể cung cấp gì ngoài việc đệm tiêu chuẩn. Các HTTP reverse proxy như Varnish nói chung là có thể cắm thêm (pluggable) và có thể cung cấp một số tính năng nâng cao hữu ích ngoài việc đệm.

Rate Limiting and Denial-of-Service Defense — Giới hạn tốc độ và phòng thủ chống từ chối dịch vụ

Few of us build sites with the expectation that we will encounter a **denial-of-service** attack. But as more and more of us build APIs, a denial of service can come simply from a developer misconfiguring a client or a site-reliability engineer accidentally running a load test against a production installation. Thus, it makes sense to add general denial-of-service defense via **rate limiting** to the caching layer. Most HTTP reverse proxies like Varnish have capabilities along this line. In particular, Varnish has a throttle module that can be configured to provide throttling based on IP address and request path, as well as whether or not a user is logged in.

Ít ai trong chúng ta xây dựng các trang web với kỳ vọng rằng mình sẽ gặp phải một cuộc tấn công từ chối dịch vụ. Nhưng khi ngày càng nhiều người trong chúng ta xây dựng API, một cuộc từ chối dịch vụ có thể đến đơn giản từ việc một nhà phát triển cấu hình sai một client hoặc một kỹ sư độ tin cậy hệ thống (SRE) vô tình chạy một bài kiểm thử tải đối với một bản cài đặt sản xuất (production). Vì vậy, việc thêm phòng thủ chống từ chối dịch vụ tổng quát thông qua giới hạn tốc độ vào tầng đệm là hợp lý. Hầu hết các HTTP reverse proxy như Varnish đều có các khả năng theo hướng này. Cụ thể, Varnish có một mô-đun throttle có thể được cấu hình để cung cấp việc điều tiết (throttling) dựa trên địa chỉ IP và đường dẫn yêu cầu, cũng như liệu một người dùng có đăng nhập hay không.

If you are deploying an API, it is generally a best practice to have a relatively small rate limit for anonymous access and then force users to log in to obtain a higher rate limit. Requiring a login provides auditing to determine who is responsible for the unexpected load, and also offers a barrier to would-be attackers who need to obtain multiple identities to launch a successful attack.

Nếu bạn đang triển khai một API, thông lệ tốt nhất nói chung là đặt một giới hạn tốc độ tương đối nhỏ cho truy cập ẩn danh và rồi buộc người dùng đăng nhập để có được giới hạn tốc độ cao hơn. Việc yêu cầu đăng nhập cung cấp khả năng kiểm toán để xác định ai chịu trách nhiệm cho tải bất ngờ, và cũng tạo ra một rào cản đối với những kẻ tấn công tiềm năng vốn cần phải có được nhiều danh tính để phát động một cuộc tấn công thành công.

When a user hits the rate limit, the server will return the 429 error code indicating that too many requests have been issued. However, many users want to understand how many requests they have left before hitting that limit. To that end, you will likely also want to populate an HTTP header with the remaining-calls information. Though there isn't a standard header for returning this data, many APIs return some variation

Khi một người dùng chạm giới hạn tốc độ, máy chủ sẽ trả về mã lỗi 429 cho biết đã có quá nhiều yêu cầu được phát đi. Tuy nhiên, nhiều người dùng muốn hiểu họ còn lại bao nhiêu yêu cầu trước khi chạm giới hạn đó. Vì mục đích đó, bạn cũng sẽ có khả năng muốn điền vào một HTTP header thông tin về số lượng lời gọi còn lại. Mặc dù không có một header tiêu chuẩn để trả về dữ liệu này, nhiều API trả về một biến thể nào đó

ofX-RateLimit-Remaining.

SSL Termination — Kết thúc SSL

In addition to performing caching for performance, one of the other common tasks performed by the **edge layer** is **SSL** termination. Even if you plan on using SSL for communication between layers in your cluster, you should still use different certificates for the edge and your internal services. Indeed, each individual internal service should use its own certificate to ensure that each layer can be rolled out independently. Unfortunately, the Varnish web cache can't be used for SSL termination, but fortunately, the **nginx** application can. Thus we want to add a third layer to our stateless application pattern, which will be a replicated layer of nginx servers that will handle SSL termination for HTTPS traffic and forward traffic on to our Varnish cache. HTTP traffic continues to travel to the Varnish web cache, and Varnish forwards traffic on to our web application, as shown in Figure 5-8.

Ngoài việc thực hiện đệm để tăng hiệu năng, một trong những tác vụ phổ biến khác được thực hiện bởi tầng biên (edge layer) là kết thúc SSL (SSL termination). Ngay cả khi bạn dự định dùng SSL cho việc giao tiếp giữa các tầng trong cụm của mình, bạn vẫn nên dùng các chứng chỉ khác nhau cho biên và cho các dịch vụ nội bộ của bạn. Thực vậy, mỗi dịch vụ

nội bộ riêng lẻ nên dùng chứng chỉ riêng của nó để đảm bảo rằng mỗi tầng có thể được tung ra một cách độc lập. Đáng tiếc là Varnish web cache không thể được dùng cho việc kết thúc SSL, nhưng may thay, ứng dụng nginx thì có thể. Do đó chúng ta muốn thêm một tầng thứ ba vào mẫu ứng dụng không trạng thái của mình, đó sẽ là một tầng nhân bản của các máy chủ nginx, sẽ xử lý việc kết thúc SSL cho lưu lượng HTTPS và chuyển tiếp lưu lượng tới Varnish cache của chúng ta. Lưu lượng HTTP tiếp tục đi tới Varnish web cache, và Varnish chuyển tiếp lưu lượng tới ứng dụng web của chúng ta, như được trình bày ở Hình 5-8.

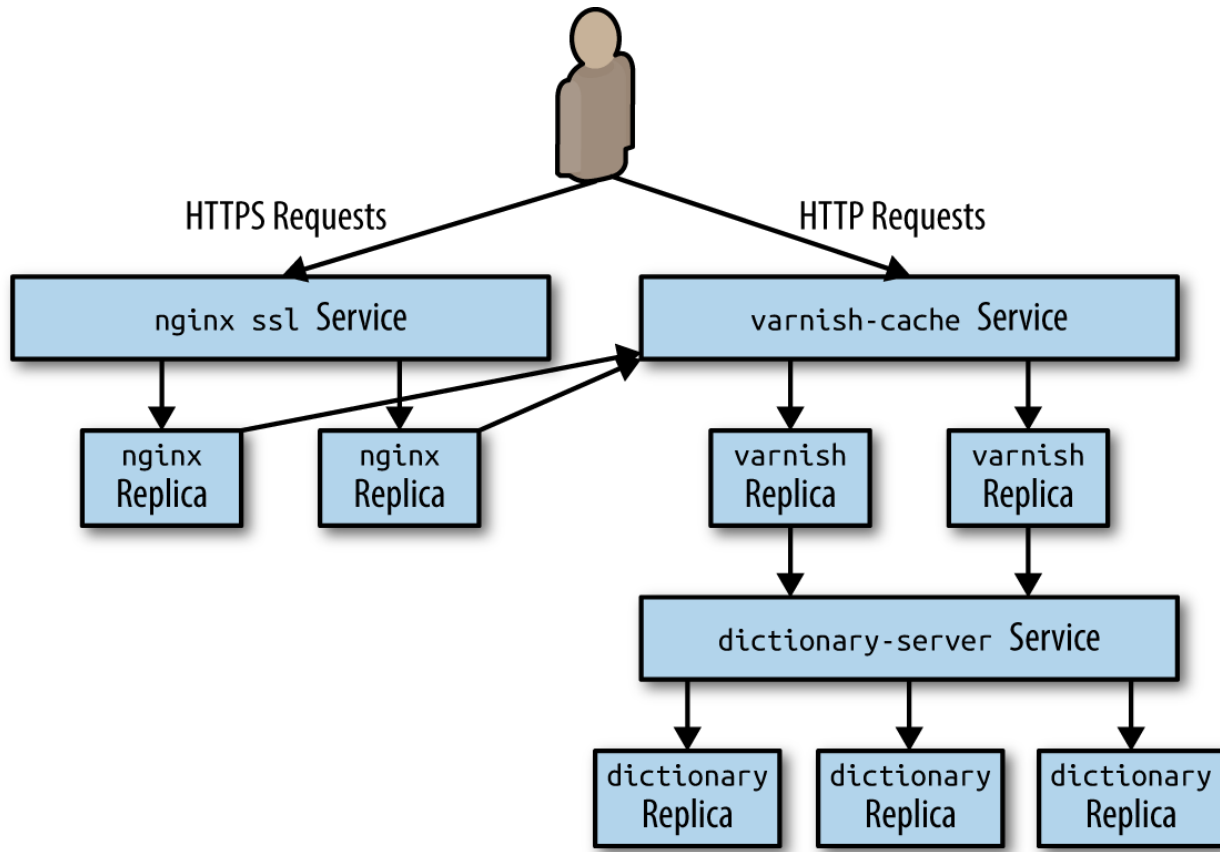


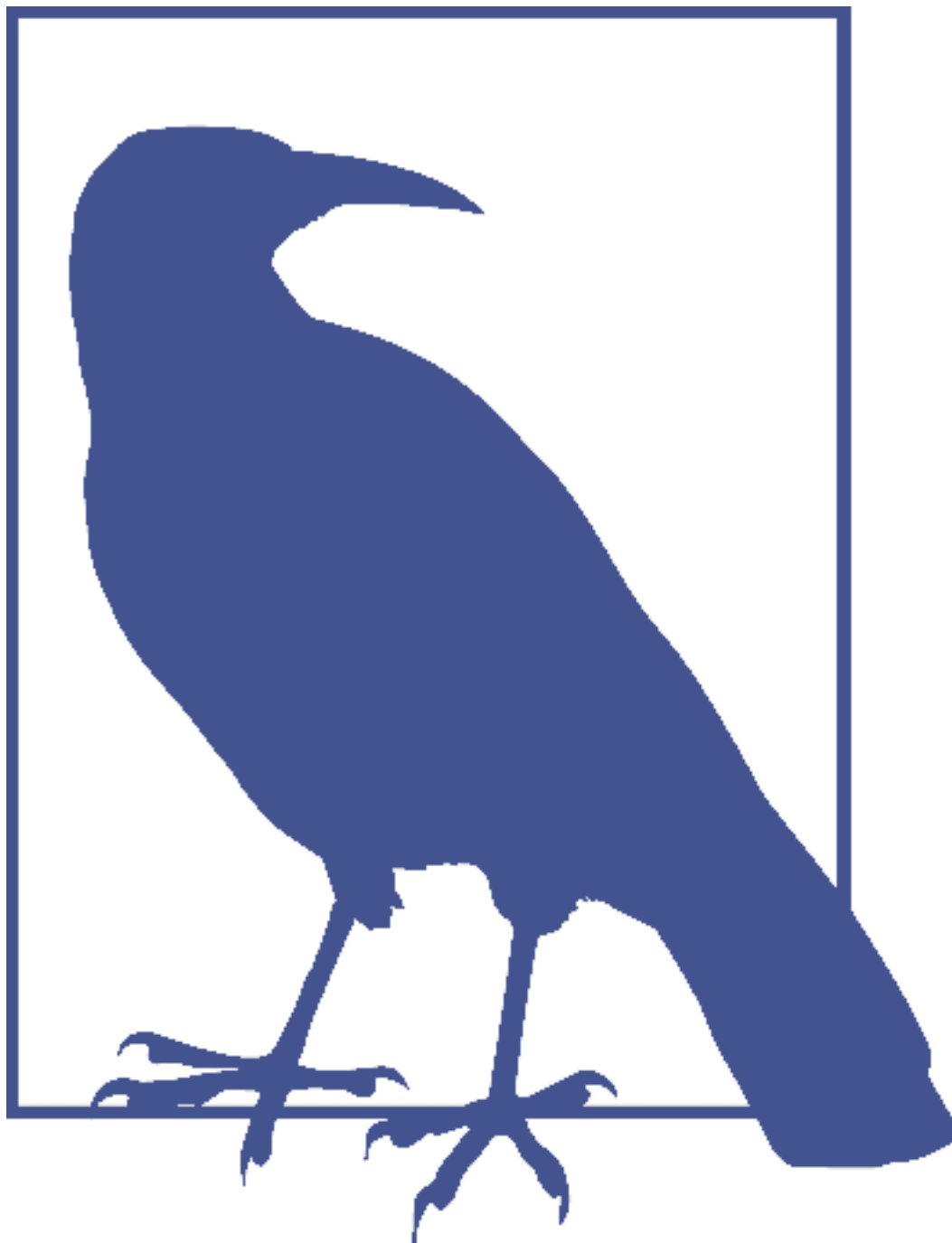
Figure 5-8. Complete replicated stateless serving example

Hình 5-8. Ví dụ phục vụ không trạng thái nhân bản hoàn chỉnh

Hands On: Deploying nginx and SSL Termination — Thực hành (Hands On): Triển khai nginx và kết thúc SSL

The following instructions describe how to add a replicated SSL terminating nginx to the replicated service and cache that we previously deployed.

Các hướng dẫn sau mô tả cách thêm một nginx kết thúc SSL được nhân bản vào dịch vụ nhân bản và cache mà chúng ta đã triển khai trước đó.



These instructions assume that you have a certificate. If you need to obtain a certificate, the easiest way to do that is via the tools at Let's Encrypt . Alternately, you can use the **openssl** tool to create them. The following instructions assume that you've named them `server.crt` (public certificate) and `server.key` (private key on the server). Such self-signed certificates will cause security alerts in modern web browsers and should never be used for production.

Các hướng dẫn này giả định rằng bạn đã có một chứng chỉ. Nếu bạn cần lấy một chứng chỉ, cách dễ nhất để làm điều đó là thông qua các công cụ tại Let's Encrypt. Hoặc, bạn có thể dùng công cụ openssl để tạo chúng. Các hướng dẫn sau giả định rằng bạn đã đặt tên chúng là `server.crt` (chứng chỉ công khai) và `server.key` (khóa riêng tư trên máy chủ). Các

chứng chỉ tự ký (self-signed) như vậy sẽ gây ra các cảnh báo bảo mật trong các trình duyệt web hiện đại và không bao giờ nên được dùng cho sản xuất (production).

The first step is to upload your certificate as a **secret** to Kubernetes:

Bước đầu tiên là tải chứng chỉ của bạn lên dưới dạng một secret tới Kubernetes:

```
kubectl create secret tls ssl --cert=server.crt --key=server.key
```

Once you have uploaded your certificate as a secret you need to create an nginx configuration to serve SSL:

Một khi bạn đã tải chứng chỉ của mình lên dưới dạng một secret, bạn cần tạo một cấu hình nginx để phục vụ SSL:

```
events {
    worker_connections 1024;
}
http {
    server {
        listen 443 ssl;
        server_name my-domain.com www.my-domain.com;

        ssl on;
        ssl_certificate      /etc/certs/tls.crt;
        ssl_certificate_key   /etc/certs/tls.key;
        location / {
            proxy_pass http://varnish-service:80;
            proxy_set_header Host $host;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

As with Varnish, you need to transform this into a ConfigMap object:

Cũng như với Varnish, bạn cần biến đổi cái này thành một đối tượng ConfigMap:

```
kubectl create configmap nginx-conf --from-file=nginx.conf
```

Now that you have a secret and an nginx configuration, it is time to create the replicated, stateless nginx layer:

Giờ khi bạn đã có một secret và một cấu hình nginx, đã đến lúc tạo tầng nginx nhân bản, không trạng thái:

```
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: nginx-ssl
spec:
  replicas: 4
  template:
    metadata:
      labels:
```

```

    app: nginx-ssl
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 443
    volumeMounts:
    - name: conf
      mountPath: /etc/nginx
    - name: certs
      mountPath: /etc/certs
  volumes:
  - name: conf
    configMap:
# This is the ConfigMap for nginx we created previously
      name: nginx-conf
  - name: certs
    secret:
# This is the secret we created above
      secretName: ssl

```

To create the replicated nginx servers, you use:

Để tạo các máy chủ nginx nhân bản, bạn dùng:

```
kubectl create -f nginx-deploy.yaml
```

Finally, you can expose this nginx SSL server with a service:

Cuối cùng, bạn có thể phơi bày (expose) máy chủ nginx SSL này bằng một service:

```

kind: Service
apiVersion: v1
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx-ssl
  type: LoadBalancer
  ports:
  - protocol: TCP
    port: 443
    targetPort: 443

```

To create this load-balancing service run:

Để tạo dịch vụ cân bằng tải này, hãy chạy:

```
kubectl create -f nginx-service.yaml
```

If you create this service on a Kubernetes cluster that supports external load balancers, this will create an externalized, public service that services traffic on a public IP address.

Nếu bạn tạo dịch vụ này trên một cụm Kubernetes hỗ trợ các bộ cân bằng tải bên ngoài, điều này sẽ tạo ra một dịch vụ công khai, được đưa ra bên ngoài, phục vụ lưu lượng trên

một địa chỉ IP công khai.

To get this IP address, you can run:

Để lấy địa chỉ IP này, bạn có thể chạy:

```
kubectl get services
```

You should then be able to access the service with your web browser.

Sau đó bạn sẽ có thể truy cập dịch vụ bằng trình duyệt web của mình.

Summary — Tóm tắt

This chapter began with a simple pattern for replicated stateless services. Then we saw how this pattern grows with two additional replicated load-balanced layers to provide caching for performance, and SSL termination for secure web serving. This complete pattern for stateless replicated serving is shown in Figure 5-8 .

Chương này bắt đầu với một mẫu đơn giản cho các dịch vụ nhân bản không trạng thái. Sau đó chúng ta đã thấy mẫu này lớn lên như thế nào với hai tầng nhân bản được cân bằng tải bổ sung để cung cấp việc đệm nhằm tăng hiệu năng, và việc kết thúc SSL cho việc phục vụ web an toàn. Mẫu hoàn chỉnh này cho việc phục vụ nhân bản không trạng thái được trình bày ở Hình 5-8.

This complete pattern can be deployed into Kubernetes using three Deployments and Service load balancers to connect the layers shown in Figure 5-8 . The complete source for these examples can be found at <https://github.com/brendandburns/designing-distributed-systems> .

Mẫu hoàn chỉnh này có thể được triển khai vào Kubernetes bằng ba Deployment và các bộ cân bằng tải Service để kết nối các tầng được trình bày ở Hình 5-8. Toàn bộ mã nguồn cho các ví dụ này có thể tìm thấy tại <https://github.com/brendandburns/designing-distributed-systems>.

CHAPTER 6 Sharded Services —

CHAPTER 6 Dịch vụ phân mảnh (Sharded Services)

In the previous chapter, we saw the value of replicating **stateless** services for **reliability**, redundancy, and scaling. This chapter considers sharded services. With the replicated services that we introduced in the preceding chapter, each **replica** was entirely homogeneous and capable of serving every request. In contrast to replicated services, with sharded services, each replica, or **shard**, is only capable of serving a subset of all requests. A load-balancing **node**, or root, is responsible for examining each request and distributing each request to the appropriate shard or shards for processing. The contrast between replicated and sharded services is represented in Figure 6-1.

Ở chương trước, chúng ta đã thấy giá trị của việc nhân bản các dịch vụ không trạng thái (stateless) để đạt độ tin cậy, tính dư thừa và khả năng mở rộng. Chương này xem xét các dịch vụ phân mảnh (sharded). Với các dịch vụ nhân bản được giới thiệu ở chương trước, mỗi bản sao (replica) hoàn toàn đồng nhất và có khả năng phục vụ mọi yêu cầu. Trái với các dịch vụ nhân bản, với dịch vụ phân mảnh, mỗi bản sao, hay mảnh (shard), chỉ có khả năng phục vụ một tập con của tất cả các yêu cầu. Một nút cân bằng tải, hay gốc (root), chịu trách nhiệm xem xét từng yêu cầu và phân phối mỗi yêu cầu đến đúng mảnh hoặc các mảnh thích hợp để xử lý. Sự tương phản giữa dịch vụ nhân bản và dịch vụ phân mảnh được minh họa trong Hình 6-1.

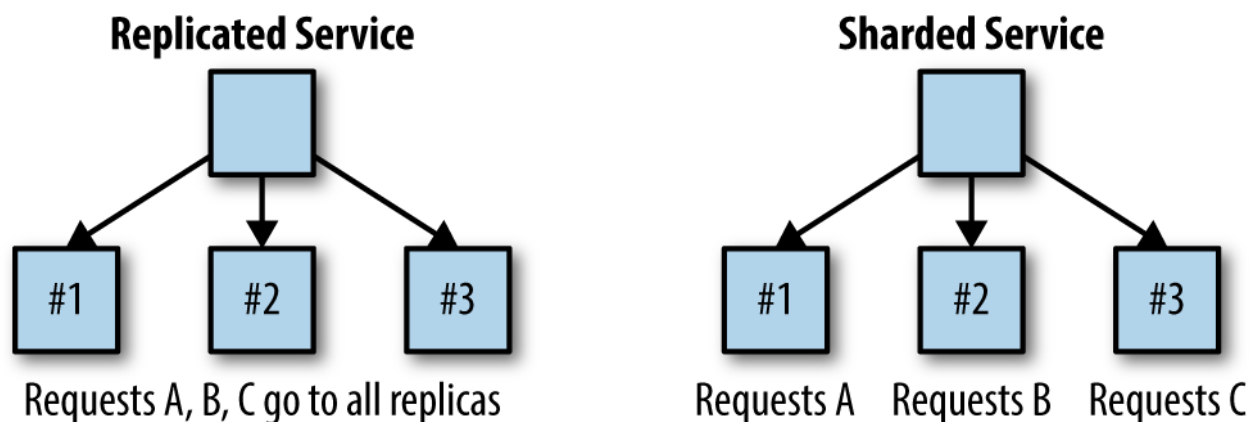


Figure 6-1. **Replicated service** versus **sharded service**

Hình 6-1. Dịch vụ nhân bản so với dịch vụ phân mảnh

Replicated services are generally used for building stateless services, whereas sharded services are generally used for building **stateful** services. The primary reason for sharding the data is because the size of the state is too large to be served by a single machine. Sharding enables you to scale a **service** in response to the size of the state that needs to be served.

Các dịch vụ nhân bản thường được dùng để xây dựng dịch vụ không trạng thái, trong khi các dịch vụ phân mảnh thường được dùng để xây dựng dịch vụ có trạng thái. Lý do chính để phân mảnh dữ liệu là vì kích thước của trạng thái quá lớn để một máy đơn lẻ có thể phục vụ. Việc phân mảnh cho phép bạn mở rộng một dịch vụ tương ứng với kích thước của trạng thái cần được phục vụ.

Sharded Caching — Sharded Caching (Đệm phân mảnh)

To completely illustrate the design of a sharded system, this section provides a deep dive into the design of a sharded caching system. A sharded cache is a cache that sits

Để minh họa đầy đủ cách thiết kế một hệ thống phân mảnh, phần này đi sâu vào thiết kế của một hệ thống đệm phân mảnh (sharded caching). Một cache phân mảnh là một cache nằm

59 between the user requests and the actual frontend implementation. A high-level diagram of the system is shown in Figure 6-2 .

59 giữa các yêu cầu của người dùng và phần triển khai frontend thực sự. Một sơ đồ tổng quát của hệ thống được trình bày trong Hình 6-2.

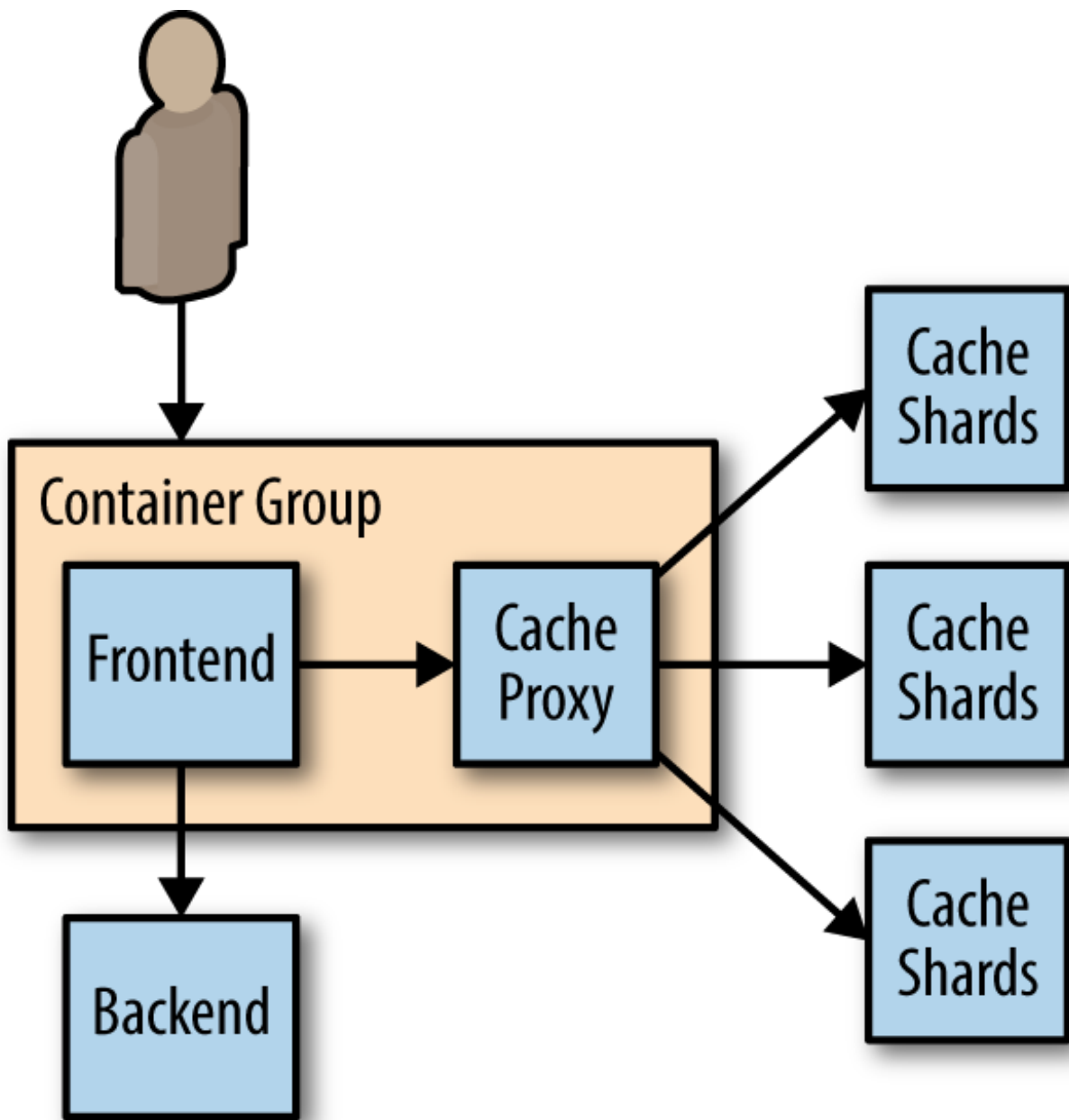


Figure 6-2. A sharded cache

Hình 6-2. Một cache phân mảnh

In Chapter 3, we discussed how an **ambassador** could be used to distribute data to a sharded service. This section discusses how to build that service. When designing a sharded cache, there are a number of design aspects to consider:

Trong Chương 3, chúng ta đã thảo luận về cách một ambassador có thể được dùng để phân phối dữ liệu đến một dịch vụ phân mảnh. Phần này thảo luận cách xây dựng dịch vụ đó. Khi thiết kế một cache phân mảnh, có một số khía cạnh thiết kế cần cân nhắc:

- Why you might need a sharded cache
- The role of the cache in your architecture

- Replicated, sharded caches
- The **sharding function**
 - Vì sao bạn có thể cần đến một cache phân mảnh
 - Vai trò của cache trong kiến trúc của bạn
 - Cache phân mảnh, nhân bản
 - Hàm phân mảnh

Why You Might Need a Sharded Cache — Vì sao bạn có thể cần đến một cache phân mảnh

As was mentioned in the introduction, the primary reason for sharding any service is to increase the size of the data being stored in the service. To understand how this helps a caching system, imagine the following system: Each cache has 10 GB of RAM available to store results, and can serve 100 requests per second (RPS). Suppose then that our service has a total of 200 GB possible results that could be returned, and an expected 1,000 RPS. Clearly, we need 10 replicas of the cache in order to satisfy 1,000 RPS (10 replicas $\times$ 100 requests per second per replica). The simplest way to deploy this service would be as a replicated service, as described in the previous chapter. But deployed this way, the distributed cache can only hold a maximum of 5% (10 GB/200 GB) of the total data set that we are serving. This is because each cache replica is independent, and thus each cache replica stores roughly the exact same data in the cache. This is great for redundancy, but pretty terrible for maximizing memory utilization. If instead, we deploy a 10-way sharded cache, we can still serve the appropriate number of RPS (10 $\times$ 100 is still 1,000), but because each cache serves a completely unique set of data, we are able to store 50% (10 $\times$ 10 GB/200 GB) of the total data set. This tenfold increase in cache storage means that the memory for the cache is much better utilized, since each key exists only in a single cache.

Như đã đề cập trong phần giới thiệu, lý do chính để phân mảnh bất kỳ dịch vụ nào là để tăng kích thước dữ liệu được lưu trữ trong dịch vụ. Để hiểu điều này giúp ích như thế nào cho một hệ thống đệm, hãy hình dung hệ thống sau: Mỗi cache có 10 GB RAM khả dụng để lưu kết quả, và có thể phục vụ 100 yêu cầu mỗi giây (RPS). Giả sử rằng dịch vụ của chúng ta có tổng cộng 200 GB kết quả khả dĩ có thể được trả về, và kỳ vọng 1.000 RPS. Rõ ràng, chúng ta cần 10 bản sao của cache để đáp ứng 1.000 RPS (10 bản sao $\times$ 100 yêu cầu mỗi giây mỗi bản sao). Cách đơn giản nhất để triển khai dịch vụ này là dưới dạng một dịch vụ nhân bản, như mô tả ở chương trước. Nhưng khi triển khai theo cách này, cache phân tán chỉ có thể chứa tối đa 5% (10 GB/200 GB) của tổng tập dữ liệu mà chúng ta đang phục vụ. Đó là vì mỗi bản sao cache là độc lập, và do đó mỗi bản sao cache lưu trữ gần như chính xác cùng một dữ liệu trong cache. Điều này tốt cho tính dư thừa, nhưng khá tệ cho việc tối đa hóa hiệu suất sử dụng bộ nhớ. Nếu thay vào đó, chúng ta triển khai một cache phân mảnh 10 chiều, chúng ta vẫn có thể phục vụ số RPS thích hợp (10 $\times$ 100 vẫn là 1.000), nhưng vì mỗi cache phục vụ một tập dữ liệu hoàn toàn duy nhất, chúng ta có thể lưu trữ 50% (10 $\times$ 10 GB/200 GB) của tổng tập dữ liệu. Mức tăng gấp mười lần dung lượng lưu trữ cache này có nghĩa là bộ nhớ cho cache được tận dụng tốt hơn nhiều, vì mỗi khóa chỉ tồn tại trong một cache duy nhất.

The Role of the Cache in System Performance — Vai trò của cache trong hiệu năng hệ thống

In Chapter 5 we discussed how caches can be used to optimize end-user performance and **latency**, but one thing that wasn't covered was the criticality of the cache to your application's performance, reliability, and stability.

Ở Chương 5 chúng ta đã thảo luận cách các cache có thể được dùng để tối ưu hóa hiệu năng và độ trễ cho người dùng cuối, nhưng có một điều chưa được đề cập là tầm quan trọng then chốt của cache đối với hiệu năng, độ tin cậy và độ ổn định của ứng dụng của bạn.

Put simply, the important question for you to consider is: If the cache were to fail, what would the impact be for your users and your service?

Nói đơn giản, câu hỏi quan trọng mà bạn cần cân nhắc là: Nếu cache hỏng, tác động đến người dùng và dịch vụ của bạn sẽ ra sao?

When we discussed the replicated cache, this question was less relevant because the cache itself was horizontally scalable, and failures of specific replicas would only lead to transient failures. Likewise, the cache could be horizontally scaled in response to increased load without impacting the end user.

Khi chúng ta thảo luận về cache nhân bản, câu hỏi này ít liên quan hơn vì bản thân cache có khả năng mở rộng theo chiều ngang, và lỗi của các bản sao cụ thể chỉ dẫn đến những thất bại nhất thời. Tương tự, cache có thể được mở rộng theo chiều ngang để đáp ứng tải tăng mà không ảnh hưởng đến người dùng cuối.

This changes when you consider sharded caches. Because a specific user or request is always mapped to the same shard, if that shard fails, that user or request will always **miss** the cache until the shard is restored. Given the nature of a cache as transient data, this miss is not inherently a problem, and your system must know how to recalculate the data. However, this recalculation is inherently slower than using the cache directly, and thus it has performance implications for your end users.

Điều này thay đổi khi bạn xét đến cache phân mảnh. Vì một người dùng hoặc yêu cầu cụ thể luôn được ánh xạ đến cùng một mảnh, nếu mảnh đó hỏng, người dùng hoặc yêu cầu đó sẽ luôn trượt cache cho đến khi mảnh được khôi phục. Xét bản chất của cache là dữ liệu nhất thời, việc trượt này về cơ bản không phải là vấn đề, và hệ thống của bạn phải biết cách tính toán lại dữ liệu. Tuy nhiên, việc tính toán lại này về bản chất chậm hơn so với dùng cache trực tiếp, và do đó nó có hệ quả về hiệu năng đối với người dùng cuối của bạn.

The performance of your cache is defined in terms of its **hit rate**. The hit rate is the percentage of the time that your cache contains the data for a user request. Ultimately, the hit rate determines the overall capacity of your **distributed system** and affects the overall capacity and performance of your system.

Hiệu năng của cache được định nghĩa theo tỷ lệ trúng (hit rate). Tỷ lệ trúng là tỷ lệ phần trăm số lần cache chứa dữ liệu cho một yêu cầu của người dùng. Cuối cùng, tỷ lệ trúng quyết định tổng năng lực của hệ phân tán của bạn và ảnh hưởng đến tổng năng lực và hiệu năng của hệ thống.

Imagine, if you will, that you have a request-serving layer that can handle 1,000 RPS. After 1,000 RPS, the system starts to return HTTP 500 errors to users. If you place a cache with a 50% hit rate in front of this request-serving layer, adding this cache increases your maximum RPS from 1,000 RPS to 2,000 RPS. To understand why this is true, you can see that of the 2,000 inbound requests, 1,000 (50%) can

be serviced by the cache, leaving 1,000 requests to be serviced by your serving layer. In this instance, the cache is fairly critical to your service, because if the cache fails, then the serving layer will be overloaded and half of all your user requests will fail. Given this, it likely makes sense to rate your service at a maximum of 1,500 RPS rather than the full 2,000 RPS. If you do this, then you can sustain a failure of half of your cache replicas and still keep your service stable.

Hãy hình dung rằng bạn có một tầng phục vụ yêu cầu có thể xử lý 1.000 RPS. Sau 1.000 RPS, hệ thống bắt đầu trả về lỗi HTTP 500 cho người dùng. Nếu bạn đặt một cache với tỷ lệ trúng 50% trước tầng phục vụ yêu cầu này, việc thêm cache này làm tăng RPS tối đa của bạn từ 1.000 RPS lên 2.000 RPS. Để hiểu vì sao điều này đúng, bạn có thể thấy rằng trong 2.000 yêu cầu đến, 1.000 (50%) có thể được phục vụ bởi cache, để lại 1.000 yêu cầu cho tầng phục vụ của bạn xử lý. Trong trường hợp này, cache khá then chốt đối với dịch vụ của bạn, vì nếu cache hỏng, tầng phục vụ sẽ bị quá tải và một nửa số yêu cầu của người dùng sẽ thất bại. Vì vậy, có lẽ hợp lý khi định mức dịch vụ của bạn ở mức tối đa 1.500 RPS thay vì toàn bộ 2.000 RPS. Nếu làm vậy, bạn có thể chịu được lỗi của một nửa số bản sao cache mà vẫn giữ dịch vụ ổn định.

But the performance of your system isn't just defined in terms of the number of requests that it can process. Your system's end-user performance is defined in terms of the latency of requests as well. A result from a cache is generally significantly faster than calculating that result from scratch. Consequently, a cache can improve the speed of requests as well as the total number of requests processed. To see why this is true, imagine that your system can serve a request from a user in 100 milliseconds. You add a cache with a 25% hit rate that can return a result in 10 milliseconds. Thus, the average latency for a request in your system is now 77.5 milliseconds. Unlike maximum requests per second, the cache simply makes your requests faster, so there is somewhat less need to worry about the fact that requests will slow down if the cache fails or is being upgraded. However, in some cases, the performance impact can cause too many user requests to pile up in request queues and ultimately time out. It's always recommended that you load test your system both with and without caches to understand the impact of the cache on the overall performance of your system.

Nhưng hiệu năng của hệ thống không chỉ được định nghĩa theo số lượng yêu cầu mà nó có thể xử lý. Hiệu năng đối với người dùng cuối của hệ thống còn được định nghĩa theo độ trễ của các yêu cầu. Một kết quả từ cache thường nhanh hơn đáng kể so với việc tính toán kết quả đó từ đầu. Do đó, một cache có thể cải thiện cả tốc độ của các yêu cầu lẫn tổng số yêu cầu được xử lý. Để thấy vì sao điều này đúng, hãy hình dung rằng hệ thống của bạn có thể phục vụ một yêu cầu từ người dùng trong 100 mili-giây. Bạn thêm một cache với tỷ lệ trúng 25% có thể trả về kết quả trong 10 mili-giây. Như vậy, độ trễ trung bình cho một yêu cầu trong hệ thống của bạn giờ là 77,5 mili-giây. Khác với số yêu cầu tối đa mỗi giây, cache đơn giản chỉ làm cho các yêu cầu của bạn nhanh hơn, nên ít cần lo lắng hơn về việc các yêu cầu sẽ chậm lại nếu cache hỏng hoặc đang được nâng cấp. Tuy nhiên, trong một số trường hợp, tác động về hiệu năng có thể khiến quá nhiều yêu cầu của người dùng dồn ứ trong hàng đợi yêu cầu và cuối cùng hết thời gian (time out). Luôn được khuyến nghị rằng bạn nên kiểm thử tải hệ thống của mình cả khi có và không có cache để hiểu tác động của cache đối với hiệu năng tổng thể của hệ thống.

Finally, it isn't just failures that you need to think about. If you need to upgrade or redeploy a sharded cache, you can not just deploy a new replica and assume it will take the load. Deploying a new version of a sharded cache will generally result in temporarily losing some capacity. Another, more advanced option is to replicate your shards.

Cuối cùng, không chỉ những lỗi là điều bạn cần nghĩ đến. Nếu bạn cần nâng cấp hoặc tái triển khai một cache phân mảnh, bạn không thể chỉ triển khai một bản sao mới và cho rằng nó sẽ gánh tải. Việc triển khai một phiên bản mới của một cache phân mảnh thường

dẫn đến việc tạm thời mất một phần năng lực. Một lựa chọn khác, nâng cao hơn, là nhân bản các mảnh của bạn.

Replicated, Sharded Caches — Cache phân mảnh, nhân bản

Sometimes your system is so dependent on a cache for latency or load that it is not acceptable to lose an entire cache shard if there is a failure or you are doing a **rollout**. Alternatively, you may have so much load on a particular cache shard that you need to scale it to handle the load. For these reasons, you may choose to deploy a sharded, replicated service. A sharded, replicated service combines the replicated service pattern described in the previous chapter with the sharded pattern described in previous sections. In a nutshell, rather than having a single server implement each shard in the cache, a replicated service is used to implement each cache shard.

Đôi khi hệ thống của bạn phụ thuộc vào cache về độ trễ hoặc tải đến mức không thể chấp nhận việc mất nguyên một mảnh cache nếu có lỗi hoặc khi bạn đang thực hiện một đợt tung ra (rollout). Hoặc, bạn có thể có quá nhiều tải lên một mảnh cache cụ thể đến mức cần mở rộng nó để xử lý tải. Vì những lý do này, bạn có thể chọn triển khai một dịch vụ phân mảnh, nhân bản. Một dịch vụ phân mảnh, nhân bản kết hợp mẫu dịch vụ nhân bản được mô tả ở chương trước với mẫu phân mảnh được mô tả ở các phần trước. Nói gọn, thay vì có một máy chủ đơn lẻ triển khai mỗi mảnh trong cache, một dịch vụ nhân bản được dùng để triển khai mỗi mảnh cache.

This design is obviously more complicated to implement and deploy, but it has several advantages over a simple sharded service. Most importantly, by replacing a single server with a replicated service, each cache shard is resilient to failures and is always present during failures. Rather than designing your system to be tolerant to performance degradation resulting from cache shard failures, you can rely on the performance improvements that the cache provides. Assuming that you are willing to over-provision shard capacity, this means that it is safe for you to do a cache rollout during peak traffic, rather than waiting for a quiet period for your service.

Thiết kế này rõ ràng phức tạp hơn để triển khai và vận hành, nhưng nó có một số lợi thế so với một dịch vụ phân mảnh đơn giản. Quan trọng nhất, bằng cách thay thế một máy chủ đơn lẻ bằng một dịch vụ nhân bản, mỗi mảnh cache có khả năng chống chịu lỗi và luôn hiện diện trong khi xảy ra lỗi. Thay vì thiết kế hệ thống của bạn để chịu đựng sự suy giảm hiệu năng do lỗi mảnh cache gây ra, bạn có thể tin tưởng vào những cải thiện hiệu năng mà cache mang lại. Giả sử bạn sẵn lòng cấp dư năng lực mảnh, điều này có nghĩa là bạn có thể an toàn thực hiện một đợt tung ra cache trong lúc lưu lượng cao điểm, thay vì chờ một giai đoạn yên ắng của dịch vụ.

Additionally, because each replicated cache shard is an independent replicated service, you can scale each cache shard in response to its load; this sort of “**hot sharding**” is discussed at the end of this chapter.

Ngoài ra, vì mỗi mảnh cache nhân bản là một dịch vụ nhân bản độc lập, bạn có thể mở rộng mỗi mảnh cache để đáp ứng tải của nó; kiểu “phân mảnh nóng” (hot sharding) này được thảo luận ở cuối chương này.

Hands On: Deploying an Ambassador and Memcache for a Sharded Cache — Thực hành (Hands On): Triển khai một Ambassador và Memcache cho một Cache phân mảnh

In Chapter 3 we saw how to deploy a sharded **Redis** service. Deploying a sharded **memcache** is similar.

Ở Chương 3 chúng ta đã thấy cách triển khai một dịch vụ Redis phân mảnh. Triển khai một memcache phân mảnh cũng tương tự.

First, we will deploy memcache as a **Kubernetes StatefulSet** :

Trước tiên, chúng ta sẽ triển khai memcache dưới dạng một Kubernetes StatefulSet:

```
apiVersion: apps/v1beta1
kind: StatefulSet
metadata:
  name: sharded-memcache
spec:
  serviceName: "memcache"
  replicas: 3
  template:
    metadata:
      labels:
        app: memcache
    spec:
      terminationGracePeriodSeconds: 10
      containers:
        - name: memcache
          image: memcached
          ports:
            - containerPort: 11211
              name: memcache
```

Save this to a file named **memcached-shards.yaml** and you can deploy this with **kubectl create -f memcached-shards.yaml** . This will create three containers running memcached.

Lưu nội dung này vào một tệp tên là memcached-shards.yaml và bạn có thể triển khai nó bằng kubectl create -f memcached-shards.yaml. Lệnh này sẽ tạo ra ba container chạy memcached.

As with the sharded Redis example, we also need to create a Kubernetes Service that will create DNS names for the replicas we have created. The service looks like this:

Như với ví dụ Redis phân mảnh, chúng ta cũng cần tạo một Kubernetes Service để tạo các tên DNS cho các bản sao mà chúng ta đã tạo. Service trông như sau:

```
apiVersion: v1
kind: Service
metadata:
  name: memcache
  labels:
    app: memcache
spec:
```

```

ports:
- port: 11211
  name: memcache
clusterIP: None
selector:
  app: memcache

```

Save this to a file named `memcached-service.yaml` and deploy it with `kubectl create -f memcached-service.yaml`. You should now have DNS entries for `memcache-0.memcache`, `memcache-1.memcache`, etc. As with Redis, we can use these names to configure **twemproxy**.

Lưu nội dung này vào một tệp tên là `memcached-service.yaml` và triển khai nó bằng `kubectl create -f memcached-service.yaml`. Giờ bạn sẽ có các mục DNS cho `memcache-0.memcache`, `memcache-1.memcache`, v.v. Như với Redis, chúng ta có thể dùng các tên này để cấu hình `twemproxy`.

```

memcache:
  listen: 127.0.0.1:11211
  hash: fnv1a_64
  distribution: ketama
  auto_eject_hosts: true
  timeout: 400
  server_retry_timeout: 2000
  server_failure_limit: 1
  servers:
    - memcache-0.memcache:11211:1
    - memcache-1.memcache:11211:1
    - memcache-2.memcache:11211:1

```

In this config, you can see that we are serving the memcache protocol on localhost: 11211 so that the application **container** can access the ambassador. We will deploy this into our ambassador **pod** using a Kubernetes **ConfigMap** object that we can create

Trong cấu hình này, bạn có thể thấy rằng chúng ta đang phục vụ giao thức memcache trên localhost:11211 để container ứng dụng có thể truy cập ambassador. Chúng ta sẽ triển khai cái này vào pod ambassador của mình bằng cách dùng một đối tượng Kubernetes ConfigMap mà chúng ta có thể tạo

with: `kubectl create configmap --from-file=nutcracker.yaml twem-config`.

Finally, all of the preparations are done, and we can deploy our ambassador example. We define a pod that looks like this:

Cuối cùng, mọi sự chuẩn bị đã xong, và chúng ta có thể triển khai ví dụ ambassador của mình. Chúng ta định nghĩa một pod trông như sau:

```

apiVersion: v1
kind: Pod
metadata:
  name: sharded-memcache-ambassador
spec:
  containers:
# This is where the application container would go, for example
# - name: nginx
#   image: nginx

```

```
# This is the ambassador container
- name: twemproxy
  image: ganomede/twemproxy
  command:
    - nutcracker
    - -c
    - /etc/config/nutcracker.yaml
    - -v
    - 7
    - -s
    - 6222
  volumeMounts:
    - name: config-volume
      mountPath: /etc/config
volumes:
- name: config-volume
  configMap:
    name: twem-config
```

You can save this to a file named `memcached-ambassador-pod.yaml`, and then deploy it with:

Bạn có thể lưu nội dung này vào một tệp tên là `memcached-ambassador-pod.yaml`, rồi triển khai nó bằng:

```
kubectl create -f memcached-ambassador-pod.yaml
```

Of course, we don't have to use the ambassador pattern if we don't want to. An alternative is to deploy a replicated **shard router** service. There are trade-offs between using an ambassador versus using a **shard routing service**. The value of the service is a reduction of complexity. You don't have to deploy the ambassador with every pod that wants to access the sharded memcache service, it can be accessed via a named and load-balanced service. The downside of a shared service is twofold. First, because it is a shared service, you will have to scale it larger as demand load increases. Second, using the shared service introduces an extra network hop that will add some latency to requests and contribute network bandwidth to the overall distributed system.

Tất nhiên, chúng ta không bắt buộc phải dùng mẫu ambassador nếu không muốn. Một lựa chọn thay thế là triển khai một dịch vụ định tuyến mảnh (shard router) nhân bản. Có những đánh đổi giữa việc dùng một ambassador so với việc dùng một dịch vụ định tuyến mảnh. Giá trị của dịch vụ là giảm độ phức tạp. Bạn không phải triển khai ambassador cùng mỗi pod muốn truy cập dịch vụ memcache phân mảnh; nó có thể được truy cập thông qua một dịch vụ được đặt tên và cân bằng tải. Mặt hạn chế của dịch vụ dùng chung gồm hai điểm. Thứ nhất, vì là dịch vụ dùng chung, bạn sẽ phải mở rộng nó lớn hơn khi tải nhu cầu tăng. Thứ hai, việc dùng dịch vụ dùng chung tạo thêm một bước nhảy mạng (network hop) làm tăng đôi chút độ trễ cho các yêu cầu và đóng góp vào băng thông mạng của toàn hệ phân tán.

To deploy a shared routing service, you need to change the twemproxy **configuration** slightly so that it listens on all interfaces, not just localhost:

Để triển khai một dịch vụ định tuyến dùng chung, bạn cần thay đổi cấu hình twemproxy một chút để nó lắng nghe trên tất cả các giao diện, không chỉ localhost:

```
memcache:
  listen: 0.0.0.0:11211
```

```

hash: fnv1a_64
distribution: ketama
auto_eject_hosts: true
timeout: 400
server_retry_timeout: 2000
server_failure_limit: 1
servers:
- memcache-0.memcache:11211:1
- memcache-1.memcache:11211:1
- memcache-2.memcache:11211:1

```

You can save this to a file named `shared-nutcrackeryaml` , and then create a corresponding ConfigMap using `kubectl` :

Bạn có thể lưu nội dung này vào một tệp tên là `shared-nutcrackeryaml`, rồi tạo một ConfigMap tương ứng bằng `kubectl`:

```
kubectl create configmap --from-file=shared-nutcracker.yaml shared-twem-config
```

Then you can turn up the replicated shard routing service as a **Deployment** :

Sau đó bạn có thể khởi động dịch vụ định tuyến mảnh nhân bản dưới dạng một Deployment:

```

apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: shared-twemproxy
spec:
  replicas: 3
  template:
    metadata:
      labels:
        app: shared-twemproxy
    spec:
      containers:
      - name: twemproxy
        image: ganomede/twemproxy
        command:
        - nutcracker
        - -c
        - /etc/config/shared-nutcracker.yaml
        - -v
        - 7
        - -s
        - 6222
        volumeMounts:
        - name: config-volume
          mountPath: /etc/config
      volumes:
      - name: config-volume
        configMap:
          name: shared-twem-config

```

If you save this to `shared-twemproxy-deploy.yaml`, you can create the replicated shard router using `kubectl`:

Nếu bạn lưu nội dung này vào `shared-twemproxy-deploy.yaml`, bạn có thể tạo shard router nhân bản bằng `kubectl`:

```
kubectl create -f shared-twemproxy-deploy.yaml
```

To complete the shard router, we have to declare a **load balancer** to process requests:

Để hoàn tất shard router, chúng ta phải khai báo một bộ cân bằng tải để xử lý các yêu cầu:

```
kind: Service
apiVersion: v1
metadata:
  name: shard-router-service
spec:
  selector:
    app: shared-twemproxy
  ports:
    - protocol: TCP
      port: 11211
      targetPort: 11211
```

This load balancer can be created using `kubectl create -f shard-router-`

`Bộ cân bằng tải này có thể được tạo bằng kubectl create -f shard-router-service.yaml.`

An Examination of Sharding Functions — Khảo sát về các hàm phân mảnh

So far we've discussed the design and deployment of both simple sharded and replicated sharded caches, but we haven't spent very much time considering how traffic is routed to different shards. Consider a sharded service where you have 10 independent shards. Given some specific user request `Req`, how do you determine which shard `S` in the range from zero to nine should be used for the request? This mapping is the responsibility of the sharding **function**. A sharding function is very similar to a hashing function, which you may have encountered when learning about hashtable data structures. Indeed, a bucket-based hashtable could be considered an example of a sharded service. Given both `Req` and `Shard`, then the role of the sharding function is to relate them together, specifically:

Cho đến giờ chúng ta đã thảo luận về thiết kế và triển khai cả cache phân mảnh đơn giản lẫn cache phân mảnh nhân bản, nhưng chúng ta chưa dành nhiều thời gian xem xét cách lưu lượng được định tuyến đến các mảnh khác nhau. Hãy xét một dịch vụ phân mảnh nơi bạn có 10 mảnh độc lập. Cho một yêu cầu cụ thể của người dùng `Req`, làm sao bạn xác định mảnh `S` nào trong khoảng từ không đến chín nên được dùng cho yêu cầu? Việc ánh xạ này là trách nhiệm của hàm phân mảnh (sharding function). Một hàm phân mảnh rất giống với một hàm băm (hashing function), thứ mà bạn có thể đã gặp khi học về các cấu trúc dữ liệu bảng băm (hashtable). Thực vậy, một bảng băm dựa trên thùng (bucket) có thể được coi là một ví dụ về dịch vụ phân mảnh. Cho cả `Req` và `Shard`, vai trò của hàm phân mảnh là liên hệ chúng với nhau, cụ thể:

`www.allitebooks.com Shard = ShardingFunction(Req)`

`www.allitebooks.com Shard = ShardingFunction(Req)`

Commonly, the sharding function is defined using a hashing function and the modulo (%) operator. Hashing functions are functions that transform an arbitrary object into an integer hash. The hash function has two important characteristics for our sharding:

Thông thường, hàm phân mảnh được định nghĩa bằng cách dùng một hàm băm và toán tử lấy phần dư (%). Các hàm băm là những hàm biến đổi một đối tượng tùy ý thành một số nguyên gọi là giá trị băm (hash). Hàm băm có hai đặc tính quan trọng cho việc phân mảnh của chúng ta:

Determinism The output should always be the same for a unique input.

Tính tất định (Determinism) Đầu ra phải luôn giống nhau cho một đầu vào duy nhất.

Uniformity The distribution of outputs across the output space should be equal.

Tính đồng đều (Uniformity) Phân bố của các đầu ra trên không gian đầu ra phải đồng đều.

For our sharded service, determinism and uniformity are the most important characteristics. Determinism is important because it ensures that a particular request R always goes to the same shard in the service. Uniformity is important because it ensures that load is evenly spread between the different shards.

Đối với dịch vụ phân mảnh của chúng ta, tính tất định và tính đồng đều là những đặc tính quan trọng nhất. Tính tất định quan trọng vì nó đảm bảo rằng một yêu cầu R cụ thể luôn đi đến cùng một mảnh trong dịch vụ. Tính đồng đều quan trọng vì nó đảm bảo rằng tải được trải đều giữa các mảnh khác nhau.

Fortunately for us, modern programming languages include a wide variety of high-quality hash functions. However, the outputs of these hash functions are often significantly larger than the number of shards in a sharded service. Consequently, we use the modulo operator (%) to **reduce** a hash function to the appropriate range. Returning to our sharded service with 10 shards, we can see that we can define our sharding function as:

May mắn cho chúng ta, các ngôn ngữ lập trình hiện đại bao gồm rất nhiều hàm băm chất lượng cao. Tuy nhiên, đầu ra của các hàm băm này thường lớn hơn đáng kể so với số mảnh trong một dịch vụ phân mảnh. Do đó, chúng ta dùng toán tử lấy phần dư (%) để thu nhỏ một hàm băm về khoảng thích hợp. Quay lại dịch vụ phân mảnh với 10 mảnh của chúng ta, chúng ta có thể thấy rằng có thể định nghĩa hàm phân mảnh của mình là:

`Shard = hash(Req) % 10`

`Shard = hash(Req) % 10`

If the output of the hash function has the appropriate properties in terms of determinism and uniformity, those properties will be preserved by the modulo operator.

Nếu đầu ra của hàm băm có các tính chất thích hợp về tính tất định và tính đồng đều, những tính chất đó sẽ được toán tử lấy phần dư bảo toàn.

Selecting a Key — Chọn khóa

Given this sharding function, it might be tempting to simply use the hashing function that is built into the programming language, hash the entire object, and call it a day. The result of this, however,

will not be a very good sharding function.

Với hàm phân mảnh này, có thể bạn sẽ bị cám dỗ chỉ đơn giản dùng hàm băm có sẵn trong ngôn ngữ lập trình, băm toàn bộ đối tượng, và coi như xong. Tuy nhiên, kết quả của việc này sẽ không phải là một hàm phân mảnh tốt.

To understand this, consider a simple HTTP request that contains three things:

Để hiểu điều này, hãy xét một yêu cầu HTTP đơn giản chứa ba thứ:

- The time of the request
- The source IP address from the client
- The HTTP request path (e.g., /some/page.html)
 - Thời điểm của yêu cầu
 - Địa chỉ IP nguồn từ phía client
 - Đường dẫn yêu cầu HTTP (ví dụ: /some/page.html)

If we use a simple object-based hashing function, `shard ( request )`, then it is clear that `{12:00, 1.2.3.4, /some/file.html}` has a different shard value than `{12:01, 5.6.7.8, /some/file.html}`. The output of the sharding function is different because the client's IP address and the time of the request are different between the two requests. But of course, in most cases, the IP address of the client and the time of the request don't impact the response to the HTTP request. Consequently, instead of hashing the entire request object, a much better sharding function would be `shard ( request.path )`. When we use `request.path` as the shard key, then we **map** both requests to the same shard, and thus the response to one request can be served out of the cache to service the other.

Nếu chúng ta dùng một hàm băm dựa trên đối tượng đơn giản, `shard(request)`, thì rõ ràng `{12:00, 1.2.3.4, /some/file.html}` có giá trị mảnh khác với `{12:01, 5.6.7.8, /some/file.html}`. Đầu ra của hàm phân mảnh khác nhau vì địa chỉ IP của client và thời điểm của yêu cầu khác nhau giữa hai yêu cầu. Nhưng tất nhiên, trong hầu hết trường hợp, địa chỉ IP của client và thời điểm của yêu cầu không tác động đến phản hồi cho yêu cầu HTTP. Do đó, thay vì băm toàn bộ đối tượng yêu cầu, một hàm phân mảnh tốt hơn nhiều sẽ là `shard(request.path)`. Khi chúng ta dùng `request.path` làm khóa phân mảnh, chúng ta ánh xạ cả hai yêu cầu đến cùng một mảnh, và như vậy phản hồi cho một yêu cầu có thể được phục vụ từ cache để đáp ứng yêu cầu kia.

Of course, sometimes client IP is important to the response that is returned from the frontend. For example, client IP may be used to look up the geographic region that the user is located in, and different content (e.g., different languages) may be returned to different IP addresses. In such cases, the previous sharding function `shard ( request.path )` will actually result in errors, since a cache request from a French IP address may be served a result page from the cache in English. In such cases, the cache function is too general , as it groups together requests that do not have identical responses.

Tất nhiên, đôi khi IP của client lại quan trọng đối với phản hồi được trả về từ frontend. Ví dụ, IP của client có thể được dùng để tra cứu vùng địa lý mà người dùng đang ở, và nội dung khác nhau (ví dụ: các ngôn ngữ khác nhau) có thể được trả về cho các địa chỉ IP khác nhau. Trong những trường hợp như vậy, hàm phân mảnh trước đó `shard(request.path)` sẽ thực sự dẫn đến lỗi, vì một yêu cầu cache từ một địa chỉ IP của Pháp có thể được phục vụ một trang kết quả từ cache bằng tiếng Anh. Trong những trường hợp như vậy, hàm cache quá tổng quát, vì nó gộp chung những yêu cầu không có phản hồi giống hệt nhau.

Given this problem, it would be tempting then to define our sharding function as `shard ( request.ip, request.path )`, but this sharding function has problems as well. It will cause two different French IP

addresses to map to different shards, thus resulting in inefficient sharding. This shard function is too specific, as it fails to group together requests that are identical. A better sharding function for this situation would be:

Trước vấn đề này, có thể bạn sẽ bị cám dỗ định nghĩa hàm phân mảnh của mình là `shard(request.ip, request.path)`, nhưng hàm phân mảnh này cũng có vấn đề. Nó sẽ khiến hai địa chỉ IP khác nhau của Pháp ánh xạ đến các mảnh khác nhau, từ đó dẫn đến việc phân mảnh kém hiệu quả. Hàm phân mảnh này quá cụ thể, vì nó không gộp chung được những yêu cầu giống hệt nhau. Một hàm phân mảnh tốt hơn cho tình huống này sẽ là:

```
shard(country(request.ip), request.path)
```

This first determines the country from the IP address, and then uses that country as part of the key for the sharding function. Thus multiple requests from France will be routed to one shard, while requests from the United States will be routed to a different shard.

Hàm này trước tiên xác định quốc gia từ địa chỉ IP, rồi dùng quốc gia đó như một phần của khóa cho hàm phân mảnh. Như vậy nhiều yêu cầu từ Pháp sẽ được định tuyến đến một mảnh, trong khi các yêu cầu từ Hoa Kỳ sẽ được định tuyến đến một mảnh khác.

Determining the appropriate key for your sharding function is vital to designing your sharded system well. Determining the correct shard key requires an understanding of the requests that you expect to see.

Việc xác định khóa thích hợp cho hàm phân mảnh của bạn là yếu tố sống còn để thiết kế tốt hệ thống phân mảnh của mình. Việc xác định khóa phân mảnh đúng đòi hỏi sự hiểu biết về các yêu cầu mà bạn kỳ vọng sẽ gặp.

Consistent Hashing Functions — Các hàm băm nhất quán (Consistent Hashing Functions)

Setting up the initial shards for a new service is relatively straightforward: you set up the appropriate shards and the roots to perform the sharding, and you are off to the races. However, what happens when you need to change the number of shards in your sharded service? Such “**re-sharding**” is often a complicated process.

Việc thiết lập các mảnh ban đầu cho một dịch vụ mới tương đối đơn giản: bạn thiết lập các mảnh thích hợp và các gốc (root) để thực hiện việc phân mảnh, và bạn đã sẵn sàng vào cuộc. Tuy nhiên, điều gì xảy ra khi bạn cần thay đổi số mảnh trong dịch vụ phân mảnh của mình? Việc “tái phân mảnh” (re-sharding) như vậy thường là một quá trình phức tạp.

To understand why this is true, consider the sharded cache previously examined. Certainly, scaling the cache from 10 to 11 replicas is straightforward to do with a **container orchestrator**, but consider the effect of changing the scaling function from `hash(Req) % 10` to `hash(Req) % 11`. When you deploy this new scaling function, a large number of requests are going to be mapped to a different shard than the one they were previously mapped to. In a sharded cache, this is going to dramatically increase your miss rate until the cache is repopulated with responses for the new requests that have been mapped to that cache shard by the new sharding function. In the worst case, rolling out a new sharding function for your sharded cache will be equivalent to a complete cache failure.

Để hiểu vì sao điều này đúng, hãy xét cache phân mảnh đã khảo sát trước đó. Chắc chắn, việc mở rộng cache từ 10 lên 11 bản sao là đơn giản với một bộ điều phối container, nhưng hãy xét tác động của việc thay đổi hàm mở rộng từ `hash(Req) % 10` thành `hash(Req) % 11`. Khi bạn triển khai hàm mở rộng mới này, một số lượng lớn yêu cầu sẽ được ánh xạ đến

một mảnh khác với mảnh mà chúng từng được ánh xạ trước đây. Trong một cache phân mảnh, điều này sẽ làm tăng đột ngột tỷ lệ trượt của bạn cho đến khi cache được nạp lại với các phản hồi cho những yêu cầu mới đã được hàm phân mảnh mới ánh xạ đến mảnh cache đó. Trong trường hợp tệ nhất, việc tung ra một hàm phân mảnh mới cho cache phân mảnh của bạn sẽ tương đương với một sự cố hỏng cache hoàn toàn.

To resolve these kinds of problems, many sharding functions use **consistent hashing** functions . Consistent hashing functions are special hash functions that are guaranteed to only remap # keys / # shards , when being resized to # shards . For example, if we use a consistent hashing function for our sharded cache, moving from 10 to 11 shards will only result in remapping < 10% (K / 11) keys. This is dramatically better than losing the entire sharded service.

Để giải quyết những loại vấn đề này, nhiều hàm phân mảnh dùng các hàm băm nhất quán (consistent hashing functions). Các hàm băm nhất quán là những hàm băm đặc biệt được đảm bảo chỉ ánh xạ lại # khóa / # mảnh, khi được thay đổi kích thước về # mảnh. Ví dụ, nếu chúng ta dùng một hàm băm nhất quán cho cache phân mảnh của mình, việc chuyển từ 10 lên 11 mảnh sẽ chỉ dẫn đến việc ánh xạ lại < 10% (K / 11) số khóa. Điều này tốt hơn rất nhiều so với việc mất toàn bộ dịch vụ phân mảnh.

Hands On: Building a Consistent HTTP Sharding Proxy — Thực hành (Hands On): Xây dựng một Proxy phân mảnh HTTP nhất quán

To shard HTTP requests, the first question to answer is what to use as the key for the sharding function. Though there are several options, a good general-purpose key is the request path as well as the fragment and query parameters (i.e., everything that makes the request unique). Note that this does not include cookies from the user or the language/location (e.g., EN_US). If your service provides extensive customization to users or their location, you will need to include them in the hash key as well.

Để phân mảnh các yêu cầu HTTP, câu hỏi đầu tiên cần trả lời là dùng gì làm khóa cho hàm phân mảnh. Mặc dù có một số lựa chọn, một khóa đa dụng tốt là đường dẫn yêu cầu cùng với phần fragment và các tham số query (tức là mọi thứ làm cho yêu cầu trở nên duy nhất). Lưu ý rằng điều này không bao gồm cookie từ người dùng hay ngôn ngữ/vị trí (ví dụ: EN_US). Nếu dịch vụ của bạn cung cấp khả năng tùy biến sâu rộng cho người dùng hoặc vị trí của họ, bạn sẽ cần đưa chúng vào khóa băm nữa.

We can use the versatile **nginx** HTTP server for our sharding **proxy**.

Chúng ta có thể dùng máy chủ HTTP đa năng nginx cho proxy phân mảnh của mình.

```
worker_processes 5;
error_log error.log;
pid nginx.pid;
worker_rlimit_nofile 8192;
events {
    worker_connections 1024;
}
http {
    # define a named 'backend' that we can use in the proxy directive
    # below.
```

```

upstream backend {
    # Has the full URI of the request and use a consistent hash
    hash $request_uri consistent
    server web-shard-1.web;
    server web-shard-2.web;
    server web-shard-3.web;
}
server {
    listen localhost:80;
    location / {
        proxy_pass http://backend;
    }
}

```

Note that we chose to use the full request URI as the key for the hash and use the key word `consistent` to indicate that we want to use a consistent hashing function.

Lưu ý rằng chúng ta đã chọn dùng toàn bộ URI yêu cầu làm khóa cho phép băm và dùng từ khóa `consistent` để chỉ định rằng chúng ta muốn dùng một hàm băm nhất quán.

Sharded, Replicated Serving — Phục vụ phân mảnh, nhân bản

Most of the examples in this chapter so far have described sharding in terms of cache serving. But, of course, caches are not the only kinds of services that can benefit from sharding. Sharding is useful when considering any sort of service where there is more data than can fit on a single machine. In contrast to previous examples, the key and sharding function are not a part of the HTTP request, but rather some context for the user.

Phần lớn các ví dụ trong chương này cho đến giờ đã mô tả việc phân mảnh dưới góc độ phục vụ cache. Nhưng tất nhiên, cache không phải là loại dịch vụ duy nhất có thể hưởng lợi từ việc phân mảnh. Phân mảnh hữu ích khi xét đến bất kỳ loại dịch vụ nào mà ở đó có nhiều dữ liệu hơn mức một máy đơn lẻ có thể chứa. Trái với các ví dụ trước, khóa và hàm phân mảnh không phải là một phần của yêu cầu HTTP, mà là một ngữ cảnh nào đó của người dùng.

For example, consider implementing a large-scale multi-player game. Such a game world is likely to be far too large to fit on a single machine. However, players who are distant from each other in this virtual world are unlikely to interact. Consequently, the world of the game can be sharded across many different machines. The sharding function is keyed off of the player's location so that all players in a particular location land on the same set of servers.

Ví dụ, hãy xét việc triển khai một trò chơi nhiều người chơi quy mô lớn. Một thế giới trò chơi như vậy nhiều khả năng quá lớn để vừa trong một máy đơn lẻ. Tuy nhiên, những người chơi ở xa nhau trong thế giới ảo này nhiều khả năng sẽ không tương tác với nhau. Do đó, thế giới của trò chơi có thể được phân mảnh trên nhiều máy khác nhau. Hàm phân mảnh được khóa theo vị trí của người chơi sao cho tất cả người chơi ở một vị trí cụ thể đáp xuống cùng một tập máy chủ.

Hot Sharding Systems — Các hệ thống phân mảnh nóng (Hot Sharding)

Ideally the load on a sharded cache will be perfectly even, but in many cases this isn't true and "hot shards" appear because organic load patterns drive more traffic to one particular shard.

Lý tưởng nhất thì tải lên một cache phân mảnh sẽ hoàn toàn đồng đều, nhưng trong nhiều trường hợp điều này không đúng và các "mảnh nóng" (hot shard) xuất hiện vì các mẫu hình tải tự nhiên dồn nhiều lưu lượng hơn đến một mảnh cụ thể.

As an example of this, consider a sharded cache for a user's photos; when a particular photo goes viral and suddenly receives a disproportionate amount of traffic, the cache shard containing that photo will become "hot." When this happens, with a replicated, sharded cache, you can scale the cache shard to respond to the increased load. Indeed, if you set up autoscaling for each cache shard, you can dynamically grow and shrink each replicated shard as the organic traffic to your service shifts around. An illustration of this process is shown in Figure 6-3 . Initially the sharded service receives equal traffic to all three shards. Then the traffic shifts so that Shard A is receiving four times as much traffic as Shard B and Shard C. The hot sharding system moves Shard B to the same machine as Shard C, and replicates Shard A to a second machine. Traffic is now, once again, equally shared between replicas.

Như một ví dụ về điều này, hãy xét một cache phân mảnh cho ảnh của người dùng; khi một bức ảnh cụ thể trở nên lan truyền (viral) và đột nhiên nhận một lượng lưu lượng không cân xứng, mảnh cache chứa bức ảnh đó sẽ trở nên "nóng". Khi điều này xảy ra, với một cache phân mảnh, nhân bản, bạn có thể mở rộng mảnh cache để đáp ứng tải tăng. Thực vậy, nếu bạn thiết lập tự động co giãn (autoscaling) cho mỗi mảnh cache, bạn có thể tăng và giảm động mỗi mảnh nhân bản khi lưu lượng tự nhiên đến dịch vụ của bạn dịch chuyển. Một minh họa cho quá trình này được trình bày trong Hình 6-3. Ban đầu dịch vụ phân mảnh nhận lưu lượng bằng nhau đến cả ba mảnh. Sau đó lưu lượng dịch chuyển sao cho Mảnh A nhận lưu lượng gấp bốn lần Mảnh B và Mảnh C. Hệ thống phân mảnh nóng chuyển Mảnh B sang cùng máy với Mảnh C, và nhân bản Mảnh A sang một máy thứ hai. Giờ đây, một lần nữa, lưu lượng được chia đều giữa các bản sao.

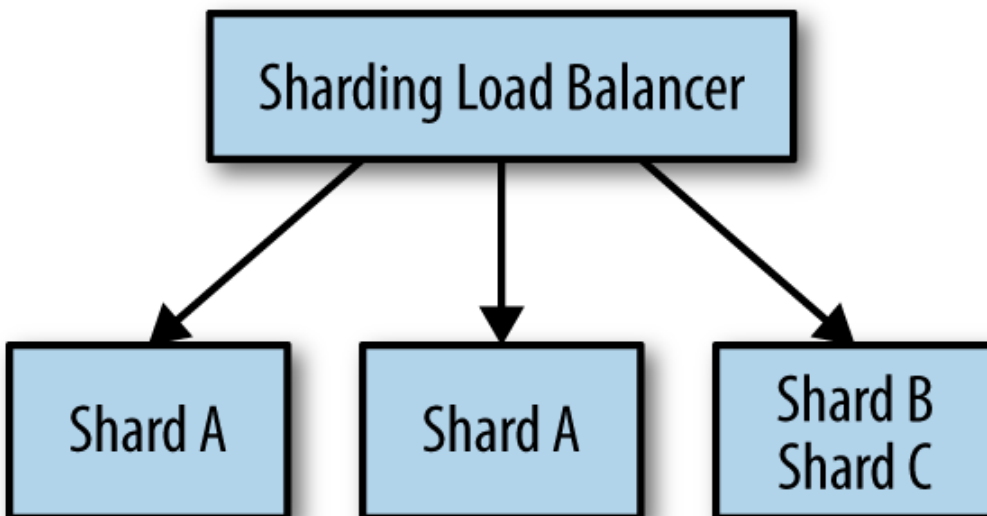
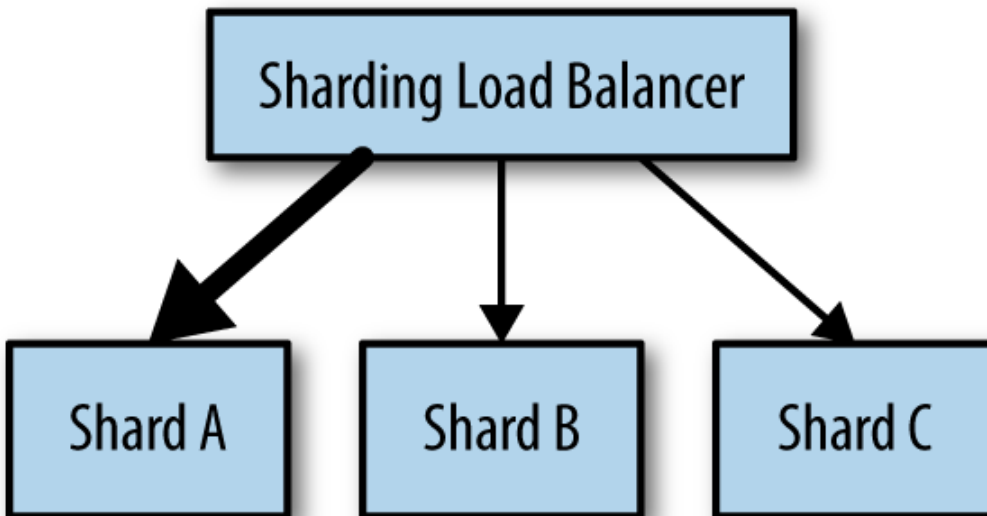
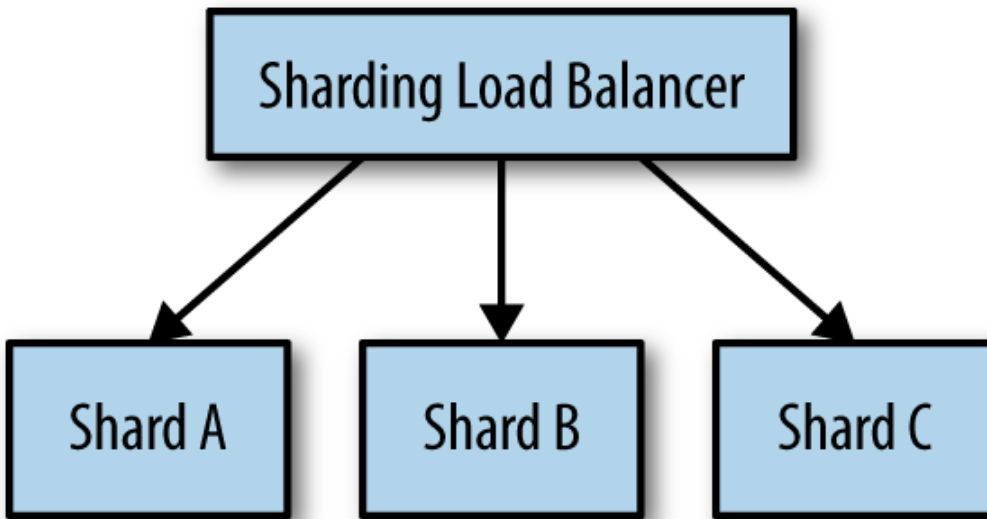


Figure 6-3. An example of a hot sharded system: initially the shards are evenly distributed, but when extra traffic comes to shard A, it is replicated to two machines, and shards B and C are combined on a single machine

Hình 6-3. Một ví dụ về hệ thống phân mảnh nóng: ban đầu các mảnh được phân bố đều, nhưng khi lưu lượng phụ trội đến với mảnh A, nó được nhân bản sang hai máy, và các mảnh B và C được gộp trên một máy đơn lẻ

CHAPTER 7 Scatter/Gather —

CHAPTER 7 Scatter/Gather

So far we've examined systems that replicate for **scalability** in terms of the number of requests processed per second (the **stateless** replicated pattern), as well as scalability for the size of the data (the sharded data pattern). In this chapter we introduce the **scatter/gather** pattern, which uses replication for scalability in terms of time. Specifically, the scatter/gather pattern allows you to achieve parallelism in servicing requests, enabling you to **service** them significantly faster than you could if you had to service them sequentially.

Cho đến giờ chúng ta đã khảo sát các hệ thống nhân bản (replicate) để mở rộng theo số lượng yêu cầu xử lý được mỗi giây (mẫu nhân bản không trạng thái), cũng như mở rộng theo kích thước dữ liệu (mẫu dữ liệu phân mảnh). Trong chương này chúng ta giới thiệu mẫu scatter/gather, vốn dùng nhân bản để mở rộng theo chiều thời gian. Cụ thể, mẫu scatter/gather cho phép bạn đạt được tính song song khi phục vụ các yêu cầu, nhờ đó phục vụ chúng nhanh hơn đáng kể so với khi phải xử lý tuần tự.

Like replicated and sharded systems, the scatter/gather pattern is a tree pattern with a root that distributes requests and leaves that process those requests. However, in contrast to replicated and sharded systems, with scatter/gather requests are simultaneously farmed out to all of the replicas in the system. Each **replica** does a small amount of processing and then returns a fraction of the result to the root. The root server then combines the various partial results together to form a single complete response to the request and then sends this request back out to the client. The scatter/gather pattern is illustrated in Figure 7-1 .

Giống như các hệ thống nhân bản và phân mảnh, mẫu scatter/gather là một mẫu dạng cây với một gốc (root) phân phối các yêu cầu và các lá (leaf) xử lý những yêu cầu đó. Tuy nhiên, khác với các hệ thống nhân bản và phân mảnh, với scatter/gather thì các yêu cầu được phân phát đồng thời tới tất cả các bản sao (replica) trong hệ thống. Mỗi bản sao xử lý một lượng nhỏ rồi trả về một phần kết quả cho gốc. Sau đó máy chủ gốc kết hợp các kết quả thành phần khác nhau lại với nhau để tạo thành một phản hồi hoàn chỉnh duy nhất cho yêu cầu, rồi gửi phản hồi này trở lại cho client. Mẫu scatter/gather được minh họa trong Hình 7-1.

Scatter/gather is quite useful when you have a large amount of mostly independent processing that is needed to handle a particular request. Scatter/gather can be seen as sharding the computation necessary to service the request, rather than sharding the data (although data sharding may be part of it as well).

Scatter/gather khá hữu ích khi bạn có một lượng lớn công việc xử lý phần lớn là độc lập cần thực hiện để phục vụ một yêu cầu cụ thể. Có thể xem scatter/gather như việc phân mảnh phần tính toán cần thiết để phục vụ yêu cầu, thay vì phân mảnh dữ liệu (mặc dù

phân mảnh dữ liệu cũng có thể là một phần của nó).

73

73

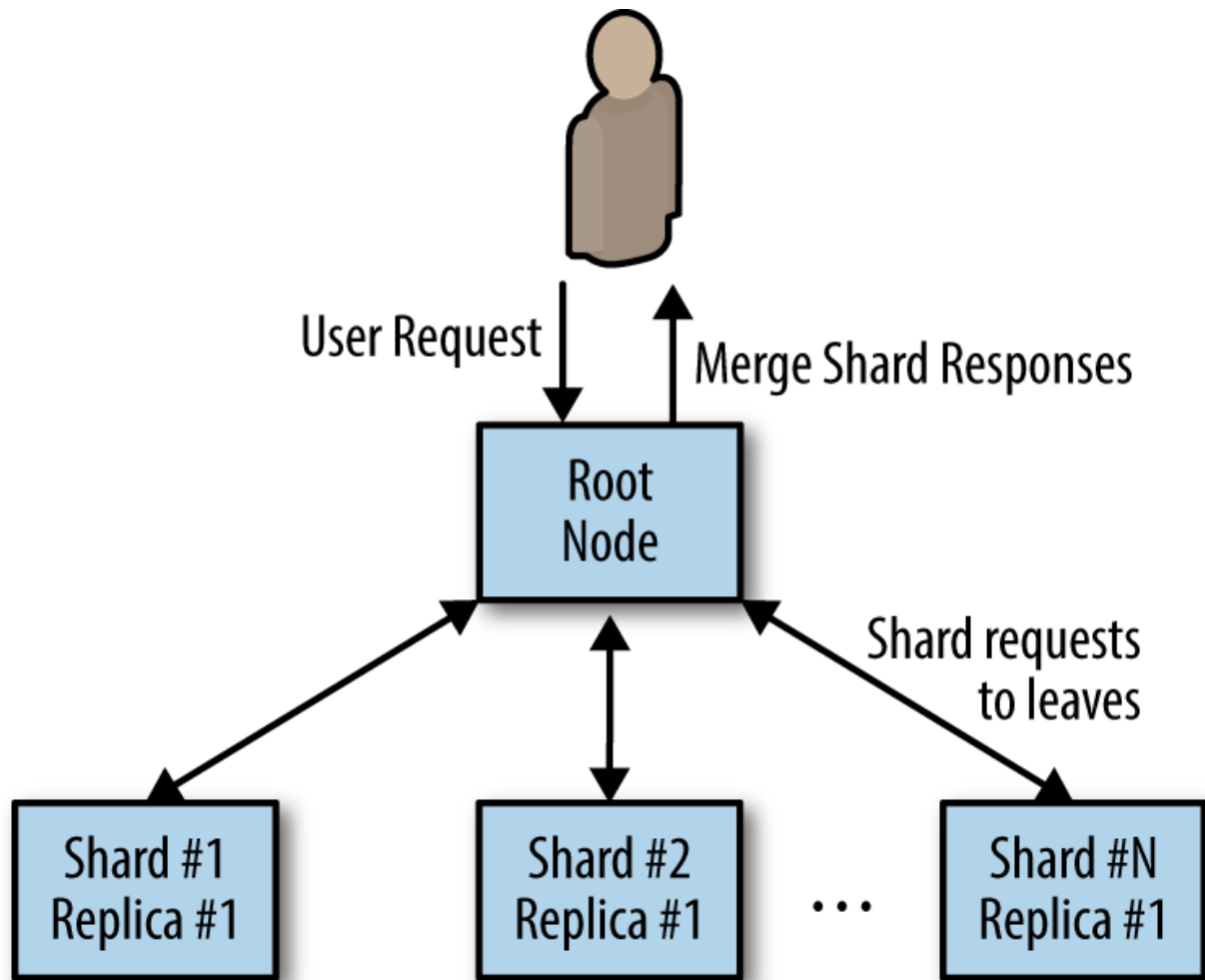


Figure 7-1. A scatter/gather pattern

Hình 7-1. Một mẫu scatter/gather

Scatter/Gather with Root Distribution — Scatter/Gather với Phân phối tại Gốc

The simplest form of scatter/gather is one in which each leaf is entirely homogenous but the work is distributed to a number of different leaves in order to improve the performance of the request. This pattern is equivalent to solving an “embarrassingly parallel” problem. The problem can be broken up into many different pieces and each piece can be put back together with all of the other pieces to form a complete answer.

Dạng đơn giản nhất của scatter/gather là dạng trong đó mỗi lá hoàn toàn đồng nhất nhưng công việc được phân phối tới nhiều lá khác nhau nhằm cải thiện hiệu năng của yêu cầu.

Mẫu này tương đương với việc giải một bài toán “song song hóa dễ dàng” (embarrassingly parallel). Bài toán có thể được chia thành nhiều phần khác nhau và mỗi phần có thể được ghép lại cùng tất cả các phần khác để tạo thành một câu trả lời hoàn chỉnh.

To understand this in more concrete terms, imagine that you need to service a user request *R* and it takes one minute for a single core to produce the answer *A* to this request. If we program a multi-threaded application, we can parallelize this request on a single machine by using multiple cores. Given this approach and a 30 core processor (yes, typically it would be a 32 core processor, but 30 makes the math cleaner), we can **reduce** the time that it takes to process a single request down to 2 seconds (60 seconds of computation split across 30 threads for computation is equal to 2 seconds). But even two seconds is pretty slow to service a user’s web request. Additionally, truly achieving a completely parallel speed up on a single process is going to be tricky as things like memory, network, or disk bandwidth start to become the bottleneck. Instead of parallelizing an application across cores on a single machine, we can use the scatter/gather pattern to parallelize requests across multiple processes on many different machines. In this way, we can improve our overall **latency** requests, since we are no longer bound by the number of cores we can get on a single machine, as well as ensure that the bottleneck in our process continues to be CPU, since the memory, network, and disk bandwidth are all spread across a number of different machines. Additionally, because every machine in the scatter/gather tree is capable of handling every request, the root of the tree can dynamically dispatch load to different nodes at different times depending on their responsiveness. If, for some reason, a particular leaf **node** is responding more slowly than other machines (e.g., it has a noisy neighbor process that is interfering with resources), then the root can dynamically redistribute load to assure a fast response.

Để hiểu điều này một cách cụ thể hơn, hãy hình dung rằng bạn cần phục vụ một yêu cầu người dùng *R* và phải mất một phút để một lõi (core) đơn lẻ tạo ra câu trả lời *A* cho yêu cầu này. Nếu chúng ta lập trình một ứng dụng đa luồng, ta có thể song song hóa yêu cầu này trên một máy duy nhất bằng cách dùng nhiều lõi. Với cách tiếp cận này và một bộ xử lý 30 lõi (vâng, thông thường sẽ là bộ xử lý 32 lõi, nhưng 30 làm phép tính gọn hơn), ta có thể giảm thời gian xử lý một yêu cầu xuống còn 2 giây (60 giây tính toán chia đều cho 30 luồng tính toán bằng 2 giây). Nhưng kể cả hai giây vẫn khá chậm để phục vụ một yêu cầu web của người dùng. Hơn nữa, việc thực sự đạt được tăng tốc song song hoàn toàn trên một tiến trình đơn lẻ sẽ rất khó khăn vì những thứ như bộ nhớ, mạng, hay băng thông đĩa bắt đầu trở thành nút thắt cổ chai. Thay vì song song hóa một ứng dụng trên nhiều lõi của một máy duy nhất, ta có thể dùng mẫu scatter/gather để song song hóa các yêu cầu trên nhiều tiến trình ở nhiều máy khác nhau. Theo cách này, ta có thể cải thiện độ trễ tổng thể của các yêu cầu, vì ta không còn bị giới hạn bởi số lõi có thể có trên một máy duy nhất, đồng thời đảm bảo rằng nút thắt cổ chai trong tiến trình của ta vẫn là CPU, vì bộ nhớ, mạng và băng thông đĩa đều được trải đều trên nhiều máy khác nhau. Ngoài ra, vì mỗi máy trong cây scatter/gather đều có khả năng xử lý mọi yêu cầu, gốc của cây có thể điều phối tải một cách động tới các nút khác nhau vào những thời điểm khác nhau tùy theo mức độ phản hồi của chúng. Nếu vì lý do nào đó một nút lá cụ thể phản hồi chậm hơn các máy khác (ví dụ, nó có một tiến trình “hàng xóm ồn ào” đang can thiệp vào tài nguyên), thì gốc có thể phân phối lại tải một cách động để đảm bảo phản hồi nhanh.

Hands On: Distributed Document Search — Thực hành (Hands On): Tìm kiếm Tài liệu Phân tán

To see an example of scatter/gather in action, consider the task of searching across a large database of documents for all documents that contain the words “cat” and “dog.” One way to perform this search

would be to open up all of the documents, read through the entire set, searching for the words in each document, and then return to the user the set of documents that contain both words.

Để thấy một ví dụ về scatter/gather trong thực tế, hãy xét tác vụ tìm kiếm trong một cơ sở dữ liệu lớn gồm các tài liệu để tìm tất cả tài liệu chứa các từ “cat” và “dog”. Một cách thực hiện việc tìm kiếm này là mở tất cả các tài liệu, đọc qua toàn bộ tập, tìm các từ trong mỗi tài liệu, rồi trả về cho người dùng tập các tài liệu chứa cả hai từ.

As you might imagine, this is quite a slow process because it requires opening and reading through a large number of files for each request. To make request processing faster, you can build an index . The index is effectively a hashtable, where the keys are individual words (e.g., “cat”) and the values are a list of documents containing that word.

Như bạn có thể hình dung, đây là một quá trình khá chậm vì nó đòi hỏi phải mở và đọc qua một số lượng lớn tệp cho mỗi yêu cầu. Để xử lý yêu cầu nhanh hơn, bạn có thể xây dựng một chỉ mục (index). Chỉ mục thực chất là một bảng băm (hashtable), trong đó các khóa là từng từ riêng lẻ (ví dụ, “cat”) và các giá trị là một danh sách các tài liệu chứa từ đó.

Now, instead of searching through every document, finding the documents that match any one word is as easy as doing a lookup in this hashtable. However, we have lost one important ability. Remember that we were looking for all documents that contained “cat” and “dog.” Since the index only has single words, not conjunctions of words, we still need to find the documents that contain both words. Luckily, this is just an intersection of the sets of documents returned for each word.

Giờ đây, thay vì tìm kiếm qua mọi tài liệu, việc tìm các tài liệu khớp với một từ bất kỳ đơn giản như thực hiện một lần tra cứu trong bảng băm này. Tuy nhiên, ta đã đánh mất một khả năng quan trọng. Hãy nhớ rằng chúng ta đang tìm tất cả các tài liệu chứa cả “cat” và “dog”. Vì chỉ mục chỉ có các từ đơn, không phải các kết hợp từ, ta vẫn cần tìm các tài liệu chứa cả hai từ. May thay, đây chỉ là phép giao của các tập tài liệu được trả về cho mỗi từ.

Given this approach, we can implement this document search as an example of the scatter/gather pattern. When a request comes in to the document search root, it parses the request and farms out two leaf machines (one for the word “cat” and one for the word “dog”). Each of these machines returns a list of documents that match one of the words, and the root node returns the list of documents containing both “cat” and “dog.”

Với cách tiếp cận này, ta có thể triển khai việc tìm kiếm tài liệu này như một ví dụ của mẫu scatter/gather. Khi một yêu cầu đến gốc tìm kiếm tài liệu, nó phân tích yêu cầu và phân phát tới hai máy lá (một cho từ “cat” và một cho từ “dog”). Mỗi máy này trả về một danh sách các tài liệu khớp với một trong hai từ, và nút gốc trả về danh sách các tài liệu chứa cả “cat” và “dog”.

A diagram of this process is shown in Figure 7-2 : the leaf returns {doc1, doc2, doc4} for “cat” and {doc1, doc3, doc4} for “dog,” so the root finds the intersection and returns {doc1, doc4} .

Một sơ đồ của quá trình này được thể hiện trong Hình 7-2: lá trả về {doc1, doc2, doc4} cho “cat” và {doc1, doc3, doc4} cho “dog”, nên gốc tìm phép giao và trả về {doc1, doc4}.

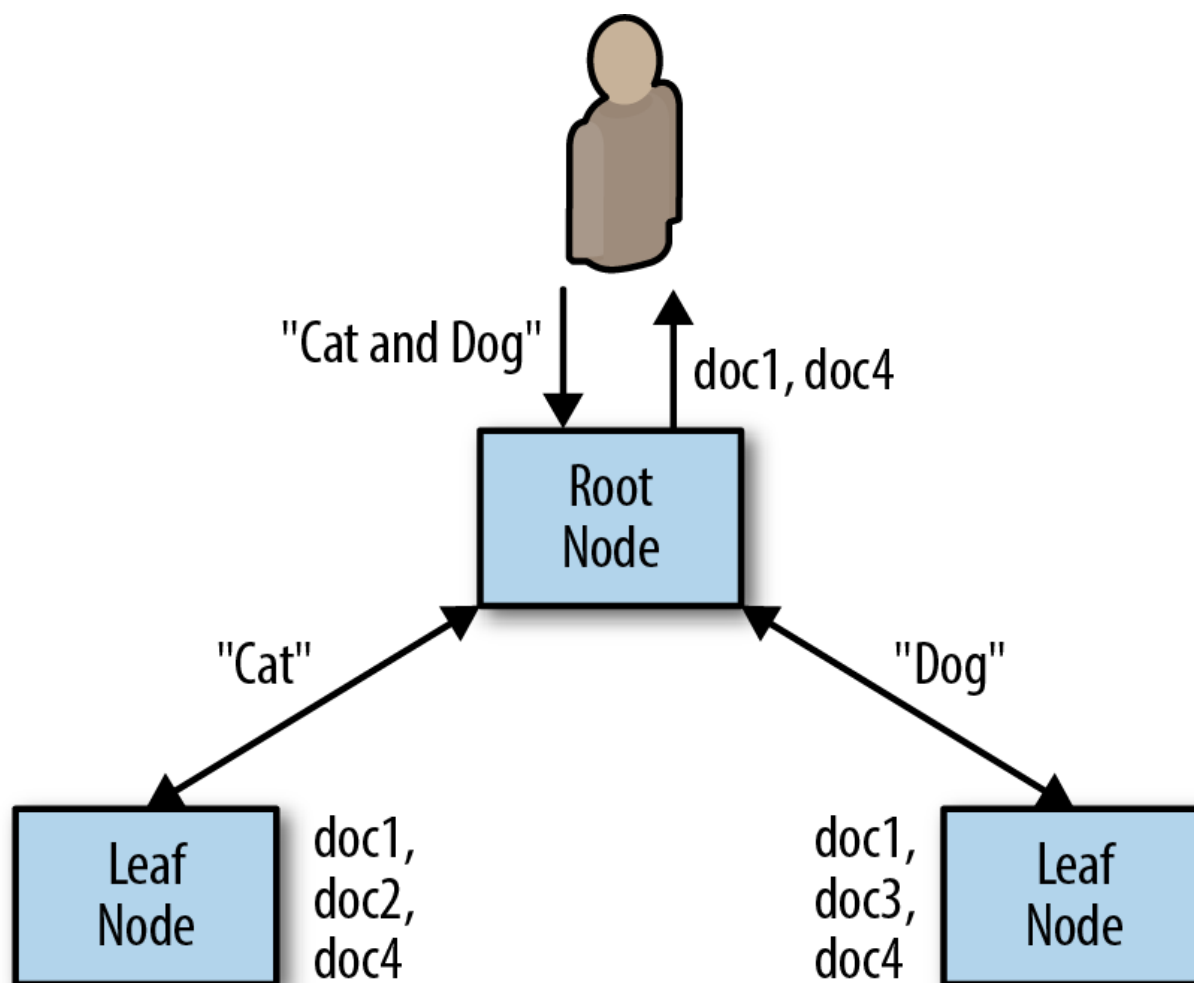


Figure 7-2. Example of a term-sharded scatter/gather system

Hình 7-2. Ví dụ về một hệ thống scatter/gather phân mảnh theo từ khóa

Scatter/Gather with Leaf Sharding — Scatter/Gather với Phân mảnh tại Lá

While applying the replicated data scatter/gather pattern allows you to reduce the processing time required for handling user requests, it doesn't allow you to scale beyond an amount of data that can be held in the memory or disk of a single machine. Much like the replicated serving pattern that was previously described, it is simple to build a replicated scatter/gather system. But at a certain data size, it is necessary to introduce sharding in order to build a system that can hold more data than can be stored on a single machine.

Mặc dù việc áp dụng mẫu scatter/gather với dữ liệu nhân bản cho phép bạn giảm thời gian xử lý cần thiết để phục vụ các yêu cầu người dùng, nó không cho phép bạn mở rộng vượt quá lượng dữ liệu có thể chứa trong bộ nhớ hoặc đĩa của một máy duy nhất. Tương tự như mẫu phục vụ nhân bản đã mô tả trước đó, việc xây dựng một hệ thống scatter/gather nhân bản là đơn giản. Nhưng đến một kích thước dữ liệu nhất định, cần phải đưa vào phân mảnh để xây dựng một hệ thống có thể chứa nhiều dữ liệu hơn mức có thể lưu trên

một máy duy nhất.

Previously, when sharding was introduced to scale replicated systems, the sharding was done at a per-request level. Some part of the request was used to determine where the request was sent. That replica then handled all of the processing for the request and the response was handed back to the user. Instead, with scatter/gather sharding, the request is sent to all of the leaf nodes (or shards) in the system. Each leaf node processes the request using the data that it has loaded in its **shard**. This partial response is then returned to the root node that requested data, and that root node merges all of the responses together to form a comprehensive response for the user.

Trước đây, khi phân mảnh được đưa vào để mở rộng các hệ thống nhân bản, việc phân mảnh được thực hiện ở mức từng yêu cầu. Một phần nào đó của yêu cầu được dùng để xác định yêu cầu được gửi đi đâu. Bản sao đó sau đó xử lý toàn bộ việc xử lý cho yêu cầu và phản hồi được trả lại cho người dùng. Ngược lại, với phân mảnh scatter/gather, yêu cầu được gửi tới tất cả các nút lá (hay các mảnh) trong hệ thống. Mỗi nút lá xử lý yêu cầu bằng dữ liệu đã nạp trong mảnh của nó. Phản hồi thành phần này sau đó được trả về cho nút gốc đã yêu cầu dữ liệu, và nút gốc đó gộp tất cả các phản hồi lại với nhau để tạo thành một phản hồi toàn diện cho người dùng.

As a concrete example of this sort of architecture, consider implementing search across a very large document set (all patents in the world, for example); in such a case, the data is too large to fit in the memory of a single machine, so instead the data is sharded across multiple replicas. For example, patents 0-100,000 might be on the first machine, 100,001-200,000 on the next machine, and so forth. (Note that this is not actually a good sharding scheme since it will continually force us to add new shards as new patents are registered. In practice, we'd likely use the patent number modulo the total number of shards.) When a user submits a request to find a particular word (e.g., “rockets”) in all of the patents in the index, that request is sent to each shard, which searches through its patent shard for patents which match the word in the query. Any matches that are found are returned to the root node in response to the shard request. The root node then collates all of these responses together into a single response that contains all the patents that match the particular word. The operation of this search index is illustrated in Figure 7-3 .

Như một ví dụ cụ thể về kiểu kiến trúc này, hãy xét việc triển khai tìm kiếm trên một tập tài liệu rất lớn (chẳng hạn, tất cả bằng sáng chế trên thế giới); trong trường hợp đó, dữ liệu quá lớn để vừa trong bộ nhớ của một máy duy nhất, nên thay vào đó dữ liệu được phân mảnh trên nhiều bản sao. Ví dụ, các bằng sáng chế 0-100.000 có thể nằm trên máy đầu tiên, 100.001-200.000 trên máy tiếp theo, v.v. (Lưu ý rằng đây thực ra không phải một sơ đồ phân mảnh tốt vì nó sẽ liên tục buộc ta phải thêm mảnh mới khi có bằng sáng chế mới được đăng ký. Trong thực tế, ta có lẽ sẽ dùng số bằng sáng chế chia lấy dư cho tổng số mảnh.) Khi một người dùng gửi yêu cầu tìm một từ cụ thể (ví dụ, “rockets”) trong tất cả các bằng sáng chế trong chỉ mục, yêu cầu đó được gửi tới mỗi mảnh, và mỗi mảnh tìm qua mảnh bằng sáng chế của nó để tìm các bằng sáng chế khớp với từ trong truy vấn. Bất kỳ kết quả khớp nào tìm được đều được trả về nút gốc để đáp lại yêu cầu phân mảnh. Sau đó nút gốc đối chiếu tất cả các phản hồi này lại thành một phản hồi duy nhất chứa tất cả các bằng sáng chế khớp với từ cụ thể đó. Hoạt động của chỉ mục tìm kiếm này được minh họa trong Hình 7-3.

Hands On: Sharded Document Search — Thực hành (Hands On): Tìm kiếm Tài liệu có Phân mảnh

The previous example scattered the different term requests across the cluster, but this only works if all of the documents are present on all of the machines in the scatter/ gather tree. If there is not enough room for all of the documents in all of the leaves in the tree, then sharding must be used to put different sets of documents onto different leaves.

Ví dụ trước đã phát tán các yêu cầu theo từng từ khác nhau ra khắp cụm, nhưng điều này chỉ hoạt động nếu tất cả các tài liệu đều hiện diện trên tất cả các máy trong cây scatter/gather. Nếu không đủ chỗ cho tất cả các tài liệu trên tất cả các lá trong cây, thì phải dùng phân mảnh để đặt các tập tài liệu khác nhau lên các lá khác nhau.

This means that when a user makes a request for all documents that match the words “cat” and “dog,” the request is actually sent out to every leaf in the scatter/gather system. Each leaf node returns the set of documents that it knows about that matches “cat” and “dog.” Previously, the root node was responsible for performing the intersection of the two sets of documents returned for two different words. In the sharded case, the root node is responsible for generating the union of all of the documents returned by all of the different shards and returning this complete set of documents back up to the user.

Điều này có nghĩa là khi một người dùng đưa ra yêu cầu tìm tất cả các tài liệu khớp với các từ “cat” và “dog”, yêu cầu thực ra được gửi ra mọi lá trong hệ thống scatter/gather. Mỗi nút lá trả về tập các tài liệu mà nó biết và khớp với “cat” và “dog”. Trước đây, nút gốc chịu trách nhiệm thực hiện phép giao của hai tập tài liệu được trả về cho hai từ khác nhau. Trong trường hợp phân mảnh, nút gốc chịu trách nhiệm tạo phép hợp của tất cả các tài liệu được trả về bởi tất cả các mảnh khác nhau và trả tập tài liệu hoàn chỉnh này lên cho người dùng.

In Figure 7-3, the first leaf serves documents 1 through 10 and returns {doc1, doc5}. The second leaf serves documents 11 through 20 and returns {doc15}. The third leaf serves documents 21 through 30 and returns {doc22, doc28}. The root combines all of these responses together into a single response and returns {doc1, doc5, doc15, doc22, doc28}.

Trong Hình 7-3, lá thứ nhất phục vụ các tài liệu 1 đến 10 và trả về {doc1, doc5}. Lá thứ hai phục vụ các tài liệu 11 đến 20 và trả về {doc15}. Lá thứ ba phục vụ các tài liệu 21 đến 30 và trả về {doc22, doc28}. Gốc kết hợp tất cả các phản hồi này lại thành một phản hồi duy nhất và trả về {doc1, doc5, doc15, doc22, doc28}.

doc22, doc28}.

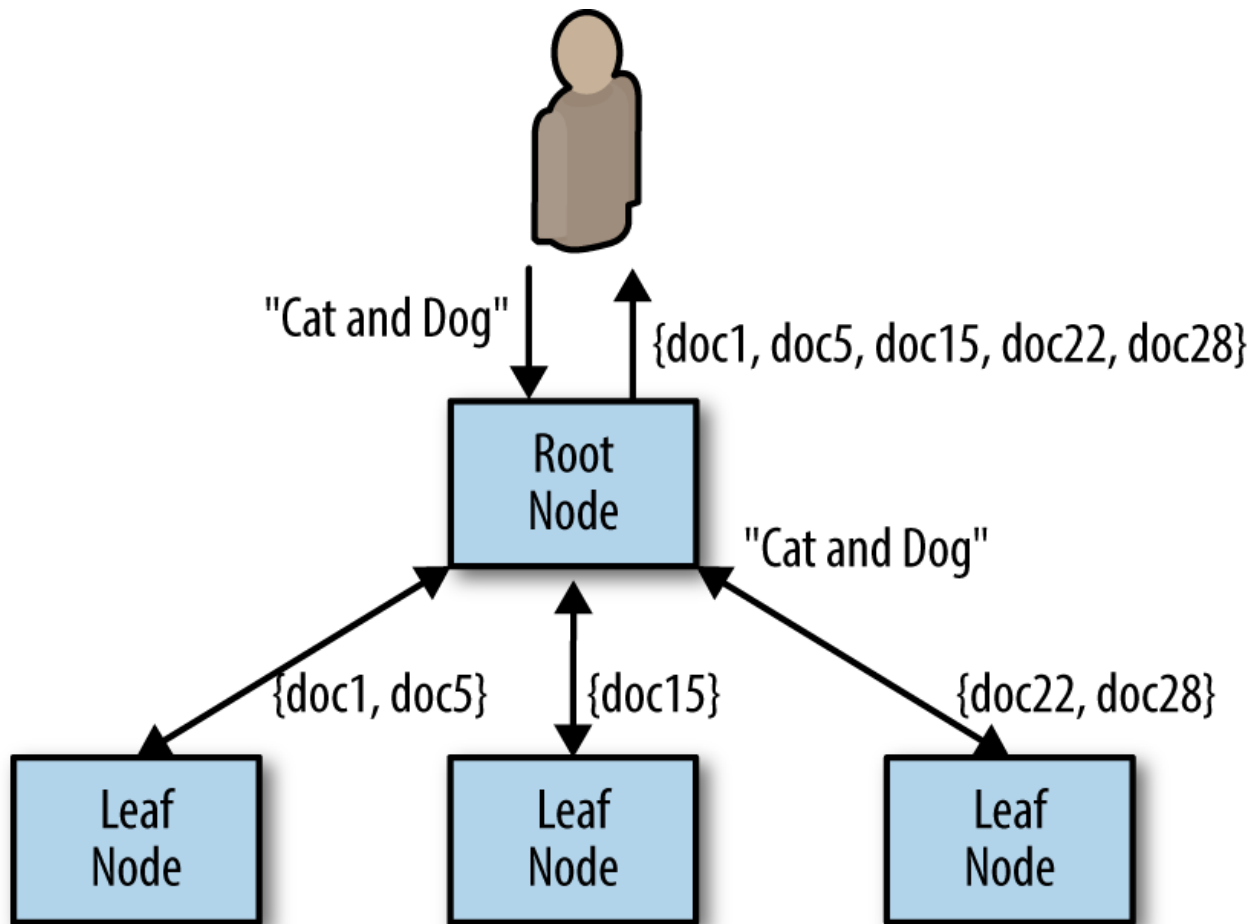


Figure 7-3. Conjunctive query executing in a scatter/gather search system

Hình 7-3. Truy vấn liên kết (conjunctive query) đang được thực thi trong một hệ thống tìm kiếm scatter/gather

Choosing the Right Number of Leaves — Chọn Số lượng Lá Phù hợp

It might seem that in the scatter/gather pattern, replicating out to a very large number of leaves would always be a good idea. You parallelize your computation and consequently reduce the clock time required to process any particular request. However, increased parallelization comes at a cost, and thus choosing the right number of leaf nodes in the scatter/gather pattern is critical to designing a performant **distributed system**.

Có vẻ như trong mẫu scatter/gather, nhân bản ra một số lượng lá rất lớn luôn là một ý tưởng tốt. Bạn song song hóa phần tính toán và nhờ đó giảm thời gian đồng hồ cần thiết để xử lý bất kỳ yêu cầu cụ thể nào. Tuy nhiên, việc tăng song song hóa đi kèm cái giá của nó, và do đó việc chọn đúng số lượng nút lá trong mẫu scatter/gather là tối quan trọng để thiết kế một hệ phân tán có hiệu năng cao.

To understand how this can happen, it's worth considering two things. The first is that processing any particular request has a certain amount of overhead. This is the time spent parsing a request, sending

HTTP across the wire, and so forth. In general, the overhead due to system request handling is constant and significantly less than the time spent in user code processing the request. Consequently, this overhead can generally be ignored when assessing the performance of the scatter/gather pattern. However, it is important to understand that the cost of this overhead scales with the number of leaf nodes in the scatter/gather pattern. Thus, even though it is low cost, as parallelization continues, this overhead eventually dominates the compute cost of your business logic. This means that the gains of parallelization are asymptotic.

Để hiểu điều này có thể xảy ra như thế nào, đáng để xét hai điều. Điều thứ nhất là việc xử lý bất kỳ yêu cầu cụ thể nào cũng có một lượng chi phí phụ (overhead) nhất định. Đó là thời gian dành để phân tích một yêu cầu, gửi HTTP qua đường truyền, v.v. Nhìn chung, chi phí phụ do hệ thống xử lý yêu cầu là hằng số và nhỏ hơn đáng kể so với thời gian dành trong mã người dùng để xử lý yêu cầu. Do đó, chi phí phụ này nói chung có thể được bỏ qua khi đánh giá hiệu năng của mẫu scatter/gather. Tuy nhiên, điều quan trọng cần hiểu là cái giá của chi phí phụ này tỉ lệ với số lượng nút lá trong mẫu scatter/gather. Vì vậy, dù chi phí thấp, khi song song hóa tiếp tục tăng, chi phí phụ này cuối cùng sẽ lấn át chi phí tính toán của logic nghiệp vụ của bạn. Điều này có nghĩa là lợi ích của song song hóa là tiệm cận (asymptotic).

In addition to the fact that adding more leaf nodes may not actually speed up processing, scatter/gather systems also suffer from the “straggler” problem. To understand how this works, it is important to remember that in a scatter/gather system, the root node waits for requests from all of the leaf nodes to return before sending a response back to the end user. Since data from every leaf node is required, the overall time it takes to process a user request is defined by the slowest leaf node that sends a response. To understand the impact of this, imagine that we have a service that has a 99th percentile latency of 2 seconds. This means that on average one request out of every 100 has a latency of 2 seconds, or put another way, there is a 1% chance that a request will take 2 seconds. This may be totally acceptable at first glance: a single user out of 100 has a slow request. However, consider how this actually works in a scatter/ gather system. Since the time of the user request is defined by the slowest response, we need to consider not a single request but all requests scattered out to the various leaf nodes.

Ngoài việc thêm nhiều nút lá hơn có thể thực ra không giúp tăng tốc việc xử lý, các hệ thống scatter/gather còn gặp phải vấn đề “kẻ tụt hậu” (straggler). Để hiểu điều này hoạt động ra sao, cần nhớ rằng trong một hệ thống scatter/gather, nút gốc chờ các yêu cầu từ tất cả các nút lá trả về trước khi gửi phản hồi lại cho người dùng cuối. Vì cần dữ liệu từ mọi nút lá, tổng thời gian để xử lý một yêu cầu người dùng được quyết định bởi nút lá phản hồi chậm nhất. Để hiểu tác động của điều này, hãy hình dung ta có một dịch vụ có độ trễ phân vị thứ 99 (99th percentile latency) là 2 giây. Điều này có nghĩa là trung bình cứ 100 yêu cầu thì có một yêu cầu có độ trễ 2 giây, hay nói cách khác, có 1% khả năng một yêu cầu sẽ mất 2 giây. Thoạt nhìn điều này có thể hoàn toàn chấp nhận được: một người dùng trong số 100 người có một yêu cầu chậm. Tuy nhiên, hãy xét xem điều này thực sự hoạt động ra sao trong một hệ thống scatter/gather. Vì thời gian của yêu cầu người dùng được quyết định bởi phản hồi chậm nhất, ta cần xét không phải một yêu cầu đơn lẻ mà là tất cả các yêu cầu được phát tán ra tới các nút lá khác nhau.

Let’s see what happens when we scatter out to five leaf nodes. In this situation, there is a 5% chance that one of these five scatter requests has a latency of 2 seconds ($0.99 \times 0.99 \times 0.99 \times 0.99 \times 0.99 == 0.95$). This means that our 99th percentile latency for individual requests becomes a 95th percentile latency for our complete scatter/gather system. And it only gets worse from there: if we scatter out to 100 leaves, then we are more or less guaranteeing that our overall latency for all requests will be 2 seconds.

Hãy xem điều gì xảy ra khi ta phát tán ra năm nút lá. Trong tình huống này, có 5% khả năng một trong năm yêu cầu phát tán này có độ trễ 2 giây ($0,99 \times 0,99 \times 0,99 \times 0,99 \times 0,99 == 0,95$). Điều này có nghĩa là độ trễ phân vị thứ 99 của ta cho các yêu cầu riêng lẻ trở thành độ trễ phân vị thứ 95 cho toàn bộ hệ thống scatter/gather. Và nó chỉ tệ hơn từ đó: nếu ta phát tán ra 100 lá, thì ta gần như chắc chắn rằng độ trễ tổng thể cho tất cả các yêu cầu sẽ là 2 giây.

Together, these complications of scatter/gather systems lead us to some conclusions:

Cùng nhau, những phức tạp này của các hệ thống scatter/gather dẫn ta đến một số kết luận:

- Increased parallelism doesn't always speed things up because of overhead on each node.
- Increased parallelism doesn't always speed things up because of the straggler problem.
- The performance of the 99th percentile is more important than in other systems because each user request actually becomes numerous requests to the service.
 - Tăng tính song song không phải lúc nào cũng tăng tốc mọi thứ vì chi phí phụ trên mỗi nút.
 - Tăng tính song song không phải lúc nào cũng tăng tốc mọi thứ vì vấn đề kẻ tụt hậu.
 - Hiệu năng ở phân vị thứ 99 quan trọng hơn so với ở các hệ thống khác vì mỗi yêu cầu người dùng thực ra trở thành nhiều yêu cầu tới dịch vụ.

The same straggler problem applies to **availability**. If you issue a request to 100 leaf nodes, and the probability that any leaf node failing is 1 in 100, you are again practically guaranteed to fail every single user request.

Cùng vấn đề kẻ tụt hậu đó cũng áp dụng cho tính sẵn sàng. Nếu bạn phát ra một yêu cầu tới 100 nút lá, và xác suất bất kỳ nút lá nào hỏng là 1 trên 100, thì bạn lại gần như chắc chắn sẽ làm hỏng mọi yêu cầu người dùng.

Scaling Scatter/Gather for Reliability and Scale — Mở rộng Scatter/Gather để Đạt Độ tin cậy và Quy mô

Of course, just as with a sharded system, having a single replica of a sharded scatter/gather system is likely not the desirable design choice. A single replica means that if it fails, all scatter/gather requests will fail for the duration that the shard is unavailable because all requests are required to be processed by all leaf nodes in the scatter/gather pattern. Likewise, upgrades will take out a percentage of your shards, so an upgrade while under user-facing load is no longer possible. Finally, the computational scale of your system will be limited by the load that any single node is capable of achieving. Ultimately, this limits your scale, and as we have seen in previous sections, you cannot simply increase the number of shards in order to improve the computational power of a scatter/gather pattern.

Tất nhiên, cũng như với một hệ thống phân mảnh, việc chỉ có một bản sao duy nhất của một hệ thống scatter/gather phân mảnh có lẽ không phải lựa chọn thiết kế mong muốn. Một bản sao duy nhất có nghĩa là nếu nó hỏng, tất cả các yêu cầu scatter/gather sẽ thất bại trong suốt thời gian mảnh đó không khả dụng, vì mọi yêu cầu đều buộc phải được xử lý bởi tất cả các nút lá trong mẫu scatter/gather. Tương tự, các lần nâng cấp sẽ làm mất một phần trăm số mảnh của bạn, nên việc nâng cấp khi đang chịu tải hướng tới người dùng không còn khả thi. Cuối cùng, quy mô tính toán của hệ thống sẽ bị giới hạn bởi mức tải mà bất kỳ nút đơn lẻ nào có khả năng đạt được. Rốt cuộc, điều này giới hạn quy mô của bạn,

và như ta đã thấy trong các phần trước, bạn không thể chỉ đơn giản tăng số mảnh để cải thiện sức mạnh tính toán của một mẫu scatter/gather.

Given these challenges of **reliability** and scale, the correct approach is to replicate each of the individual shards so that instead of a single instance at each leaf node, there is a **replicated service** that implements each leaf shard. This replicated, sharded scatter/gather pattern is shown in Figure 7-4 .

Trước những thách thức về độ tin cậy và quy mô này, cách tiếp cận đúng đắn là nhân bản từng mảnh riêng lẻ sao cho thay vì một thực thể duy nhất tại mỗi nút lá, có một dịch vụ nhân bản hiện thực mỗi mảnh lá. Mẫu scatter/gather nhân bản, phân mảnh này được thể hiện trong Hình 7-4.

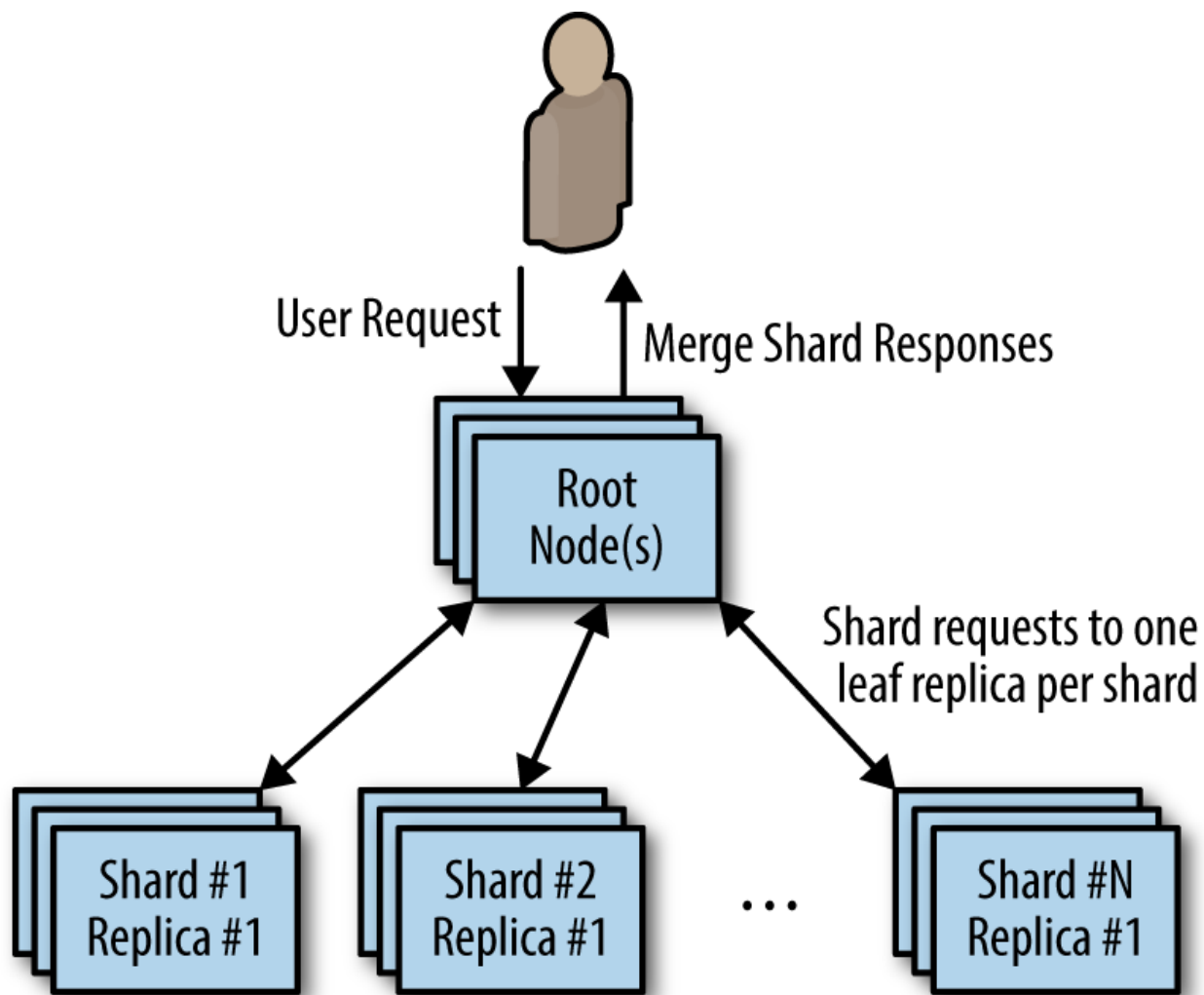


Figure 7-4. A sharded, replicated scatter/gather system

Hình 7-4. Một hệ thống scatter/gather phân mảnh, nhân bản

Built this way, each leaf request from the root is actually load balanced across all healthy replicas of the shard. This means that if there are any failures, they won't result in a user-visible outage for your system. Likewise, you can safely perform an upgrade under load, since each replicated shard can be upgraded one replica at a time. Indeed, you can perform the upgrade across multiple shards simultaneously, depending on how quickly you want to perform the upgrade.

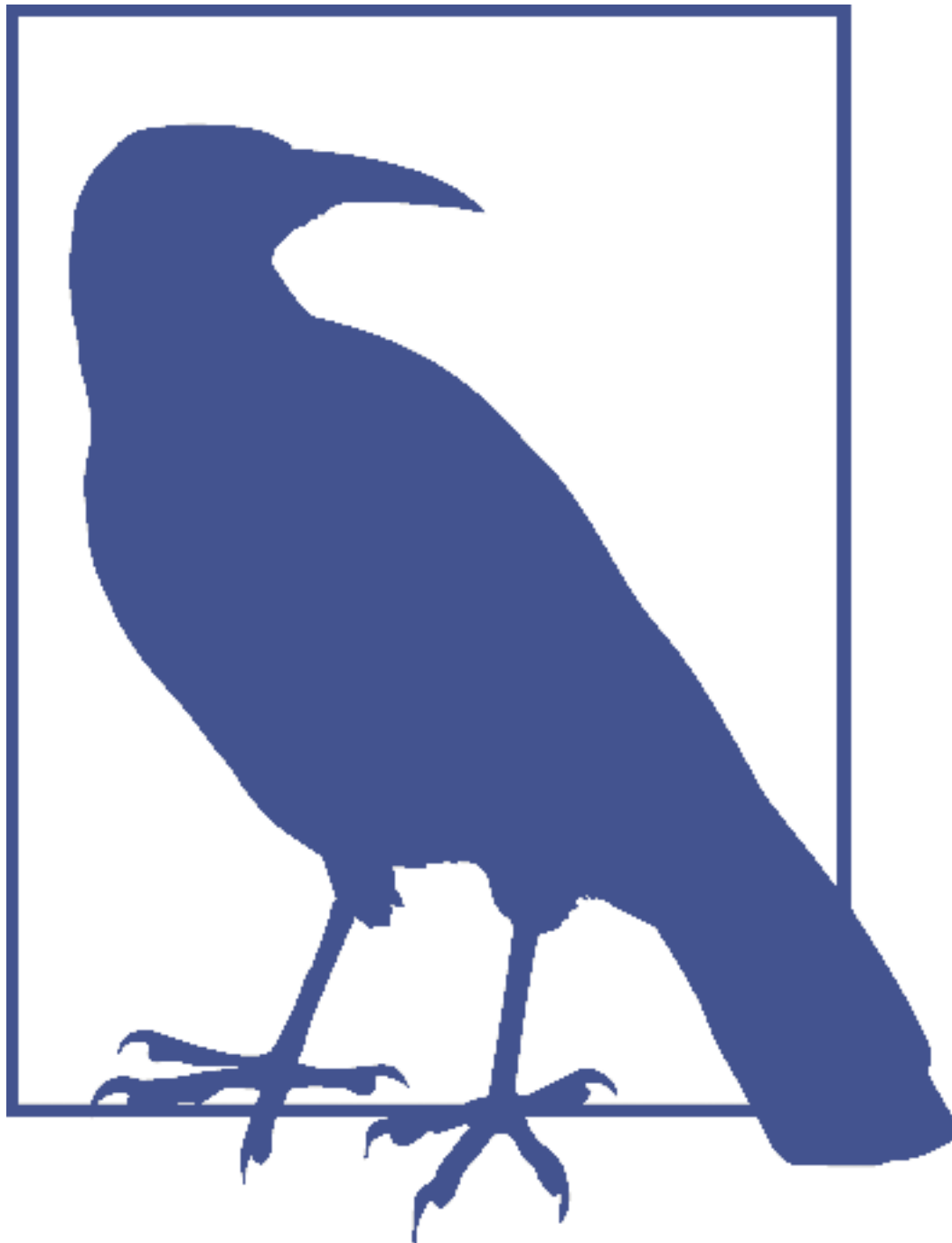
Được xây dựng theo cách này, mỗi yêu cầu lá từ gốc thực ra được cân bằng tải trên tất cả các bản sao khỏe mạnh của mảnh. Điều này có nghĩa là nếu có bất kỳ sự cố nào, chúng sẽ không gây ra một sự gián đoạn mà người dùng nhìn thấy cho hệ thống của bạn. Tương tự, bạn có thể an toàn thực hiện một lần nâng cấp khi đang chịu tải, vì mỗi mảnh nhân bản có thể được nâng cấp từng bản sao một. Thật vậy, bạn có thể thực hiện nâng cấp trên nhiều mảnh đồng thời, tùy thuộc vào mức độ nhanh chóng mà bạn muốn thực hiện nâng cấp.

CHAPTER 8 Functions and Event-Driven Processing —

CHAPTER 8 Hàm và Xử lý hướng Sự kiện

So far, we have examined design for systems with long-running computation. The servers that handle user requests are always up and running. This pattern is the right one for many applications that are under heavy load, keep a large amount of data in memory, or require some sort of **background processing**. However, there is a class of applications that might only need to temporarily come into existence to handle a single request, or simply need to respond to a specific **event**. This style of request or **event-driven** application design has flourished recently as large-scale public cloud providers have developed **function-as-a-service** (FaaS) products. More recently, FaaS implementations have also emerged running on top of cluster orchestrators in private cloud or physical environments. This chapter describes emerging architectures for this new style of computing. In many cases, FaaS is a component in a broader architecture rather than a complete solution.

Cho đến nay, chúng ta đã xem xét cách thiết kế cho các hệ thống có khối lượng tính toán chạy lâu dài. Các máy chủ xử lý yêu cầu của người dùng luôn ở trạng thái sẵn sàng và đang chạy. Mẫu này phù hợp với nhiều ứng dụng chịu tải nặng, giữ một lượng lớn dữ liệu trong bộ nhớ, hoặc cần một dạng xử lý nền nào đó. Tuy nhiên, có một lớp ứng dụng có thể chỉ cần xuất hiện tạm thời để xử lý một yêu cầu đơn lẻ, hoặc đơn giản là cần phản hồi một sự kiện cụ thể. Phong cách thiết kế ứng dụng hướng yêu cầu hoặc hướng sự kiện (event-driven) này đã nở rộ gần đây khi các nhà cung cấp đám mây công cộng quy mô lớn phát triển các sản phẩm Hàm-như-Dịch-vụ (FaaS). Gần đây hơn, các hiện thực FaaS cũng đã xuất hiện chạy trên các bộ điều phối cụm (cluster orchestrator) trong môi trường đám mây riêng hoặc môi trường vật lý. Chương này mô tả các kiến trúc đang nổi lên cho phong cách điện toán mới này. Trong nhiều trường hợp, FaaS là một thành phần trong một kiến trúc rộng lớn hơn chứ không phải là một giải pháp hoàn chỉnh.



Oftentimes, FaaS is referred to as **serverless** computing. And while this is true (you don't see the servers in FaaS) it's worth differentiating between event-driven FaaS and the broader notion of serverless computing. Indeed, serverless computing can apply to a wide variety of computing services; for example, a multi-tenant **container orchestrator (container-as-a-service)** is serverless but not event-driven. Conversely, an **open source** FaaS running on a cluster of physical machines that you own and administer is event-driven but not serverless. Understanding this distinction enables you to determine when event-driven, serverless, or both is the right choice for your application.

FaaS thường được gọi là điện toán serverless. Và mặc dù điều này đúng (bạn không nhìn

thấy các máy chủ trong FaaS), việc phân biệt giữa FaaS hướng sự kiện và khái niệm rộng hơn về điện toán serverless vẫn đáng được lưu tâm. Thật vậy, điện toán serverless có thể áp dụng cho rất nhiều loại dịch vụ điện toán khác nhau; ví dụ, một bộ điều phối container đa người thuê (container-as-a-service) là serverless nhưng không hướng sự kiện. Ngược lại, một FaaS mã nguồn mở chạy trên một cụm máy vật lý mà bạn sở hữu và quản trị thì hướng sự kiện nhưng không phải serverless. Hiểu được sự khác biệt này giúp bạn xác định khi nào hướng sự kiện, serverless, hay cả hai mới là lựa chọn đúng cho ứng dụng của mình.

81

81

Determining When FaaS Makes Sense — Xác định Khi nào FaaS Hợp lý

As with many tools for developing a **distributed system**, it can be tempting to see a particular solution like event-driven processing as a universal hammer. However, the truth is that it is best suited to a particular set of problems. Within a particular context it is a powerful tool, but stretching it to fit all applications or systems will lead to overly complicated, brittle designs. Especially since FaaS is such a new computing tool, before discussing specific design patterns, it is worth discussing the benefits, limitations, and optimal situations for employing event-driven computing.

Cũng như nhiều công cụ để phát triển một hệ phân tán, ta dễ bị cám dỗ xem một giải pháp cụ thể như xử lý hướng sự kiện là một chiếc búa vạn năng. Tuy nhiên, sự thật là nó phù hợp nhất với một tập vấn đề cụ thể. Trong một ngữ cảnh nhất định, nó là một công cụ mạnh mẽ, nhưng cố nong nó cho vừa mọi ứng dụng hay hệ thống sẽ dẫn đến những thiết kế quá phức tạp và dễ vỡ. Đặc biệt vì FaaS là một công cụ điện toán còn rất mới, trước khi bàn về các mẫu thiết kế cụ thể, việc thảo luận về lợi ích, hạn chế và các tình huống tối ưu để dùng điện toán hướng sự kiện là rất đáng giá.

The Benefits of FaaS — Lợi ích của FaaS

The benefits of FaaS are primarily for the developer. It dramatically simplifies the distance from code to running service. Because there is no **artifact** to create or push beyond the source code itself, FaaS makes it simple to go from code on a laptop or web browser to running code in the cloud.

Lợi ích của FaaS chủ yếu dành cho lập trình viên. Nó rút ngắn đáng kể khoảng cách từ mã nguồn đến dịch vụ đang chạy. Vì không có sản phẩm tạo tác (artifact) nào cần tạo ra hay đẩy đi ngoài chính mã nguồn, FaaS giúp việc đi từ mã trên laptop hay trình duyệt web đến mã đang chạy trên đám mây trở nên đơn giản.

Likewise, the code that is deployed is managed and scaled automatically. As more traffic is loaded onto the service, more instances of the function are created to handle that increase in traffic. If a function fails due to application or machine failures, it is automatically restarted on some other machine.

Tương tự, mã được triển khai sẽ được quản lý và mở rộng tự động. Khi lưu lượng đổ vào dịch vụ tăng lên, nhiều thực thể (instance) của hàm hơn sẽ được tạo ra để xử lý mức tăng lưu lượng đó. Nếu một hàm thất bại do lỗi ứng dụng hoặc lỗi máy, nó sẽ tự động được khởi động lại trên một máy khác.

Finally, much like containers, functions are an even more granular building block for designing distributed systems. Functions are **stateless** and thus any system you build on top of functions is inherently more modular and decoupled than a similar system built into a single binary. But, of course, this is also the challenge of developing systems in FaaS. The decoupling is both a strength and a weakness. The following section describes some of the challenges that come from developing systems using FaaS.

Cuối cùng, giống như container, hàm là một khối xây dựng còn chi tiết hơn nữa để thiết kế hệ phân tán. Hàm là không trạng thái (stateless) và do đó bất kỳ hệ thống nào bạn xây dựng trên nền hàm vốn dĩ sẽ mô-đun hóa và tách rời (decoupled) hơn so với một hệ thống tương tự được tích hợp vào một tệp nhị phân duy nhất. Nhưng, tất nhiên, đây cũng là thách thức của việc phát triển hệ thống trong FaaS. Việc tách rời vừa là điểm mạnh vừa là điểm yếu. Phần tiếp theo mô tả một số thách thức nảy sinh từ việc phát triển hệ thống bằng FaaS.

The Challenges of FaaS — Những Thách thức của FaaS

As described in the previous section, developing systems using FaaS forces you to strongly decouple each piece of your service. Each function is entirely independent. The only communication is across the network, and each function instance cannot have local memory, requiring all states to be stored in a storage service. This forced decoupling can improve the agility and speed with which you can develop services, but it can also significantly complicate the operations of the same service.

Như đã mô tả ở phần trước, phát triển hệ thống bằng FaaS buộc bạn phải tách rời triệt để từng mảnh trong dịch vụ của mình. Mỗi hàm là hoàn toàn độc lập. Giao tiếp duy nhất là qua mạng, và mỗi thực thể hàm không thể có bộ nhớ cục bộ, đòi hỏi mọi trạng thái phải được lưu trong một dịch vụ lưu trữ. Việc tách rời cưỡng bức này có thể cải thiện sự nhanh nhẹn và tốc độ phát triển dịch vụ, nhưng nó cũng có thể làm phức tạp hóa đáng kể việc vận hành cùng dịch vụ đó.

In particular, it is often quite difficult to obtain a comprehensive view of your service, determine how the various functions integrate with one another, and understand when things go wrong, and why they go wrong. Additionally, the request-based and serverless nature of functions means that certain problems are quite difficult to detect. As an example, consider the following functions:

Cụ thể, thường khá khó để có được một cái nhìn toàn diện về dịch vụ của bạn, để xác định các hàm khác nhau tích hợp với nhau như thế nào, và để hiểu khi nào mọi thứ trục trặc, và tại sao chúng trục trặc. Ngoài ra, bản chất dựa trên yêu cầu và serverless của các hàm khiến một số vấn đề khá khó phát hiện. Ví dụ, hãy xem xét các hàm sau:

- `functionA()` which calls `functionB()`
- `functionB()` which calls `functionC()`
- `functionC()` which calls back to `functionA()`
 - `functionC()` gọi ngược lại `functionA()`

Now consider what happens when a request comes into any of these functions: it kicks off an infinite loop that only terminates when the original request times out (and possibly not even then) or when you run out of money to pay for requests in the system. Obviously, the above example is quite contrived, but it is actually quite difficult to detect in your code. Since each function is radically decoupled from the other functions, there is no real representation of the dependencies or interactions between different functions. These problems are not unsolvable, and I expect that

as FaaS mature, more analysis and debugging tools will provide a richer experience to understand how and why an application comprised of FaaS is performing the way that it does.

Giờ hãy xem điều gì xảy ra khi một yêu cầu đến bất kỳ hàm nào trong số này: nó khởi động một vòng lặp vô hạn chỉ kết thúc khi yêu cầu gốc hết thời gian chờ (và có thể cũng không kết thúc kể cả khi đó) hoặc khi bạn cạn tiền để trả cho các yêu cầu trong hệ thống. Hiển nhiên, ví dụ trên khá gượng ép, nhưng thực tế lại khá khó phát hiện trong mã của bạn. Vì mỗi hàm được tách rời triệt để khỏi các hàm khác, không có biểu diễn thực sự nào về sự phụ thuộc hay tương tác giữa các hàm khác nhau. Những vấn đề này không phải không giải được, và tôi kỳ vọng rằng khi các FaaS trưởng thành hơn, sẽ có nhiều công cụ phân tích và gỡ lỗi cung cấp trải nghiệm phong phú hơn để hiểu một ứng dụng cấu thành từ FaaS đang vận hành theo cách của nó như thế nào và vì sao.

For now, when adopting FaaS, you must be vigilant to adopt rigorous monitoring and alerting for how your system is behaving so that you can detect situations and correct them before they become significant problems. Of course, the complexity introduced by monitoring flies somewhat in the face of the simplicity of deploying to FaaS, which is friction that your developers must overcome.

Hiện tại, khi áp dụng FaaS, bạn phải luôn cảnh giác để áp dụng việc giám sát và cảnh báo nghiêm ngặt về hành vi của hệ thống, để bạn có thể phát hiện các tình huống và sửa chữa chúng trước khi chúng trở thành vấn đề lớn. Tất nhiên, độ phức tạp do việc giám sát mang lại phần nào đi ngược với sự đơn giản của việc triển khai lên FaaS, và đó là trở ngại mà các lập trình viên của bạn phải vượt qua.

The Need for Background Processing — Nhu cầu Xử lý Nền

FaaS is inherently an event-based application model. Functions are executed in response to discrete events that occur and **trigger** the execution of the functions. Additionally, because of the serverless nature of the implementation of these services, the runtime of any particular function instance is generally time bounded. This means that FaaS is usually a poor fit for situations that require processing. Examples of such background processing might be transcoding a video, compressing log files, or other sorts of low-priority, long-running computations. In many cases, it is possible to set up a scheduled trigger that synthetically generates events in your functions on a particular schedule. Though this is a good fit for responding to temporal events (e.g., firing a text-message alarm to wake someone up), it is still not sufficient infrastructure for generic background processing. To achieve that, you need to launch your code in an environment that supports long-running processes. And this generally means switching to a **pay-per-consumption** rather than **pay-per-request** model for the parts of your application that do background processing.

FaaS vốn dĩ là một mô hình ứng dụng dựa trên sự kiện. Hàm được thực thi để phản hồi các sự kiện rời rạc xảy ra và kích hoạt việc thực thi hàm. Ngoài ra, do bản chất serverless của hiện thực các dịch vụ này, thời gian chạy của bất kỳ thực thể hàm cụ thể nào nhìn chung đều bị giới hạn về thời gian. Điều này có nghĩa là FaaS thường không phù hợp với những tình huống đòi hỏi xử lý kéo dài. Ví dụ về kiểu xử lý nền như vậy có thể là chuyển mã (transcode) một video, nén các tệp nhật ký, hoặc các loại tính toán chạy lâu, ưu tiên thấp khác. Trong nhiều trường hợp, có thể thiết lập một trình kích hoạt theo lịch (scheduled trigger) tự động sinh ra sự kiện trong các hàm của bạn theo một lịch trình nhất định. Mặc dù điều này phù hợp để phản hồi các sự kiện theo thời gian (ví dụ, kích hoạt một báo thức tin nhắn để đánh thức ai đó), nó vẫn chưa phải là hạ tầng đủ cho xử lý nền tổng quát. Để đạt được điều đó, bạn cần chạy mã của mình trong một môi trường hỗ trợ tiến trình chạy lâu dài. Và điều này nhìn chung có nghĩa là chuyển sang mô hình trả-theo-mức-tiêu-thụ

thay vì trả-theo-yêu-cầu cho những phần ứng dụng của bạn thực hiện xử lý nền.

The Need to Hold Data in Memory — Nhu cầu Giữ Dữ liệu trong Bộ nhớ

In addition to the operational challenges, there are some architectural limitations that make FaaS ill-suited for some types of applications. The first of these limitations is the need to have a significant amount of data loaded into memory in order to process user requests. There are a variety of services (e.g., serving a search index of documents) that require a great deal of data to be loaded in memory in order to service user requests. Even with a relatively fast storage layer, loading such data can take significantly longer than the desired time to service a user request. Because with FaaS, the function itself may be dynamically spun up in response to a user request while the user is waiting, the need to load a lot of detail may significantly impact the **latency** that the user perceives while interacting with your service. Of course, once your FaaS has been created, it may handle a large number of requests, so this loading cost can be amortized across a large number of requests. But if you have a sufficient number of requests to keep a function active, then it's likely you are overpaying for the requests you are processing.

Ngoài các thách thức vận hành, còn có một số hạn chế kiến trúc khiến FaaS không phù hợp với một số loại ứng dụng. Hạn chế đầu tiên trong số này là nhu cầu nạp một lượng lớn dữ liệu vào bộ nhớ để xử lý các yêu cầu của người dùng. Có nhiều loại dịch vụ khác nhau (ví dụ, phục vụ một chỉ mục tìm kiếm tài liệu) đòi hỏi rất nhiều dữ liệu phải được nạp vào bộ nhớ để phục vụ các yêu cầu của người dùng. Ngay cả với một tầng lưu trữ tương đối nhanh, việc nạp lượng dữ liệu như vậy có thể mất nhiều thời gian hơn đáng kể so với thời gian mong muốn để phục vụ một yêu cầu của người dùng. Vì với FaaS, bản thân hàm có thể được khởi tạo động để phản hồi một yêu cầu của người dùng trong khi người dùng đang chờ, nhu cầu nạp nhiều dữ liệu có thể tác động đáng kể đến độ trễ mà người dùng cảm nhận khi tương tác với dịch vụ của bạn. Tất nhiên, một khi FaaS của bạn đã được tạo ra, nó có thể xử lý một số lượng lớn yêu cầu, nên chi phí nạp này có thể được phân bổ (amortize) trên một số lượng lớn yêu cầu. Nhưng nếu bạn có đủ lượng yêu cầu để giữ một hàm luôn hoạt động, thì rất có khả năng bạn đang trả thừa cho các yêu cầu mà mình đang xử lý.

The Costs of Sustained Request-Based Processing — Chi phí của Xử lý Dựa trên Yêu cầu Liên tục

The cost model of public cloud FaaS is based on per-request pricing. This approach is great if you only have a few requests per minute or hour. In such a situation, you are idle most of the time, and given a pay-per-request model, you are only paying for the time when your service is actively serving requests. In contrast, if you service requests via a long-running service either in a container or a virtual machine, then you are always paying for processor cycles that is largely sitting around waiting for a user request.

Mô hình chi phí của FaaS đám mây công cộng dựa trên định giá theo từng yêu cầu. Cách tiếp cận này tuyệt vời nếu bạn chỉ có một vài yêu cầu mỗi phút hoặc mỗi giờ. Trong tình huống đó, bạn rảnh rỗi gần như suốt thời gian, và với mô hình trả-theo-yêu-cầu, bạn chỉ trả cho khoảng thời gian khi dịch vụ thực sự đang phục vụ yêu cầu. Ngược lại, nếu bạn phục vụ yêu cầu thông qua một dịch vụ chạy lâu dài trong một container hoặc một máy

ảo, thì bạn luôn trả tiền cho các chu kỳ xử lý mà phần lớn chỉ ngồi không chờ một yêu cầu của người dùng.

However, as a service grows, the number of requests that you are servicing grows to the point where you can keep a processor continuously active servicing user requests. At this point, the economics of a pay-per-request model start to become bad, and only get worse because the cost of cloud virtual machines generally decreases as you add more cores (and also via committed resources like reservations or sustained use discounts), whereas the cost per-request largely grows linearly with the number of requests.

Tuy nhiên, khi một dịch vụ phát triển, số lượng yêu cầu bạn đang phục vụ tăng đến mức bạn có thể giữ một bộ xử lý hoạt động liên tục để phục vụ các yêu cầu của người dùng. Đến thời điểm này, kinh tế học của mô hình trả-theo-yêu-cầu bắt đầu trở nên tệ, và chỉ càng tệ hơn vì chi phí máy ảo đám mây nhìn chung giảm khi bạn thêm nhiều lõi (và cũng nhờ các tài nguyên cam kết như đặt trước hay chiết khấu sử dụng liên tục), trong khi chi phí trên mỗi yêu cầu phần lớn tăng tuyến tính theo số lượng yêu cầu.

Consequently, as your service grows and evolves, it's highly likely that your use of FaaS will evolve as well. One ideal way to scale FaaS is to run an open source FaaS that runs on a container orchestrator like **Kubernetes**. That way, you can still take advantage of the developer benefits of FaaS, while taking advantage of the pricing models of virtual machines.

Do đó, khi dịch vụ của bạn phát triển và tiến hóa, rất có khả năng cách bạn dùng FaaS cũng sẽ tiến hóa theo. Một cách lý tưởng để mở rộng FaaS là chạy một FaaS mã nguồn mở chạy trên một bộ điều phối container như Kubernetes. Bằng cách đó, bạn vẫn có thể tận dụng các lợi ích cho lập trình viên của FaaS, đồng thời tận dụng các mô hình định giá của máy ảo.

Patterns for FaaS — Các Mẫu cho FaaS

In addition to understanding the trade-offs in deploying event-driven or FaaS architectures as part of your distributed system, understanding the best ways to deploy FaaS is critical to the design of a successful system. This section describes some canonical patterns for incorporating FaaS.

Bên cạnh việc hiểu các đánh đổi trong việc triển khai các kiến trúc hướng sự kiện hoặc FaaS như một phần của hệ phân tán, việc hiểu các cách tốt nhất để triển khai FaaS là yếu tố then chốt cho thiết kế của một hệ thống thành công. Phần này mô tả một số mẫu kinh điển để tích hợp FaaS.

The Decorator Pattern: Request or Response Transformation — Mẫu Decorator: Biến đổi Yêu cầu hoặc Phản hồi

FaaS is ideal for deploying simple functions that can take an input, transform it into an output, and then pass it on to a different service. This general pattern can be used to augment or decorate HTTP requests to or from a different service. A basic illustration of this pattern is shown in Figure 8-1 .

FaaS lý tưởng cho việc triển khai các hàm đơn giản có thể nhận một đầu vào, biến đổi nó thành một đầu ra, rồi chuyển nó tới một dịch vụ khác. Mẫu tổng quát này có thể được dùng để bổ sung hoặc trang trí (decorate) các yêu cầu HTTP đến hoặc đi từ một dịch vụ khác. Một minh họa cơ bản cho mẫu này được thể hiện trong Hình 8-1.

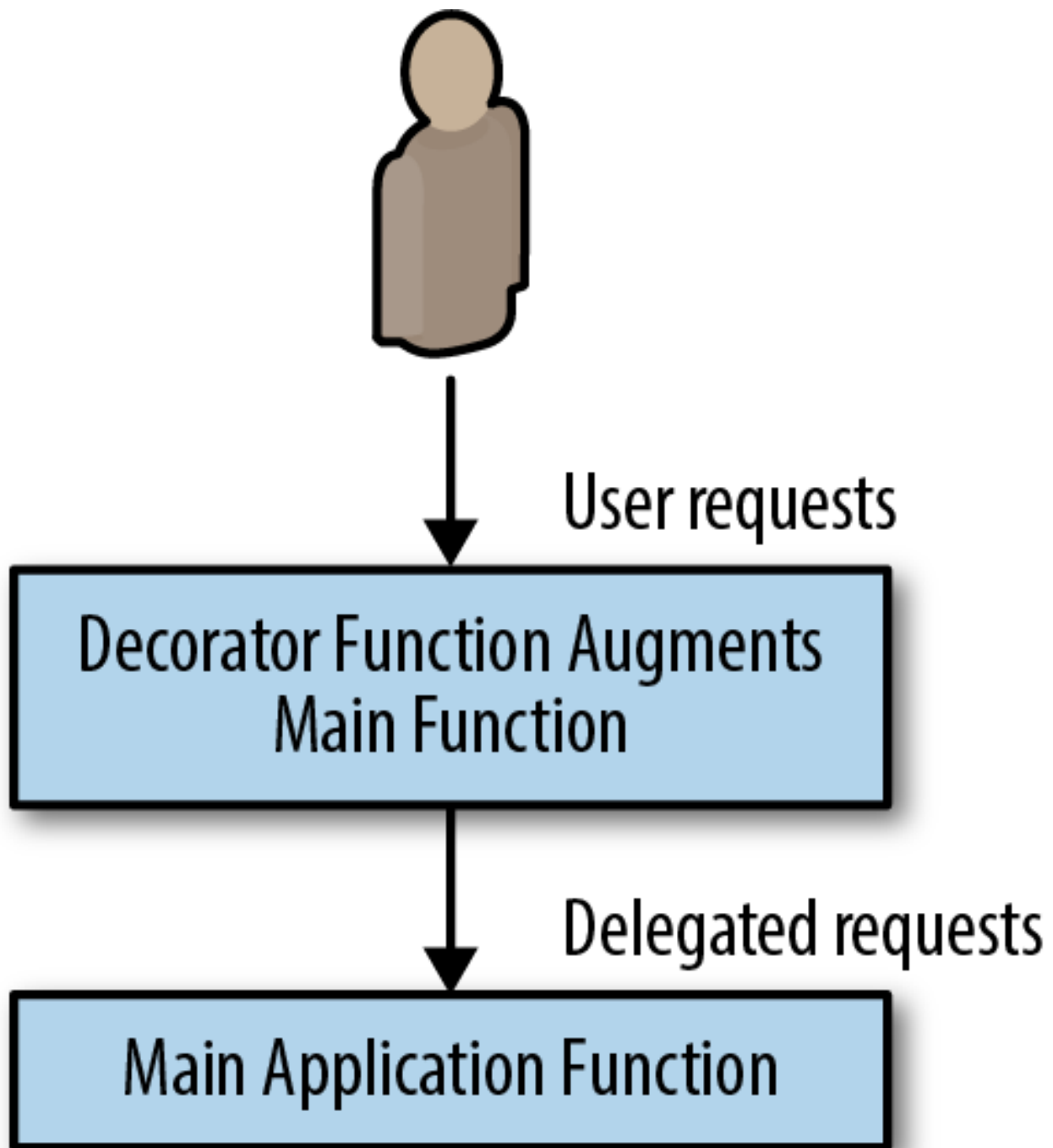


Figure 8-1. The **decorator pattern** applied to HTTP APIs

Hình 8-1. Mẫu decorator áp dụng cho các API HTTP

Interestingly, there are several analogies to this pattern in programming languages. In particular, the decorator pattern from Python is a close analogue for the services that a request or response decorator can perform. Because decoration transformations are generally stateless, and also because they are often added after the fact to existing code as the service evolves, they are ideal services to implement via FaaS. Additionally, the lightness of FaaS means that you can experiment with a

variety of different decorators before finally adopting one and pulling it more completely into the implementation of the service.

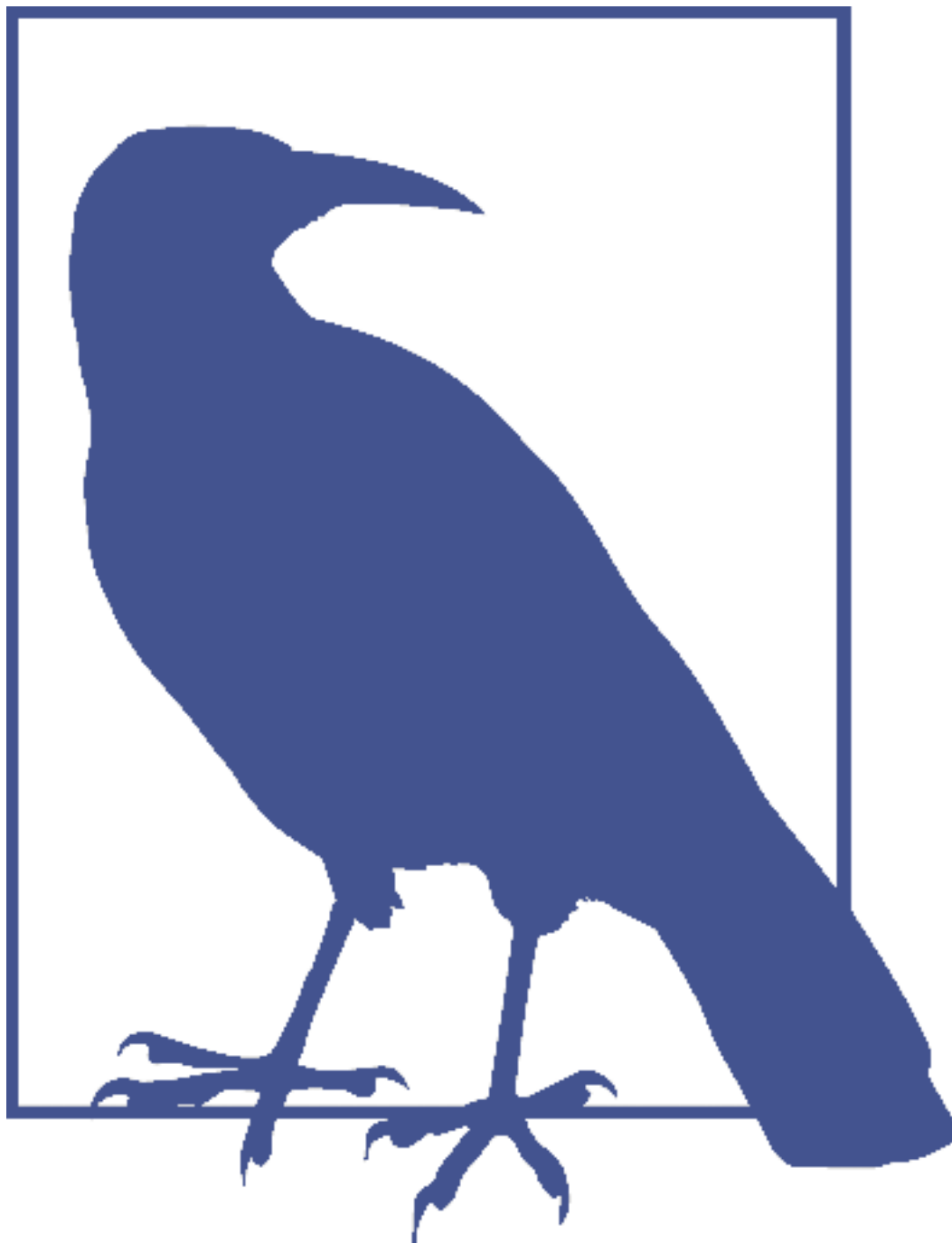
Thú vị là có một vài điểm tương đồng với mẫu này trong các ngôn ngữ lập trình. Cụ thể, decorator trong Python là một điểm tương đồng gần gũi với những dịch vụ mà một bộ trang trí yêu cầu hoặc phản hồi có thể thực hiện. Vì các phép biến đổi trang trí nhìn chung là không trạng thái, và cũng vì chúng thường được thêm vào sau khi mã đã tồn tại khi dịch vụ tiến hóa, chúng là những dịch vụ lý tưởng để hiện thực qua FaaS. Ngoài ra, sự nhẹ nhàng của FaaS có nghĩa là bạn có thể thử nghiệm nhiều bộ trang trí khác nhau trước khi cuối cùng chọn một cái và đưa nó vào hiện thực của dịch vụ một cách trọn vẹn hơn.

A great example of the value of the decorator pattern is adding defaults to the input to an HTTP RESTful **API**. In many cases in the API, there are fields whose values should have sane defaults if they are left empty. For example, you may want a field to default to true, but it's difficult to accomplish this in classical JSON, because the default value for a field is null, which is generally understood to be false. To resolve this, we can add defaulting logic either in the front of the API server or within the application code itself (e.g., if (field == null) field = true). However, both of these solutions are somewhat unappealing since the defaulting mechanism is fairly conceptually independent from the handling of the request. Instead, we can use the FaaS decorator pattern to transform the request in between the user and the service implementation.

Một ví dụ tuyệt vời về giá trị của mẫu decorator là việc thêm các giá trị mặc định vào đầu vào của một API HTTP RESTful. Trong nhiều trường hợp ở API, có những trường mà giá trị của chúng nên có giá trị mặc định hợp lý nếu chúng bị để trống. Ví dụ, bạn có thể muốn một trường mặc định là true, nhưng khó đạt được điều này trong JSON cổ điển, vì giá trị mặc định cho một trường là null, vốn thường được hiểu là false. Để giải quyết điều này, ta có thể thêm logic gán mặc định hoặc ở phía trước máy chủ API hoặc bên trong chính mã ứng dụng (ví dụ, if (field == null) field = true). Tuy nhiên, cả hai giải pháp này đều phần nào kém hấp dẫn vì cơ chế gán mặc định khá độc lập về mặt khái niệm với việc xử lý yêu cầu. Thay vào đó, ta có thể dùng mẫu decorator của FaaS để biến đổi yêu cầu ở giữa người dùng và hiện thực dịch vụ.

Given the previous discussion of the **adapter** pattern in the **single-node** section, you may be wondering why we don't simply package this defaulting as an adapter container. And this is a totally reasonable approach, but it does mean that we are going to couple the scale of the defaulting service with the API service itself. The defaulting is actually a lightweight operation, and we are likely to need far fewer instances of it than the service itself to handle the load.

Với phần thảo luận trước đây về mẫu adapter trong phần đơn nút (single-node), bạn có thể đang thắc mắc tại sao chúng ta không đơn giản đóng gói việc gán mặc định này thành một container adapter. Và đây là một cách tiếp cận hoàn toàn hợp lý, nhưng nó đồng nghĩa với việc chúng ta sẽ ràng buộc quy mô của dịch vụ gán mặc định với chính dịch vụ API. Việc gán mặc định thực ra là một thao tác nhẹ, và ta có khả năng cần ít thực thể của nó hơn nhiều so với chính dịch vụ để xử lý tải.



For the examples in this chapter, we are going to use the **kubeless** FaaS framework. Kubeless is deployed on top of the Kubernetes container orchestration service. Assuming that you have provisioned a Kubernetes cluster, you can install Kubeless from its releases page . Once you have the kubeless binary installed, you can install it into your cluster with the following commands: kubeless

Đối với các ví dụ trong chương này, chúng ta sẽ dùng framework FaaS kubeless. Kubeless được triển khai trên nền dịch vụ điều phối container Kubernetes. Giả sử bạn đã cấp phát (provision) một cụm Kubernetes, bạn có thể cài đặt Kubeless từ trang releases của nó. Một khi đã cài đặt tệp nhị phân kubeless, bạn có thể cài nó vào cụm của mình bằng các lệnh sau: kubeless

install.

Kubeless installs itself as a native Kubernetes third-party API. This means that once it is installed, you can use the native **kubectrl** command-line tool. For example, you can see deployed functions using `kubectrl get functions`. Currently, you should have no functions deployed.

Kubeless tự cài đặt như một API bên thứ ba (third-party API) gốc của Kubernetes. Điều này có nghĩa là một khi nó được cài, bạn có thể dùng công cụ dòng lệnh `kubectrl` gốc. Ví dụ, bạn có thể xem các hàm đã triển khai bằng `kubectrl get functions`. Hiện tại, bạn sẽ chưa có hàm nào được triển khai.

Hands On: Adding Request Defaulting Prior to Request Processing — Thực hành: Thêm Gán mặc định cho Yêu cầu Trước khi Xử lý Yêu cầu

To demonstrate the utility of the decorator pattern for FaaS, consider the task of adding default values to a RESTful function call if the values are missing. This is quite straightforward to do using FaaS. We'll write the defaulting function using the Python programming language:

Để minh họa tính hữu ích của mẫu decorator cho FaaS, hãy xem xét tác vụ thêm các giá trị mặc định vào một lời gọi hàm RESTful nếu các giá trị bị thiếu. Điều này khá dễ thực hiện bằng FaaS. Chúng ta sẽ viết hàm gán mặc định bằng ngôn ngữ lập trình Python:

```
# Simple handler function for adding default values
def handler(context):
    # Get the input value
    obj = context.json
    # If the 'name' field is not present, set it randomly
    if obj.get("name", None) is None:
        obj["name"] = random_name()
    # If the 'color' field is not present, set it to 'blue'
    if obj.get("color", None) is None:
        obj["color"] = "blue"
    # Call the actual API, potentially with the new default
    # values, and return the result
    return call_my_api(obj)
```

Save this function in a file named `defaults.py`. You obviously will want to update the `call_my_api` code so that it points to the actual API you want to call. Once you have finished writing the code, this defaulting function can be installed as a kubeless function using:

Lưu hàm này vào một tệp tên là `defaults.py`. Hiển nhiên bạn sẽ muốn cập nhật mã `call_my_api` để nó trỏ đến API thực tế mà bạn muốn gọi. Một khi đã viết xong mã, hàm gán mặc định này có thể được cài đặt như một hàm kubeless bằng:

```
kubeless function deploy add-defaults \
  --runtime python27 \
  --handler defaults.handler \
  --from-file defaults.py \
  --trigger-http
```

If you want to test the handling of this function, you can also use the kubeless tool:

Nếu bạn muốn kiểm thử việc xử lý của hàm này, bạn cũng có thể dùng công cụ kubeless:

```
kubeless function call add-defaults --data '{"name": "foo"}'
```

The decorator pattern shows just how easy it is to adapt and extend existing APIs with additional features like validation or defaulting.

Mẫu decorator cho thấy việc thích nghi và mở rộng các API hiện có với các tính năng bổ sung như xác thực (validation) hay gán mặc định dễ dàng đến mức nào.

Handling Events — Xử lý Sự kiện

While most systems are request driven, handling a steady stream of user and API requests, many other systems are more event-driven in nature. The differentiation, in my mind at least, between a request and an event have to do with the notion of session . Requests are part of a larger series of interactions or sessions; generally each user request is part of a larger interaction with a complete web application or API. Events , as I see them, instead tend to be single-instance and asynchronous in nature. Events are important and need to be properly handled, but they are fired off from a main interaction and responded to some time later. Examples of events include a user signing up for a new service (which might trigger a welcome email, someone uploading a file to a shared folder (which might send notifications to everyone who has access to the folder), or even a machine being about to reboot (which might notify an operator or automated system to take appropriate action).

Mặc dù hầu hết hệ thống là hướng yêu cầu, xử lý một luồng yêu cầu đều đặn từ người dùng và API, nhiều hệ thống khác lại mang bản chất hướng sự kiện hơn. Sự khác biệt, ít nhất là theo tôi, giữa một yêu cầu và một sự kiện liên quan đến khái niệm phiên (session). Yêu cầu là một phần của một chuỗi tương tác hoặc phiên lớn hơn; nhìn chung mỗi yêu cầu của người dùng là một phần của một tương tác lớn hơn với một ứng dụng web hoặc API hoàn chỉnh. Sự kiện, theo cách tôi nhìn nhận, thay vào đó thường mang bản chất đơn lẻ và bất đồng bộ. Sự kiện rất quan trọng và cần được xử lý đúng cách, nhưng chúng được bắn ra từ một tương tác chính và được phản hồi sau đó một khoảng thời gian. Ví dụ về sự kiện gồm một người dùng đăng ký một dịch vụ mới (có thể kích hoạt một email chào mừng), ai đó tải một tệp lên thư mục chia sẻ (có thể gửi thông báo đến tất cả những người có quyền truy cập thư mục đó), hoặc thậm chí một máy sắp khởi động lại (có thể thông báo cho một người vận hành hoặc một hệ thống tự động để thực hiện hành động phù hợp).

Because these events tend to be largely independent and stateless in nature, and because the rate of events can be highly variable, they are ideal candidates for event-driven and FaaS architectures. In this role, they are often deployed alongside a production application server as augmentation to the main user experience, or to handle some sort of reactive, background processing. Additionally, because new events are often dynamically added to the service, the lightweight nature of deploying functions is a good match for defining new event handlers. Likewise, because each event is conceptually independent, the forced decoupling of a functions-based system actually helps **reduce** the conceptual complexity by enabling a developer to focus on the steps required to handle just a single type of event.

Vì các sự kiện này có xu hướng phần lớn độc lập và không trạng thái về bản chất, và vì tốc độ phát sinh sự kiện có thể biến động rất mạnh, chúng là những ứng viên lý tưởng cho các kiến trúc hướng sự kiện và FaaS. Trong vai trò này, chúng thường được triển khai cùng với một máy chủ ứng dụng sản xuất như một sự bổ sung cho trải nghiệm chính của người dùng, hoặc để xử lý một dạng xử lý nền phản ứng (reactive) nào đó. Ngoài ra, vì các sự kiện mới thường được thêm vào dịch vụ một cách động, bản chất nhẹ nhàng của việc

triển khai hàm rất phù hợp để định nghĩa các trình xử lý sự kiện mới. Tương tự, vì mỗi sự kiện về mặt khái niệm là độc lập, việc tách rời cưỡng bức của một hệ thống dựa trên hàm thực sự giúp giảm độ phức tạp khái niệm bằng cách cho phép lập trình viên tập trung vào các bước cần thiết để xử lý chỉ một loại sự kiện duy nhất.

A concrete example of integrating an event-based component to an existing service is implementing **two-factor authentication**. In this case, the event is the user logging into a service. The service can generate an event for this action, fire it into a function-based handler that takes the code and the user's contact information, and sends the two-factor code via text message.

Một ví dụ cụ thể về việc tích hợp một thành phần dựa trên sự kiện vào một dịch vụ hiện có là hiện thực xác thực hai yếu tố (two-factor authentication). Trong trường hợp này, sự kiện là việc người dùng đăng nhập vào một dịch vụ. Dịch vụ có thể sinh ra một sự kiện cho hành động này, bắn nó vào một trình xử lý dựa trên hàm nhận mã và thông tin liên hệ của người dùng, và gửi mã hai yếu tố qua tin nhắn văn bản.

Hands On: Implementing Two-Factor Authentication — Thực hành: Hiện thực Xác thực Hai Yếu tố

Two-factor authentication requires that the user both have something that they know (e.g., a password) as well as something that they possess (e.g., a phone) to be able to log in to the system. Two-factor authentication is significantly more secure than passwords alone since it requires two different security compromises (a thief learning your password and a thief stealing your phone) to enable a true security problem.

Xác thực hai yếu tố đòi hỏi người dùng phải vừa có thứ họ biết (ví dụ, một mật khẩu) vừa có thứ họ sở hữu (ví dụ, một chiếc điện thoại) để có thể đăng nhập vào hệ thống. Xác thực hai yếu tố an toàn hơn đáng kể so với chỉ dùng mật khẩu vì nó đòi hỏi hai sự xâm phạm bảo mật khác nhau (kẻ trộm biết được mật khẩu của bạn và kẻ trộm lấy được điện thoại của bạn) mới gây ra một vấn đề bảo mật thực sự.

When considering how to implement two-factor authentication, one of the challenges is how to handle the request to generate a random code and register it with the login service as well as send the text message. It is possible to add this code to the main login web server. But it is complicated and monolithic, and forces the act of sending a text message, which can have some latency, to be inline with the code that renders the login web page. This latency produces a substandard user experience.

Khi cân nhắc cách hiện thực xác thực hai yếu tố, một trong những thách thức là làm thế nào để xử lý yêu cầu sinh ra một mã ngẫu nhiên và đăng ký nó với dịch vụ đăng nhập cũng như gửi tin nhắn văn bản. Có thể thêm mã này vào máy chủ web đăng nhập chính. Nhưng nó phức tạp và nguyên khối (monolithic), và buộc hành động gửi tin nhắn văn bản, vốn có thể có độ trễ nào đó, phải nằm chung dòng (inline) với mã render trang web đăng nhập. Độ trễ này tạo ra trải nghiệm người dùng dưới mức tiêu chuẩn.

A better option is to register a FaaS to asynchronously generate the random number, register it with the login service, and send the number to the user's phone. In this way, the login server can simply fire an asynchronous web-hook request to a FaaS, and that FaaS can handle the somewhat slow and asynchronous task of registering the two-factor code and sending the text message.

Một lựa chọn tốt hơn là đăng ký một FaaS để sinh số ngẫu nhiên một cách bất đồng bộ, đăng ký nó với dịch vụ đăng nhập, và gửi số đó đến điện thoại của người dùng. Bằng cách này, máy chủ đăng nhập có thể chỉ cần bắn một yêu cầu web-hook bất đồng bộ tới một

FaaS, và FaaS đó có thể xử lý tác vụ có phần chậm và bất đồng bộ là đăng ký mã hai yếu tố và gửi tin nhắn văn bản.

To see how this works in practice, consider the following code:

Để xem cách này hoạt động trong thực tế, hãy xem xét đoạn mã sau:

```
def two_factor(context):
    # Generate a random six digit code
    code = random.randint(100000, 999999)
    # Register the code with the login service
    user = context.json["user"]
    register_code_with_login_service(user, code)
    # Use the twilio library to send texts
    account = "my-account-sid"
    token = "my-token"
    client = twilio.rest.Client(account, token)
    user_number = context.json["phoneNumber"]
    msg = "Hello {} your authentication code is: {}".format(user, code)
    message = client.api.account.messages.create(to=user_number,
                                                  from_="+12065251212",
                                                  body=msg)

    return {"status": "ok"}
```

We can then register this FaaS with kubeless :

Sau đó chúng ta có thể đăng ký FaaS này với kubeless:

```
kubeless function deploy add-two-factor \
  --runtime python27 \
  --handler two_factor.two_factor \
  --from-file two_factor.py \
  --trigger-http
```

This function can then be made asynchronously from client-side JavaScript whenever the user successfully provides their password. The web UX can then immediately display a page to enter the code, and the user (once they receive the code as a text message) can supply it to the service, where the code has already been registered via our FaaS.

Hàm này sau đó có thể được gọi một cách bất đồng bộ từ JavaScript phía client bất cứ khi nào người dùng cung cấp mật khẩu thành công. Giao diện web khi đó có thể ngay lập tức hiển thị một trang để nhập mã, và người dùng (một khi họ nhận được mã dưới dạng tin nhắn văn bản) có thể cung cấp nó cho dịch vụ, nơi mã đã được đăng ký sẵn thông qua FaaS của chúng ta.

Again, developing a simple, asynchronous, event-based service that is triggered whenever a user logs in is made dramatically simpler using FaaS.

Một lần nữa, việc phát triển một dịch vụ đơn giản, bất đồng bộ, dựa trên sự kiện được kích hoạt mỗi khi người dùng đăng nhập trở nên đơn giản hơn đáng kể nhờ FaaS.

Event-Based Pipelines — Pipeline Dựa trên Sự kiện

There are some applications that are inherently easier to think about in terms of a **pipeline** of decoupled events. These event pipelines often resemble the flowcharts of old. They can be

represented as a directed graph of connected event syncs. In the event pipeline pattern, each node is a different function or **webhook**, and the edges linking the graph together are HTTP or other network calls to the function/webhook. In general, there is no shared state between the different pieces of the pipeline, but there may be a context or other reference point that can be used to look up information in shared storage.

Có một số ứng dụng vốn dĩ dễ hình dung hơn dưới dạng một pipeline gồm các sự kiện được tách rời. Các pipeline sự kiện này thường giống các lưu đồ (flowchart) thời xưa. Chúng có thể được biểu diễn dưới dạng một đồ thị có hướng gồm các điểm tiếp nhận sự kiện được kết nối với nhau. Trong mẫu pipeline sự kiện, mỗi nút là một hàm hoặc webhook khác nhau, và các cạnh nối đồ thị lại với nhau là các lời gọi HTTP hoặc lời gọi mạng khác tới hàm/webhook. Nhìn chung, không có trạng thái chia sẻ giữa các mảnh khác nhau của pipeline, nhưng có thể có một ngữ cảnh (context) hoặc điểm tham chiếu khác có thể được dùng để tra cứu thông tin trong bộ lưu trữ chia sẻ.

So what is the difference between this type of pipeline and a “microservices” architecture? There are two central differences. The first is the main difference between functions in general and long-running services, which is that an **event-based pipeline** is by its very nature event-driven. Conversely, a microservices architecture features a collection of long-running services. Additionally, event-driven pipelines may be highly asynchronous and diverse in the things that they connect together. For example, while it is difficult to see how a human approving a ticket in a ticketing system like Jira could be integrated into a microservices application, it’s quite easy to see how that event could be incorporated into an event-driven pipeline.

Vậy sự khác biệt giữa loại pipeline này và một kiến trúc “microservices” là gì? Có hai sự khác biệt cốt lõi. Thứ nhất là sự khác biệt chính giữa hàm nói chung và các dịch vụ chạy lâu dài, đó là một pipeline dựa trên sự kiện về bản chất của nó là hướng sự kiện. Ngược lại, một kiến trúc microservices đặc trưng bởi một tập hợp các dịch vụ chạy lâu dài. Ngoài ra, các pipeline hướng sự kiện có thể mang tính bất đồng bộ cao và đa dạng về những thứ mà chúng kết nối lại với nhau. Ví dụ, trong khi khó hình dung làm sao một con người phê duyệt một ticket trong một hệ thống quản lý ticket như Jira lại có thể được tích hợp vào một ứng dụng microservices, thì lại khá dễ hình dung làm sao sự kiện đó có thể được đưa vào một pipeline hướng sự kiện.

As an example of this, imagine a pipeline in which the original event is code being submitted into a source control system. This event then triggers a build. The build may take multiple minutes to complete, and when it does, it fires an event to a build analysis function. This function takes different actions if the build is successful or fails. If the build succeeded, a ticket is created for a human to approve it to be pushed to production. Once the ticket is closed, the act of closing is an event that triggers the actual push to production. If the build failed, a bug is filed on the failure, and the event pipeline terminates.

Lấy ví dụ minh họa, hãy hình dung một pipeline trong đó sự kiện ban đầu là mã được gửi (submit) vào một hệ thống quản lý mã nguồn. Sự kiện này sau đó kích hoạt một bản build. Bản build có thể mất nhiều phút để hoàn thành, và khi xong, nó bắn một sự kiện đến một hàm phân tích build. Hàm này thực hiện các hành động khác nhau tùy theo build thành công hay thất bại. Nếu build thành công, một ticket được tạo ra để một con người phê duyệt nó được đẩy lên production. Một khi ticket được đóng, hành động đóng ticket là một sự kiện kích hoạt việc đẩy thực sự lên production. Nếu build thất bại, một bug được lập về sự cố đó, và pipeline sự kiện kết thúc.

Hands On: Implementing a Pipeline for New-User Signup — Thực hành: Hiện thực một Pipeline cho Đăng ký Người dùng Mới

Consider the task of implementing a new-user signup flow. When a new user account is created, there are certain things that are always done, such as sending a welcome email. And there are some things that are optionally done, such as registering a user to receive product updates (sometimes known as “spam”) via their email.

Hãy xem xét tác vụ hiện thực một luồng đăng ký người dùng mới. Khi một tài khoản người dùng mới được tạo, có những việc luôn được thực hiện, chẳng hạn gửi email chào mừng. Và có một số việc được thực hiện tùy chọn, chẳng hạn đăng ký cho người dùng nhận các cập nhật sản phẩm (đôi khi gọi là “spam”) qua email của họ.

One approach to implementing this logic would be to place everything into a single monolithic user-creation server. However, this factoring means that a single team owns the entirety of the user-creation service, and that the entire experience is deployed as a single service. Both of these mean that it is more difficult to perform experiments or make changes to the user experience.

Một cách tiếp cận để hiện thực logic này là đặt mọi thứ vào một máy chủ tạo người dùng nguyên khối duy nhất. Tuy nhiên, cách phân chia này có nghĩa là một đội duy nhất sở hữu toàn bộ dịch vụ tạo người dùng, và toàn bộ trải nghiệm được triển khai như một dịch vụ duy nhất. Cả hai điều này đều có nghĩa là việc thực hiện các thử nghiệm hay thay đổi trải nghiệm người dùng trở nên khó khăn hơn.

Consider, instead, implementing the user login experience as an event pipeline with a series of FaaS. In this factoring, the user-creation function is actually unaware of the details of what happens on user login. Instead, the main user-creation service simply has two lists:

Thay vào đó, hãy xem xét việc hiện thực trải nghiệm đăng nhập của người dùng như một pipeline sự kiện với một loạt FaaS. Trong cách phân chia này, hàm tạo người dùng thực ra không hề biết về chi tiết những gì xảy ra khi người dùng đăng nhập. Thay vào đó, dịch vụ tạo người dùng chính chỉ đơn giản có hai danh sách:

- A list of required actions (e.g., sending a welcome mail)
- A list of optional actions (e.g., subscribing the user to a mailing list)
 - Một danh sách các hành động bắt buộc (ví dụ, gửi một thư chào mừng)
 - Một danh sách các hành động tùy chọn (ví dụ, đăng ký người dùng vào một danh sách gửi thư)

Each of these actions is also implemented as a FaaS, and the list of actions is actually just a list of webhooks. Consequently, the main user creation function looks like this:

Mỗi hành động này cũng được hiện thực như một FaaS, và danh sách các hành động thực ra chỉ là một danh sách các webhook. Do đó, hàm tạo người dùng chính trông như sau:

```
def create_user(context):
    # For required event handlers, call them universally
    for key, value in required.items():
        call_function(value.webhook, context.json)
    # For optional event handlers, check and call them
    # conditionally
    for key, value in optional.items():
        if context.json.get(key, None) is not None:
            call_function(value.webhook, context.json)
```

Now we can also use FaaS to implement each of these handlers:

Giờ chúng ta cũng có thể dùng FaaS để hiện thực từng trình xử lý này:

```
def email_user(context):
    # Get the user name
    user = context.json['username']
    msg = 'Hello {} thanks for joining my awesome service!'.format(user)
    send_email(msg, context.json['email'])
def subscribe_user(context):
    # Get the user name
    email = context.json['email']
    subscribe_user(email)
```

Factored in this way, each FaaS is simple, containing only a few lines of code and focused on implementing one specific piece of functionality. This microservices-based approach is simple to write but might lead to complexity if we actually had to deploy and manage three different microservices. This is where FaaS can shine, since it makes it trivially easy to host these small code snippets. Additionally, by visualizing our user-creation flow as an event-driven pipeline, it is also straightforward to have a high-level understanding of what exactly happens on user login, simply by following the flow of the context through the various functions in the pipeline.

Được phân chia theo cách này, mỗi FaaS đều đơn giản, chỉ chứa vài dòng mã và tập trung vào hiện thực một mảnh chức năng cụ thể. Cách tiếp cận dựa trên microservices này đơn giản để viết nhưng có thể dẫn đến phức tạp nếu chúng ta thực sự phải triển khai và quản lý ba microservice khác nhau. Đây là nơi FaaS có thể tỏa sáng, vì nó khiến việc lưu trữ các đoạn mã nhỏ này trở nên dễ dàng đến mức tầm thường. Ngoài ra, bằng cách hình dung luồng tạo người dùng của chúng ta như một pipeline hướng sự kiện, cũng rất dễ để có một hiểu biết ở mức cao về chính xác điều gì xảy ra khi người dùng đăng nhập, đơn giản bằng cách dõi theo luồng của ngữ cảnh đi qua các hàm khác nhau trong pipeline.

CHAPTER 9 Ownership Election —

CHAPTER 9 Bầu chọn Quyền sở hữu

The previous patterns that we have seen have been about distributing requests in order to scale requests per second, the state being served, or the time to process a request. This final chapter on multi-**node** serving patterns is about how you scale assignment. In many different systems, there is a notion of **ownership** where a specific process owns a specific task. We have previously seen this in the context of sharded and hot-sharded systems where specific instances owned specific sections of the sharded key space.

Các mẫu thiết kế mà chúng ta đã xem trước đây đều xoay quanh việc phân phối yêu cầu nhằm mở rộng số yêu cầu trên giây, lượng trạng thái được phục vụ, hoặc thời gian xử lý một yêu cầu. Chương cuối cùng này về các mẫu phục vụ đa nút bàn về cách bạn mở rộng việc gán nhiệm vụ (assignment). Trong nhiều hệ thống khác nhau, có một khái niệm về quyền sở hữu (ownership) trong đó một tiến trình cụ thể sở hữu một tác vụ cụ thể. Trước đây chúng ta đã thấy điều này trong bối cảnh các hệ thống phân mảnh (sharded) và phân mảnh nóng (hot-sharded), nơi các thực thể cụ thể sở hữu những phần cụ thể của không gian khóa phân mảnh.

In the context of a single server, ownership is generally straightforward to achieve because there is only a single application that is establishing ownership, and it can use well-established in-process locks to ensure that only a single actor owns a particular **shard** or context. However, restricting ownership to a single application limits **scalability**, since the task can't be replicated, and **reliability**, since if the task fails, it is unavailable for a period of time. Consequently, when ownership is required in your system, you need to develop a **distributed system** for establishing ownership.

Trong bối cảnh một máy chủ đơn lẻ, quyền sở hữu nhìn chung khá dễ đạt được vì chỉ có một ứng dụng duy nhất thiết lập quyền sở hữu, và nó có thể dùng các khóa (lock) trong tiến trình đã được kiểm chứng để đảm bảo chỉ một actor duy nhất sở hữu một mảnh hoặc ngữ cảnh nhất định. Tuy nhiên, giới hạn quyền sở hữu ở một ứng dụng duy nhất sẽ hạn chế khả năng mở rộng, vì tác vụ không thể được nhân bản, và hạn chế độ tin cậy, vì nếu tác vụ thất bại thì nó sẽ không khả dụng trong một khoảng thời gian. Do đó, khi hệ thống của bạn cần quyền sở hữu, bạn phải phát triển một hệ phân tán để thiết lập quyền sở hữu đó.

A general diagram of distributed ownership is shown in Figure 9-1. In the diagram, there are three replicas that could be the owner or master. Initially, the first **replica** is the master. Then that replica fails, and replica number three then becomes the master. Finally, replica number one recovers and returns to the group, but replica three remains as the master/owner.

Một sơ đồ tổng quát về quyền sở hữu phân tán được minh họa trong Hình 9-1. Trong sơ đồ, có ba bản sao (replica) có thể trở thành chủ sở hữu hoặc master. Ban đầu, bản sao đầu

tiên là master. Sau đó bản sao này thất bại, và bản sao số ba trở thành master. Cuối cùng, bản sao số một phục hồi và quay lại nhóm, nhưng bản sao số ba vẫn là master/chủ sở hữu.

93

93

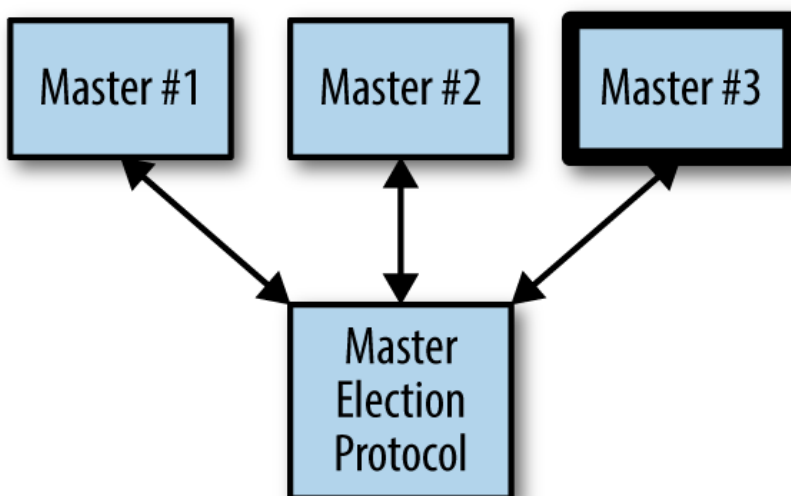
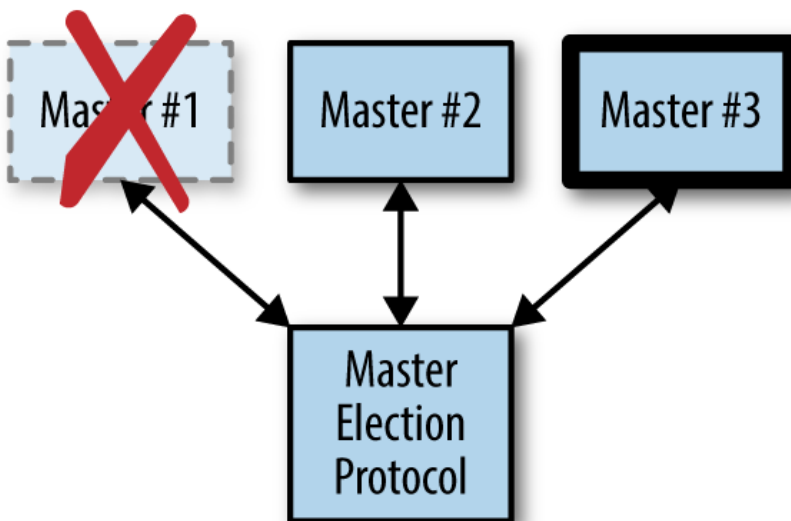
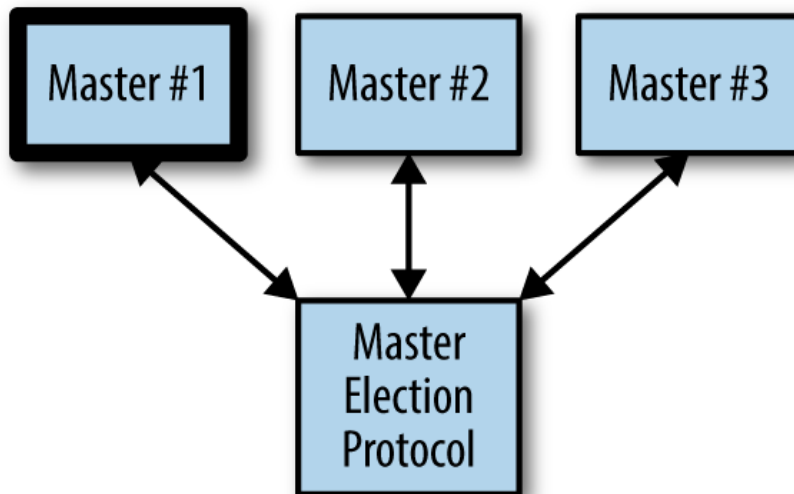


Figure 9-1. A **master election** protocol in operation: initially the first master is selected, but when it fails, the third master takes over

Hình 9-1. Giao thức bầu chọn master đang hoạt động: ban đầu master đầu tiên được chọn, nhưng khi nó thất bại, master thứ ba tiếp quản

Often, establishing distributed ownership is both the most complicated and most important part of designing a reliable distributed system.

Thông thường, việc thiết lập quyền sở hữu phân tán vừa là phần phức tạp nhất vừa là phần quan trọng nhất trong việc thiết kế một hệ phân tán đáng tin cậy.

Determining If You Even Need Master Election — Xác định Liệu Bạn Có Thực Sự Cần Bầu chọn Master Hay Không

The simplest form of ownership is to just have a single replica of the **service**. Since there is only one instance running at a time, that instance implicitly owns everything without any need for election. This has advantages of simplifying your application and **deployment**, but it has disadvantages in terms of downtime and reliability. However, for many applications, the simplicity of this singleton pattern may be worth the reliability trade-off. Let's look at this further.

Dạng quyền sở hữu đơn giản nhất là chỉ có một bản sao duy nhất của dịch vụ. Vì tại một thời điểm chỉ có một thực thể đang chạy, thực thể đó ngầm định sở hữu mọi thứ mà không cần bầu chọn. Cách này có ưu điểm là đơn giản hóa ứng dụng và việc triển khai, nhưng lại có nhược điểm về thời gian ngừng hoạt động (downtime) và độ tin cậy. Tuy nhiên, với nhiều ứng dụng, sự đơn giản của mẫu singleton này có thể đáng để đánh đổi với độ tin cậy. Hãy cùng xem xét kỹ hơn.

Assuming that you run your singleton in a **container** orchestration system like **Kubernetes**, you have the following guarantees:

Giả sử bạn chạy singleton của mình trong một hệ thống điều phối container (orchestration) như Kubernetes, bạn sẽ có những đảm bảo sau:

- If the container crashes, it will automatically be restarted
- If the container hangs, and you implement a **health check**, it will automatically be restarted
- If the machine fails, the container will be moved to a different machine
 - Nếu container bị crash, nó sẽ tự động được khởi động lại
 - Nếu container bị treo, và bạn có triển khai kiểm tra sức khỏe (health check), nó sẽ tự động được khởi động lại
 - Nếu máy bị hỏng, container sẽ được chuyển sang một máy khác

Because of these guarantees, a singleton of a service running in a **container orchestrator** has pretty good uptime. To take the definition of “pretty good” a little further, let's examine what happens in each of these failure modes. If the container process fails or the container hangs, your application will be restarted in a few seconds. Assuming your container crashes once a day, this is roughly three to four nines of uptime (2 seconds of downtime / day $\approx$ 99.99% uptime). If your container crashes less often, it's even better than that. If your machine fails, it takes a while for Kubernetes to decide that the machine has failed and move it over to a different machine; let's assume that takes around 5 minutes. Given that, if every machine in your cluster fails every day, then your service will have two nines of uptime. And honestly, if every machine in your cluster fails every day, then you have way worse problems than the uptime of your master-elected service.

Nhờ những đảm bảo này, một singleton của dịch vụ chạy trong bộ điều phối container có thời gian hoạt động (uptime) khá tốt. Để hiểu rõ hơn định nghĩa “khá tốt” này, hãy xem xét điều gì xảy ra trong từng chế độ thất bại. Nếu tiến trình container thất bại hoặc container bị treo, ứng dụng của bạn sẽ được khởi động lại trong vài giây. Giả sử container của bạn crash một lần mỗi ngày, con số này tương đương khoảng ba đến bốn số chín về uptime (2 giây downtime / ngày $\sim$ 99,99% uptime). Nếu container của bạn crash ít hơn thì còn tốt hơn nữa. Nếu máy của bạn hỏng, Kubernetes sẽ mất một khoảng thời gian để quyết định rằng máy đã hỏng và chuyển nó sang máy khác; giả sử quá trình đó mất khoảng 5 phút. Với giả định đó, nếu mỗi máy trong cụm (cluster) của bạn hỏng mỗi ngày, thì dịch vụ của bạn sẽ có uptime hai số chín. Và thành thật mà nói, nếu mỗi máy trong cụm của bạn hỏng mỗi ngày, thì bạn còn gặp những vấn đề tệ hơn nhiều so với uptime của dịch vụ được bầu chọn master.

It's worth considering, of course, that there are more reasons for downtime than just failures. When you are rolling out new software, it takes time to download and start the new version. With a singleton, you cannot have both old and new versions running at the same time, so you will need to take down the old version for the duration of the upgrade, which may be several minutes if your image is large. Consequently, if you deploy daily and it takes 2 minutes to upgrade your software, you will only be able to run a two nines service, and if you deploy hourly, it won't even be a single nine service. Of course, there are ways that you can speed up your deployment by pre-pulling the new image onto the machine before you run the update. This can **reduce** the time it takes to deploy a new version to a few seconds, but the trade-off is added complexity, which was what we were trying to avoid in the first place.

Tất nhiên, đáng để cân nhắc rằng còn có nhiều lý do gây downtime khác ngoài việc thất bại. Khi bạn tung ra phần mềm mới, sẽ mất thời gian để tải về và khởi động phiên bản mới. Với một singleton, bạn không thể chạy đồng thời cả phiên bản cũ và mới, nên bạn sẽ phải hạ phiên bản cũ trong suốt thời gian nâng cấp, có thể kéo dài vài phút nếu image của bạn lớn. Do đó, nếu bạn triển khai hằng ngày và mất 2 phút để nâng cấp phần mềm, bạn sẽ chỉ chạy được một dịch vụ hai số chín, và nếu bạn triển khai hằng giờ thì thậm chí còn không đạt nổi một số chín. Tất nhiên, có những cách để tăng tốc việc triển khai bằng cách kéo trước (pre-pull) image mới về máy trước khi chạy bản cập nhật. Cách này có thể giảm thời gian triển khai một phiên bản mới xuống còn vài giây, nhưng cái giá phải trả là sự phức tạp tăng thêm, vốn là điều mà ngay từ đầu chúng ta đang cố tránh.

Regardless, there are many applications (e.g., background asynchronous processing) where such an **SLA** is an acceptable trade-off for application simplicity. One of the key components of designing a distributed system is deciding when the “distributed” part is actually unnecessarily complex. But there are certainly situations where high **availability** (four+ nines) is a critical component of the application, and in such systems you need to run multiple replicas of the service, where only one replica is the designated owner. The design of these types of systems is described in the sections that follow.

Dù sao đi nữa, có nhiều ứng dụng (ví dụ, xử lý bất đồng bộ ở chế độ nền) mà ở đó một SLA như vậy là một sự đánh đổi chấp nhận được để đổi lấy sự đơn giản của ứng dụng. Một trong những thành phần then chốt khi thiết kế một hệ phân tán là quyết định khi nào phần “phân tán” thực ra là phức tạp một cách không cần thiết. Nhưng chắc chắn có những tình huống mà tính sẵn sàng cao (bốn số chín trở lên) là thành phần thiết yếu của ứng dụng, và trong những hệ thống như vậy bạn cần chạy nhiều bản sao của dịch vụ, trong đó chỉ một bản sao là chủ sở hữu được chỉ định. Việc thiết kế các loại hệ thống này được mô tả trong các phần tiếp theo.

The Basics of Master Election — Những Điều Cơ Bản Về Bầu chọn Master

Imagine that there is a service Foo with three replicas: Foo-1 , Foo-2 , and Foo-3 . There is also some object Bar that must only be “owned” by one of the replicas (e.g., Foo-1) at a time. Often this replica is called the master , hence the term master election used to describe the process of how this master is selected as well as how a new master is selected if that master fails.

Hãy hình dung có một dịch vụ Foo với ba bản sao: Foo-1, Foo-2 và Foo-3. Cũng có một đối tượng Bar nào đó mà tại một thời điểm chỉ được “sở hữu” bởi một trong các bản sao (ví dụ, Foo-1). Thông thường bản sao này được gọi là master, do đó có thuật ngữ bầu chọn master (master election) dùng để mô tả quá trình chọn ra master này cũng như cách một master mới được chọn nếu master đó thất bại.

There are two ways to implement this master election. This first is to implement a distributed consensus algorithm like Paxos or RAFT, but the complexity of these algorithms make them beyond the scope of this book and not worthwhile to implement. Implementing one of these algorithms is akin to implementing locks on top of assembly code compare-and-swap instructions. It’s an interesting exercise for an undergraduate computer science course, but it is not something that is generally worth doing in practice.

Có hai cách để triển khai việc bầu chọn master này. Cách thứ nhất là triển khai một thuật toán đồng thuận phân tán (distributed consensus) như Paxos hoặc RAFT, nhưng độ phức tạp của các thuật toán này khiến chúng nằm ngoài phạm vi của cuốn sách và không đáng để tự triển khai. Triển khai một trong các thuật toán này cũng giống như triển khai khóa (lock) dựa trên các lệnh compare-and-swap ở cấp hợp ngữ (assembly). Đó là một bài tập thú vị cho một khóa học khoa học máy tính bậc đại học, nhưng nói chung không phải là thứ đáng làm trong thực tế.

Fortunately, there are a large number of distributed key-value stores that have implemented such consensus algorithms for you. At a general level, these systems provide a replicated, reliable data store and the primitives necessary to build more complicated locking and election abstractions on top. Examples of these distributed stores include etcd, ZooKeeper, and consul. The basic primitives that these systems provide is the ability to perform a compare-and-swap operation for a particular key. If you haven’t seen compare-and-swap before, it is basically an atomic operation that looks like this:

May mắn thay, có rất nhiều kho key-value (key-value store) phân tán đã triển khai sẵn các thuật toán đồng thuận như vậy cho bạn. Ở mức tổng quát, các hệ thống này cung cấp một kho dữ liệu được nhân bản, đáng tin cậy, cùng với các nguyên thủy (primitive) cần thiết để xây dựng các trừu tượng khóa và bầu chọn phức tạp hơn ở bên trên. Ví dụ về các kho phân tán này gồm etcd, ZooKeeper và consul. Nguyên thủy cơ bản mà các hệ thống này cung cấp là khả năng thực hiện thao tác compare-and-swap cho một khóa cụ thể. Nếu bạn chưa từng thấy compare-and-swap, về cơ bản đó là một thao tác nguyên tử (atomic) trông như thế này:

```
var lock = sync.Mutex{}
var store = map[string]string{}
func compareAndSwap(key, nextValue, currentValue string) (bool, error) {
    lock.Lock()
    defer lock.Unlock()
    if _, found := store[key]; found {
        if len(currentValue) == 0 {
```

```

        store[key] = nextValue
        return true, nil
    }
    return false, fmt.Errorf("Expected value %s for key %s, but
found empty", currentValue, key)
}
if store[key] == currentValue {
    store[key] = nextValue
    return true, nil
}
return false, nil
}

```

Compare-and-swap atomically writes a new value if the existing value matches the expected value. If the value doesn't match, it returns false. If the value doesn't exist and currentValue is not null, it returns an error.

Compare-and-swap ghi một giá trị mới một cách nguyên tử nếu giá trị hiện có khớp với giá trị kỳ vọng. Nếu giá trị không khớp, nó trả về false. Nếu giá trị không tồn tại và currentValue không phải null, nó trả về một lỗi.

In addition to compare-and-swap, the key-value stores allow you to set a time-to-live (TTL) for a key. Once the TTL expires, the key is set back to empty.

Ngoài compare-and-swap, các kho key-value cho phép bạn đặt thời gian sống (time-to-live, TTL) cho một khóa. Khi TTL hết hạn, khóa được đặt trở lại trạng thái rỗng.

Put together, these functions are sufficient to implement a variety of distributed synchronization primitives.

Kết hợp lại, các hàm này đủ để triển khai nhiều nguyên thủy đồng bộ hóa phân tán khác nhau.

Hands On: Deploying etcd — Thực hành: Triển khai etcd

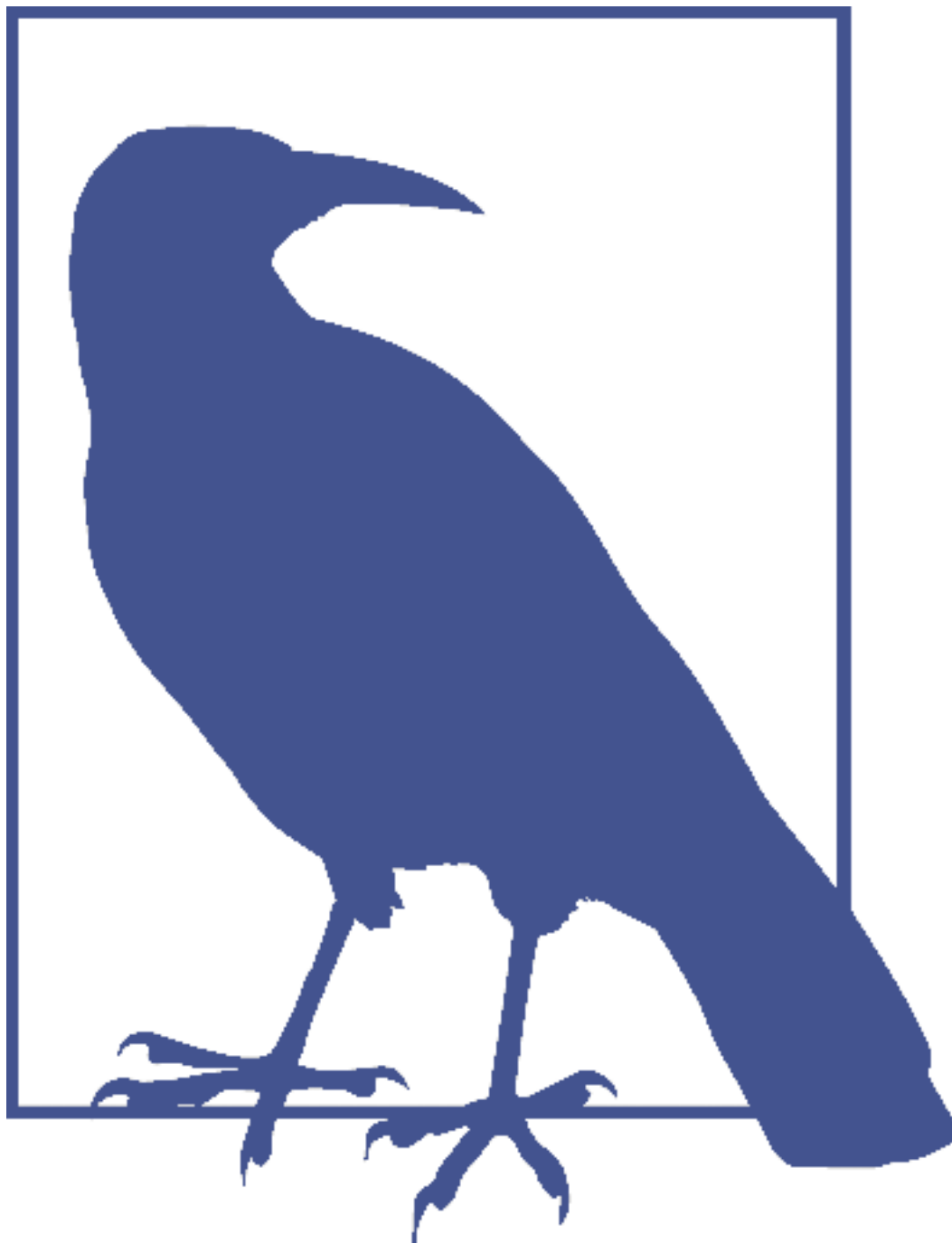
etcd is a **distributed lock** server developed by CoreOS. It is robust and proven in production at high scale, and is used by a variety of projects including Kubernetes.

etcd là một máy chủ khóa phân tán do CoreOS phát triển. Nó mạnh mẽ và đã được kiểm chứng trong môi trường sản xuất ở quy mô lớn, và được nhiều dự án sử dụng, bao gồm cả Kubernetes.

Deploying etcd has fortunately become quite easy due to the development of two different **open source** projects:

May mắn là việc triển khai etcd đã trở nên khá dễ dàng nhờ sự phát triển của hai dự án mã nguồn mở khác nhau:

- Helm : a Kubernetes package manager supported by Microsoft Azure
 - Helm: một trình quản lý gói (package manager) cho Kubernetes được Microsoft Azure hỗ trợ
- The etcd operator developed by CoreOS
 - Toán tử etcd (etcd operator) do CoreOS phát triển



Operators are an interesting topic being explored by CoreOS. An operator is an online program that runs inside your container orchestrator with the express purpose of running one or more applications. The operator is responsible for creating, scaling, and maintaining the successful operation of the program. Users configure the application through a desired state **API**. For example, the etcd operator is in charge of monitoring etcd itself. Operators are still a new idea but represent an important new direction in building reliable distributed systems.

Operator là một chủ đề thú vị đang được CoreOS khám phá. Một operator là một chương trình trực tuyến (online) chạy bên trong bộ điều phối container của bạn với mục đích rõ ràng là vận hành một hoặc nhiều ứng dụng. Operator chịu trách nhiệm tạo, mở rộng và

duy trì việc vận hành thành công của chương trình. Người dùng cấu hình ứng dụng thông qua một API trạng thái mong muốn (desired state). Ví dụ, etcd operator chịu trách nhiệm giám sát chính etcd. Operator vẫn là một ý tưởng mới nhưng đại diện cho một hướng đi mới quan trọng trong việc xây dựng các hệ phân tán đáng tin cậy.

To deploy the etcd operator for CoreOS, we're going to use the helm package management tool. Helm is an open source package manager that is part of the Kubernetes project, and was developed by Deis. Deis was acquired by Microsoft Azure in 2017 and Microsoft continues to support the further open source development of Helm.

Để triển khai etcd operator cho CoreOS, chúng ta sẽ dùng công cụ quản lý gói helm. Helm là một trình quản lý gói mã nguồn mở, là một phần của dự án Kubernetes, và do Deis phát triển. Deis được Microsoft Azure mua lại vào năm 2017 và Microsoft tiếp tục hỗ trợ việc phát triển mã nguồn mở của Helm.

If this is your first time using helm , you need to install the helm tool, following the instructions here: <https://github.com/kubernetes/helm/releases> .

Nếu đây là lần đầu bạn dùng helm, bạn cần cài đặt công cụ helm, theo hướng dẫn tại đây: <https://github.com/kubernetes/helm/releases>.

Once you have the helm tool installed in your environment, you can install the etcd operator using helm , as follows:

Sau khi đã cài công cụ helm trong môi trường của mình, bạn có thể cài etcd operator bằng helm như sau:

```
# Initialize helm
helm init
# Install the etcd operator
helm install stable/etcd-operator
```

Once the operator is installed, it creates a custom Kubernetes resource to represent the etcd cluster. The operator is running, but there are no etcd clusters yet. To create an etcd cluster, you need to create a declarative **configuration**:

Sau khi operator được cài đặt, nó tạo một tài nguyên Kubernetes tùy chỉnh (custom resource) để biểu diễn cụm etcd. Operator đang chạy, nhưng chưa có cụm etcd nào. Để tạo một cụm etcd, bạn cần tạo một cấu hình khai báo (declarative):

```
apiVersion:"etcd.coreos.com/v1beta1"
kind:"Cluster"
metadata:
# Whatever name you want here
  name:"my-etcd-cluster"
spec:
# 1, 3, 5 are the options for size
  size: 3
# The version of etcd to install
  version:"3.1.0"
```

Save this configuration to etcd-cluster.yaml and then create the cluster using **kubectl**

Lưu cấu hình này vào etcd-cluster.yaml rồi tạo cụm bằng kubectl

```
create -f etcd-cluster.yaml.
```

Creating this cluster will cause the the operator to create pods for the replicas of the etcd cluster. You can see the running replicas using:

Việc tạo cụm này sẽ khiến operator tạo các pod cho các bản sao của cụm etcd. Bạn có thể xem các bản sao đang chạy bằng:

```
kubectl get pods
```

Once all three replicas are running, you can get their endpoints using:

Sau khi cả ba bản sao đang chạy, bạn có thể lấy các endpoint của chúng bằng:

```
export ETCD_ENDPOINTS=$(kubectl get endpoints example-etcd-cluster
-o=jsonpath={.subsets[*].addresses[*].ip}:2379,")
```

You can then store something into etcd using:

Sau đó bạn có thể lưu một thứ gì đó vào etcd bằng:

```
kubectl exec my-etcd-cluster-0000 -- sh -c "ETCD_API=3 etcdctl
--endpoints=${ETCD_ENDPOINTS} set foo bar"
```

Implementing Locks — Triển khai Khóa (Lock)

The simplest form of synchronization is the **mutual exclusion lock** (aka Mutex). Anyone who has done concurrent programming on a single machine is familiar with locks, and the same concept can be applied to distributed replicas. Instead of local memory and assembly instructions, these distributed locks can be implemented in terms of the distributed key-value stores described previously.

Dạng đồng bộ hóa đơn giản nhất là khóa loại trừ tương hỗ (mutual exclusion lock, hay Mutex). Bất kỳ ai từng lập trình đồng thời (concurrent) trên một máy đơn lẻ đều quen thuộc với khóa, và cùng khái niệm đó có thể áp dụng cho các bản sao phân tán. Thay vì dùng bộ nhớ cục bộ và các lệnh hợp ngữ, các khóa phân tán này có thể được triển khai dựa trên các kho key-value phân tán đã mô tả ở trên.

As with locks in memory, the first step is to acquire the lock:

Cũng giống như khóa trong bộ nhớ, bước đầu tiên là giành lấy (acquire) khóa:

```
func (Lock l) simpleLock() boolean {
    // compare and swap "1" for "0"
    locked, _ = compareAndSwap(l.lockName, "1", "0")
    return locked
}
```

But of course, it's possible that the lock doesn't already exist, because we are the first to claim it, so we need to handle that case, too:

Nhưng tất nhiên, có khả năng khóa chưa tồn tại, vì chúng ta là người đầu tiên đòi nó, nên chúng ta cũng cần xử lý trường hợp đó:

```
func (Lock l) simpleLock() boolean {
    // compare and swap "1" for "0"
    locked, error = compareAndSwap(l.lockName, "1", "0")
    // lock doesn't exist, try to write "1" with a previous value of
    // non-existent
```

```

if error != nil {
    locked, _ = compareAndSwap(l.lockName, "1", nil)
}
return locked
}

```

Traditional lock implementations block until the lock is acquired, so we actually want something like this:

Các cách triển khai khóa truyền thống sẽ chặn (block) cho đến khi giành được khóa, nên thực ra chúng ta muốn một thứ như thế này:

```

func (Lock l) lock() {
    while (!l.simpleLock()) {
        sleep(2)
    }
}

```

This implementation, though simple, has the problem that you will always wait at least a second after the lock is released before you acquire the lock. Fortunately, many key-value stores let you watch for changes instead of polling, so you can implement:

Cách triển khai này, dù đơn giản, lại có vấn đề là bạn sẽ luôn phải chờ ít nhất một giây sau khi khóa được giải phóng thì mới giành được nó. May mắn là nhiều kho key-value cho phép bạn theo dõi (watch) các thay đổi thay vì hỏi vòng (polling), nên bạn có thể triển khai:

```

func (Lock l) lock() {
    while (!l.simpleLock()) {
        waitForChanges(l.lockName)
    }
}

```

Given this locking **function**, we can also implement unlock:

Với hàm khóa này, chúng ta cũng có thể triển khai unlock (mở khóa):

```

func (Lock l) unlock() {
    compareAndSwap(l.lockName, "0", "1")
}

```

You might now think that we are done, but remember that we are building this for a distributed system. A process could fail in the middle of holding the lock, and at that point there is no one left to release it. In such a situation, our system will become stuck. To resolve this, we take advantage of the TTL functionality of the **key-value store**. We change our simpleLock function so that it always writes with a TTL, so if we don't unlock within a given time, the lock will automatically unlock.

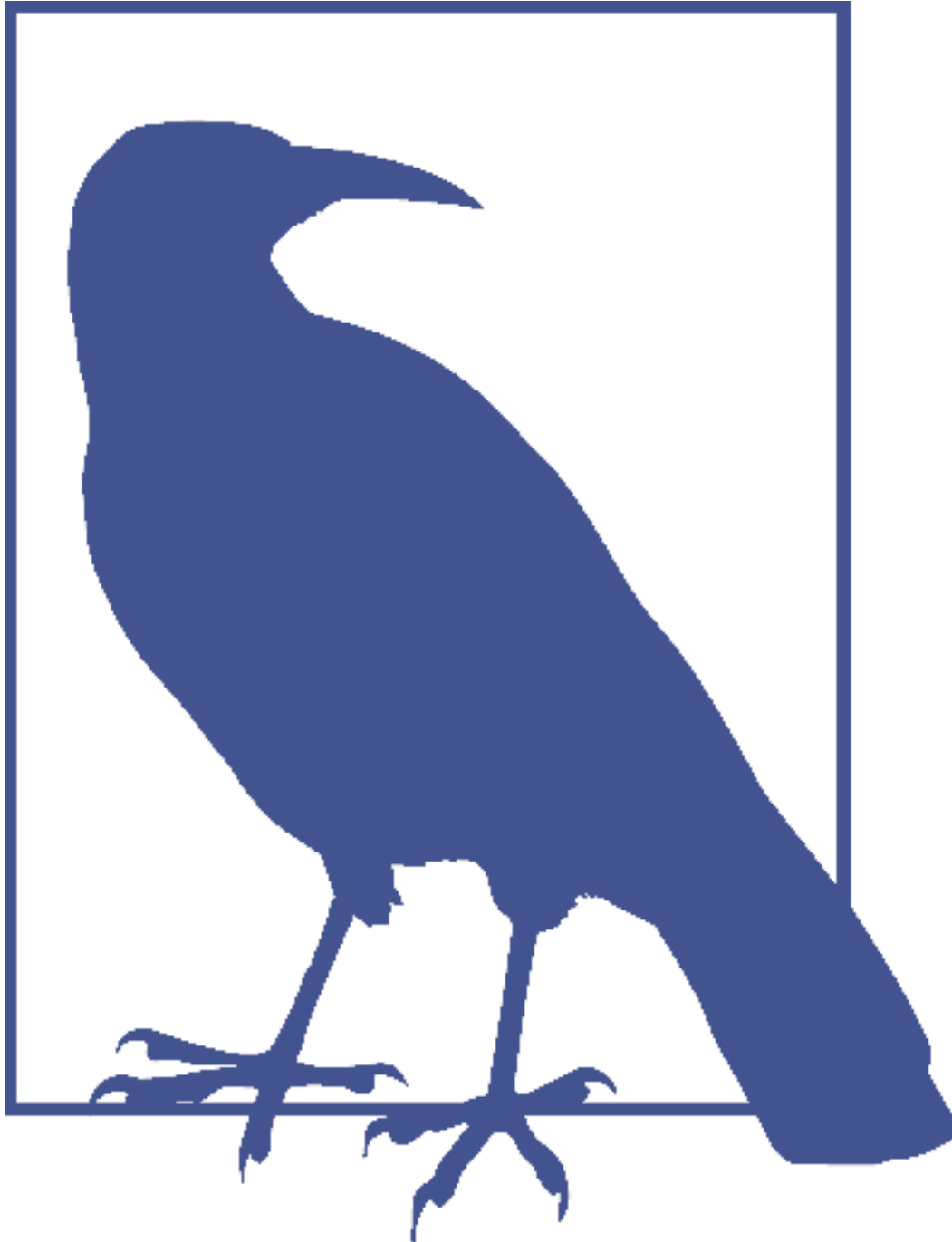
Bây giờ bạn có thể nghĩ rằng chúng ta đã xong, nhưng hãy nhớ rằng chúng ta đang xây dựng thứ này cho một hệ phân tán. Một tiến trình có thể thất bại ngay giữa lúc đang giữ khóa, và tại thời điểm đó không còn ai để giải phóng nó. Trong tình huống như vậy, hệ thống của chúng ta sẽ bị kẹt. Để giải quyết, chúng ta tận dụng chức năng TTL của kho key-value. Chúng ta thay đổi hàm simpleLock để nó luôn ghi kèm một TTL, sao cho nếu chúng ta không mở khóa trong một khoảng thời gian nhất định, khóa sẽ tự động được mở.

```

func (Lock l) simpleLock() boolean {
    // compare and swap "1" for "0"
    locked, error = compareAndSwap(l.lockName, "1", "0", l.ttl)
}

```

```
// lock doesn't exist, try to write "1" with a previous value of
// non-existent
if error != nil {
    locked, _ = compareAndSwap(l.lockName, "1", nil, l.ttl)
}
return locked
}
```



When using distributed locks, it is critical to ensure that any processing you do doesn't last longer than the TTL of the lock. One good practice is to set a watchdog timer when you acquire the lock. The

watchdog contains an assertion that will crash your program if the TTL of the lock expires before you have called unlock .

Khi dùng khóa phân tán, điều then chốt là phải đảm bảo bất kỳ xử lý nào bạn thực hiện không kéo dài hơn TTL của khóa. Một thực hành tốt là đặt một bộ đếm thời gian giám sát (watchdog timer) khi bạn giành được khóa. Watchdog chứa một khẳng định (assertion) sẽ làm crash chương trình của bạn nếu TTL của khóa hết hạn trước khi bạn gọi unlock.

By adding TTL to our locks, we have actually introduced a bug into our unlock function. Consider the following scenario:

Bằng việc thêm TTL vào các khóa, thực ra chúng ta đã đưa một lỗi (bug) vào hàm unlock. Hãy xem xét tình huống sau:

1. Process-1 obtains the lock with TTL t .
2. Process-1 runs really slowly for some reason, for longer than t .
3. The lock expires.
4. Process-2 acquires the lock, since Process-1 has lost it due to TTL.
5. Process-1 finishes and calls unlock .
6. Process-3 acquires the lock.

1. Process-1 giành khóa với TTL t .
2. Vì lý do nào đó, Process-1 chạy rất chậm, lâu hơn t .
3. Khóa hết hạn.
4. Process-2 giành khóa, vì Process-1 đã mất nó do TTL.
5. Process-1 hoàn thành và gọi unlock.
6. Process-3 giành khóa.

At this point, Process-1 believes that it has unlocked the lock that it held at the beginning; it doesn't understand that it has actually lost the lock due to TTL, and in fact unlocked the lock held by Process-2. Then Process-3 comes along and also grabs the lock. Now both Process-2 and Process-3 both believe they own the lock, and hilarity ensues.

Tại thời điểm này, Process-1 tin rằng nó đã mở khóa mà nó nắm giữ từ đầu; nó không hiểu rằng thực ra nó đã mất khóa do TTL, và trên thực tế đã mở khóa do Process-2 đang giữ. Rồi Process-3 xuất hiện và cũng chớp lấy khóa. Giờ thì cả Process-2 lẫn Process-3 đều tin rằng mình sở hữu khóa, và sự hỗn loạn buồn cười xảy ra.

Fortunately, the key-value store provides a resource version for every write that is performed. Our lock function can store this resource version and augment compareAnd Swap to ensure that not only is the value as expected, but the resource version is the same as when the lock operation occurred. This changes our simple Lock function to look like this:

May mắn thay, kho key-value cung cấp một phiên bản tài nguyên (resource version) cho mỗi lần ghi được thực hiện. Hàm khóa của chúng ta có thể lưu phiên bản tài nguyên này và bổ sung cho compareAndSwap để đảm bảo rằng không chỉ giá trị đúng như kỳ vọng, mà phiên bản tài nguyên cũng giống với lúc thao tác khóa diễn ra. Điều này làm cho hàm simpleLock của chúng ta trông như thế này:

```
func (Lock l) simpleLock() boolean {
    // compare and swap "1" for "0"
    locked, l.version, error = compareAndSwap(l.lockName, "1", "0", l.ttl)
    // lock doesn't exist, try to write "1" with a previous value of
    // non-existent
    if error != null {
```

```

    locked, l.version, _ = compareAndSwap(l.lockName, "1", null, l.ttl)
}
return locked
}

```

And the unlock function then looks like this:

Và hàm unlock khi đó trông như thế này:

```

func (Lock l) unlock() {
    compareAndSwap(l.lockName, "0", "1", l.version)
}

```

This ensures that the lock is only unlocked if the TTL has not expired.

Điều này đảm bảo khóa chỉ được mở nếu TTL chưa hết hạn.

Hands On: Implementing Locks in etcd — Thực hành: Triển khai Khóa trong etcd

To implement locks in etcd, you can use a key as the name of the lock and pre-condition writes to ensure that only one lock holder is allowed at a time. For simplicity, we'll use the etcdctl command line to lock and unlock the lock. In reality, of course, you would want to use a programming language; there are etcd clients for most popular programming languages.

Để triển khai khóa trong etcd, bạn có thể dùng một khóa (key) làm tên của khóa (lock) và thiết lập điều kiện tiên quyết (pre-condition) cho các lần ghi nhằm đảm bảo chỉ một bên giữ khóa được phép tại một thời điểm. Để cho đơn giản, chúng ta sẽ dùng dòng lệnh etcdctl để khóa và mở khóa. Tất nhiên, trong thực tế bạn sẽ muốn dùng một ngôn ngữ lập trình; có sẵn các etcd client cho hầu hết các ngôn ngữ lập trình phổ biến.

Let's start by creating a lock named my-lock :

Hãy bắt đầu bằng việc tạo một khóa tên là my-lock:

```

kubectl exec my-etcd-cluster-0000 -- sh -c \
"ETCD_API=3 etcdctl --endpoints=${ETCD_ENDPOINTS} set my-lock unlocked"

```

This creates a key in etcd named my-lock and sets the initial value to unlocked .

Lệnh này tạo một key trong etcd tên là my-lock và đặt giá trị ban đầu là unlocked.

Now let's suppose that Alice and Bob both want to take ownership of my-lock . Alice and Bob both try to write their name to the lock, using a precondition that the value of the lock is unlocked .

Bây giờ giả sử cả Alice và Bob đều muốn giành quyền sở hữu my-lock. Cả Alice và Bob đều cố ghi tên mình vào khóa, với điều kiện tiên quyết là giá trị của khóa đang là unlocked.

Alice first runs:

Alice chạy trước:

```

kubectl exec my-etcd-cluster-0000 -- sh -c \
"ETCD_API=3 etcdctl --endpoints=${ETCD_ENDPOINTS} \
set --swap-with-value unlocked my-lock alice"

```

And obtains the lock. Now Bob attempts to obtain the lock:

Và giành được khóa. Bây giờ Bob thử giành khóa:

```
kubectl exec my-etcd-cluster-0000 -- sh -c \  
"ETCD_API=3 etcdctl --endpoints=${ETCD_ENDPOINTS} \  
set --swap-with-value unlocked my-lock bob"  
Error: 101: Compare failed ([unlocked != alice]) [6]
```

You can see that Bob's attempt to claim the lock has failed, since Alice currently owns the lock.

Bạn có thể thấy nỗ lực đòi khóa của Bob đã thất bại, vì hiện tại Alice đang sở hữu khóa.

To unlock the lock, Alice writes unlocked with a precondition value of alice :

Để mở khóa, Alice ghi unlocked với giá trị điều kiện tiên quyết là alice:

```
kubectl exec my-etcd-cluster-0000 -- sh -c \  
"ETCD_API=3 etcdctl --endpoints=${ETCD_ENDPOINTS} \  
set --swap-with-value alice my-lock unlocked"
```

Implementing Ownership — Triển khai Quyền sở hữu

While locks are great for establishing temporary ownership of some critical component, sometimes you want to take ownership for the duration of the time that the component is running. For example, in a highly available deployment of Kubernetes, there are multiple replicas of the scheduler but only one replica is actively making scheduling decisions. Further, once it becomes the active scheduler, it remains the active scheduler until that process fails for some reason.

Trong khi khóa rất tốt để thiết lập quyền sở hữu tạm thời đối với một thành phần quan trọng nào đó, đôi khi bạn muốn giữ quyền sở hữu trong suốt khoảng thời gian thành phần đó đang chạy. Ví dụ, trong một triển khai Kubernetes có tính sẵn sàng cao, có nhiều bản sao của bộ lập lịch (scheduler) nhưng chỉ một bản sao thực sự đang đưa ra các quyết định lập lịch. Hơn nữa, một khi nó trở thành scheduler đang hoạt động, nó vẫn là scheduler đang hoạt động cho đến khi tiến trình đó thất bại vì lý do nào đó.

Obviously, one way to do this would be to extend the TTL for the lock to a very long period (say a week or longer), but this has the significant downside that if the current lock owner fails, a new lock owner wouldn't be chosen until the TTL expired a week later.

Hiển nhiên, một cách để làm điều này là kéo dài TTL của khóa lên một khoảng rất dài (chẳng hạn một tuần hoặc lâu hơn), nhưng cách này có nhược điểm lớn là nếu chủ sở hữu khóa hiện tại thất bại, một chủ sở hữu khóa mới sẽ không được chọn cho đến khi TTL hết hạn một tuần sau đó.

Instead, we need to create a **renewable lock**, which can be periodically renewed by the owner so that the lock can be retained for an arbitrary period of time.

Thay vào đó, chúng ta cần tạo một khóa gia hạn được (renewable lock), thứ có thể được chủ sở hữu gia hạn định kỳ để khóa có thể được giữ trong một khoảng thời gian tùy ý.

We can extend the existing Lock that we defined previously to create a renewable lock, which enables the lock holder to renew the lock:

Chúng ta có thể mở rộng kiểu Lock đã định nghĩa trước đó để tạo một khóa gia hạn được (renewable lock), cho phép bên giữ khóa gia hạn khóa:

```
func (Lock l) renew() boolean {  
    locked, _ = compareAndSwap(l.lockName, "1", "1", l.version, ttl)
```

```
    return locked
}
```

Of course, you probably want to do this repeatedly in a separate thread so that you hold onto the lock indefinitely. Notice that the lock is renewed every $\text{ttl}/2$ seconds; that way there is significantly less risk that the lock will accidentally expire due to timing subtleties:

Tất nhiên, bạn có thể sẽ muốn làm điều này lặp đi lặp lại trong một luồng (thread) riêng để giữ khóa vô thời hạn. Lưu ý rằng khóa được gia hạn mỗi $\text{ttl}/2$ giây; bằng cách đó nguy cơ khóa vô tình hết hạn do những tinh tế về thời điểm (timing) giảm đi đáng kể:

```
for {
    if !l.renew() {
        handleLockLost()
    }
    sleep(ttl/2)
}
```

Of course, you need to implement the `handleLockLost()` function so that it terminates all activity that required the lock in the first place. In a container orchestration system, the easiest way to do this may simply be to terminate the application and let the orchestrator restart it. This is safe, because some other replica has grabbed the lock in the interim, and when the restarted application comes back online it will become a secondary listener waiting for the lock to become free.

Tất nhiên, bạn cần triển khai hàm `handleLockLost()` sao cho nó chấm dứt mọi hoạt động vốn cần đến khóa ngay từ đầu. Trong một hệ thống điều phối container, cách dễ nhất để làm điều này có thể đơn giản là chấm dứt ứng dụng và để bộ điều phối khởi động lại nó. Cách này an toàn, vì một bản sao khác đã chộp lấy khóa trong khoảng thời gian đó, và khi ứng dụng được khởi động lại quay lại trực tuyến, nó sẽ trở thành một bên lắng nghe phụ (secondary listener) chờ đến khi khóa trở nên rảnh.

Hands On: Implementing Leases in etcd — Thực hành: Triển khai Hợp đồng thuê (Lease) trong etcd

To see how we implement leases using etcd, we will return to our earlier locking example and add the `-ttl=` flag to our lock create and update calls. The `ttl` flag defines a time after which the lock that we create is deleted. Because the lock disappears after the `ttl` expires, instead of creating with the value of `unlocked`, we will assume that the absence of the lock means that it is unlocked. To do this, we use the `mk` command instead of the `set` command. `etcdctl mk` only succeeds if the key does not currently exist.

Để xem cách chúng ta triển khai hợp đồng thuê (lease) bằng etcd, chúng ta sẽ quay lại ví dụ khóa trước đó và thêm cờ `-ttl=` vào các lệnh tạo và cập nhật khóa. Cờ `ttl` định nghĩa một khoảng thời gian, sau đó khóa mà chúng ta tạo sẽ bị xóa. Vì khóa biến mất sau khi `ttl` hết hạn, thay vì tạo với giá trị `unlocked`, chúng ta sẽ giả định rằng sự vắng mặt của khóa nghĩa là nó đang được mở. Để làm điều này, chúng ta dùng lệnh `mk` thay vì lệnh `set`. `etcdctl mk` chỉ thành công nếu key hiện chưa tồn tại.

Thus, to lock a leased lock, Alice executes:

Như vậy, để khóa một khóa kiểu thuê (leased lock), Alice thực thi:

```
kubectl exec my-etcd-cluster-0000 -- \
sh -c "ETCD_API=3 etcdctl --endpoints=${ETCD_ENDPOINTS} \
```

```
--ttl=10 mk my-lock alice"
```

This creates a leased lock with a duration of 10 seconds.

Lệnh này tạo một khóa kiểu thuê với thời hạn 10 giây.

For Alice to continue to hold the lock, she needs to execute:

Để Alice tiếp tục giữ khóa, cô cần thực thi:

```
kubectl exec my-etcd-cluster-0000 -- \
  sh -c "ETCD_API=3 etcdctl --endpoints=${ETCD_ENDPOINTS} \
    set --ttl=10 --swap-with-value alice my-lock alice"
```

It may seem odd that Alice is continually rewriting her own name into the lock, but this is the way the lock **lease** is extended beyond the 10-second TTL.

Có vẻ kỳ lạ khi Alice cứ liên tục ghi lại tên mình vào khóa, nhưng đây chính là cách hợp đồng thuê khóa được gia hạn vượt quá TTL 10 giây.

If, for some reason, the TTL expires, then the lock update will fail, and Alice will go back to creating the lock using the etcd mk command, or Bob may also use the mk command to obtain the lock for himself. Bob will likewise need to set and update the lock every 10 seconds to maintain ownership.

Nếu vì lý do nào đó TTL hết hạn, thì lệnh cập nhật khóa sẽ thất bại, và Alice sẽ quay lại tạo khóa bằng lệnh etcd mk, hoặc Bob cũng có thể dùng lệnh mk để tự giành lấy khóa cho mình. Tương tự, Bob sẽ cần đặt và cập nhật khóa mỗi 10 giây để duy trì quyền sở hữu.

Handling Concurrent Data Manipulation — Xử lý Thao tác Dữ liệu Đồng thời

Even with all of the locking mechanisms we have described, it is still possible for two replicas to simultaneously believe they hold the lock for a very brief period of time. To understand how this can happen, imagine that the original lock holder becomes so overwhelmed that its processor stops running for minutes at a time. This can happen on extremely overscheduled machines. In such a case, the lock will time out and some other replica will own the lock. Now the processor frees up the replica that was the original lock holder. Obviously, the `handleLockLost()` function will quickly be called, but there will be a brief period where the replica still believes it holds the lock. Although such an **event** is fairly unlikely, systems need to be built to be robust to such occurrences. The first step to take is to double-check that the lock is still held, using a function like this:

Ngay cả với tất cả các cơ chế khóa mà chúng ta đã mô tả, vẫn có khả năng hai bản sao cùng tin rằng mình đang giữ khóa trong một khoảng thời gian rất ngắn. Để hiểu điều này có thể xảy ra như thế nào, hãy hình dung bên giữ khóa ban đầu trở nên quá tải đến mức bộ xử lý của nó ngừng chạy trong nhiều phút liền. Điều này có thể xảy ra trên những máy bị xếp lịch quá tải (overscheduled) nghiêm trọng. Trong trường hợp như vậy, khóa sẽ hết thời gian (time out) và một bản sao khác sẽ sở hữu khóa. Giờ thì bộ xử lý giải phóng bản sao vốn là bên giữ khóa ban đầu. Hiển nhiên, hàm `handleLockLost()` sẽ nhanh chóng được gọi, nhưng sẽ có một khoảng thời gian ngắn mà bản sao vẫn tin rằng nó đang giữ khóa. Mặc dù một sự kiện như vậy khá khó xảy ra, các hệ thống cần được xây dựng đủ vững chắc để chịu được những tình huống như thế. Bước đầu tiên cần làm là kiểm tra lại rằng khóa vẫn đang được giữ, dùng một hàm như thế này:

```
func (Lock l) isLocked() boolean {
  return l.locked && l.lockTime + 0.75 * l.ttl > now()
```

```
}
```

If this function executes prior to any code that needs to be protected by a lock, then the probability of two masters being active is significantly reduced, but—it is important to note—it is not completely eliminated. The lock timeout could always occur between the time that the lock was checked and the guarded code was executed. To protect against these scenarios, the system that is being called from the replica needs to validate that the replica sending a request is actually still the master. To do this, the hostname of the replica holding the lock is stored in the key-value store in addition to the state of the lock. That way, others can double-check that a replica asserting that it is the master is in fact the master.

Nếu hàm này thực thi trước bất kỳ đoạn mã nào cần được khóa bảo vệ, thì xác suất hai master cùng hoạt động giảm đi đáng kể, nhưng — điều quan trọng cần lưu ý — nó không được loại bỏ hoàn toàn. Tình trạng khóa hết thời gian luôn có thể xảy ra trong khoảng giữa thời điểm khóa được kiểm tra và thời điểm đoạn mã được bảo vệ được thực thi. Để phòng vệ trước các kịch bản này, hệ thống đang được bản sao gọi tới cần xác thực rằng bản sao gửi yêu cầu thực sự vẫn đang là master. Để làm điều này, tên máy (hostname) của bản sao đang giữ khóa được lưu trong kho key-value bên cạnh trạng thái của khóa. Bằng cách đó, các bên khác có thể kiểm tra lại xem một bản sao đang khẳng định mình là master có thực sự là master hay không.

This system diagram is shown in Figure 9-2 . In the first image, shard2 is the owner of the lock, and when a request is sent to the **worker**, the worker double-checks with the lock server and validates that shard2 is actually the current owner.

Sơ đồ hệ thống này được minh họa trong Hình 9-2. Trong hình đầu tiên, shard2 là chủ sở hữu của khóa, và khi một yêu cầu được gửi đến worker, worker kiểm tra lại với máy chủ khóa (lock server) và xác thực rằng shard2 thực sự là chủ sở hữu hiện tại.

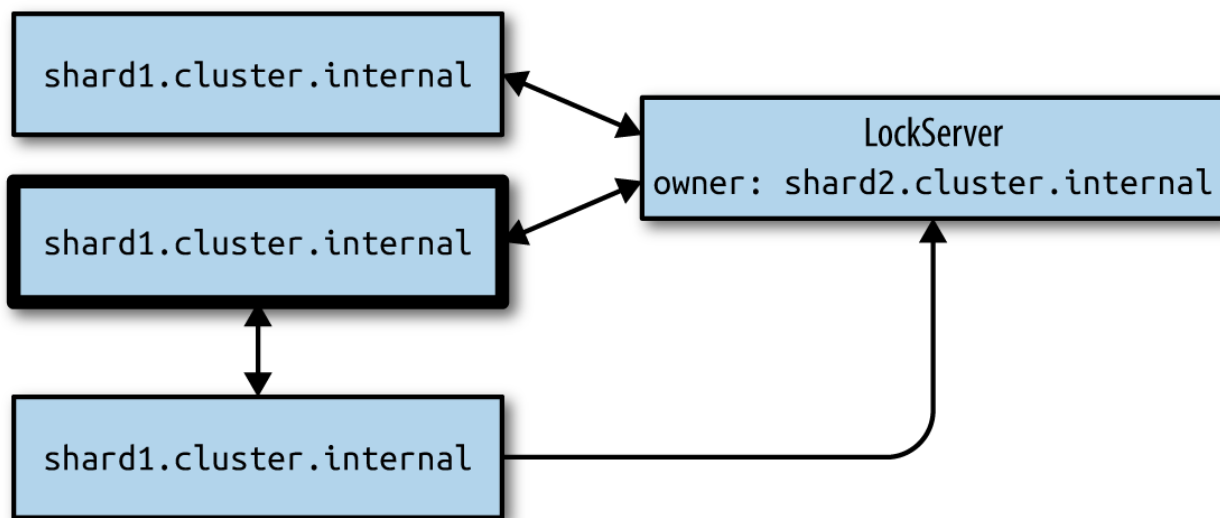


Figure 9-2. A worker double-checking to validate that the requester who sent a message is actually the current owner of the shard

Hình 9-2. Một worker kiểm tra lại để xác thực rằng bên gửi yêu cầu thực sự là chủ sở hữu hiện tại của mảnh (shard)

In the second case, shard2 has lost ownership of the lock, but it has not yet realized this so it continues to send requests to the worker node. This time, when the worker node receives a request from shard2

, it double-checks with the lock service and realizes that shard2 is no longer the lock owner, and thus the requests are rejected.

Trong trường hợp thứ hai, shard2 đã mất quyền sở hữu khóa, nhưng nó chưa nhận ra điều này nên vẫn tiếp tục gửi yêu cầu đến nút worker. Lần này, khi nút worker nhận một yêu cầu từ shard2, nó kiểm tra lại với dịch vụ khóa và nhận ra rằng shard2 không còn là chủ sở hữu khóa nữa, và do đó các yêu cầu bị từ chối.

To add one final further complicating wrinkle, it's always possible that ownership could be obtained, lost, and then re-obtained by the system, which could actually cause a request to succeed when it should actually be rejected. To understand how this is possible, consider the following sequence of events:

Để thêm một tình tiết rắc rối cuối cùng nữa, luôn có khả năng quyền sở hữu được giành lấy, bị mất, rồi lại được giành lại bởi hệ thống, điều này thực ra có thể khiến một yêu cầu thành công trong khi lẽ ra nó phải bị từ chối. Để hiểu điều này có thể xảy ra như thế nào, hãy xem xét chuỗi sự kiện sau:

1. Shard-1 obtains ownership to become master.
2. Shard-1 sends a request R1 as master at time T1 .
3. The network hiccups and delivery of R1 is delayed.
4. Shard-1 fails TTL because of the network and loses lock to Shard-2.
5. Shard-2 becomes master and sends a request R2 at time T2 .
6. Request R2 is received and processed.
7. Shard-2 crashes and loses ownership back to Shard-1.
8. Request R1 finally arrives, and Shard-1 is the current master, so it is accepted, but this is bad because R2 has already been processed.

1. Shard-1 giành quyền sở hữu để trở thành master.
2. Shard-1 gửi một yêu cầu R1 với tư cách master tại thời điểm T1.
3. Mạng bị trục trặc và việc giao R1 bị trì hoãn.
4. Shard-1 thất bại TTL do vấn đề mạng và mất khóa vào tay Shard-2.
5. Shard-2 trở thành master và gửi một yêu cầu R2 tại thời điểm T2.
6. Yêu cầu R2 được nhận và xử lý.
7. Shard-2 crash và để quyền sở hữu trở lại về Shard-1.
8. Yêu cầu R1 cuối cùng cũng đến, và Shard-1 đang là master hiện tại, nên nó được chấp nhận, nhưng điều này tệ vì R2 đã được xử lý rồi.

Such sequences of events seem byzantine, but in reality, in any large system they occur with disturbing frequency. Fortunately, this is similar to the case described previously, which we resolved with the resource version in etcd. We can do the same thing here. In addition to storing the name of the current owner in etcd, we also send the resource version along with each request. So in the previous example, R1 becomes (R1, Version1) . Now when the request is received, the double-check validates both the current owner and the resource version of the request. If either match fails, the request is rejected. This patches up this example.

Những chuỗi sự kiện như vậy nghe có vẻ rối rắm (byzantine), nhưng trên thực tế, trong bất kỳ hệ thống lớn nào chúng xảy ra với tần suất đáng lo ngại. May mắn thay, trường hợp này tương tự trường hợp đã mô tả trước đó, mà chúng ta đã giải quyết bằng phiên bản tài nguyên (resource version) trong etcd. Chúng ta có thể làm điều tương tự ở đây. Ngoài việc lưu tên của chủ sở hữu hiện tại trong etcd, chúng ta cũng gửi kèm phiên bản tài nguyên với mỗi yêu cầu. Vậy nên trong ví dụ trước, R1 trở thành (R1, Version1). Giờ khi yêu cầu được nhận, bước kiểm tra lại sẽ xác thực cả chủ sở hữu hiện tại lẫn phiên bản tài nguyên của yêu cầu. Nếu một trong hai phép so khớp thất bại, yêu cầu bị từ chối. Điều này vá lại

ví dụ trên.

PART III Batch Computational Patterns — PART III Các mẫu tính toán theo lô

The preceding chapter described patterns for reliable, long-running server applications. This section describes patterns for **batch processing**. In contrast to long-running applications, batch processes are expected to only run for a short period of time. Examples of a batch process include generating aggregation of user telemetry data, analyzing sales data for daily or weekly reporting, or transcoding video files. Batch processes are generally characterized by the need to process large amounts of data quickly using parallelism to speed up the processing. The most famous pattern for distributed batch processing is the **MapReduce** pattern, which has become an entire industry in itself. However, there are several other patterns that are useful for batch processing, which are described in the following chapters.

Chương trước đã mô tả các mẫu dành cho ứng dụng máy chủ chạy dài hạn, tin cậy. Phần này mô tả các mẫu cho xử lý theo lô (batch). Khác với ứng dụng chạy dài hạn, tiến trình theo lô được kỳ vọng chỉ chạy trong một khoảng thời gian ngắn. Ví dụ về một tiến trình theo lô bao gồm tổng hợp dữ liệu telemetry của người dùng, phân tích dữ liệu bán hàng cho báo cáo ngày hoặc tuần, hoặc chuyển mã (transcode) các tệp video. Tiến trình theo lô nhìn chung được đặc trưng bởi nhu cầu xử lý lượng lớn dữ liệu một cách nhanh chóng bằng cách dùng tính song song để tăng tốc độ xử lý. Mẫu nổi tiếng nhất cho xử lý theo lô phân tán là mẫu MapReduce, vốn đã trở thành một ngành công nghiệp riêng. Tuy nhiên, còn nhiều mẫu khác hữu ích cho xử lý theo lô, sẽ được mô tả trong các chương sau.

CHAPTER 10 Work Queue Systems — CHAPTER 10 Hệ thống hàng đợi công việc

The simplest form of batch processing is a **work queue**. In a work queue system, there is a batch of work to be performed. Each piece of work is wholly independent of the other and can be processed without any interactions. Generally, the goals of the work queue system are to ensure that each piece of work is processed within a certain amount of time. Workers are scaled up or scaled down to ensure that the work can be handled. An illustration of a generic work queue is shown in Figure 10-1.

Dạng đơn giản nhất của xử lý theo lô là hàng đợi công việc (work queue). Trong một hệ thống hàng đợi công việc, có một lô công việc cần thực hiện. Mỗi đơn vị công việc hoàn toàn độc lập với các đơn vị khác và có thể được xử lý mà không cần bất kỳ tương tác nào. Nhìn chung, mục tiêu của hệ thống hàng đợi công việc là đảm bảo rằng mỗi đơn vị công việc được xử lý trong một khoảng thời gian nhất định. Các worker được mở rộng (scale up) hoặc thu hẹp (scale down) để đảm bảo công việc có thể được xử lý. Một minh họa về hàng đợi công việc tổng quát được thể hiện trong Hình 10-1.

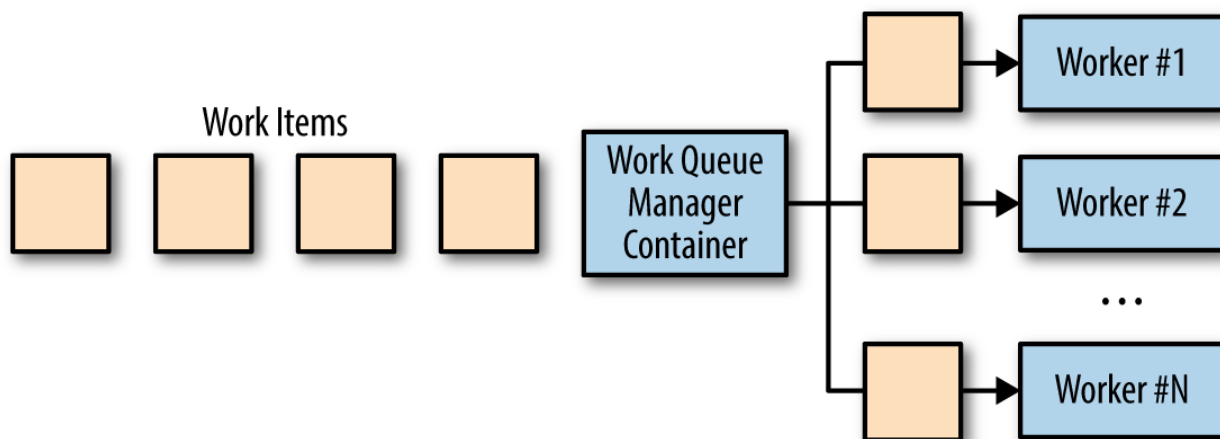


Figure 10-1. A generic work queue

Hình 10-1. Một hàng đợi công việc tổng quát

A Generic Work Queue System — Một hệ thống hàng đợi công việc tổng quát

The work queue is an ideal way to demonstrate the power of **distributed system** patterns. Most of the logic in the work queue is wholly independent of the actual work being done, and in many cases the delivery of the work can be performed in an independent manner as well. To illustrate this point, consider the work queue illustrated in Figure 10-1 . If we take a look at it again and identify functionality that can be provided by a shared set of library containers , it becomes apparent that most of the implementation of a containerized work queue can be shared across a wide variety of users, as shown in Figure 10-2 .

Hàng đợi công việc là một cách lý tưởng để minh họa sức mạnh của các mẫu hệ phân tán. Phần lớn logic trong hàng đợi công việc hoàn toàn độc lập với công việc thực tế đang được thực hiện, và trong nhiều trường hợp, việc cung cấp công việc cũng có thể được thực hiện một cách độc lập. Để minh họa điểm này, hãy xem xét hàng đợi công việc được minh họa trong Hình 10-1. Nếu nhìn lại nó và xác định những chức năng có thể được cung cấp bởi một tập hợp các container thư viện (library) dùng chung, thì rõ ràng là phần lớn việc triển khai một hàng đợi công việc đóng gói container có thể được chia sẻ cho nhiều loại người dùng khác nhau, như thể hiện trong Hình 10-2.

109

109

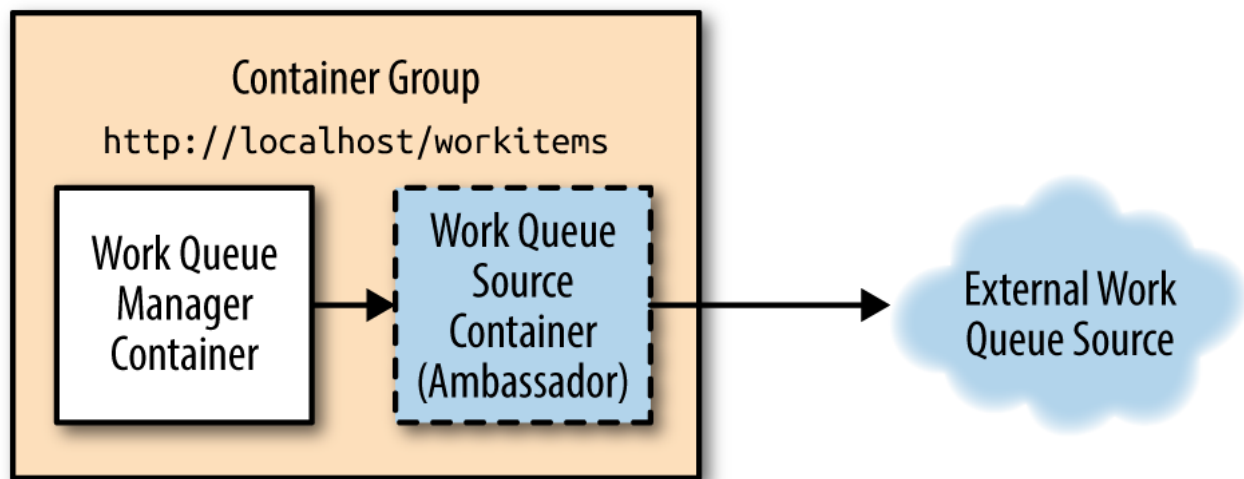


Figure 10-2. The same work queue as shown in Figure 10-1 , but this time using reusable containers. The reusable system containers are shown in white while the user-supplied **container** is shown in grey/blue.

Hình 10-2. Cùng một hàng đợi công việc như trong Hình 10-1, nhưng lần này sử dụng các container tái sử dụng được. Các container hệ thống tái sử dụng được thể hiện màu trắng còn container do người dùng cung cấp được thể hiện màu xám/xanh.

Building a reusable container-based work queue requires the definition of interfaces between the generic library containers and the user-defined application logic. In the containerized work queue, there are two interfaces: the **source container interface**, which provides a stream of work items that need processing, and the **worker container interface**, which knows how to actually process a **work item**.

Việc xây dựng một hàng đợi công việc dựa trên container tái sử dụng được đòi hỏi phải định nghĩa các giao diện giữa các container thư viện tổng quát và logic ứng dụng do người dùng định nghĩa. Trong hàng đợi công việc đóng gói container, có hai giao diện: giao diện container nguồn (source container interface), cung cấp một luồng các đơn vị công việc cần xử lý, và giao diện container worker (worker container interface), vốn biết cách thực sự xử lý một đơn vị công việc.

The Source Container Interface — Giao diện container nguồn

To operate, every work queue needs a collection of work items that need processing. There are many different sources of work items the work queue, depending on the specific application of the work queue. However, once the set of items has been obtained, the actual operation of the work queue is quite generic. Consequently, we can separate the application-specific queue source logic from the generic queue processing logic. Given the previously defined patterns of container groups, this can be seen as an example of the **ambassador** pattern defined previously. The generic work queue container is the primary application container, and the application-specific source container is the ambassador that proxies the generic work queue's requests out to the concrete definition of the work queue out in the real world. This container group is illustrated in Figure 10-3 .

Để hoạt động, mọi hàng đợi công việc đều cần một tập hợp các đơn vị công việc cần xử lý. Có nhiều nguồn cung cấp đơn vị công việc khác nhau cho hàng đợi công việc, tùy thuộc vào ứng dụng cụ thể của hàng đợi công việc. Tuy nhiên, một khi tập hợp các đơn vị đã được lấy về, thì hoạt động thực tế của hàng đợi công việc lại khá tổng quát. Do đó, ta có thể tách logic nguồn hàng đợi đặc thù theo ứng dụng ra khỏi logic xử lý hàng đợi tổng quát. Với các mẫu nhóm container đã định nghĩa trước đó, điều này có thể được xem là một ví dụ của mẫu ambassador đã định nghĩa trước đây. Container hàng đợi công việc tổng quát là container ứng dụng chính, còn container nguồn đặc thù theo ứng dụng là ambassador làm trung gian (proxy) chuyển các yêu cầu của hàng đợi công việc tổng quát ra tới định nghĩa cụ thể của hàng đợi công việc ngoài thế giới thực. Nhóm container này được minh họa trong Hình 10-3.

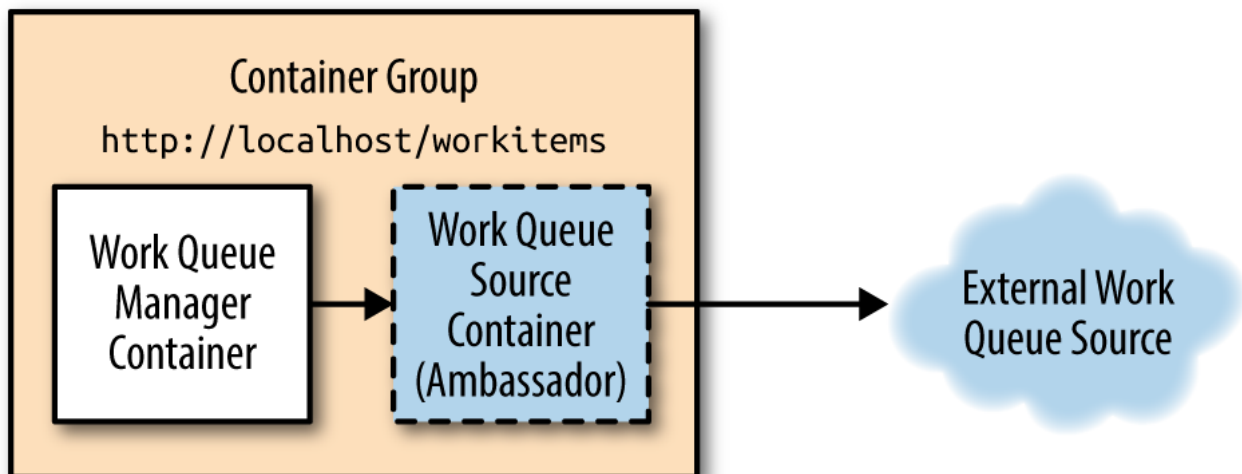


Figure 10-3. The work queue container group Interestingly, while the ambassador container is clearly application-specific, there is also a variety of generic implementations of the work queue source **API**. For example, a source might be a list of pictures stored in a cloud storage API, a collection of files stored on network storage, or a queue in an pub/sub system like Kafka or **Redis**. In these cases,

though the user chooses the particular work queue ambassador that fits their scenario, they should be reusing a single generic “library” implementation of the container itself. This minimizes work and maximizes code reuse.

Hình 10-3. Nhóm container hàng đợi công việc. Điều thú vị là, mặc dù container ambassador rõ ràng là đặc thù theo ứng dụng, vẫn có một loạt các cách triển khai tổng quát cho API nguồn hàng đợi công việc. Ví dụ, một nguồn có thể là danh sách hình ảnh lưu trong một API lưu trữ đám mây, một tập hợp các tệp lưu trên bộ lưu trữ mạng, hoặc một hàng đợi trong hệ thống pub/sub như Kafka hay Redis. Trong những trường hợp này, dù người dùng chọn ambassador hàng đợi công việc cụ thể phù hợp với kịch bản của họ, họ vẫn nên tái sử dụng một cách triển khai “thư viện” tổng quát duy nhất của chính container đó. Điều này giảm thiểu công sức và tối đa hóa việc tái sử dụng code.

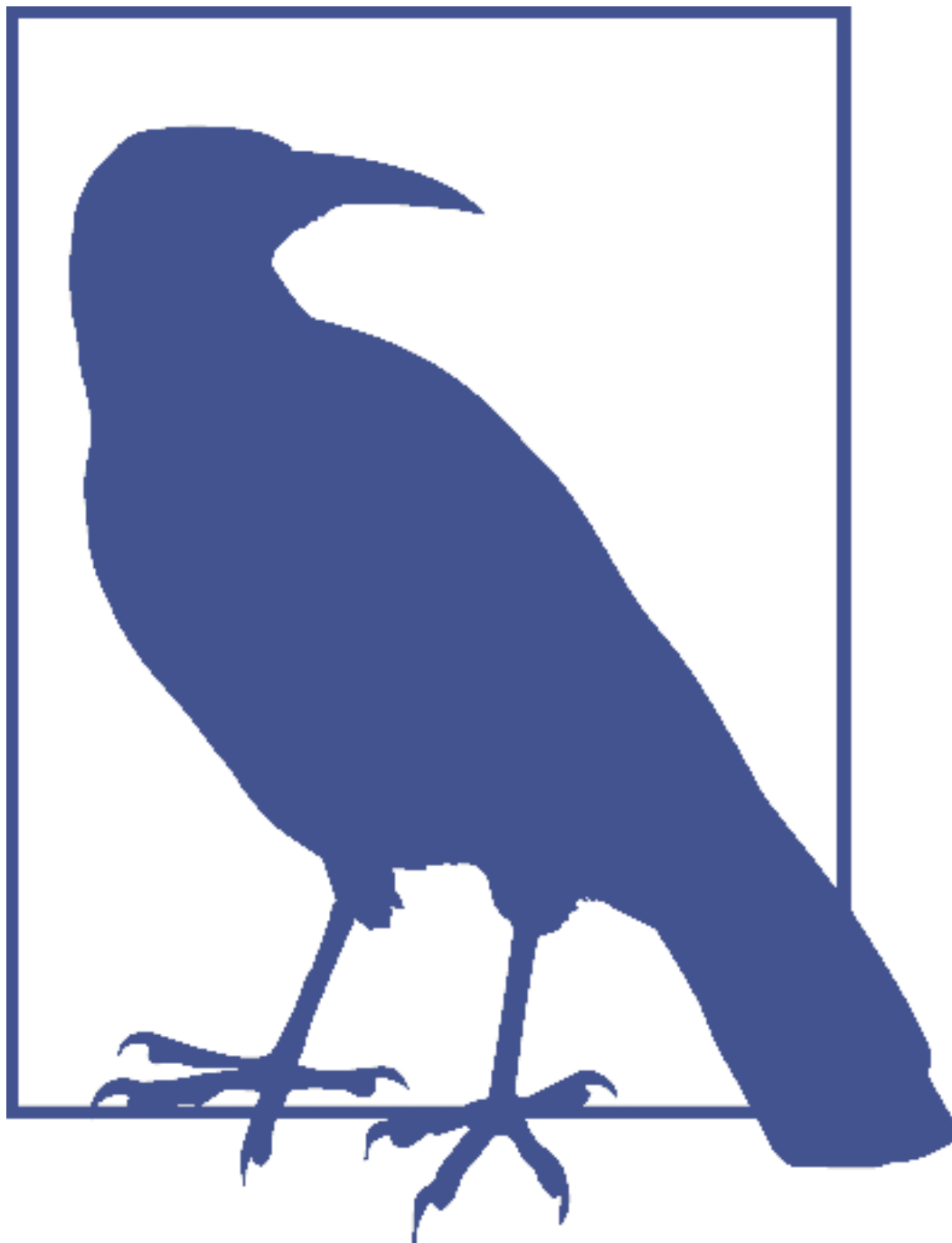
Work queue API

API hàng đợi công việc

Given this coordination between the generic work-queue manager and the application-specific ambassador, we need a formal definition of the **interface** between the two containers. Though there are a variety of different protocols, an HTTP RESTful API is both the easiest to implement as well as the de facto standard for such an interface. The master work queue expects the ambassador to implement the following URLs:

Với sự phối hợp này giữa trình quản lý hàng đợi công việc tổng quát và ambassador đặc thù theo ứng dụng, ta cần một định nghĩa chính thức cho giao diện giữa hai container. Mặc dù có nhiều giao thức khác nhau, một API HTTP RESTful vừa dễ triển khai nhất vừa là tiêu chuẩn trên thực tế (de facto) cho loại giao diện như vậy. Trình quản lý hàng đợi công việc chính (master) kỳ vọng ambassador triển khai các URL sau:

- GET <http://localhost/api/v1/items>
- GET <http://localhost/api/v1/items/>
 - GET <http://localhost/api/v1/items>
 - GET <http://localhost/api/v1/items/>



You might wonder why we include a `v1` in the API definition. Will there ever be a `v2` of this interface? It may not seem logical, but it costs very little to version your API when you initially define it. Refactoring versioning onto an API without it, on the other hand, is very expensive. Consequently, it is a best practice to always add versions to your APIs even if you're not sure they will ever change. Better safe than sorry.

Bạn có thể tự hỏi vì sao ta lại đưa `v1` vào trong định nghĩa API. Liệu có bao giờ xuất hiện `v2` của giao diện này không? Nghe có vẻ không hợp lý, nhưng việc đánh phiên bản (version) cho API ngay khi mới định nghĩa nó tốn rất ít chi phí. Ngược lại, việc tái cấu trúc để thêm phiên bản vào một API vốn không có lại rất tốn kém. Do đó, một thực hành tốt nhất là luôn

thêm phiên bản cho API ngay cả khi bạn không chắc chúng sẽ thay đổi. Cần tắc vô áy náy.

The `/items/` URL returns a complete list of all items:

URL `/items/` trả về danh sách đầy đủ tất cả các đơn vị:

```
{
  kind: ItemList,
  apiVersion: v1,
  items: [
    "item-1",
    "item-2",
    ...
  ]
}
```

The `/items/` URL provides the details for a specific item:

URL `/items/` cung cấp chi tiết cho một đơn vị cụ thể:

```
{
  kind: Item,
  apiVersion: v1,
  data: {
    "some": "json",
    "object": "here",
  }
}
```

Importantly, you will notice that this API doesn't have any affordances for recording that a work item has been processed. We could have designed a more complicated API and then pushed more implementation into the ambassador container, but remember, the goal of this effort is to place as much of the generic implementation inside of the generic work queue manager as possible. To that end, the work queue manager itself is responsible for tracking which items have been processed and which items remain to be processed.

Quan trọng là, bạn sẽ nhận thấy API này không có bất kỳ cơ chế nào để ghi nhận rằng một đơn vị công việc đã được xử lý. Ta có thể đã thiết kế một API phức tạp hơn rồi đẩy thêm phần triển khai vào container ambassador, nhưng hãy nhớ rằng, mục tiêu của nỗ lực này là đặt càng nhiều phần triển khai tổng quát vào bên trong trình quản lý hàng đợi công việc tổng quát càng tốt. Vì lẽ đó, chính trình quản lý hàng đợi công việc chịu trách nhiệm theo dõi những đơn vị nào đã được xử lý và những đơn vị nào còn chờ xử lý.

The item details are obtained from this API and the `item.data` field is passed along to the **worker** interface for processing.

Chi tiết đơn vị được lấy về từ API này và trường `item.data` được chuyển tiếp sang giao diện **worker** để xử lý.

The Worker Container Interface — Giao diện container worker

Once a particular work item has been obtained by the work queue manager, it needs to be processed by a worker. This is the second container interface in our generic work queue. This container and interface are slightly different than the previous work queue source interface for a few reasons. The

first is that it is a one-off API: a single call is made to begin the work, and no other API calls are made throughout the life of the worker container. Secondly, the worker container is not inside a container group with the work queue manager. Instead, it is launched via a container orchestration API and scheduled to its own container group. This means that the work queue manager has to make a remote call to the worker container in order to start work. It also means that we may need to be more careful about security to prevent a malicious user in our cluster from injecting extra work into the system.

Một khi một đơn vị công việc cụ thể đã được trình quản lý hàng đợi công việc lấy về, nó cần được một worker xử lý. Đây là giao diện container thứ hai trong hàng đợi công việc tổng quát của chúng ta. Container và giao diện này hơi khác so với giao diện nguồn hàng đợi công việc trước đó vì một vài lý do. Thứ nhất, đây là một API dùng một lần (one-off): chỉ một lời gọi duy nhất được thực hiện để bắt đầu công việc, và không có lời gọi API nào khác trong suốt vòng đời của container worker. Thứ hai, container worker không nằm trong cùng một nhóm container với trình quản lý hàng đợi công việc. Thay vào đó, nó được khởi chạy thông qua một API điều phối container và được lập lịch vào nhóm container riêng của nó. Điều này có nghĩa là trình quản lý hàng đợi công việc phải thực hiện một lời gọi từ xa tới container worker để bắt đầu công việc. Nó cũng có nghĩa là ta có thể cần cẩn trọng hơn về bảo mật để ngăn một người dùng độc hại trong cụm (cluster) của chúng ta chen thêm công việc vào hệ thống.

With the work queue source API, we used a simple HTTP-based API for sending items back to the work queue manager. This was because we needed to make repeated calls to the API, and security wasn't a concern since everything was running on localhost. With the worker container, we only need to make a single call, and we want to ensure that other users in the system can't accidentally or maliciously add work to our workers. Consequently, for the worker container, we will use a file-based API. Namely, when the worker container is created, it will receive an environment variable named `WORK_ITEM_FILE`; this will point to a file in the container's local filesystem, where the data field from a work item has been written to a file. Concretely, as you will see below, this API can be implemented via a **Kubernetes ConfigMap** object that can be mounted into a container group as a file, as illustrated in Figure 10-4.

Với API nguồn hàng đợi công việc, ta đã dùng một API dựa trên HTTP đơn giản để gửi các đơn vị trở lại trình quản lý hàng đợi công việc. Đó là vì ta cần thực hiện các lời gọi lặp đi lặp lại tới API, và bảo mật không phải là mối lo vì mọi thứ đều chạy trên localhost. Với container worker, ta chỉ cần thực hiện một lời gọi duy nhất, và ta muốn đảm bảo rằng những người dùng khác trong hệ thống không thể vô tình hay cố ý thêm công việc vào các worker của ta. Do đó, với container worker, ta sẽ dùng một API dựa trên tệp (file-based). Cụ thể, khi container worker được tạo, nó sẽ nhận một biến môi trường tên là `WORK_ITEM_FILE`; biến này sẽ trỏ tới một tệp trong hệ thống tệp cục bộ của container, nơi trường data của một đơn vị công việc đã được ghi ra tệp. Cụ thể hơn, như bạn sẽ thấy bên dưới, API này có thể được triển khai thông qua một đối tượng ConfigMap của Kubernetes vốn có thể được gắn (mount) vào một nhóm container dưới dạng tệp, như minh họa trong Hình 10-4.

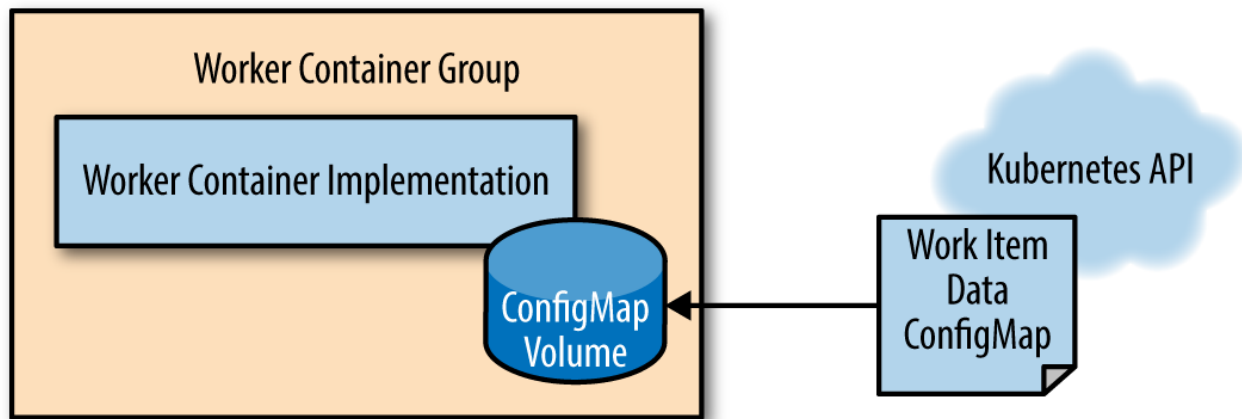


Figure 10-4. The work queue worker API

Hình 10-4. API worker của hàng đợi công việc

This file-based API pattern is also easier for the container to implement. Often a work queue worker is simply a shell script across a few command line tools. In that context, spinning up a web server to manage the work to perform is an unnecessary complexity. As was true with the work queue source implementation, most of the worker containers will be special-purpose container images built for specific work queue applications, but there are also generic workers that can be applied to multiple different work queue applications.

Mẫu API dựa trên tệp này cũng dễ triển khai hơn cho container. Thường thì một worker hàng đợi công việc chỉ đơn giản là một shell script gọi qua vài công cụ dòng lệnh. Trong bối cảnh đó, việc dựng lên một web server để quản lý công việc cần thực hiện là một sự phức tạp không cần thiết. Cũng giống như với cách triển khai nguồn hàng đợi công việc, phần lớn các container worker sẽ là các image container chuyên dụng được xây dựng cho những ứng dụng hàng đợi công việc cụ thể, nhưng cũng có những worker tổng quát có thể được áp dụng cho nhiều ứng dụng hàng đợi công việc khác nhau.

Consider the example of a work queue worker that downloads a file from cloud storage and runs a shell script with that file as input, and then copies its output back up to cloud storage. Such a container can be mostly generic but then have the specific script to execute supplied to it as a runtime parameter. In this way, most of the work of file handling can be shared by multiple users/work queues and only the specifics of file processing need to be supplied by the end user.

Hãy xem xét ví dụ về một worker hàng đợi công việc tải một tệp từ bộ lưu trữ đám mây và chạy một shell script với tệp đó làm đầu vào, rồi sao chép đầu ra trở lại lên bộ lưu trữ đám mây. Một container như vậy có thể phần lớn là tổng quát nhưng sau đó được cung cấp script cụ thể cần thực thi như một tham số lúc chạy (runtime parameter). Theo cách này, phần lớn công việc xử lý tệp có thể được nhiều người dùng/hàng đợi công việc chia sẻ, và chỉ những phần cụ thể của việc xử lý tệp mới cần do người dùng cuối cung cấp.

The Shared Work Queue Infrastructure — Hạ tầng hàng đợi công việc dùng chung

Given implementations of the two container interfaces described previously, what is left to implement our reusable work queue implementation? The basic algorithm for the work queue is fairly straightforward:

Với các cách triển khai của hai giao diện container đã mô tả trước đó, còn lại điều gì cần triển khai cho hàng đợi công việc tái sử dụng được của chúng ta? Thuật toán cơ bản của hàng đợi công việc khá đơn giản:

1. Load the available work by calling into source container interface.
 2. Consult with work queue state to determine which work items have been processed or are being processed currently.
 3. For these items, spawn jobs that use the worker container interface to process the work item.
 4. When one of these worker containers finishes successfully, record that the work item has been completed.
1. Nạp các công việc khả dụng bằng cách gọi vào giao diện container nguồn.
 2. Tham chiếu với trạng thái hàng đợi công việc để xác định những đơn vị công việc nào đã được xử lý hoặc đang được xử lý hiện tại.
 3. Với các đơn vị này, sinh ra các job dùng giao diện container worker để xử lý đơn vị công việc.
 4. Khi một trong các container worker này hoàn thành thành công, ghi nhận rằng đơn vị công việc đã hoàn tất.

While this algorithm is simple to express in words, it is somewhat more complicated to implement in reality. Fortunately for us, the Kubernetes **container orchestrator** contains a number of features that make it significantly easier to implement. Namely, Kubernetes contains a **Job** object that allows for the reliable execution of the work queue. The Job can be configured to either run the worker container once or to run it until it completes successfully. If the worker container is set to run to completion, then even if a machine in the cluster fails, the job will eventually be run to success. This dramatically simplifies the task of building a work queue because the orchestrator takes on responsibility for the reliable operation of each work item.

Mặc dù thuật toán này dễ diễn đạt bằng lời, việc triển khai nó trong thực tế lại có phần phức tạp hơn. May mắn cho chúng ta, bộ điều phối container Kubernetes chứa một số tính năng giúp việc triển khai dễ dàng hơn đáng kể. Cụ thể, Kubernetes chứa một đối tượng Job cho phép thực thi hàng đợi công việc một cách tin cậy. Job có thể được cấu hình để hoặc chạy container worker một lần, hoặc chạy nó cho đến khi hoàn thành thành công. Nếu container worker được đặt chạy đến khi hoàn tất, thì ngay cả khi một máy trong cụm gặp sự cố, job rồi cũng sẽ được chạy đến thành công. Điều này đơn giản hóa đáng kể nhiệm vụ xây dựng một hàng đợi công việc vì bộ điều phối đảm nhận trách nhiệm về hoạt động tin cậy của mỗi đơn vị công việc.

Additionally, Kubernetes has annotations for each Job object that enable us to mark each job with the work item it is processing. This enables us to understand which items are being processed as well as those that have completed in either failure or success.

Ngoài ra, Kubernetes có các chú thích (annotation) cho mỗi đối tượng Job cho phép ta đánh dấu mỗi job với đơn vị công việc mà nó đang xử lý. Điều này cho phép ta hiểu được những đơn vị nào đang được xử lý cũng như những đơn vị nào đã hoàn tất, dù thất bại hay thành công.

Put together, this means that we can implement a work queue on top of the Kubernetes orchestration layer without using any storage of our own. This dramatically simplifies the task of building the work queue infrastructure.

Gộp lại, điều này có nghĩa là ta có thể triển khai một hàng đợi công việc trên nền tảng điều phối Kubernetes mà không cần dùng bất kỳ kho lưu trữ nào của riêng mình. Điều này đơn giản hóa đáng kể nhiệm vụ xây dựng hạ tầng hàng đợi công việc.

Thus, the expanded operation of our work queue container looks like this:

Như vậy, hoạt động mở rộng của container hàng đợi công việc của chúng ta trông như sau:

Repeat forever

Get the list of work items from the work source container interface. Get the list of all jobs that have been created to **service** this work queue. Difference these lists to find the set of work items that haven't been processed.

Lấy danh sách các đơn vị công việc từ giao diện container nguồn công việc. Lấy danh sách tất cả các job đã được tạo để phục vụ hàng đợi công việc này. Lấy hiệu (difference) của hai danh sách này để tìm ra tập các đơn vị công việc chưa được xử lý.

For these unprocessed items, create new Job objects that spawn the appropriate worker container.

Với những đơn vị chưa xử lý này, tạo các đối tượng Job mới sinh ra container worker thích hợp.

Here is a simple Python script that implements this work queue:

Đây là một script Python đơn giản triển khai hàng đợi công việc này:

```
import requests
import json
from kubernetes import client, config
import time
namespace = "default"
def make_container(item, obj):
    container = client.V1Container()
    container.image = "my/worker-image"
    container.name = "worker"
    return container
def make_job(item):
    response = requests.get("http://localhost:8000/items/{}".format(item))
    obj = json.loads(response.text)
    job = client.V1Job()
    job.metadata = client.V1ObjectMeta()
    job.metadata.name = item

    job.spec = client.V1JobSpec()
    job.spec.template = client.V1PodTemplate()
    job.spec.template.spec = client.V1PodTemplateSpec()
    job.spec.template.spec.restart_policy = "Never"
    job.spec.template.spec.containers = [
        make_container(item, obj)
    ]
    return job
def update_queue(batch):
    response = requests.get("http://localhost:8000/items")
    obj = json.loads(response.text)
    items = obj['items']
    ret = batch.list_namespaced_job(namespace, watch=False)
    for item in items:
        found = False
```

```

    for i in ret.items:
        if i.metadata.name == item:
            found = True
    if not found:
        # This function creates the job object, omitted for
        # brevity
        job = make_job(item)
        batch.create_namespaced_job(namespace, job)
config.load_kube_config()
batch = client.BatchV1Api()
while True:
    update_queue(batch)
    time.sleep(10)

```

Hands On: Implementing a Video Thumbnailer — Thực hành (Hands On): Triển khai bộ tạo thumbnail video

To provide a concrete example of how we might use a work queue, consider the task of generating thumbnails for videos. These thumbnails help users determine which videos they want to watch.

Để đưa ra một ví dụ cụ thể về cách ta có thể dùng một hàng đợi công việc, hãy xem xét nhiệm vụ tạo thumbnail cho các video. Những thumbnail này giúp người dùng xác định họ muốn xem video nào.

To implement this video thumbnailer, we need two different user containers. The first is the work item source container. The simplest way for this to work is for the work items to appear on a shared disk, such as a Network File System (NFS) share. The work item source simply lists the files in this directory and returns them to the caller. Here's a simple **node** program that does this:

Để triển khai bộ tạo thumbnail video này, ta cần hai container người dùng khác nhau. Cái thứ nhất là container nguồn đơn vị công việc. Cách đơn giản nhất để nó hoạt động là các đơn vị công việc xuất hiện trên một đĩa dùng chung, chẳng hạn một chia sẻ Network File System (NFS). Nguồn đơn vị công việc chỉ đơn giản liệt kê các tệp trong thư mục này và trả chúng về cho phía gọi. Đây là một chương trình node đơn giản làm điều đó:

```

const http = require('http');
const fs = require('fs');
const port = 8080;

const path = process.env.MEDIA_PATH;
const requestHandler = (request, response) => {
  console.log(request.url);
  fs.readdir(path + '/*.mp4', (err, items) => {
    var msg = {
      'kind': 'ItemList',
      'apiVersion': 'v1',
      'items': []
    };
    if (!items) {
      return msg;
    }
  });
}

```

```

for (var i = 0; i < items.length; i++) {
  msg.items.push(items[i]);
}
response.end(JSON.stringify(msg));
});
}
const server = http.createServer(requestHandler);
server.listen(port, (err) => {
  if (err) {
    return console.log('Error starting server', err);
  }
  console.log(`server is active on ${port}`)
});

```

This source of defines the queue of movies to **thumbnail**. We use the `ffmpeg` utility to actually perform the thumbnailing work.

Nguồn này định nghĩa hàng đợi các bộ phim cần tạo thumbnail. Ta dùng tiện ích `ffmpeg` để thực sự thực hiện công việc tạo thumbnail.

You can create a container that uses the following as its command line:

Bạn có thể tạo một container dùng đoạn sau làm dòng lệnh của nó:

```
ffmpeg -i ${INPUT_FILE} -frames:v 100 thumb.png
```

This command will take one frame every 100 frames (that's the `-frames:v 100` flag) and output it as a PNG file (e.g., `thumb1.png`, `thumb2.png`, etc.).

Lệnh này sẽ lấy một khung hình mỗi 100 khung hình (đó là cờ `-frames:v 100`) và xuất nó ra dưới dạng tệp PNG (ví dụ `thumb1.png`, `thumb2.png`, v.v.).

You can implement this image processing using an existing `ffmpeg` **Docker** image. The `jrottenberg/ffmpeg` Docker image is a popular choice.

Bạn có thể triển khai việc xử lý hình ảnh này bằng một image Docker `ffmpeg` có sẵn. Image Docker `jrottenberg/ffmpeg` là một lựa chọn phổ biến.

By defining a simple source container as well as an even simpler worker container, we can clearly see the power and utility of a generic, container-based queuing system. It dramatically reduces the time/distance between an idea for implementing a work queue and the corresponding concrete implementation.

Bằng cách định nghĩa một container nguồn đơn giản cũng như một container worker còn đơn giản hơn nữa, ta có thể thấy rõ sức mạnh và tính tiện ích của một hệ thống hàng đợi dựa trên container tổng quát. Nó giảm đáng kể khoảng cách thời gian/quãng đường giữa một ý tưởng triển khai hàng đợi công việc và cách triển khai cụ thể tương ứng.

Dynamic Scaling of the Workers — Co giãn động số lượng worker

The previously described work queue is great for processing work items as quickly as they arrive in the work queue, but this can lead to bursty resource loads being placed onto a container orchestrator cluster. This is good if you have a lot of different workloads that will burst at different times and thus keep your infrastructure evenly utilized. But if you don't have a sufficient number of different

workloads, this **feast or famine** approach to scaling your work queue might require that you over-provision resources to support the bursts that will lay idle (and cost too much money) while you don't have work to perform.

Hàng đợi công việc đã mô tả trước đó rất tuyệt cho việc xử lý các đơn vị công việc nhanh ngay khi chúng đến hàng đợi, nhưng điều này có thể dẫn đến những tải tài nguyên dồn dập (bursty) đặt lên cụm bộ điều phối container. Điều này tốt nếu bạn có nhiều khối lượng công việc (workload) khác nhau dồn lên vào những thời điểm khác nhau, qua đó giữ cho hạ tầng của bạn được sử dụng đều đặn. Nhưng nếu bạn không có đủ số lượng workload khác nhau, thì cách tiếp cận cố gắng kiểu “no dồn ứ” này có thể đòi hỏi bạn phải cấp dư (over-provision) tài nguyên để hỗ trợ các đợt dồn, vốn sẽ nằm không (và tốn quá nhiều tiền) trong lúc bạn không có việc để làm.

To address this problem, you can limit the overall number of Job objects that your work queue is willing to create. This will naturally serve to limit the number of work items you process in parallel and consequentially limit the maximum amount of resources that you use at a particular time. However, doing this will increase the time to completion (**latency**) for each work item being completed when under heavy load. If the load is bursty, then this is probably okay because you can use the slack times to catch up with the backlog that developed during a burst of usage. However, if your steady-state usage is too high, your work queue may never be able to catch up and the time to completion will simply get longer and longer.

Để giải quyết vấn đề này, bạn có thể giới hạn tổng số đối tượng Job mà hàng đợi công việc của bạn sẵn sàng tạo ra. Điều này một cách tự nhiên sẽ giúp giới hạn số đơn vị công việc bạn xử lý song song, và do đó giới hạn lượng tài nguyên tối đa mà bạn sử dụng tại một thời điểm cụ thể. Tuy nhiên, làm như vậy sẽ tăng thời gian hoàn thành (độ trễ) cho mỗi đơn vị công việc khi hệ thống chịu tải nặng. Nếu tải có tính dồn dập, thì điều này có lẽ ổn vì bạn có thể dùng những lúc rảnh để bắt kịp lượng tồn đọng (backlog) phát sinh trong một đợt sử dụng dồn dập. Tuy nhiên, nếu mức sử dụng ở trạng thái ổn định (steady-state) của bạn quá cao, thì hàng đợi công việc của bạn có thể sẽ không bao giờ bắt kịp và thời gian hoàn thành sẽ ngày càng dài ra.

When your work queue is faced with this situation, you need to have it dynamically adjust itself to increase the parallelism that it is willing to create (and correspondingly the resources it is willing to use) so that it can keep up with the incoming work. Fortunately, there are mathematical formulas that we can use to determine when we need to dynamically scale up our work queue.

Khi hàng đợi công việc của bạn đối mặt với tình huống này, bạn cần để nó tự điều chỉnh động nhằm tăng mức song song mà nó sẵn sàng tạo ra (và tương ứng là tài nguyên nó sẵn sàng sử dụng) để có thể theo kịp lượng công việc đến. May mắn thay, có những công thức toán học mà ta có thể dùng để xác định khi nào ta cần mở rộng động hàng đợi công việc.

Consider a work queue where a new work item arrives an average of once every minute, and each work item takes an average of 30 seconds to complete. Such a system is capable of keeping up with all of the work it receives. Even if a large batch of work arrives all at once and creates a backlog, on average the work queue processes two work items for every one work item that arrives, and thus it will be able to gradually work through its backlog.

Hãy xem xét một hàng đợi công việc trong đó một đơn vị công việc mới đến trung bình một lần mỗi phút, và mỗi đơn vị công việc mất trung bình 30 giây để hoàn thành. Một hệ thống như vậy có khả năng theo kịp toàn bộ công việc nó nhận được. Ngay cả khi một lô lớn công việc đến cùng lúc và tạo ra lượng tồn đọng, thì trung bình hàng đợi công việc xử lý hai đơn vị công việc cho mỗi một đơn vị công việc đến, và do đó nó sẽ có thể dần dần giải quyết hết lượng tồn đọng của mình.

If, instead, a new work item arrives on average once every minute and it takes an average of one minute to process each work item, then the system is perfectly balanced, but it does not handle variance well. It can catch up with bursts—but it will take a while, and it has no slack or capacity to absorb a sustained increase in the rate at which new work items arrive. This is probably not an ideal way to run, as some safety margin for growth and other sustained increases in work (or unexpected slowdowns in processing) is needed to preserve a stable system.

Ngược lại, nếu một đơn vị công việc mới đến trung bình một lần mỗi phút và mất trung bình một phút để xử lý mỗi đơn vị, thì hệ thống cân bằng hoàn hảo, nhưng nó không xử lý tốt biến thiên (variance). Nó có thể bắt kịp các đợt dồn — nhưng sẽ mất một lúc, và nó không có dư địa hay năng lực để hấp thụ một sự gia tăng kéo dài về tốc độ đến của các đơn vị công việc mới. Đây có lẽ không phải là cách vận hành lý tưởng, vì cần có một biên độ an toàn nào đó cho tăng trưởng và các gia tăng kéo dài khác về công việc (hoặc những lần chậm trễ xử lý ngoài dự kiến) để duy trì một hệ thống ổn định.

Finally, consider a system in which a work item arrives every minute and each item takes 2 minutes to process. In such a system, we are always losing ground. The queue of work will grow without bound and the latency of any one item in the queue will grow toward infinity (and our users will become very frustrated).

Cuối cùng, hãy xem xét một hệ thống trong đó một đơn vị công việc đến mỗi phút và mỗi đơn vị mất 2 phút để xử lý. Trong một hệ thống như vậy, ta luôn bị thụt lùi. Hàng đợi công việc sẽ tăng không giới hạn và độ trễ của bất kỳ đơn vị nào trong hàng đợi sẽ tăng dần đến vô cùng (và người dùng của chúng ta sẽ trở nên cực kỳ bức bối).

Thus, we can keep track of both of these metrics for our work queue, and the average time between work items over an extended period of time ($\# \text{ work items} / 24 \text{ hours}$) will give us the **interarrival time** for new work. We can also keep track of the average time to process any one item once we start working on it (not counting any time in the queue). To have a stable work queue, we need to adjust the number of resources so that the time to process any item is less than the interarrival time of new items. If we are processing work items in parallel, we also divide the processing time for a work item by the parallelism. For example, if each item takes one minute to process but we process four items in parallel, the effective time to process one item is 15 seconds, and thus we can sustain an interarrival period of 16 or more seconds.

Như vậy, ta có thể theo dõi cả hai chỉ số này cho hàng đợi công việc của mình, và thời gian trung bình giữa các đơn vị công việc trong một khoảng thời gian dài ($\# \text{ đơn vị công việc} / 24 \text{ giờ}$) sẽ cho ta thời gian giữa các lần đến (interarrival time) của công việc mới. Ta cũng có thể theo dõi thời gian trung bình để xử lý bất kỳ một đơn vị nào sau khi ta bắt đầu làm việc với nó (không tính thời gian nằm trong hàng đợi). Để có một hàng đợi công việc ổn định, ta cần điều chỉnh số lượng tài nguyên sao cho thời gian xử lý bất kỳ đơn vị nào nhỏ hơn thời gian giữa các lần đến của các đơn vị mới. Nếu ta đang xử lý các đơn vị công việc song song, ta cũng chia thời gian xử lý của một đơn vị công việc cho mức song song. Ví dụ, nếu mỗi đơn vị mất một phút để xử lý nhưng ta xử lý bốn đơn vị song song, thì thời gian xử lý hiệu dụng của một đơn vị là 15 giây, và do đó ta có thể duy trì một khoảng thời gian giữa các lần đến từ 16 giây trở lên.

This approach makes it fairly straightforward to build an **autoscaler** to dynamically size up our work queue. Sizing down the work queue is somewhat trickier, but you can use the same math as well as a heuristic for the amount of spare capacity for the safety margin you want to maintain. For example, you can **reduce** the parallelism until the processing time for an item is 90% of the interarrival time for new items.

Cách tiếp cận này khiến cho việc xây dựng một bộ tự co giãn (autoscaler) để định cỡ động

hàng đợi công việc trở nên khá đơn giản. Việc thu nhỏ hàng đợi công việc có phần khó hơn một chút, nhưng bạn có thể dùng cùng phép toán cũng như một heuristic về lượng năng lực dự phòng cho biên độ an toàn mà bạn muốn duy trì. Ví dụ, bạn có thể giảm mức song song cho đến khi thời gian xử lý một đơn vị bằng 90% thời gian giữa các lần đến của các đơn vị mới.

The Multi-Worker Pattern — Mẫu đa worker

One of the themes of this book has been the use of containers for encapsulation and reuse of code. The same holds true for the work queue patterns described in this chapter. In addition to the patterns for reusing containers for driving the work queue itself, you can also reuse multiple different containers to compose a worker implementation. Suppose, for example, that you have three different types of work that you want to perform on a particular work queue item. For example, you might want to detect faces in an image, tag those faces with identities, and then blur the faces in the image. You could write a single worker to perform this complete set of tasks, but this would be a bespoke solution that would not be reusable the next time you want to identify something else, such as cars, yet still provide the same blurring.

Một trong những chủ đề xuyên suốt cuốn sách này là việc dùng container để đóng gói và tái sử dụng code. Điều tương tự cũng đúng với các mẫu hàng đợi công việc được mô tả trong chương này. Ngoài các mẫu để tái sử dụng container cho việc vận hành chính hàng đợi công việc, bạn cũng có thể tái sử dụng nhiều container khác nhau để hợp thành một cách triển khai worker. Giả sử, ví dụ, bạn có ba loại công việc khác nhau mà bạn muốn thực hiện trên một đơn vị công việc cụ thể của hàng đợi. Chẳng hạn, bạn có thể muốn phát hiện khuôn mặt trong một ảnh, gắn nhãn danh tính cho các khuôn mặt đó, rồi làm mờ các khuôn mặt trong ảnh. Bạn có thể viết một worker duy nhất để thực hiện trọn bộ các tác vụ này, nhưng đó sẽ là một giải pháp đo ni đóng giày không tái sử dụng được vào lần tiếp theo khi bạn muốn nhận dạng thứ gì khác, chẳng hạn xe hơi, mà vẫn cung cấp cùng tính năng làm mờ.

To achieve this kind of code reuse, the **multi-worker pattern** is something of a specialization of the **adapter** pattern described in previous chapters. In this case, the multi-worker pattern transforms a collection of different worker containers into a single unified container that implements the worker interface, yet delegates the actual work to a collection of different, reusable containers. This process is illustrated in Figure 10-5 .

Để đạt được kiểu tái sử dụng code này, mẫu đa worker (multi-worker) phần nào là một dạng đặc biệt hóa của mẫu adapter đã mô tả ở các chương trước. Trong trường hợp này, mẫu đa worker biến đổi một tập hợp các container worker khác nhau thành một container hợp nhất duy nhất triển khai giao diện worker, nhưng lại ủy thác công việc thực tế cho một tập hợp các container khác nhau, tái sử dụng được. Quá trình này được minh họa trong Hình 10-5.

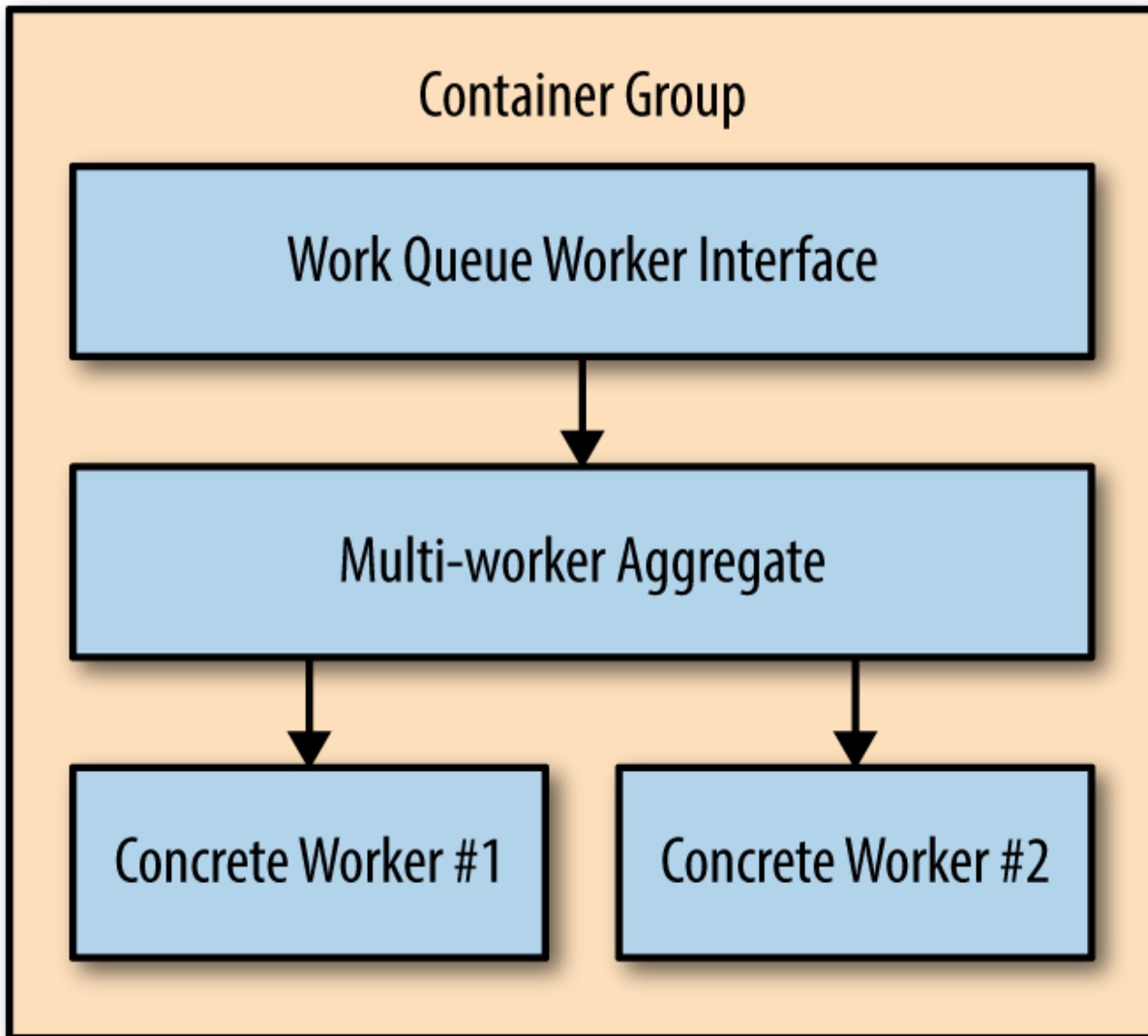


Figure 10-5. The multi-worker aggregator pattern as a group of containers

Hình 10-5. Mẫu tổng hợp đa worker dưới dạng một nhóm container

Because of this code reuse, the composition of multiple different worker containers means an increase in the reuse of code and a reduction in effort for people designing batch-oriented distributed systems.

Nhờ việc tái sử dụng code này, việc hợp thành nhiều container worker khác nhau đồng nghĩa với việc gia tăng mức tái sử dụng code và giảm công sức cho những người thiết kế các hệ phân tán hướng theo lô.

CHAPTER 11 Event-Driven Batch Processing — CHAPTER 11 Xử Lý Theo Lô Hướng Sự Kiện

In the previous chapter, we saw a generic framework for **work queue** processing, as well as a number of example applications of simple work queue processing. Work queues are great for enabling individual transformations of one input to one output. However, there are a number of batch applications where you want to perform more than a single action, or you may need to generate multiple different outputs from a single data input. In these cases, you start to link work queues together so that the output of one work queue becomes the input to one or more other work queues, and so on. This forms a series of processing steps that respond to events, with the events being the completion of the preceding step in the work queue that came before it.

Ở chương trước, chúng ta đã thấy một khung tổng quát cho việc xử lý hàng đợi công việc (work queue), cùng với một số ví dụ ứng dụng đơn giản của xử lý hàng đợi công việc. Hàng đợi công việc rất phù hợp để cho phép thực hiện từng biến đổi riêng lẻ từ một đầu vào sang một đầu ra. Tuy nhiên, có không ít ứng dụng theo lô (batch) mà ở đó bạn muốn thực hiện nhiều hơn một hành động duy nhất, hoặc bạn cần tạo ra nhiều đầu ra khác nhau từ một đầu vào dữ liệu duy nhất. Trong những trường hợp như vậy, bạn bắt đầu liên kết các hàng đợi công việc lại với nhau sao cho đầu ra của một hàng đợi công việc trở thành đầu vào cho một hoặc nhiều hàng đợi công việc khác, và cứ thế tiếp tục. Điều này tạo thành một chuỗi các bước xử lý phản ứng theo sự kiện, với sự kiện chính là việc hoàn thành bước đứng trước nó trong hàng đợi công việc trước đó.

These sort of **event-driven** processing systems are often called workflow systems, since there is a flow of work through a directed, acyclic graph that describes the various stages and their coordination. A basic illustration of such a system is shown in Figure 11-1 .

Những hệ thống xử lý hướng sự kiện (event-driven) kiểu này thường được gọi là hệ thống luồng công việc (workflow), bởi vì có một dòng công việc chảy qua một đồ thị có hướng, phi chu trình (directed, acyclic graph) mô tả các giai đoạn khác nhau và sự phối hợp giữa chúng. Một minh họa cơ bản về một hệ thống như vậy được trình bày trong Hình 11-1.

The most straightforward application of this type of system simply chains the output of one queue to the input of the next queue. But as systems become more complicated there are a series of different patterns that emerge for linking a series of work queues together. Understanding and designing in terms of these patterns is important for comprehending how the system is working. The operation of an **event-driven** batch processor is similar to event-driven FaaS. Consequently, without an overall blueprint for how the different event queues relate to each other, it can be hard to fully understand how the system is operating.

Ứng dụng đơn giản nhất của loại hệ thống này chỉ đơn thuần nối đầu ra của một hàng đợi vào đầu vào của hàng đợi tiếp theo. Nhưng khi các hệ thống trở nên phức tạp hơn, sẽ xuất hiện một loạt các mẫu khác nhau để liên kết một chuỗi hàng đợi công việc lại với nhau. Việc hiểu và thiết kế dựa trên các mẫu này rất quan trọng để nắm bắt được cách hệ thống vận hành. Cách hoạt động của một bộ xử lý theo lô hướng sự kiện tương tự như FaaS hướng sự kiện. Do đó, nếu không có một bản thiết kế tổng thể cho việc các hàng đợi sự kiện khác nhau liên hệ với nhau như thế nào, sẽ rất khó để hiểu trọn vẹn cách hệ thống đang vận hành.

121

121

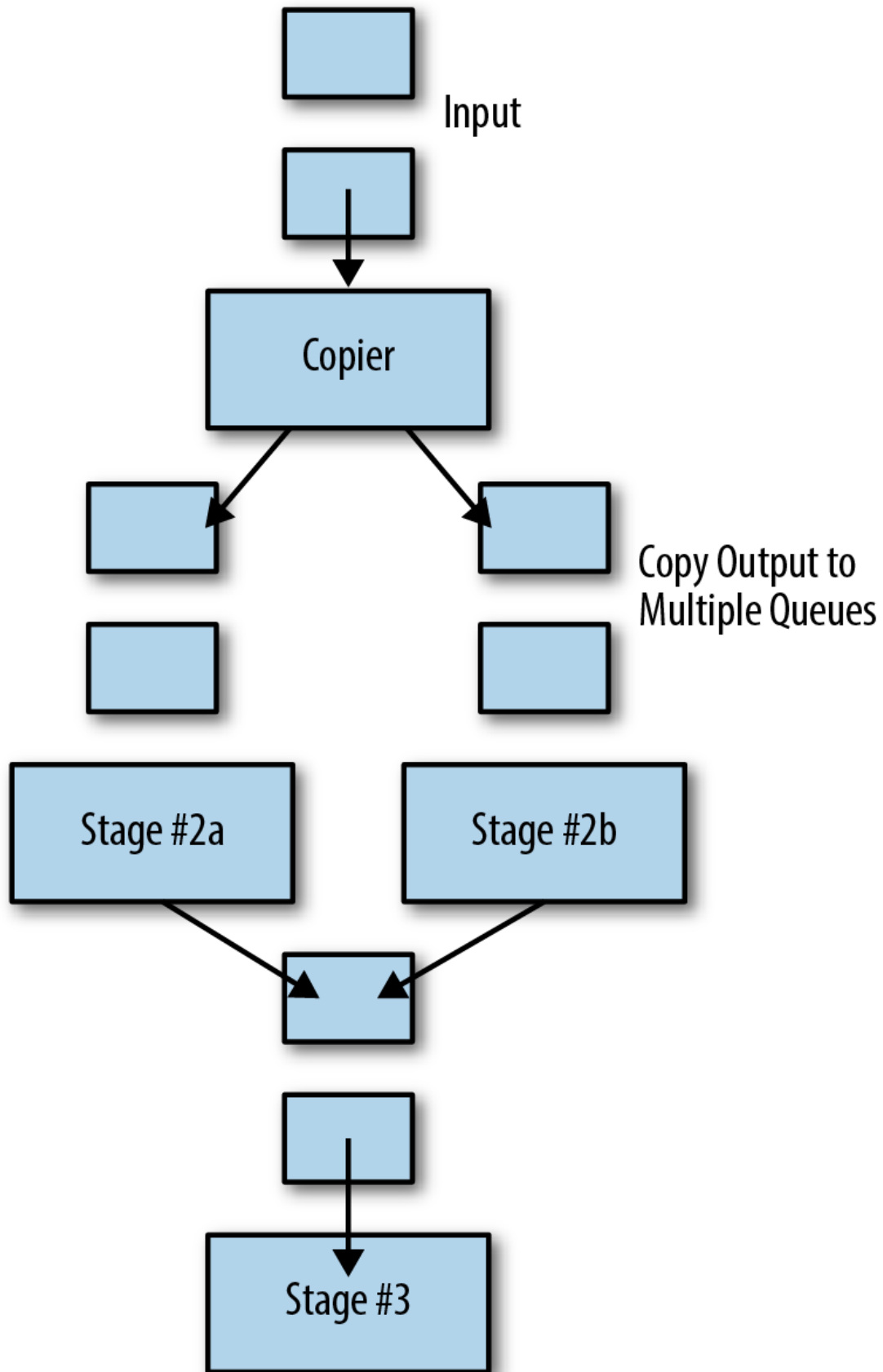


Figure 11-1. This workflow combines copying work into multiple queues (Stage 2a, 2b) parallel processing of those queues, and combining the result back into a single queue (Stage 3)

Hình 11-1. Luồng công việc này kết hợp việc sao chép công việc vào nhiều hàng đợi (Giai đoạn 2a, 2b), xử lý song song các hàng đợi đó, và gộp kết quả trở lại thành một hàng đợi duy nhất (Giai đoạn 3)

Patterns of Event-Driven Processing — Các Mẫu Xử Lý Hướng Sự Kiện

Beyond the simple work queue described in the previous chapter, there are a number of patterns for linking work queues together. The simplest pattern—one where the output of a single queue becomes the input to a second queue—is straightforward enough that we won’t cover it here. We will describe patterns that involve the coordination of multiple different queues or the modification of the output of one or more work queues.

Ngoài hàng đợi công việc đơn giản đã mô tả ở chương trước, còn có một số mẫu để liên kết các hàng đợi công việc lại với nhau. Mẫu đơn giản nhất—mẫu mà ở đó đầu ra của một hàng đợi trở thành đầu vào của hàng đợi thứ hai—đủ rõ ràng đến mức chúng ta sẽ không trình bày ở đây. Chúng ta sẽ mô tả các mẫu liên quan đến việc phối hợp nhiều hàng đợi khác nhau hoặc việc sửa đổi đầu ra của một hay nhiều hàng đợi công việc.

Copier — Copier

The first pattern for coordinating work queues is a copier. The **job** of a copier is to take a single stream of work items and duplicate it out into two or more identical streams. This pattern is useful when there are multiple different pieces of work to be done on the same **work item**. An example of this might be rendering a video. When rendering a video, there are a variety of different formats that are useful depending on where the video is intended to be shown. There might be a 4-KB high-resolution format for playing off of a hard drive, a 1080-pixel rendering for digital streaming, a low-resolution format for streaming to mobile users on slow networks, and an animated GIF **thumbnail** for displaying in a movie-picking user **interface**. All of these work items can be modeled as separate work queues for each render, but the input to each work item is identical. An illustration of the **copier pattern** applied to transcoding is shown in Figure 11-2 .

Mẫu đầu tiên để phối hợp các hàng đợi công việc là copier (mẫu sao chép). Nhiệm vụ của một copier là lấy một luồng các mục công việc duy nhất và nhân đôi nó thành hai hoặc nhiều luồng giống hệt nhau. Mẫu này hữu ích khi có nhiều phần việc khác nhau cần thực hiện trên cùng một mục công việc. Một ví dụ về điều này có thể là việc kết xuất (render) một video. Khi kết xuất một video, có nhiều định dạng khác nhau hữu ích tùy thuộc vào nơi video dự định được trình chiếu. Có thể có một định dạng độ phân giải cao 4-KB để phát từ ổ cứng, một bản kết xuất 1080-pixel để phát trực tuyến số, một định dạng độ phân giải thấp để phát trực tuyến cho người dùng di động trên mạng chậm, và một ảnh thu nhỏ GIF động để hiển thị trong giao diện người dùng chọn phim. Tất cả các mục công việc này có thể được mô hình hóa thành các hàng đợi công việc riêng biệt cho mỗi lần kết xuất, nhưng đầu vào cho mỗi mục công việc lại giống hệt nhau. Một minh họa về mẫu copier áp dụng cho việc chuyển mã (transcoding) được trình bày trong Hình 11-2.

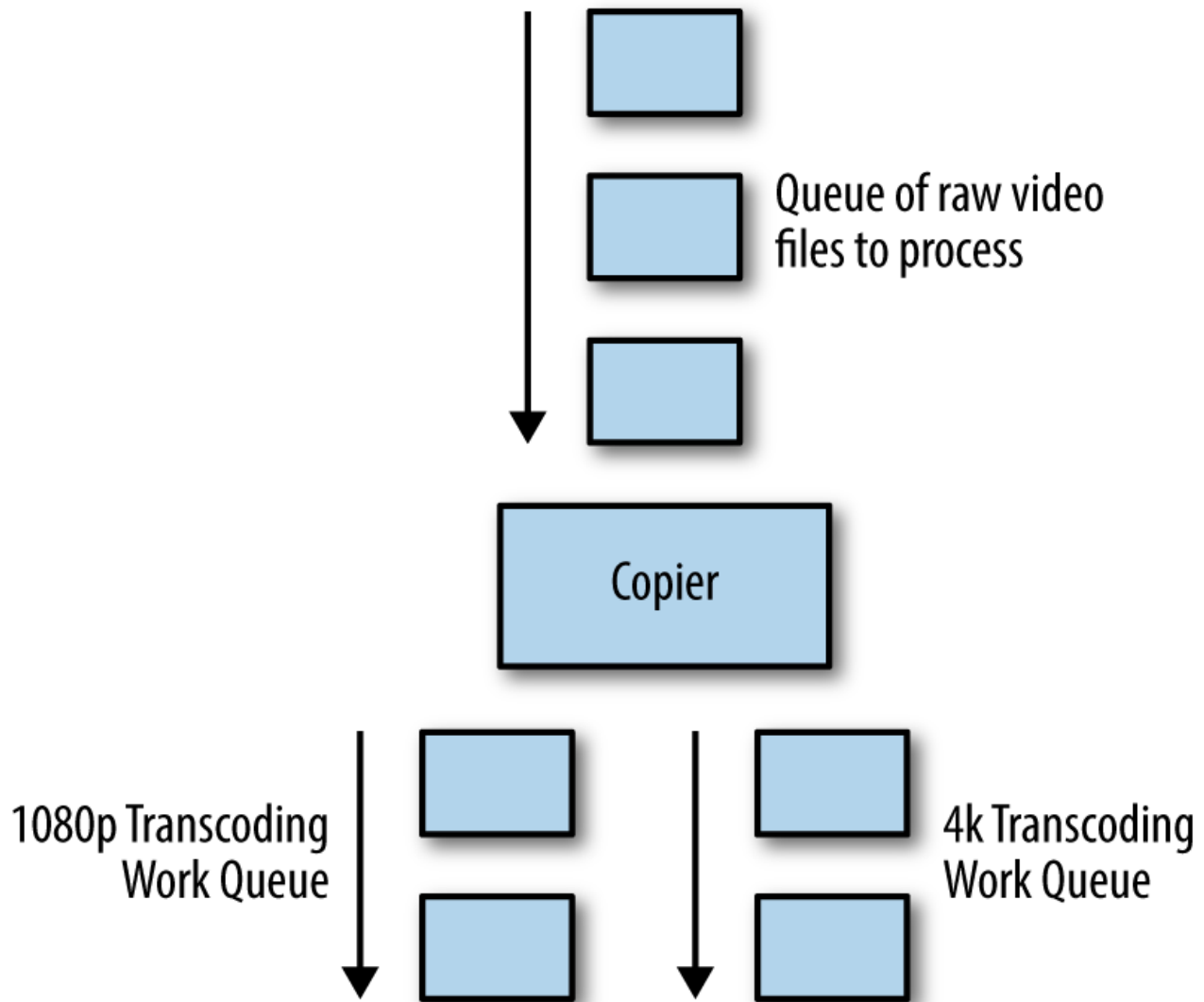


Figure 11-2. The copier batch pattern for transcoding

Hình 11-2. Mẫu theo lô copier dùng cho việc chuyển mã

Filter — Filter

The second pattern for event-driven **batch processing** is a filter. The role of a filter is to **reduce** a stream of work items to a smaller stream of work items by filtering out work items that don't meet particular criteria. As an example of this, consider setting up a batch workflow that handles new users signing up for a **service**. Some set of those users will have ticked the checkbox that indicates that they wish to be contacted via email for promotions and other information. In such a workflow, you can filter the set of newly signed-up users to only be those who have explicitly opted into being contacted.

Mẫu thứ hai cho xử lý theo lô hướng sự kiện là filter (mẫu lọc). Vai trò của một filter là thu nhỏ một luồng các mục công việc thành một luồng nhỏ hơn bằng cách lọc bỏ những mục công việc không đáp ứng các tiêu chí cụ thể. Lấy ví dụ về điều này, hãy xét việc thiết lập

một luồng công việc theo lô xử lý những người dùng mới đăng ký dịch vụ. Một số trong những người dùng đó sẽ đã tích vào ô đánh dấu thể hiện rằng họ muốn được liên hệ qua email để nhận khuyến mãi và các thông tin khác. Trong một luồng công việc như vậy, bạn có thể lọc tập hợp những người dùng vừa mới đăng ký để chỉ còn lại những người đã chủ động chọn (opt in) việc được liên hệ.

Ideally you would compose a filter work queue source as an **ambassador** that wraps up an existing work queue source. The original source **container** provides the complete list of items to be worked on, and the filter container then adjusts that list based on the filter criteria and only returns those filtered results to the work queue infrastructure. An illustration of this use of the **adapter** pattern is shown in Figure 11-3 .

Lý tưởng nhất là bạn nên cấu thành một nguồn hàng đợi công việc dạng filter như một ambassador bao bọc một nguồn hàng đợi công việc hiện có. Container nguồn gốc cung cấp danh sách đầy đủ các mục cần xử lý, sau đó container filter điều chỉnh danh sách đó dựa trên tiêu chí lọc và chỉ trả về những kết quả đã lọc cho hạ tầng hàng đợi công việc. Một minh họa về cách dùng mẫu adapter này được trình bày trong Hình 11-3.

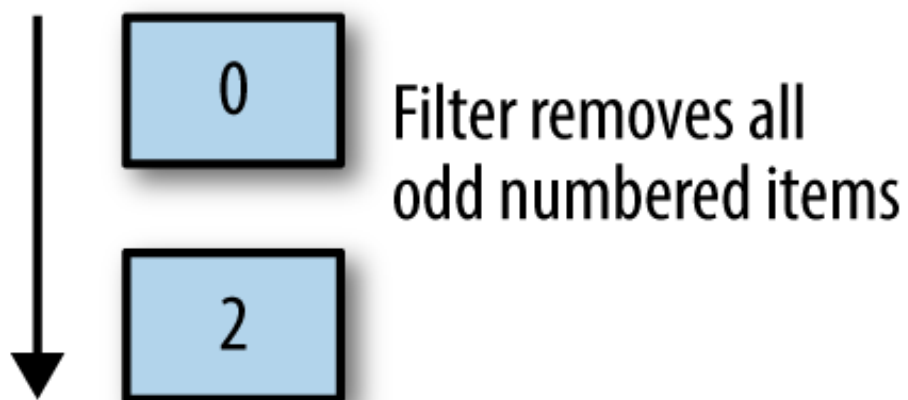
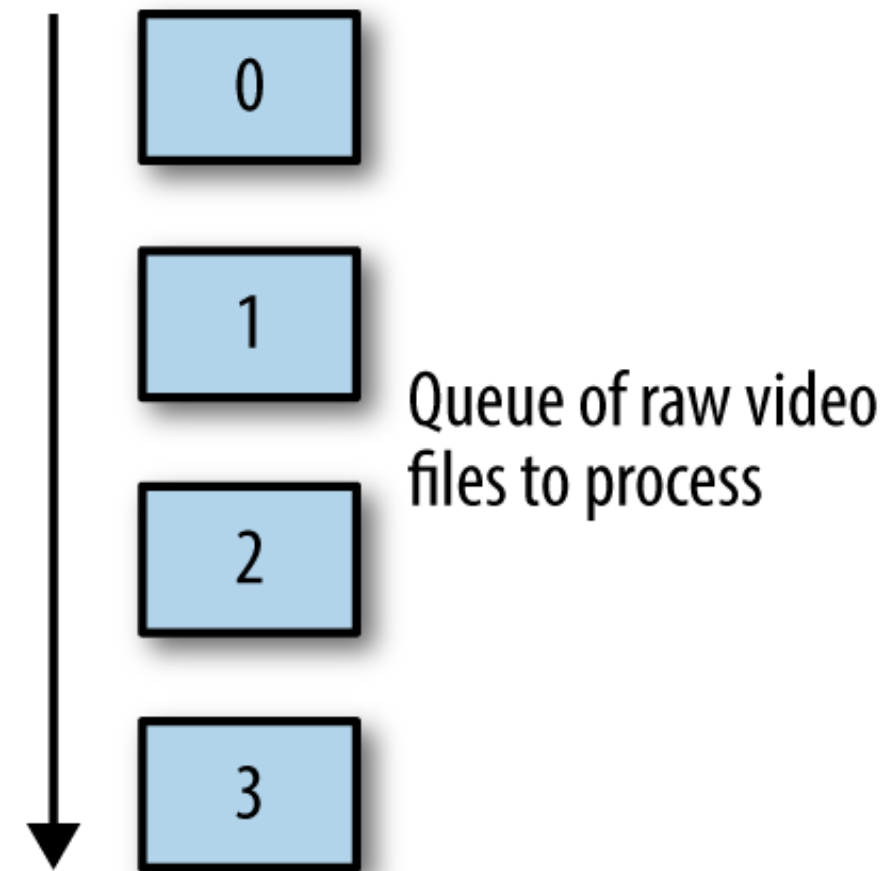


Figure 11-3. An example of a **filter pattern** that removes all odd-numbered work items

Hình 11-3. Một ví dụ về mẫu filter loại bỏ tất cả các mục công việc có số thứ tự lẻ

Splitter — Splitter

Sometimes you don't want to just filter things out by dropping them on the floor, but rather you have two different kinds of input present in your set of work items and you want to divide them into two separate work queues without dropping any of them. For this task, you want to use a **splitter**. The role of a splitter is to evaluate some criteria—just like a filter—but instead of eliminating input, the splitter sends different inputs to different queues based on that criteria.

Đôi khi bạn không muốn chỉ đơn giản lọc bỏ mọi thứ bằng cách vứt chúng đi, mà thay vào đó trong tập hợp các mục công việc của bạn có hai loại đầu vào khác nhau và bạn muốn chia chúng thành hai hàng đợi công việc riêng biệt mà không bỏ đi mục nào. Cho nhiệm vụ này, bạn nên dùng một splitter (bộ tách). Vai trò của một splitter là đánh giá một số tiêu chí nào đó—giống như một filter—nhưng thay vì loại bỏ đầu vào, splitter gửi các đầu vào khác nhau tới các hàng đợi khác nhau dựa trên tiêu chí đó.

An example of an application of the splitter pattern is processing online orders where people can receive shipping notifications either by email or text message. Given a work queue of items that have been shipped, the splitter divides it into two different queues: one that is responsible for sending emails and another devoted to sending text messages. A splitter can also be a copier if it sends the same output to multiple queues, such as when a user selects both text messages and email notifications in the previous example. It is interesting to note that a splitter can actually also be implemented by a copier and two different filters. But the splitter pattern is a more compact representation that captures the job of the splitter more succinctly. An example of using the splitter pattern to send shipping notifications to users is shown in Figure 11-4 .

Một ví dụ về ứng dụng của mẫu splitter là xử lý các đơn hàng trực tuyến, nơi mọi người có thể nhận thông báo vận chuyển hoặc qua email hoặc qua tin nhắn văn bản. Cho một hàng đợi công việc gồm các mục đã được vận chuyển, splitter chia nó thành hai hàng đợi khác nhau: một hàng đợi chịu trách nhiệm gửi email và một hàng đợi khác dành cho việc gửi tin nhắn văn bản. Một splitter cũng có thể đồng thời là một copier nếu nó gửi cùng một đầu ra tới nhiều hàng đợi, chẳng hạn như khi một người dùng chọn cả thông báo bằng tin nhắn văn bản lẫn email trong ví dụ trước. Một điều thú vị đáng lưu ý là một splitter thực ra cũng có thể được hiện thực bằng một copier và hai filter khác nhau. Nhưng mẫu splitter là một cách biểu diễn cô đọng hơn, nắm bắt nhiệm vụ của splitter một cách súc tích hơn. Một ví dụ về việc dùng mẫu splitter để gửi thông báo vận chuyển tới người dùng được trình bày trong Hình 11-4.

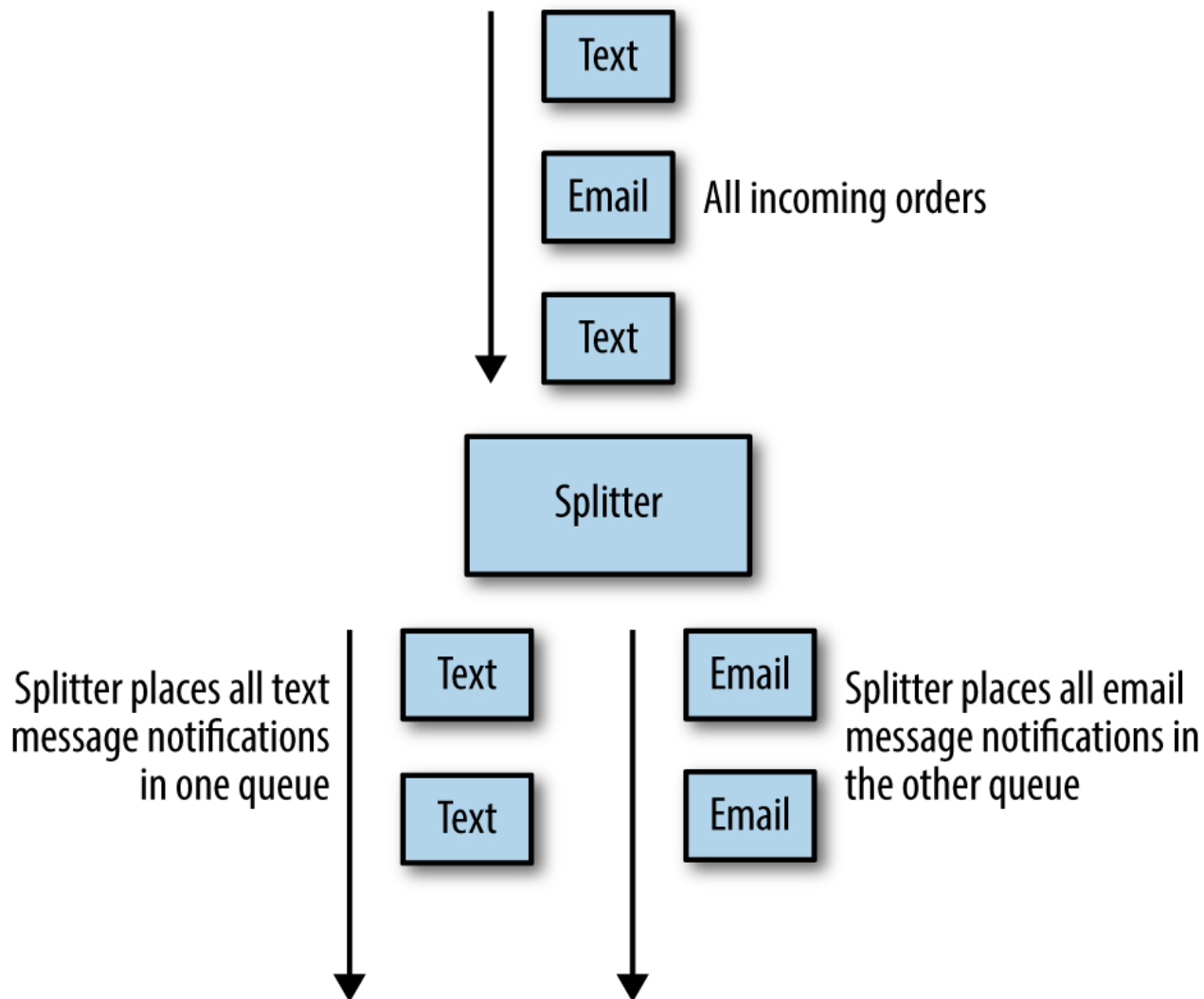


Figure 11-4. An example of the batch splitter pattern splitting shipping notifications into two different queues

Hình 11-4. Một ví dụ về mẫu splitter theo lô tách các thông báo vận chuyển thành hai hàng đợi khác nhau

Sharder — Sharder

A slightly more generic form of splitter is a sharder. Much like the sharded server that we saw in earlier chapters, the role of a sharder in a workflow is to divide up a single queue into an evenly divided collection of work items based upon some sort of **sharding function**. There are several different reasons why you might consider sharding your workflow. One of the first is for **reliability**. If you **shard** your work queue, then the failure of a single workflow due to a bad update, infrastructure failure, or other problem only affects a fraction of your service.

Một dạng tổng quát hơn đôi chút của splitter là sharder. Giống như sharded server (máy chủ phân mảnh) mà chúng ta đã thấy ở các chương trước, vai trò của một sharder trong một luồng công việc là chia một hàng đợi duy nhất thành một tập hợp các mục công việc được phân chia đồng đều dựa trên một loại hàm phân mảnh (sharding function) nào đó.

Có một số lý do khác nhau khiến bạn có thể cân nhắc phân mảnh luồng công việc của mình. Một trong những lý do đầu tiên là để tăng độ tin cậy. Nếu bạn phân mảnh hàng đợi công việc, thì sự cố của một luồng công việc đơn lẻ do một bản cập nhật lỗi, sự cố hạ tầng, hoặc vấn đề khác sẽ chỉ ảnh hưởng đến một phần dịch vụ của bạn.

For example, imagine that you push a bad update to your **worker** container, which causes your workers to crash and your queue to stop processing work items. If you only have a single work queue that is processing items, then you will have a complete outage for your service with all users affected. If, instead, you have sharded your work queue into four different shards, you have the opportunity to do a staged **rollout** of your new worker container. Assuming you catch the failure in the first phase of the staged rollout, sharding your queue into four different shards means that only one quarter of your users would be affected.

Ví dụ, hãy hình dung bạn đẩy một bản cập nhật lỗi tới container worker của mình, khiến các worker bị sập và hàng đợi của bạn ngừng xử lý các mục công việc. Nếu bạn chỉ có một hàng đợi công việc duy nhất đang xử lý các mục, thì bạn sẽ gặp sự cố mất dịch vụ hoàn toàn với toàn bộ người dùng bị ảnh hưởng. Thay vào đó, nếu bạn đã phân mảnh hàng đợi công việc thành bốn mảnh khác nhau, bạn có cơ hội thực hiện một đợt tung ra theo từng giai đoạn (staged rollout) cho container worker mới. Giả sử bạn bắt được lỗi trong giai đoạn đầu tiên của đợt tung ra theo từng giai đoạn, việc phân mảnh hàng đợi thành bốn mảnh khác nhau có nghĩa là chỉ một phần tư người dùng của bạn bị ảnh hưởng.

An additional reason to shard your work queue is to more evenly distribute work across different resources. If you don't really care which region or datacenter is used to process a particular set of work items, you can use a sharder to evenly spread work across multiple datacenters to even out utilization of all datacenters/regions. As with updates, spreading your work queue across multiple failure regions also has the benefit of providing reliability against datacenter or region failures. An illustration of a sharded queue when everything is working correctly is shown in Figure 11-5 .

Một lý do bổ sung để phân mảnh hàng đợi công việc là để phân bổ công việc đồng đều hơn trên các tài nguyên khác nhau. Nếu bạn không thực sự quan tâm vùng (region) hay trung tâm dữ liệu nào được dùng để xử lý một tập hợp các mục công việc cụ thể, bạn có thể dùng một sharder để trải đều công việc trên nhiều trung tâm dữ liệu nhằm cân bằng mức sử dụng của tất cả các trung tâm dữ liệu/vùng. Cũng như với việc cập nhật, việc trải hàng đợi công việc của bạn trên nhiều vùng sự cố (failure region) cũng có lợi ích là cung cấp độ tin cậy chống lại sự cố của trung tâm dữ liệu hoặc vùng. Một minh họa về một hàng đợi đã phân mảnh khi mọi thứ hoạt động bình thường được trình bày trong Hình 11-5.

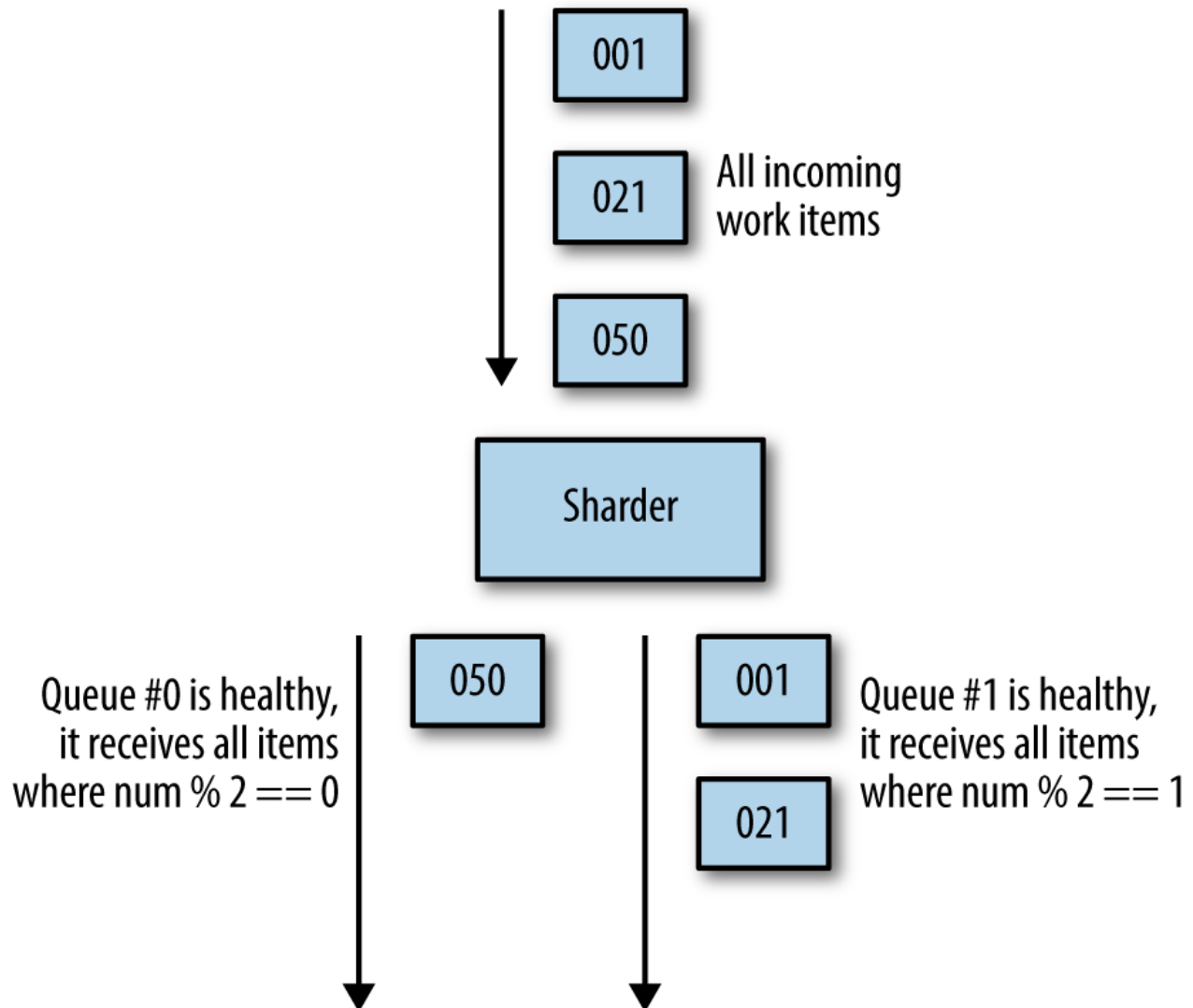


Figure 11-5. An example of the sharding pattern in a healthy operation

Hình 11-5. Một ví dụ về mẫu phân mảnh trong tình trạng hoạt động lành mạnh

When the number of healthy shards is reduced due to failures, the sharding algorithm dynamically adjusts to send work to the remaining healthy work queues, even if only a single queue remains. This is illustrated in Figure 11-6 .

Khi số lượng mảnh lành mạnh giảm đi do các sự cố, thuật toán phân mảnh sẽ tự động điều chỉnh để gửi công việc tới các hàng đợi công việc lành mạnh còn lại, ngay cả khi chỉ còn lại một hàng đợi duy nhất. Điều này được minh họa trong Hình 11-6.

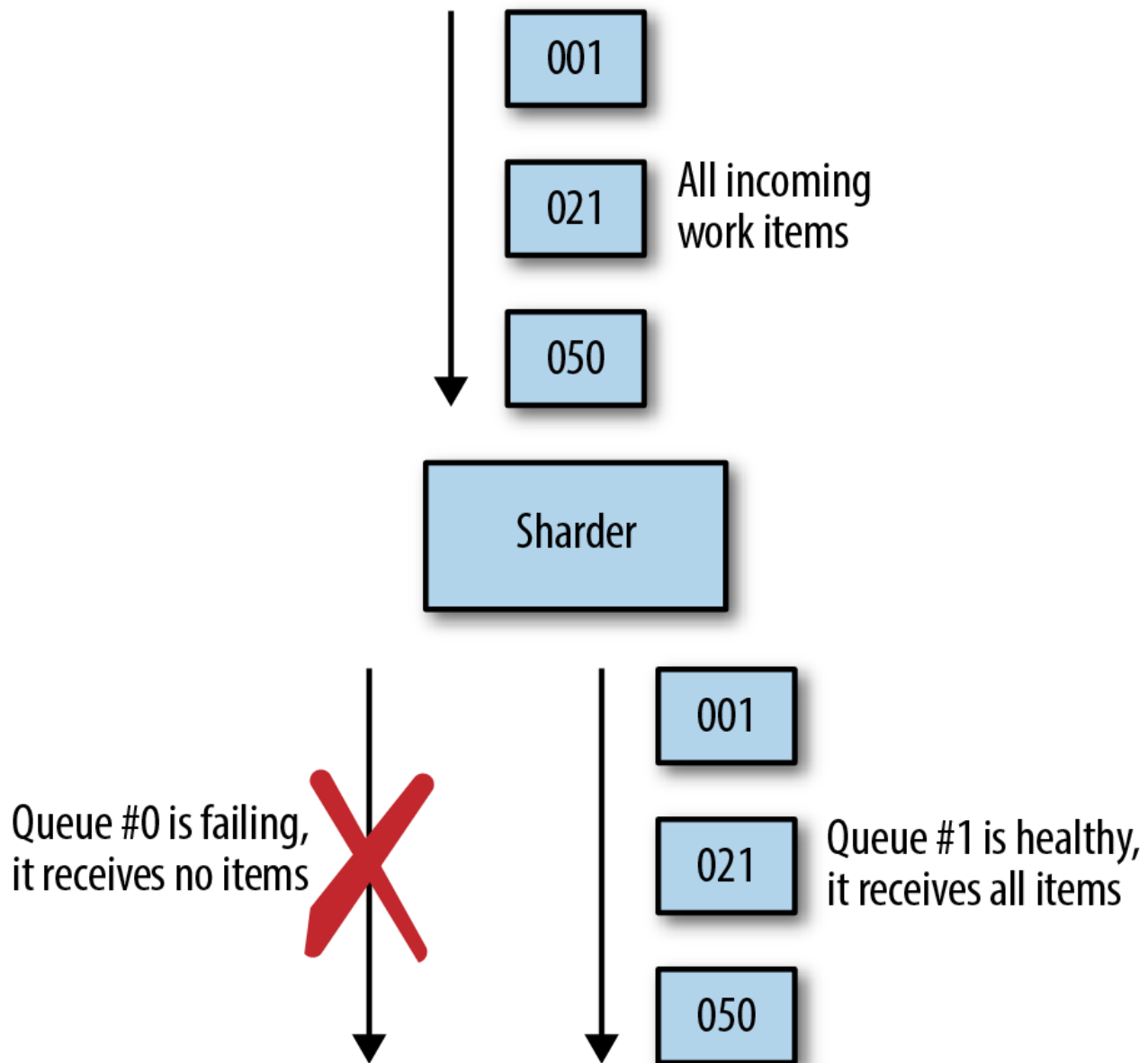


Figure 11-6. When one work queue is unhealthy the remaining work spills over to a different queue

Hình 11-6. Khi một hàng đợi công việc không lành mạnh, phần công việc còn lại tràn sang một hàng đợi khác

Merger — Merger

The last pattern for event-driven or workflow batch systems is a **merger**. A merger is the opposite of a copier; the job of a merger is to take two different work queues and turn them into a single work queue. Suppose, for example, that you have a large number of different source repositories all adding new commits at the same time. You want to take each of these commits and perform a build-and-test for it. It is not scalable to create a separate build infrastructure for each source repository. We can model each of the different source repositories as a separate work queue source that provides a set

of commits. We can transform all of these different work queue inputs into a single merged set of inputs using a merger adapter. This merged stream of commits is then the single source to the build system that performs the actual build. The merger is another great example of the adapter pattern, though in this case, the adapter is actually adapting multiple running source containers into a single merged source. This multi-adapter pattern is shown in Figure 11-7 .

Mẫu cuối cùng cho các hệ thống theo lô hướng sự kiện hoặc luồng công việc là merger (bộ gộp). Một merger là đối lập của một copier; nhiệm vụ của một merger là lấy hai hàng đợi công việc khác nhau và biến chúng thành một hàng đợi công việc duy nhất. Giả sử, ví dụ, bạn có một lượng lớn các kho mã nguồn (source repository) khác nhau đồng loạt thêm các commit mới cùng một lúc. Bạn muốn lấy mỗi commit này và thực hiện một quá trình build-and-test (dựng và kiểm thử) cho nó. Sẽ không khả mở (scalable) nếu tạo một hạ tầng build riêng cho mỗi kho mã nguồn. Chúng ta có thể mô hình hóa mỗi kho mã nguồn khác nhau như một nguồn hàng đợi công việc riêng biệt cung cấp một tập hợp các commit. Chúng ta có thể biến đổi tất cả những đầu vào hàng đợi công việc khác nhau này thành một tập hợp đầu vào đã gộp duy nhất bằng cách dùng một adapter merger. Luồng commit đã gộp này sau đó là nguồn duy nhất cho hệ thống build thực hiện quá trình build thực tế. Merger là một ví dụ tuyệt vời nữa về mẫu adapter, mặc dù trong trường hợp này, adapter thực ra đang chuyển đổi nhiều container nguồn đang chạy thành một nguồn đã gộp duy nhất. Mẫu đa adapter này được trình bày trong Hình 11-7.

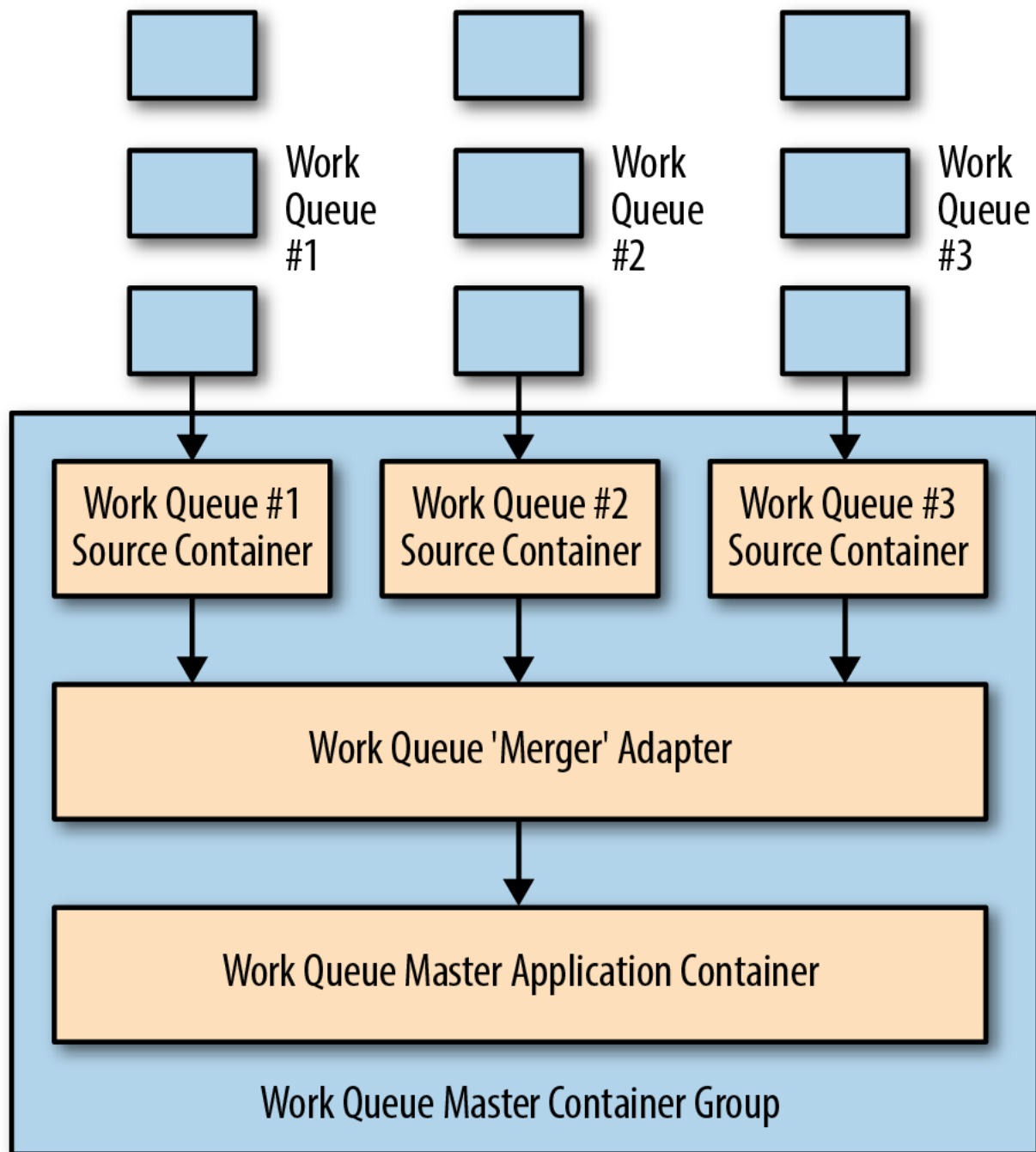


Figure 11-7. Using multiple levels of containers to adapt multiple independent work queues into a single shared queue

Hình 11-7. Sử dụng nhiều tầng container để chuyển đổi nhiều hàng đợi công việc độc lập thành một hàng đợi dùng chung duy nhất

Hands On: Building an Event-Driven Flow for New User Sign-Up — Thực hành (Hands On): Xây Dựng Một Luồng Hướng Sự Kiện Cho Việc Đăng Ký Người Dùng Mới

A concrete example of a workflow helps show how these patterns can be put together to form a complete operating system. The problem this example will consider is a new-user signup flow.

Một ví dụ cụ thể về một luồng công việc giúp cho thấy các mẫu này có thể được ghép lại với nhau như thế nào để tạo thành một hệ thống vận hành hoàn chỉnh. Bài toán mà ví dụ này sẽ xem xét là một luồng đăng ký người dùng mới.

Imagine that our user acquisition funnel has two stages. The first is user verification. After a new user signs up, the user then has to receive an email notification to validate their email. Once the user validates their email, they are sent a confirmation email. Then they are optionally registered for email, text message, both, or neither for notifications.

Hãy hình dung rằng phễu thu hút người dùng (user acquisition funnel) của chúng ta có hai giai đoạn. Giai đoạn đầu tiên là xác minh người dùng. Sau khi một người dùng mới đăng ký, người dùng đó phải nhận được một thông báo email để xác thực email của họ. Một khi người dùng xác thực email của họ, họ được gửi một email xác nhận. Sau đó họ được tùy chọn đăng ký nhận thông báo qua email, tin nhắn văn bản, cả hai, hoặc không gì cả.

The first step in the event-driven workflow is the generation of the verification email. To achieve this reliably, we will use the shard pattern to shard users across multiple different geographic failure zones. This ensures that we will continue to process new user signups, even in the presence of partial failures. Each work queue shard sends a verification email to the end user. At this point, this substage of the workflow is complete. This first stage of the flow is illustrated in Figure 11-8 .

Bước đầu tiên trong luồng công việc hướng sự kiện là việc tạo email xác minh. Để đạt được điều này một cách tin cậy, chúng ta sẽ dùng mẫu shard để phân mảnh người dùng trên nhiều vùng sự cố địa lý (geographic failure zone) khác nhau. Điều này đảm bảo rằng chúng ta sẽ tiếp tục xử lý việc đăng ký người dùng mới, ngay cả khi có những sự cố bộ phận. Mỗi mảnh hàng đợi công việc gửi một email xác minh tới người dùng cuối. Tại điểm này, giai đoạn con này của luồng công việc đã hoàn tất. Giai đoạn đầu tiên này của luồng được minh họa trong Hình 11-8.

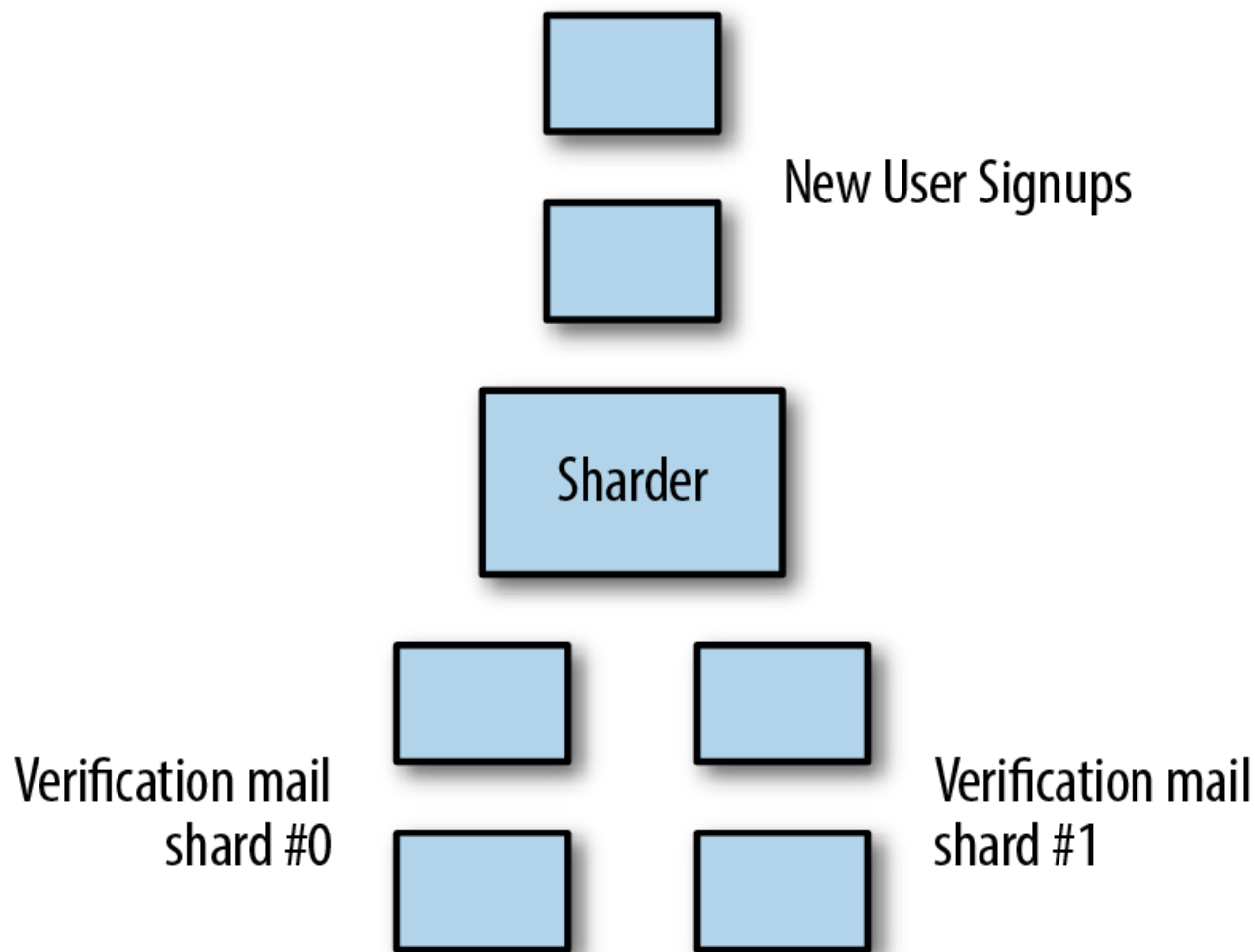


Figure 11-8. The first stage of the workflow for user sign-up

Hình 11-8. Giai đoạn đầu tiên của luồng công việc cho việc đăng ký người dùng

The workflow begins again when we receive a verification email from the end user. These emails become new events in a separate (but clearly related) workflow that sends welcome emails and sets up notifications. The first stage of this workflow is an example of the copier pattern, where the user is copied into two work queues. The first work queue is responsible for sending the welcome email, and the second work queue is responsible for setting up user notifications.

Luồng công việc bắt đầu lại khi chúng ta nhận được một email xác minh từ người dùng cuối. Những email này trở thành các sự kiện mới trong một luồng công việc riêng biệt (nhưng rõ ràng có liên hệ) chuyên gửi email chào mừng và thiết lập thông báo. Giai đoạn đầu tiên của luồng công việc này là một ví dụ về mẫu copier, nơi người dùng được sao chép vào hai hàng đợi công việc. Hàng đợi công việc thứ nhất chịu trách nhiệm gửi email chào mừng, và hàng đợi công việc thứ hai chịu trách nhiệm thiết lập thông báo cho người dùng.

Once the work items have been duplicated between the queues, the email-sending queue simply takes care of sending an email message, and the workflow exits. But remember that because of the use of the copier pattern, there is still an additional copy of the event active in our workflow. This copier triggers an additional work queue to handle notification settings. This work queue feeds into

an example of the filter pattern, which splits the work queue into separate email and text message notification queues. These specific queues register the user for email, text, or both notifications.

Một khi các mục công việc đã được nhân đôi giữa các hàng đợi, hàng đợi gửi email chỉ đơn giản lo việc gửi một thông điệp email, và luồng công việc thoát ra. Nhưng hãy nhớ rằng vì việc dùng mẫu copier, vẫn còn một bản sao bổ sung của sự kiện đang hoạt động trong luồng công việc của chúng ta. Copier này kích hoạt một hàng đợi công việc bổ sung để xử lý các thiết lập thông báo. Hàng đợi công việc này đưa vào một ví dụ về mẫu filter, vốn tách hàng đợi công việc thành các hàng đợi thông báo email và tin nhắn văn bản riêng biệt. Những hàng đợi cụ thể này đăng ký cho người dùng nhận thông báo qua email, tin nhắn văn bản, hoặc cả hai.

The remainder of this workflow is shown in Figure 11-9 .

Phần còn lại của luồng công việc này được trình bày trong Hình 11-9.

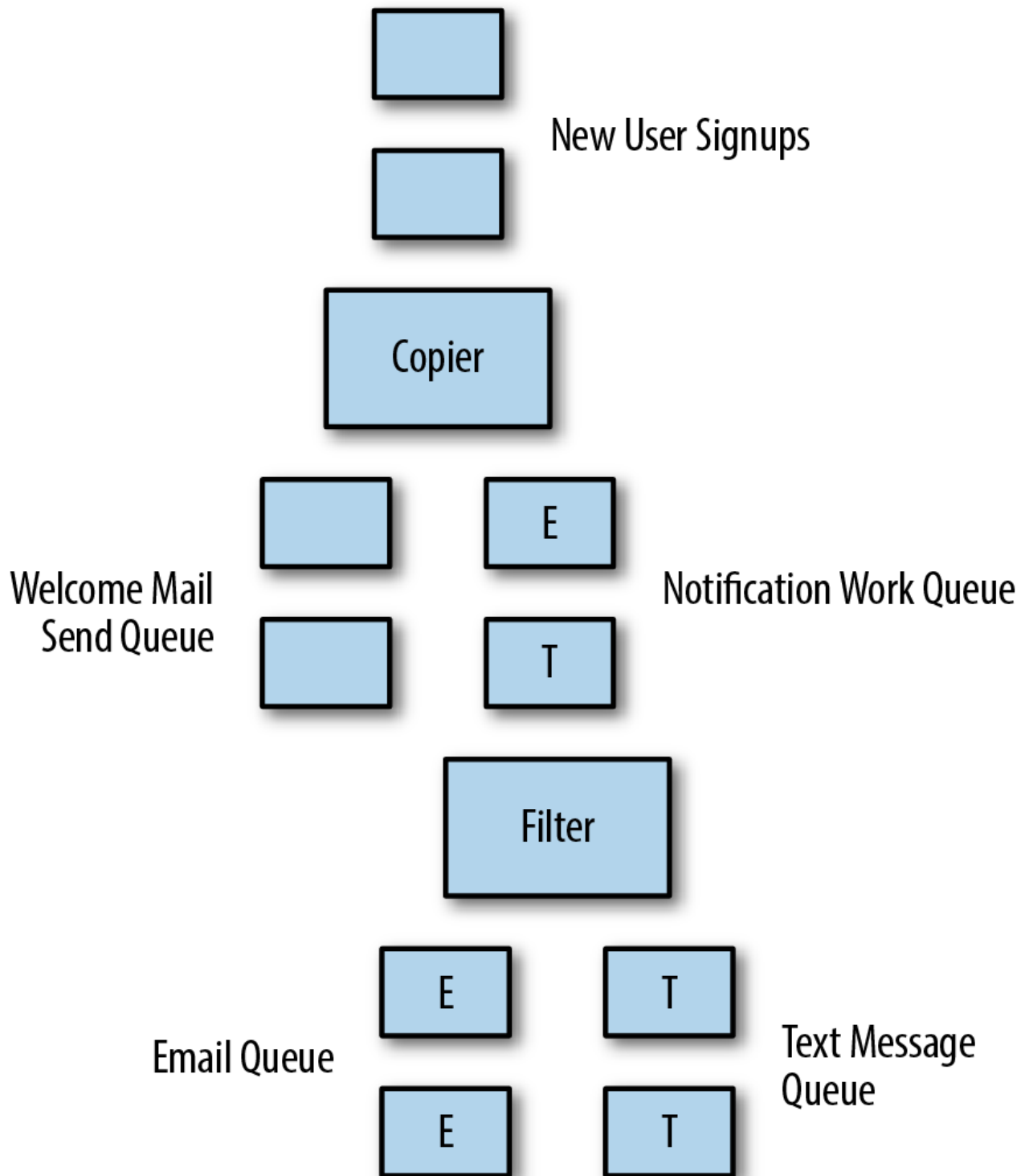


Figure 11-9. The user notification and welcome email work queue

Hình 11-9. Hàng đợi công việc gửi thông báo cho người dùng và email chào mừng

Publisher/Subscriber Infrastructure — Hạ Tầng Publisher/Subscriber

We have seen a variety of abstract patterns for linking together different event-driven batch processing patterns. But when it comes time to actually build such a system, we need to figure out how to manage the stream of data that passes through the event-driven workflow. The simplest thing to do would be to simply write each element in the work queue to a particular directory on a local filesystem, and then have each stage monitor that directory for input. But of course doing this with a local filesystem limits our workflow to operating on a single **node**. We can introduce a network filesystem to distribute files to multiple nodes, but this introduces increasing complexity both in our code and in the **deployment** of the batch workflow.

Chúng ta đã thấy nhiều mẫu trừu tượng để liên kết các mẫu xử lý theo lô hướng sự kiện khác nhau lại với nhau. Nhưng khi đến lúc thực sự xây dựng một hệ thống như vậy, chúng ta cần tìm ra cách quản lý dòng dữ liệu đi qua luồng công việc hướng sự kiện. Cách đơn giản nhất có thể làm là chỉ đơn thuần ghi mỗi phần tử trong hàng đợi công việc vào một thư mục cụ thể trên hệ thống tệp cục bộ, rồi để mỗi giai đoạn theo dõi thư mục đó để lấy đầu vào. Nhưng tất nhiên việc làm điều này với một hệ thống tệp cục bộ giới hạn luồng công việc của chúng ta hoạt động trên một nút duy nhất. Chúng ta có thể đưa vào một hệ thống tệp mạng để phân phối các tệp tới nhiều nút, nhưng điều này lại làm tăng độ phức tạp cả trong mã của chúng ta lẫn trong việc triển khai luồng công việc theo lô.

Instead, a popular approach to building a workflow like this is to use a publisher/ subscriber (pub/sub) **API** or service. A pub/sub API allows a user to define a collection of queues (sometimes called topics). One or more publishers publishes messages to these queues. Likewise, one or more subscribers is listening to these queues for new messages. When a message is published, it is reliably stored by the queue and subsequently delivered to subscribers in a reliable manner.

Thay vào đó, một cách tiếp cận phổ biến để xây dựng một luồng công việc như thế này là dùng một API hoặc dịch vụ publisher/subscriber (pub/sub). Một API pub/sub cho phép người dùng định nghĩa một tập hợp các hàng đợi (đôi khi được gọi là topic). Một hoặc nhiều publisher xuất bản (publish) các thông điệp tới những hàng đợi này. Tương tự, một hoặc nhiều subscriber lắng nghe những hàng đợi này để chờ các thông điệp mới. Khi một thông điệp được xuất bản, nó được hàng đợi lưu trữ một cách tin cậy và sau đó được chuyển tới các subscriber một cách tin cậy.

At this point, most public clouds feature a pub/sub API such as Azure's EventGrid or Amazon's Simple Queue Service. Additionally, the **open source** Kafka project provides a very popular pub/sub implementation that you can run on your own hardware as well as on cloud virtual machines. For the remainder of this overview of pub/sub APIs we'll use Kafka for our examples, but they are relatively simple to port to alternate pub/sub APIs.

Tại thời điểm này, hầu hết các đám mây công cộng (public cloud) đều có một API pub/sub như EventGrid của Azure hoặc Simple Queue Service của Amazon. Ngoài ra, dự án mã nguồn mở Kafka cung cấp một bản hiện thực pub/sub rất phổ biến mà bạn có thể chạy trên phần cứng của riêng mình cũng như trên các máy ảo đám mây. Trong phần còn lại của tổng quan về các API pub/sub này, chúng ta sẽ dùng Kafka cho các ví dụ, nhưng chúng tương đối đơn giản để chuyển đổi (port) sang các API pub/sub khác.

Hands On: Deploying Kafka — Thực hành (Hands On): Triển Khai Kafka

There are obviously many ways to deploy Kafka, and one of the easiest ways is to run it as a container using a **Kubernetes** cluster and the Helm package manager.

Hiển nhiên có nhiều cách để triển khai Kafka, và một trong những cách dễ nhất là chạy nó như một container bằng cách dùng một cụm Kubernetes và trình quản lý gói Helm.

Helm is a package manager for Kubernetes that makes it easy to deploy and manage prepackaged, off-the-shelf applications like Kafka. If you don't already have the helm command line tool installed, you can install it from <https://helm.sh>.

Helm là một trình quản lý gói cho Kubernetes giúp dễ dàng triển khai và quản lý các ứng dụng đóng gói sẵn, có sẵn (off-the-shelf) như Kafka. Nếu bạn chưa có công cụ dòng lệnh helm được cài đặt, bạn có thể cài nó từ <https://helm.sh>.

Once the helm tool is on your machine, you need to initialize it. Initializing Helm deploys a cluster-side component named tiller to your cluster and installs some templates to your local filesystem:

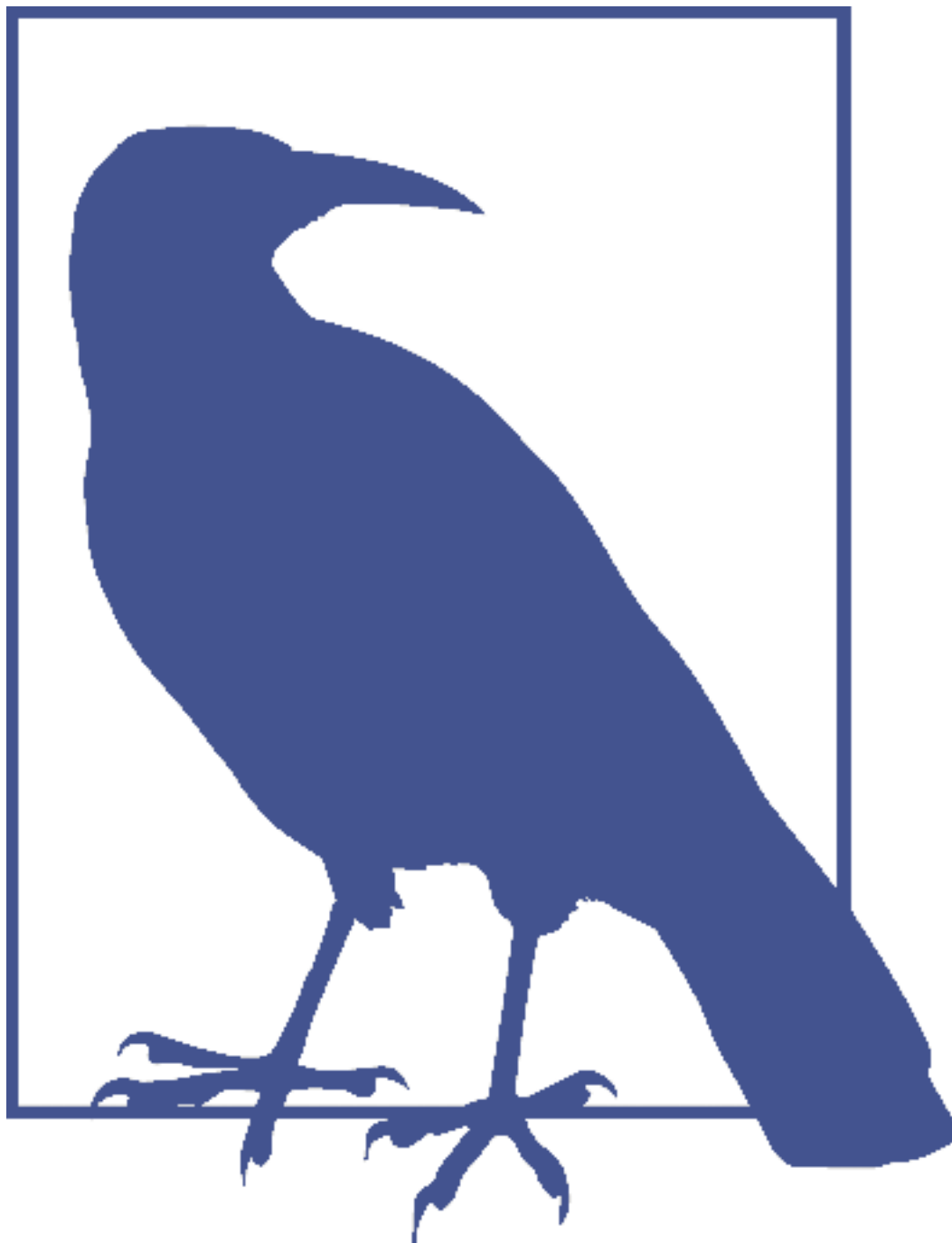
Một khi công cụ helm đã có trên máy của bạn, bạn cần khởi tạo nó. Việc khởi tạo Helm triển khai một thành phần phía cụm tên là tiller tới cụm của bạn và cài đặt một số mẫu (template) vào hệ thống tệp cục bộ của bạn:

```
helm init
```

Now that helm is initialized, you can install Kafka using this command:

Bây giờ khi helm đã được khởi tạo, bạn có thể cài đặt Kafka bằng lệnh này:

```
helm repo add incubator http://storage.googleapis.com/kubernetes-charts-incubator
helm install --name kafka-service incubator/kafka
```



Helm templates have different levels of production hardening and support. stable templates are the most strictly vetted and supported, whereas incubator templates like Kafka are more experimental and have less production mileage. Regardless, incubator templates are useful for quick proof of concepts as well as a place to start from when implementing a production deployment of a Kubernetes-based service.

Các mẫu Helm có nhiều mức độ tối luyện cho môi trường sản xuất (production hardening) và hỗ trợ khác nhau. Các mẫu stable được kiểm duyệt và hỗ trợ nghiêm ngặt nhất, trong khi các mẫu incubator như Kafka có tính thử nghiệm hơn và có ít quãng đường chạy thực tế trong môi trường sản xuất hơn. Dù vậy, các mẫu incubator vẫn hữu ích cho các bằng

chứng khái niệm (proof of concept) nhanh cũng như là điểm khởi đầu khi triển khai một bản triển khai sản xuất của một dịch vụ dựa trên Kubernetes.

Once you have Kafka up and running, you can create a topic to publish to. Generally in batch processing, you're going to use a topic to represent the output of one module in your workflow. This output is likely to be the input for another module in the workflow.

Một khi bạn đã có Kafka chạy lên, bạn có thể tạo một topic để xuất bản tới. Nhìn chung trong xử lý theo lô, bạn sẽ dùng một topic để biểu diễn đầu ra của một module trong luồng công việc của mình. Đầu ra này có khả năng sẽ là đầu vào cho một module khác trong luồng công việc.

For example, if you are using the Sharder pattern described previously, you would have a topic for each of the output shards. If you called your output Photos and you chose to have three shards, then you would have three topics: Photos-1, Photos-2, and Photos-3. Your sharder module would output messages to the appropriate topic, after applying the sharding **function**.

Ví dụ, nếu bạn đang dùng mẫu Sharder đã mô tả trước đó, bạn sẽ có một topic cho mỗi mảnh đầu ra. Nếu bạn gọi đầu ra của mình là Photos và bạn chọn có ba mảnh, thì bạn sẽ có ba topic: Photos-1, Photos-2, và Photos-3. Module sharder của bạn sẽ xuất các thông điệp tới đúng topic, sau khi áp dụng hàm phân mảnh.

Here's how you create a topic. First, create a container in the cluster so that we can access Kafka:

Đây là cách bạn tạo một topic. Trước tiên, hãy tạo một container trong cụm để chúng ta có thể truy cập Kafka:

```
for x in 0 1 2; do
  kubectl run kafka --image=solsson/kafka:0.11.0.0 --rm --attach --command -- \
    ./bin/kafka-topics.sh --create --zookeeper kafka-service-zookeeper:2181 \
    --replication-factor 3 --partitions 10 --topic photos-$x
done
```

Note that there are two interesting parameters in addition to the topic name and the zookeeper service. They are `--replication-factor` and `--partitions`. The replication factor is how many different machines messages in the topic will be replicated to. This is the redundancy that is available in case things crash. A value of 3 or 5 is recommended. The second parameter is the number of partitions for the topic. The number of partitions represents the maximum distribution of the topic onto multiple machines for purposes of **load balancing**. In this case, since there are 10 partitions, there can be at most 10 different replicas of the topic for load balancing.

Lưu ý rằng có hai tham số thú vị ngoài tên topic và dịch vụ zookeeper. Đó là `--replication-factor` và `--partitions`. Hệ số nhân bản (replication factor) là số lượng máy khác nhau mà các thông điệp trong topic sẽ được nhân bản tới. Đây là phần dự thừa sẵn sàng phòng khi mọi thứ bị sập. Khuyến nghị một giá trị là 3 hoặc 5. Tham số thứ hai là số lượng phân vùng (partition) cho topic. Số lượng phân vùng biểu diễn mức phân bố tối đa của topic lên nhiều máy nhằm mục đích cân bằng tải. Trong trường hợp này, vì có 10 phân vùng, có thể có tối đa 10 bản sao khác nhau của topic để cân bằng tải.

Now that we have created a topic, we can send messages to that topic:

Bây giờ khi chúng ta đã tạo một topic, chúng ta có thể gửi các thông điệp tới topic đó:

```
kubectl run kafka-producer --image=solsson/kafka:0.11.0.0 --rm -it --command -- \
  ./bin/kafka-console-producer.sh --broker-list kafka-service-kafka:9092 \
  --topic photos-1
```

Once that command is up and connected, you should see the Kafka prompt and you can then send messages to the topic(s). To receive messages, you can run:

Một khi lệnh đó chạy lên và đã kết nối, bạn sẽ thấy dấu nhắc (prompt) Kafka và sau đó bạn có thể gửi các thông điệp tới (các) topic. Để nhận các thông điệp, bạn có thể chạy:

```
kubect1 run kafka-consumer --image=solsson/kafka:0.11.0.0 --rm -it --command -- \
  ./bin/kafka-console-consumer.sh --bootstrap-server kafka-service-kafka:9092\
  --topic photos-1 \
  --from-beginning
```

Of course, running these command lines only gives you a taste of how to communicate via Kafka messages. To build a real-world event-driven batch processing system, you would likely use a proper programming language and Kafka SDK to access the service. But on the other hand, never underestimate the power of a good Bash script!

Tất nhiên, việc chạy các dòng lệnh này chỉ cho bạn nếm thử một chút về cách giao tiếp qua các thông điệp Kafka. Để xây dựng một hệ thống xử lý theo lô hướng sự kiện thực tế, bạn có khả năng sẽ dùng một ngôn ngữ lập trình đúng nghĩa và SDK của Kafka để truy cập dịch vụ. Nhưng mặt khác, đừng bao giờ đánh giá thấp sức mạnh của một script Bash tốt!

This section has shown how installing Kafka into your Kubernetes cluster can dramatically simplify the task of building a work queue based system.

Phần này đã cho thấy việc cài đặt Kafka vào cụm Kubernetes của bạn có thể đơn giản hóa đáng kể nhiệm vụ xây dựng một hệ thống dựa trên hàng đợi công việc như thế nào.

CHAPTER 12 Coordinated Batch Processing — CHAPTER 12 Xử lý theo lô có điều phối

The previous chapter described a number of patterns for splitting and chaining queues together to achieve more complex **batch processing**. Duplicating and producing multiple different outputs is often an important part of batch processing, but sometimes it is equally important to pull multiple outputs back together in order to generate some sort of aggregate output. A generic illustration of such a pattern is shown in Figure 12-1 .

Chương trước đã mô tả một số mẫu để tách và nối các hàng đợi (queue) lại với nhau nhằm đạt được khả năng xử lý theo lô (batch) phức tạp hơn. Việc nhân bản và tạo ra nhiều đầu ra khác nhau thường là một phần quan trọng của xử lý theo lô, nhưng đôi khi việc kéo nhiều đầu ra trở lại với nhau để tạo ra một dạng đầu ra tổng hợp nào đó cũng quan trọng không kém. Một minh họa tổng quát cho mẫu như vậy được trình bày trong Hình 12-1.

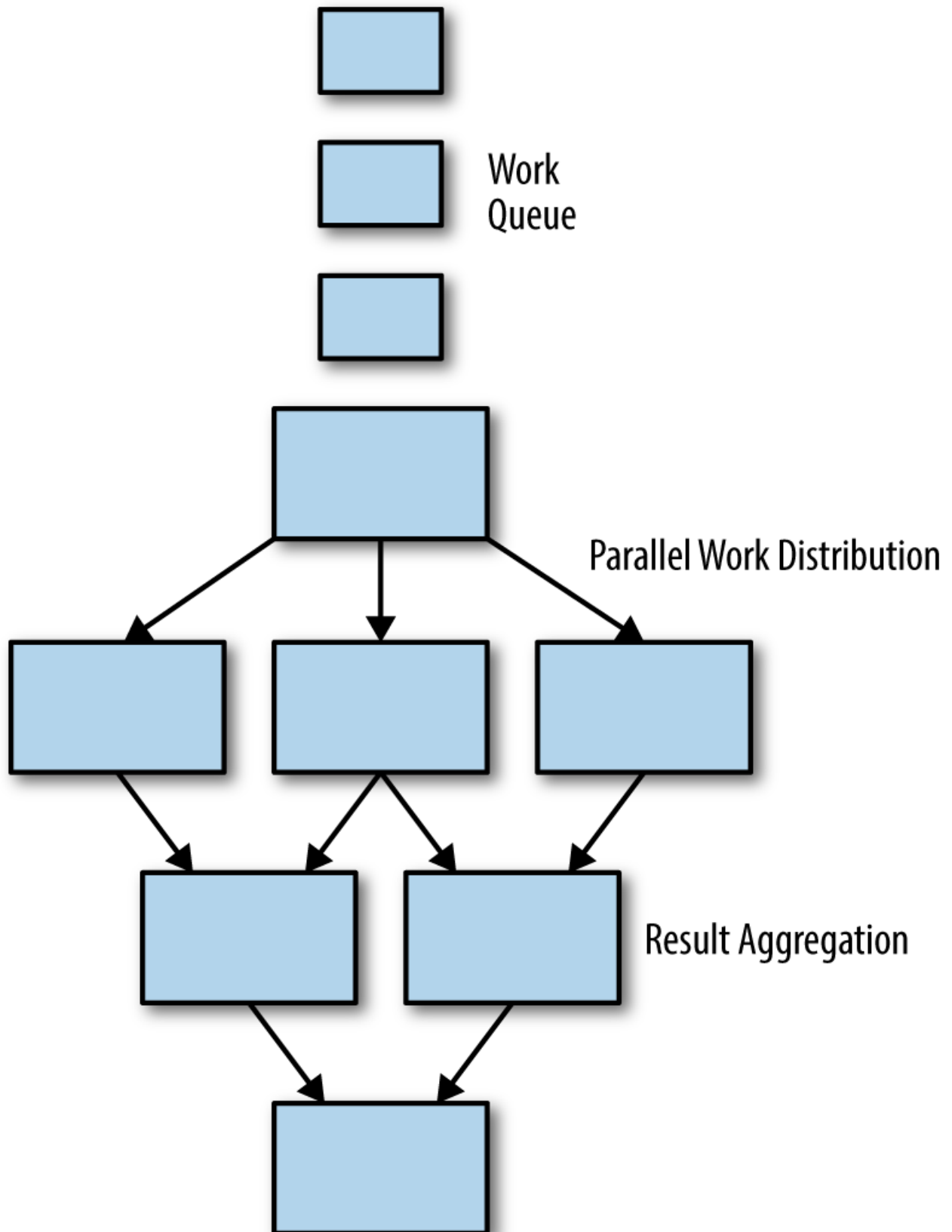


Figure 12-1. A generic parallel work distribution and result aggregation batch system

Hình 12-1. Một hệ thống theo lô tổng quát phân phối công việc song song và tổng hợp kết quả

133 Probably the most canonical example of this aggregation is the **reduce** part of the **MapReduce** pattern. It's easy to see that the **map** step is an example of sharding a **work queue**, and the reduce step is an example of coordinated processing that eventually reduces a large number of outputs down to a single aggregate response. However, there are a number of different aggregate patterns for batch processing, and this chapter discusses a number of them in addition to real-world applications.

133 Có lẽ ví dụ kinh điển nhất về phép tổng hợp này là phần reduce của mẫu MapReduce. Dễ thấy rằng bước map là một ví dụ về việc phân mảnh (shard) một hàng đợi công việc (work queue), còn bước reduce là một ví dụ về xử lý có điều phối, dần dần rút gọn một số lượng lớn đầu ra xuống thành một phản hồi tổng hợp duy nhất. Tuy nhiên, có một số mẫu tổng hợp khác nhau cho xử lý theo lô, và chương này sẽ bàn về một số mẫu trong đó bên cạnh các ứng dụng thực tế.

Join (or Barrier Synchronization) — Join (hay Đồng bộ hóa rào chắn - Barrier Synchronization)

In previous chapters, we saw patterns for breaking up work and distributing it in parallel on multiple nodes. In particular, we saw how a sharded work queue could distribute work in parallel to a number of different work queue shards. However, sometimes when processing a workflow, it is necessary to have the complete set of work available to you before you move on to the next stage of the workflow.

Trong các chương trước, chúng ta đã thấy các mẫu để chia nhỏ công việc và phân phối chúng song song trên nhiều nút (node). Cụ thể, ta đã thấy một hàng đợi công việc được phân mảnh (sharded work queue) có thể phân phối công việc song song đến nhiều mảnh hàng đợi công việc khác nhau như thế nào. Tuy nhiên, đôi khi khi xử lý một quy trình làm việc (workflow), ta cần có toàn bộ tập công việc sẵn sàng trước khi chuyển sang giai đoạn tiếp theo của quy trình.

One option for doing this was shown in the previous chapter, which was to merge multiple queues together. However, merge simply blends the output of two work queues into a single work queue for additional processing. While the merge pattern is sufficient in some cases, it does not ensure that a complete dataset is present prior to the beginning of processing. This means that there can be no guarantees about the completeness of the processing being performed, as well as no opportunity to compute aggregate statistics for all of the elements that have been processed.

Một lựa chọn để làm điều này đã được trình bày trong chương trước, đó là gộp nhiều hàng đợi lại với nhau. Tuy nhiên, phép gộp (merge) chỉ đơn thuần trộn đầu ra của hai hàng đợi công việc thành một hàng đợi công việc duy nhất để xử lý tiếp. Mặc dù mẫu gộp là đủ trong một số trường hợp, nó không đảm bảo rằng một tập dữ liệu hoàn chỉnh đã hiện diện trước khi bắt đầu xử lý. Điều này có nghĩa là không thể có bất kỳ đảm bảo nào về tính đầy đủ của quá trình xử lý đang được thực hiện, cũng như không có cơ hội để tính toán các thống kê tổng hợp cho tất cả các phần tử đã được xử lý.

Instead, we need a stronger, coordinated primitive for batch data processing, and that primitive is the **join** pattern. Join is similar to joining a thread. The basic idea is that all of the work is happening in parallel, but work items aren't released out of the join until all of the work items that are processed in parallel are completed. This is also generally known as barrier synchronization in concurrent programming. An illustration of the join pattern for a coordinated batch is shown in Figure 12-2 .

Thay vào đó, chúng ta cần một nguyên thủy (primitive) mạnh hơn, có điều phối cho xử lý dữ liệu theo lô, và nguyên thủy đó là mẫu join. Join tương tự như việc join một thread. Ý tưởng cơ bản là tất cả công việc đều diễn ra song song, nhưng các mục công việc không được giải phóng khỏi join cho đến khi tất cả các mục công việc được xử lý song song đã hoàn thành. Điều này thường được biết đến với tên đồng bộ hóa rào chắn (barrier synchronization) trong lập trình đồng thời. Một minh họa cho mẫu join cho một lô có điều phối được trình bày trong Hình 12-2.

Coordination through join ensures that no data is missing before some sort of aggregation phase is performed (e.g., finding the sum of some value in a set). The value of the join is that it ensures that all of the data in the set is present. The downside of the join pattern is that it requires that all data be processed by a previous stage before subsequent computation can begin. This reduces the parallelism that is possible in the batch workflow, and thus increases the overall **latency** of running the workflow.

Sự điều phối thông qua join đảm bảo rằng không có dữ liệu nào bị thiếu trước khi một dạng pha tổng hợp nào đó được thực hiện (ví dụ: tìm tổng của một giá trị nào đó trong một tập). Giá trị của join là nó đảm bảo rằng tất cả dữ liệu trong tập đều hiện diện. Nhược điểm của mẫu join là nó đòi hỏi tất cả dữ liệu phải được giai đoạn trước xử lý xong trước khi việc tính toán tiếp theo có thể bắt đầu. Điều này làm giảm mức độ song song có thể đạt được trong quy trình theo lô, và do đó làm tăng độ trễ tổng thể của việc chạy quy trình.

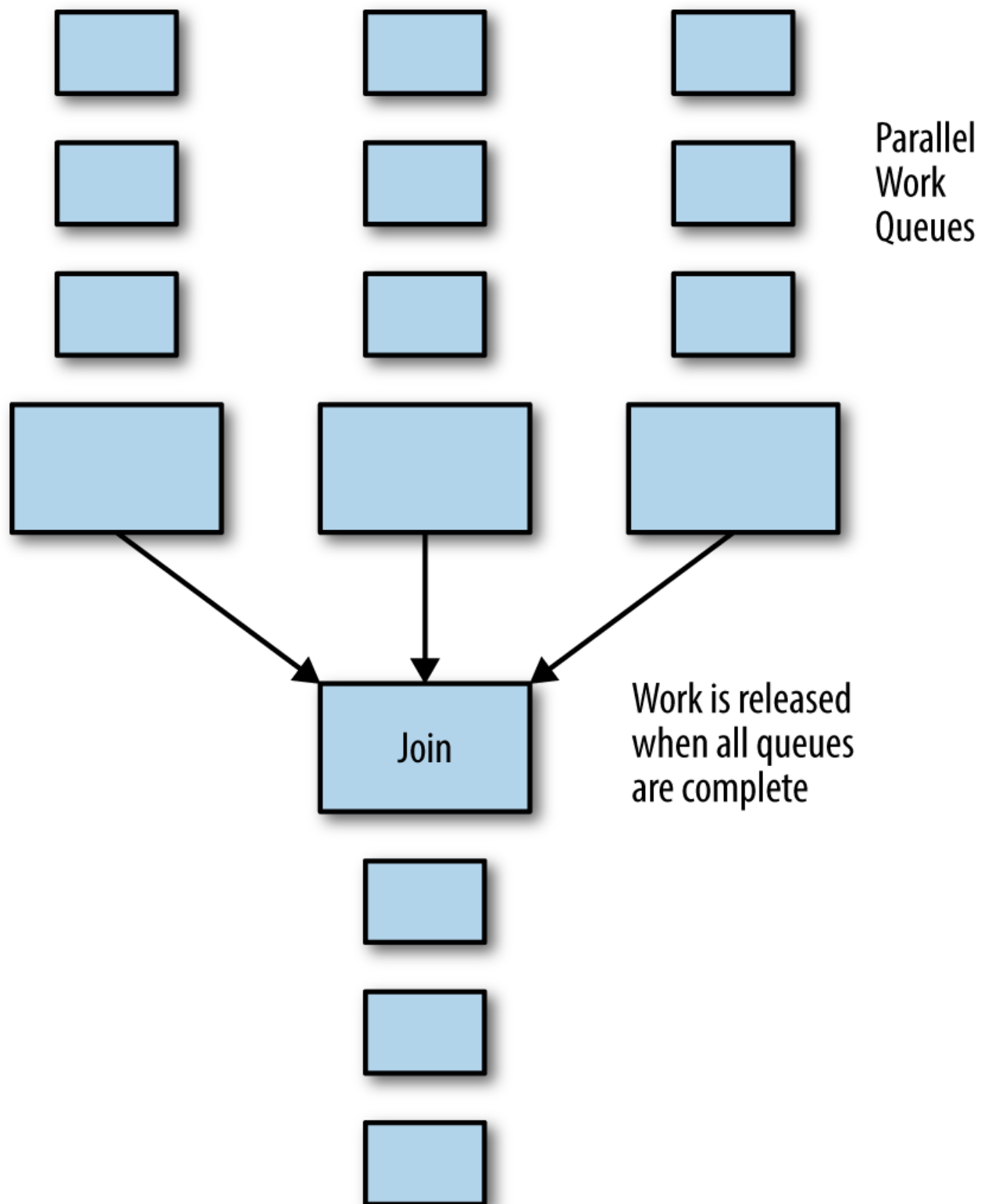


Figure 12-2. The join pattern for batch processing

Hình 12-2. Mẫu join cho xử lý theo lô

Reduce — Reduce

If sharding a work queue is an example of the map phase of the canonical map/reduce algorithm, then what remains is the reduce phase. Reduce is an example of a coordinated batch processing pattern because it can happen regardless of how the input is split up, and it is used similar to join ; that is, to group together the parallel output of a number of different batch operations on different pieces of data.

Nếu việc phân mảnh một hàng đợi công việc là một ví dụ về pha map của thuật toán map/reduce kinh điển, thì phần còn lại là pha reduce. Reduce là một ví dụ về mẫu xử lý theo lô có điều phối vì nó có thể diễn ra bất kể đầu vào được chia nhỏ như thế nào, và nó được sử dụng tương tự như join; tức là để gom lại với nhau đầu ra song song của một số thao tác theo lô khác nhau trên các phần dữ liệu khác nhau.

However, in contrast to the join pattern described previously, the goal of reduce is not to wait until all data has been processed, but rather to optimistically merge together all of the parallel data items into a single comprehensive representation of the full set.

Tuy nhiên, khác với mẫu join đã mô tả ở trên, mục tiêu của reduce không phải là chờ cho đến khi tất cả dữ liệu đã được xử lý, mà là gộp lại quan tất cả các mục dữ liệu song song lại thành một biểu diễn toàn diện duy nhất của toàn bộ tập.

With the reduce pattern, each step in the reduce merges several different outputs into a single output. This stage is called “reduce” because it reduces the total number of outputs. Additionally, it reduces the data from a complete data item to simply the representative data necessary for producing the answer to a specific batch computation. Because the reduce phase operates on a range of input, and produces a similar output, the reduce phase can be repeated as many or as few times as necessary in order to successfully reduce the output down to a single output for the entire data set.

Với mẫu reduce, mỗi bước trong reduce gộp một vài đầu ra khác nhau thành một đầu ra duy nhất. Giai đoạn này được gọi là “reduce” (rút gọn) vì nó rút gọn tổng số đầu ra. Ngoài ra, nó rút gọn dữ liệu từ một mục dữ liệu hoàn chỉnh xuống chỉ còn dữ liệu đại diện cần thiết để tạo ra câu trả lời cho một phép tính theo lô cụ thể. Vì pha reduce hoạt động trên một dải đầu vào và tạo ra một đầu ra tương tự, pha reduce có thể được lặp lại nhiều hay ít lần tùy theo nhu cầu để rút gọn thành công đầu ra xuống còn một đầu ra duy nhất cho toàn bộ tập dữ liệu.

This is a fortunate contrast to the join pattern, because unlike join, it means that reduce can be started in parallel while there is still processing going on as part of the map/**shard** phase. Of course, in order to produce a complete output, all of the data must be processed eventually, but the ability to begin early means that the batch computation executes more quickly overall.

Đây là một điểm tương phản may mắn với mẫu join, bởi vì khác với join, nó có nghĩa là reduce có thể được khởi động song song trong khi vẫn còn quá trình xử lý đang diễn ra như một phần của pha map/shard. Tất nhiên, để tạo ra một đầu ra hoàn chỉnh, cuối cùng tất cả dữ liệu đều phải được xử lý, nhưng khả năng bắt đầu sớm có nghĩa là phép tính theo lô tổng thể được thực thi nhanh hơn.

Hands On: Count — Thực hành (Hands On): Đếm

To understand how the reduce pattern works, consider the task of counting the number of instances of a particular word in a book. We can first use sharding to divide up the work of counting words into a number of different work queues. As an example, we could create 10 different sharded work

queues with 10 different people responsible for counting words in each queue. We can shard the book among these 10 work queues by looking at the page number. All pages that end in the number 1 will go to the first queue, all pages that end in the number 2 will go to the second, and so forth.

Để hiểu cách mẫu reduce hoạt động, hãy xem xét nhiệm vụ đếm số lần xuất hiện của một từ cụ thể trong một cuốn sách. Trước tiên ta có thể dùng phân mảnh (sharding) để chia nhỏ công việc đếm từ thành một số hàng đợi công việc khác nhau. Ví dụ, ta có thể tạo 10 hàng đợi công việc được phân mảnh khác nhau với 10 người khác nhau chịu trách nhiệm đếm từ trong mỗi hàng đợi. Ta có thể phân mảnh cuốn sách giữa 10 hàng đợi công việc này bằng cách nhìn vào số trang. Tất cả các trang có số tận cùng là 1 sẽ đi vào hàng đợi thứ nhất, tất cả các trang có số tận cùng là 2 sẽ đi vào hàng đợi thứ hai, và cứ thế.

Once all of the people have finished processing their pages, they write down their results on a piece of paper. For example, they might write:

Khi tất cả mọi người đã xử lý xong các trang của mình, họ viết kết quả ra một tờ giấy. Ví dụ, họ có thể viết:

```
a: 50
the: 17
cat: 2
airplane: 1
...
```

This can be output to the reduce phase. Remember that the reduce pattern reduces by combining two or more outputs into a single output.

Cái này có thể được xuất ra cho pha reduce. Hãy nhớ rằng mẫu reduce rút gọn bằng cách kết hợp hai hay nhiều đầu ra thành một đầu ra duy nhất.

Given a second output:

Cho một đầu ra thứ hai:

```
a: 30
the: 25
dog: 4
airplane: 2
...
```

The reduction proceeds by summing up all of the counts for the various words, in this example producing:

Quá trình rút gọn tiến hành bằng cách cộng tất cả các số đếm cho các từ khác nhau, trong ví dụ này tạo ra:

```
a: 80
the 42
dog: 4
cat: 2
airplane: 3
...
```

It's clear to see that this reduction phase can be repeated on the output of previous reduce phases until there is only a single reduced output left. This is valuable since this means that reductions can be performed in parallel.

Rõ ràng dễ thấy rằng pha rút gọn này có thể được lặp lại trên đầu ra của các pha reduce trước đó cho đến khi chỉ còn lại một đầu ra rút gọn duy nhất. Điều này có giá trị vì nó có nghĩa là các phép rút gọn có thể được thực hiện song song.

Ultimately, in this example you can see that the output of the reduction will be a single output with the count of all of the various words that are present in the book.

Cuối cùng, trong ví dụ này bạn có thể thấy rằng đầu ra của phép rút gọn sẽ là một đầu ra duy nhất với số đếm của tất cả các từ khác nhau hiện diện trong cuốn sách.

Sum — Sum

A similar but slightly different form of reduction is the summation of a collection of different values. This is like counting, but rather than simply counting one for every value, you actually add together a value that is present in the original output data.

Một dạng rút gọn tương tự nhưng hơi khác là phép tính tổng của một tập hợp các giá trị khác nhau. Điều này giống như đếm, nhưng thay vì chỉ đơn thuần đếm một đơn vị cho mỗi giá trị, bạn thực sự cộng dồn một giá trị có trong dữ liệu đầu ra gốc.

Suppose, for example, you want to sum the total population of the United States. Assume that you will do this by measuring the population in every town and then summing them all together.

Ví dụ, giả sử bạn muốn tính tổng dân số của Hoa Kỳ. Giả sử bạn sẽ làm điều này bằng cách đo dân số ở mỗi thị trấn rồi cộng tất cả lại với nhau.

A first step might be to shard the work into work queues of towns, sharded by state. This is a great first sharding, but it's clear that even when distributed in parallel, it would take a single person a long time to count the number of people in every town. Consequently, we perform a second sharding to another set of work queues, this time by county.

Bước đầu tiên có thể là phân mảnh công việc thành các hàng đợi công việc của các thị trấn, phân mảnh theo bang. Đây là một cách phân mảnh đầu tiên rất tốt, nhưng rõ ràng rằng ngay cả khi được phân phối song song, một người vẫn sẽ mất rất nhiều thời gian để đếm số người trong mọi thị trấn. Do đó, chúng ta thực hiện phân mảnh lần thứ hai sang một tập hàng đợi công việc khác, lần này theo quận (county).

At this point, we have parallelized first to the level of states, then to the level of counties, and then each work queue in each county produces a stream of outputs of (town, population) tuples.

Đến đây, chúng ta đã song song hóa trước tiên đến cấp độ bang, rồi đến cấp độ quận, và sau đó mỗi hàng đợi công việc trong mỗi quận tạo ra một luồng đầu ra gồm các bộ (thị trấn, dân số).

Now that we are producing output, the reduce pattern can kick in.

Giờ khi chúng ta đang tạo ra đầu ra, mẫu reduce có thể bắt đầu phát huy tác dụng.

In this case, the reduce doesn't even really need to be aware of the two-level sharding that we performed. It is sufficient for the reduce to simply grab two or more output items, such as (Seattle, 4,000,000) and (Northampton, 25,000) , and sum them together to produce a new output (Seattle-Northampton, 4,025,000) . It's clear to see that, like counting, this reduction can be performed an arbitrary number of times with the same code running at each interval, and at the end, there will only be a single output containing the complete population of the United States. Importantly, again, nearly all of the computation required is happening in parallel.

Trong trường hợp này, reduce thậm chí không thực sự cần phải biết về việc phân mảnh hai cấp mà chúng ta đã thực hiện. Chỉ cần reduce đơn thuần lấy hai hay nhiều mục đầu ra, chẳng hạn (Seattle, 4.000.000) và (Northampton, 25.000), rồi cộng chúng lại với nhau để tạo ra một đầu ra mới (Seattle-Northampton, 4.025.000) là đủ. Rõ ràng dễ thấy rằng, giống như đếm, phép rút gọn này có thể được thực hiện một số lần tùy ý với cùng một đoạn mã chạy ở mỗi lần, và đến cuối cùng sẽ chỉ còn một đầu ra duy nhất chứa toàn bộ dân số của Hoa Kỳ. Một lần nữa, điều quan trọng là gần như toàn bộ phép tính cần thiết đều đang diễn ra song song.

Histogram — Histogram

As a final example of the reduce pattern, consider that while we are counting the population of the United States via parallel sharding/mapping and reducing, we also want to build a model of the average American family. To do this, we want to develop a histogram of family size; that is, a model that estimates the total number of families with zero to 10 children. We will perform our multi-level sharding exactly as before (indeed, we can likely use the same workers).

Như một ví dụ cuối cùng về mẫu reduce, hãy xét rằng trong khi chúng ta đang đếm dân số Hoa Kỳ thông qua việc phân mảnh/map song song và reduce, chúng ta cũng muốn xây dựng một mô hình về gia đình Mỹ trung bình. Để làm điều này, chúng ta muốn phát triển một biểu đồ tần suất (histogram) về quy mô gia đình; tức là một mô hình ước lượng tổng số gia đình có từ không đến 10 con. Chúng ta sẽ thực hiện việc phân mảnh đa cấp y hệt như trước (thực vậy, ta có thể dùng chính những worker đó).

However, this time, the output of the data collection phase is a histogram per town.

Tuy nhiên, lần này, đầu ra của pha thu thập dữ liệu là một histogram cho mỗi thị trấn.

0: 15%
1: 25%
2: 50%
3: 10%
4: 5%

From the previous examples, we can see that if we apply the reduce pattern, we should be able to combine all of these histograms to develop a comprehensive picture of the United States. At first blush, it may seem quite difficult to understand how to merge these histograms, but when combined with the population data from the summation example, we can see that if we multiply each histogram by its relative population, then we can obtain the total population for each item being merged. If we then divide this new total by the sum of the merged populations, it is clear that we can merge and update multiple different histograms into a single output. Given this, we can apply the reduce pattern as many times as necessary until a single output is produced.

Từ các ví dụ trước, ta có thể thấy rằng nếu áp dụng mẫu reduce, ta sẽ có thể kết hợp tất cả các histogram này để phát triển một bức tranh toàn diện về Hoa Kỳ. Thoạt nhìn, có vẻ khá khó hiểu làm thế nào để gộp các histogram này, nhưng khi kết hợp với dữ liệu dân số từ ví dụ tính tổng, ta có thể thấy rằng nếu nhân mỗi histogram với dân số tương đối của nó, thì ta có thể thu được tổng dân số cho mỗi mục đang được gộp. Sau đó nếu ta chia tổng mới này cho tổng của các dân số đã gộp, rõ ràng ta có thể gộp và cập nhật nhiều histogram khác nhau thành một đầu ra duy nhất. Với điều này, ta có thể áp dụng mẫu reduce nhiều lần tùy theo nhu cầu cho đến khi tạo ra một đầu ra duy nhất.

Hands On: An Image Tagging and Processing Pipeline — Thực hành (Hands On): Một pipeline gắn thẻ và xử lý ảnh

To see how coordinated batch processing can be used to accomplish a larger batch task, consider the **job** of tagging and processing a set of images. Let us assume that we have a large collection of images of highways at rush hour, and we want to count both the numbers of cars, trucks, and motorcycles, as well as distribution of the colors of each of the cars. Let us also suppose that there is a preliminary step to blur the license plates of all of the cars to preserve anonymity.

Để thấy cách xử lý theo lô có điều phối có thể được dùng để hoàn thành một tác vụ theo lô lớn hơn, hãy xét công việc gắn thẻ và xử lý một tập ảnh. Giả sử rằng chúng ta có một bộ sưu tập lớn các ảnh về đường cao tốc vào giờ cao điểm, và ta muốn đếm cả số lượng ô tô, xe tải, xe máy, cũng như phân bố màu sắc của mỗi chiếc ô tô. Cũng giả sử rằng có một bước sơ bộ để làm mờ biển số của tất cả các xe nhằm bảo toàn tính ẩn danh.

The images are delivered to us as a series of HTTPS URLs where each URL points to a raw image. The first stage in the **pipeline** is to find and blur the license plates. To simplify each task in the work queue, we will have one **worker** that detects a license plate, and a second worker that blurs that location in the image. We will combine these two different worker containers into a single **container** group using the **multi-worker pattern** described in the previous chapter. This separation of concerns may seem unnecessary, but it is useful given that the workers for blurring images can be reused to blur other outputs (e.g., people's faces).

Các ảnh được chuyển đến chúng ta dưới dạng một loạt URL HTTPS, trong đó mỗi URL trỏ tới một ảnh thô. Giai đoạn đầu tiên trong pipeline là tìm và làm mờ biển số xe. Để đơn giản hóa từng tác vụ trong hàng đợi công việc, chúng ta sẽ có một worker phát hiện biển số xe, và một worker thứ hai làm mờ vị trí đó trong ảnh. Chúng ta sẽ kết hợp hai container worker khác nhau này thành một nhóm container duy nhất bằng mẫu multi-worker đã mô tả trong chương trước. Việc tách bạch mối quan tâm này thoạt nhìn có vẻ không cần thiết, nhưng nó hữu ích vì các worker dùng để làm mờ ảnh có thể được tái sử dụng để làm mờ các đầu ra khác (ví dụ: khuôn mặt người).

Additionally, to ensure **reliability** and to maximize parallel processing, we will shard the images across multiple worker queues. This complete workflow for sharded image blurring is shown in Figure 12-3.

Ngoài ra, để đảm bảo độ tin cậy và tối đa hóa xử lý song song, chúng ta sẽ phân mảnh các ảnh trên nhiều hàng đợi worker. Toàn bộ quy trình làm mờ ảnh được phân mảnh này được trình bày trong Hình 12-3.

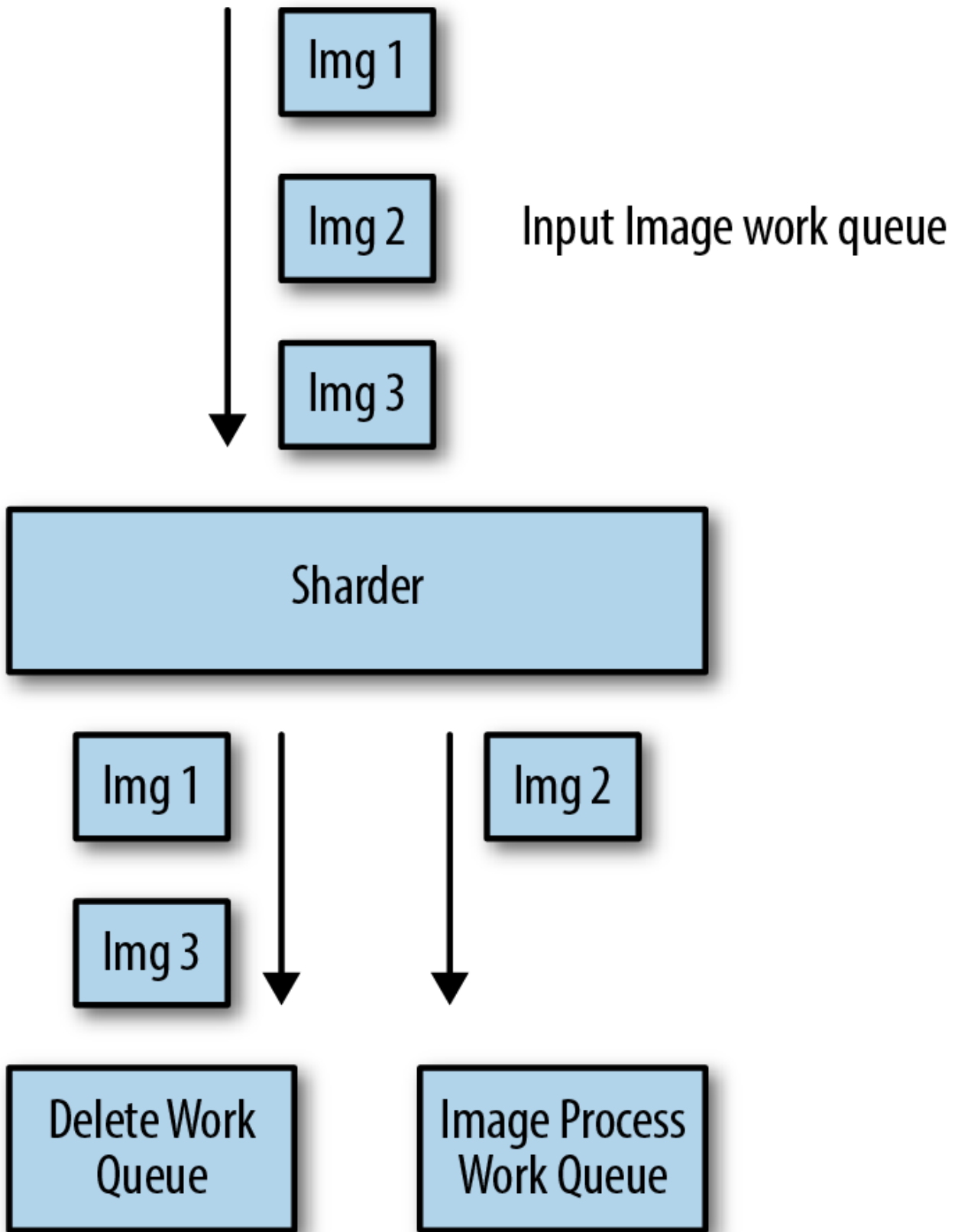


Figure 12-3. The sharded work queue and the multiple blurring shards

Hình 12-3. Hàng đợi công việc được phân mảnh và nhiều mảnh làm mờ

Once each image has been successfully blurred, we will upload it to a different location, and we will then delete the originals. However, we don't want to delete the original until all of the images have been successfully blurred in case there is some sort of catastrophic failure and we need to rerun this entire pipeline. Thus, to wait for all of the blurring to complete, we use the join pattern to merge the output of all of the sharded blurring work queues into a single queue that will only release its items after all of the shards have completed the work.

Khi mỗi ảnh đã được làm mờ thành công, chúng ta sẽ tải nó lên một vị trí khác, và sau đó sẽ xóa các ảnh gốc. Tuy nhiên, chúng ta không muốn xóa ảnh gốc cho đến khi tất cả các ảnh đã được làm mờ thành công, phòng trường hợp có một dạng sự cố thảm khốc nào đó và ta cần chạy lại toàn bộ pipeline này. Vì vậy, để chờ tất cả việc làm mờ hoàn thành, chúng ta dùng mẫu join để gộp đầu ra của tất cả các hàng đợi công việc làm mờ được phân mảnh thành một hàng đợi duy nhất, hàng đợi này chỉ giải phóng các mục của nó sau khi tất cả các mảnh đã hoàn thành công việc.

Now we are ready to delete the original images as well as begin work on car model and color detection. Again, we want to maximize the **throughput** of this pipeline, so we will use the **copier pattern** from the previous chapter to duplicate the work queue items to two different queues:

Giờ chúng ta đã sẵn sàng xóa các ảnh gốc cũng như bắt đầu công việc nhận diện mẫu xe và màu xe. Một lần nữa, chúng ta muốn tối đa hóa thông lượng của pipeline này, nên ta sẽ dùng mẫu copier từ chương trước để nhân đôi các mục hàng đợi công việc sang hai hàng đợi khác nhau:

- A work queue that deletes the original images
- A work queue that identifies the type of vehicle (car, truck, motorcycle) and the color of the vehicle
 - Một hàng đợi công việc xóa các ảnh gốc
 - Một hàng đợi công việc nhận diện loại xe (ô tô, xe tải, xe máy) và màu của xe

Figure 12-4 shows these stages of the processing pipeline.

Hình 12-4 trình bày các giai đoạn này của pipeline xử lý.

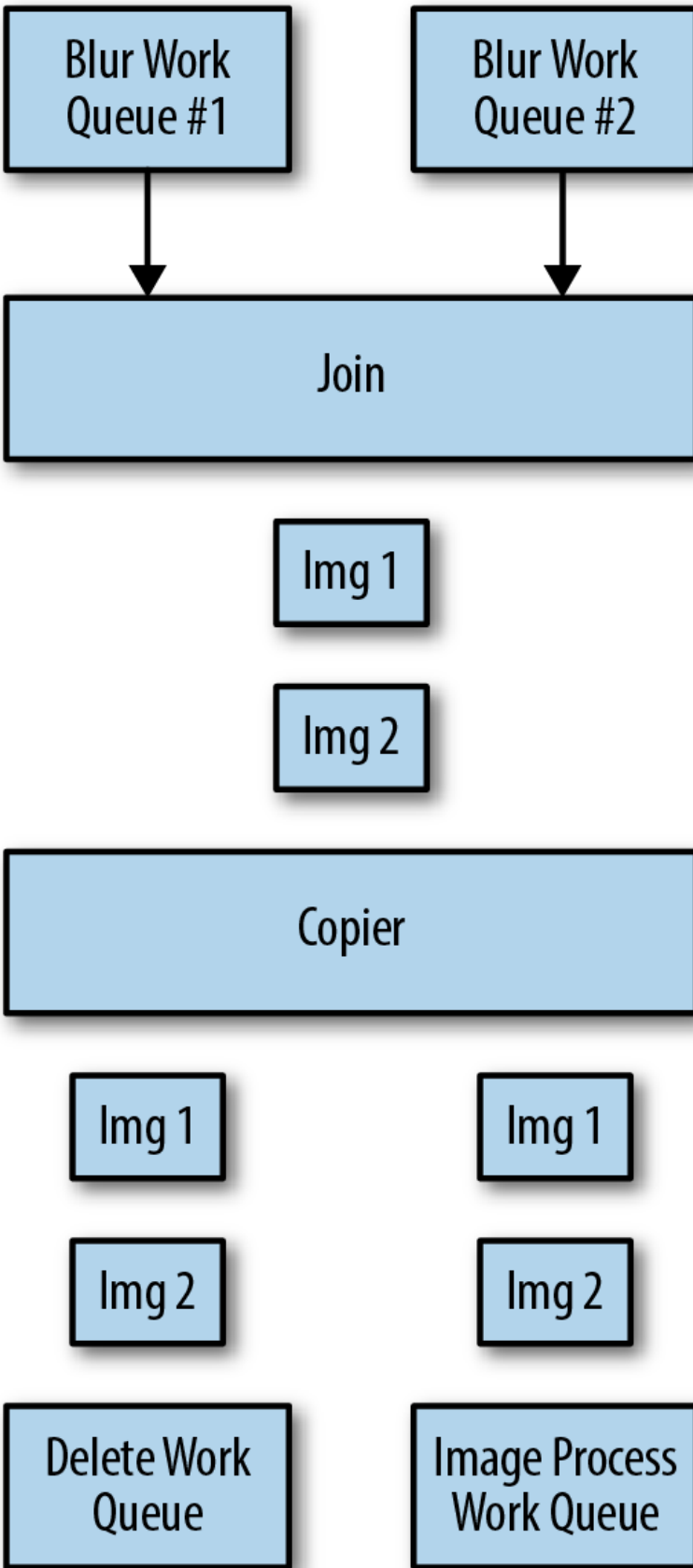


Figure 12-4. The output join, copier, deletion, and image recognition parts of the pipeline

Hình 12-4. Các phần join đầu ra, copier, xóa, và nhận diện ảnh của pipeline

Finally we need to design the queue that identifies vehicles and colors and aggregates these statistics into a final count. To do this, we first again apply the shard pattern to distribute the work out to a number of queues. Each of these queues has two different workers: one that identifies the location and type of each vehicle and one that identifies the color of a region. We will again join these together using the multi-worker pattern described in the previous chapter. As before, the separation of code into different containers enables us to reuse the color detection container for multiple tasks beyond identifying the color of the cars.

Cuối cùng chúng ta cần thiết kế hàng đợi nhận diện xe và màu sắc rồi tổng hợp các thống kê này thành một số đếm cuối cùng. Để làm điều này, trước tiên chúng ta lại áp dụng mẫu shard để phân phối công việc ra một số hàng đợi. Mỗi hàng đợi này có hai worker khác nhau: một worker nhận diện vị trí và loại của mỗi xe, và một worker nhận diện màu của một vùng. Chúng ta sẽ lại join chúng lại với nhau bằng mẫu multi-worker đã mô tả trong chương trước. Như trước, việc tách mã thành các container khác nhau cho phép chúng ta tái sử dụng container nhận diện màu cho nhiều tác vụ khác ngoài việc nhận diện màu của các xe.

The output of this work queue is a JSON tuple that looks like this:

Đầu ra của hàng đợi công việc này là một bộ JSON trông như thế này:

```
{
  "vehicles": {
    "car": 12,
    "truck": 7,
    "motorcycle": 4
  },
  "colors": {
    "white": 8,
    "black": 3,
    "blue": 6,
    "red": 6
  }
}
```

This data represents the information found in a single image. To aggregate all of this data together, we will use the reduce pattern described previously and made famous by MapReduce to sum everything together just as we did in the count example above. At the end, this reduce pipeline stage produces the final count of images and colors found in the complete set of images.

Dữ liệu này biểu diễn thông tin tìm thấy trong một ảnh đơn lẻ. Để tổng hợp tất cả dữ liệu này lại với nhau, chúng ta sẽ dùng mẫu reduce đã mô tả ở trên và được MapReduce làm cho nổi tiếng để cộng mọi thứ lại với nhau y như chúng ta đã làm trong ví dụ đếm ở trên. Đến cuối cùng, giai đoạn reduce của pipeline này tạo ra số đếm cuối cùng của các ảnh và các màu tìm thấy trong toàn bộ tập ảnh.

CHAPTER 13 Conclusion: A New Beginning? — CHAPTER 13 Kết luận: Một khởi đầu mới?

Every company, regardless of its origins, is becoming a digital company. This transformation requires the delivery of APIs and services to be consumed by mobile applications, devices in the internet of things (IoT), or even autonomous vehicles and systems. The increasing criticality of these systems means that it is necessary for these online systems to be built for redundancy, fault tolerance, and high **availability**. At the same time, the requirements of business necessitate rapid agility to develop and roll out new software, iterate on existing applications, or experiment with new user interfaces and APIs. The confluence of these requirements has led to an order of magnitude increase in the number of distributed systems that need to be built.

Mọi công ty, bất kể xuất phát điểm của mình, đều đang trở thành một công ty số. Quá trình chuyển đổi này đòi hỏi phải cung cấp các API và dịch vụ để được tiêu thụ bởi các ứng dụng di động, các thiết bị trong Internet vạn vật (IoT), hay thậm chí các phương tiện và hệ thống tự hành. Mức độ trọng yếu ngày càng tăng của các hệ thống này đồng nghĩa rằng những hệ thống trực tuyến này cần được xây dựng để đảm bảo dự phòng, dung sai lỗi và tính sẵn sàng cao. Đồng thời, các yêu cầu của doanh nghiệp đòi hỏi sự linh hoạt nhanh chóng để phát triển và tung ra phần mềm mới, lặp lại trên các ứng dụng hiện có, hoặc thử nghiệm các giao diện người dùng và API mới. Sự hội tụ của những yêu cầu này đã dẫn đến mức tăng theo cấp số nhân về số lượng hệ phân tán cần được xây dựng.

The task of building these systems is still far too difficult. The overall cost of developing, updating, and maintaining such a system is far too high. Likewise, the set of people with the capabilities and skills to build such applications is far too small to address the growing need.

Việc xây dựng những hệ thống này vẫn còn quá khó khăn. Tổng chi phí để phát triển, cập nhật và bảo trì một hệ thống như vậy vẫn còn quá cao. Tương tự, nhóm những người có đủ năng lực và kỹ năng để xây dựng các ứng dụng như vậy lại quá nhỏ so với nhu cầu đang ngày một tăng.

Historically, when these situations presented themselves in software development and technology, new abstraction layers and patterns of software development emerged to make building software faster, easier, and more reliable. This first occurred with the development of the first compilers and programming languages. Later, the development of **object-oriented programming** languages and managed code occurred. Likewise, at each of these moments, these technical developments crystallized the distillation of the knowledge and practices of experts into a series of algorithms and patterns that could be applied by a much wider group of practitioners. Technological advancement combined with the establishment of patterns democratized the process of developing software and

expanded the set of developers who could build applications on the new platform. This in turn led to the development of more applications and application diversity, which in turn expanded the market for these developer's skills.

Trong lịch sử, mỗi khi những tình huống như thế này xuất hiện trong phát triển phần mềm và công nghệ, các tầng trừu tượng và mẫu phát triển phần mềm mới đã ra đời để giúp việc xây dựng phần mềm trở nên nhanh hơn, dễ hơn và đáng tin cậy hơn. Điều này lần đầu xảy ra với sự ra đời của những trình biên dịch và ngôn ngữ lập trình đầu tiên. Về sau là sự xuất hiện của các ngôn ngữ lập trình hướng đối tượng và mã được quản lý (managed code). Tương tự, tại mỗi thời điểm đó, những bước phát triển kỹ thuật này đã kết tinh việc chất lọc kiến thức và thực hành của các chuyên gia thành một loạt thuật toán và mẫu thiết kế mà một nhóm người hành nghề rộng lớn hơn nhiều có thể áp dụng. Tiến bộ công nghệ kết hợp với việc thiết lập các mẫu thiết kế đã dân chủ hóa quá trình phát triển phần mềm và mở rộng nhóm lập trình viên có thể xây dựng ứng dụng trên nền tảng mới. Điều này lại dẫn đến việc phát triển nhiều ứng dụng hơn và đa dạng ứng dụng hơn, từ đó lại mở rộng thị trường cho kỹ năng của những lập trình viên này.

143 Again, we find ourselves at a moment of technological transformation. The need for distributed systems far exceeds our ability to deliver them. Fortunately, the development of technology has produced another set of tools to further expand the pool of developers capable of building these distributed systems. The recent development of containers and **container** orchestration has brought tools that enable rapid, easier development of distributed systems. With luck, these tools, when combined with the patterns and practices described in this book, can enhance and improve the distributed systems built by current developers, and more importantly develop a whole new expanded group of developers capable of building these systems.

143 Một lần nữa, chúng ta lại thấy mình đang ở một thời khắc chuyển đổi công nghệ. Nhu cầu về hệ phân tán vượt xa khả năng cung cấp chúng của chúng ta. May mắn thay, sự phát triển công nghệ đã tạo ra thêm một bộ công cụ mới để mở rộng hơn nữa nhóm lập trình viên có khả năng xây dựng các hệ phân tán này. Sự phát triển gần đây của container và bộ điều phối container (orchestrator) đã mang đến những công cụ cho phép phát triển hệ phân tán nhanh hơn và dễ dàng hơn. Với một chút may mắn, những công cụ này, khi kết hợp với các mẫu thiết kế và thực hành được mô tả trong cuốn sách này, có thể nâng cao và cải thiện các hệ phân tán được xây dựng bởi các lập trình viên hiện tại, và quan trọng hơn là phát triển một nhóm lập trình viên hoàn toàn mới và mở rộng có khả năng xây dựng những hệ thống này.

Patterns like sidecars, ambassadors, sharded services, FaaS, work queues, and more can form the foundation on which modern distributed systems are built. **Distributed system** developers should no longer be building their systems from scratch as individuals but rather collaborating together on reusable, shared implementations of canonical patterns that form the basis of all of the systems we collectively deploy. This will enable us to meet the demands of today's reliable, scalable APIs and services and empower a new set of applications and services for the future.

Các mẫu thiết kế như sidecar, ambassador, dịch vụ phân mảnh (sharded), FaaS, hàng đợi công việc (work queue) và nhiều mẫu khác có thể tạo nên nền tảng để xây dựng các hệ phân tán hiện đại. Lập trình viên hệ phân tán không nên tiếp tục xây dựng hệ thống của mình từ con số không với tư cách cá nhân, mà thay vào đó nên cộng tác cùng nhau trên những triển khai tái sử dụng, dùng chung của các mẫu thiết kế chuẩn mực vốn tạo nên cơ sở của mọi hệ thống mà chúng ta cùng nhau triển khai. Điều này sẽ giúp chúng ta đáp ứng được nhu cầu về các API và dịch vụ đáng tin cậy, có khả năng mở rộng của ngày nay, đồng thời trao sức mạnh cho một lớp ứng dụng và dịch vụ mới cho tương lai.